

Unit #: 1

Unit Name: Problem Solving Fundamentals

Topic: Introduction to C and C Features

Course objectives:

The objective(s) of this course is to make students

COBj1: Acquire knowledge on how to solve relevant and logical problems using computing machine

COBj2: Map algorithmic solutions to relevant features of C programming language constructs

COBj3: Gain knowledge about C constructs and it's associated eco-system

COBj4: Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors

COBj5: Get insights about testing and debugging C Programs

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs

CO2: Understand, analyse and apply text processing and string manipulation methods using C Arrays, Pointers and functions

CO3: Understand prioritized scheduling and implement the same using C structures

CO4: Understand and apply sorting techniques using advanced C constructs

CO5: Understand and evaluate portable programming techniques using preprocessor directives and conditional compilation of C Programs

Let us answer few questions before we start with C.

Q1. What is a Computer Programming Language (CPL)?

- Any set of rules that converts strings, or graphical program elements in the case of visual programming languages, to various kinds of **machine code output**.
- A **CPL** is an **artificial language** that can be used to control the behaviour of a machine, particularly a computer.
- **CPLs**, like human languages, are defined through the use of syntactic and semantic rules, to determine structure and meaning respectively.
- **CPLs** are used to implement **algorithms**.
- **CPLs allow us to give instructions to a computer** in a language the computer understands.
- **Formal computer language or constructed language** designed to communicate instructions to a machine, particularly a computer (wiki definition).
- Can be used to create programs **to control the behaviour of a machine**.

Q2. Why CPL?

- Advance our ability to develop real algorithms.
- Majority of CPLs come with a lot of features for the Computer Programmers - CP.
- CPLs can be used in a proper way to get the best results.
- Improve Customization of our Current Coding.
- By using basic features of the existing CPL we can simplify things to program a better option to write resourceful codes.
- There is no compulsion of writing code in a specific way, but rather is the usage of features used and clarity of the concept.

Q3. Why so many Programming Languages?

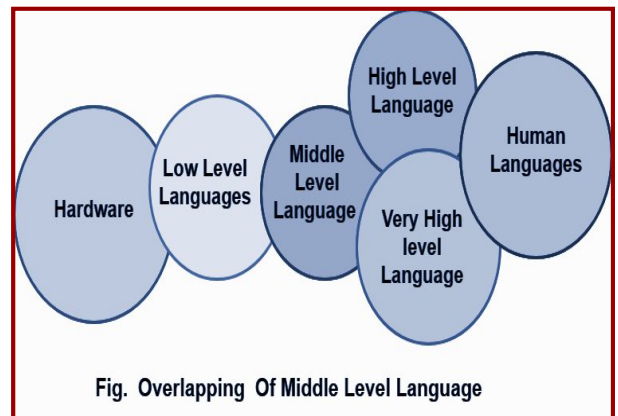
To choose the right language for a given problem. Example Domains are listed – Web Browsers, Social Networks, Image Viewer, Facebook.com, Bing, Google.com, Games, Various Operating Systems

Q4. What are the different Levels of Programming Language?

- **Low Level**

- Binary codes which CPU executes
- Programmer's responsibility is more
- Machine Language
- **Middle Level**
 - Offers basic data structure and array definition, but the programmers should take care of the operations
 - C and C++
- **High Level**
 - Programmer concentrates on the algorithm and programming itself
 - Java , Python, Pascal

<i>High Level</i>	<i>Middle Level</i>	<i>Low Level</i>
High level languages provide almost everything that the programmer might need to do as already built into the language	Middle level languages don't provide all the built-in functions found in high level languages, but provides all building blocks that we need to produce the result we want	Low level languages provides nothing other than access to the machines basic instruction set
Examples: Java, Python	C, C++	Assembler



Q5. What is the meaning of Paradigm?

A programming paradigm is a **style, or “way,” of programming**. Some languages make it easy to write in some paradigms but not all of them

Imperative: Programming with an explicit sequence of commands that update state –

Example: python

Declarative: Programming by specifying the result you want, not how to get it

Example: LISP, SQL

Structured: Programming with clean, goto-free, nested control structures

Example: C Language

Procedural: Imperative programming with procedure calls

Example: C Language.

Functional (Applicative): Programming with function calls that avoid any global state

Examples: Scheme, Haskell, Miranda and JavaScript.

Function-Level (Combinator): Programming with no variables at all

Examples: Scheme, Haskell, Miranda and JavaScript.

Object-Oriented: Programming by defining objects that send messages to each other. Objects have their own internal (encapsulated) state and public interfaces.

Class-based: Objects get state and behavior based on membership in a class.

Prototype-based: Objects get behavior from a prototype object.

Examples: Java, C++, Python

Event-Driven: Programming with emitters and listeners of asynchronous actions.

Flow-Driven: Programming processes communicating with each other over predefined channels.

Logic (Rule-based): Programming by specifying a set of facts and rules. An engine infers the answers to questions.

Constraint: Programming by specifying a set of constraints. An engine finds the values that meet the constraints.

Aspect-Oriented: Programming cross-cutting concerns applied transparently.

Reflective: Programming by manipulating the program elements themselves.

Array: Programming with powerful array operators that usually make loops unnecessary.

Q6: Which Languages are used while Developing Whatsapp? Think. !

Erlang, JqGrid, Libphonenumber, LightOpenId, PHP5, Yaws and many more...

Q7. Why should one learn C? What are the advantages of 'C'? Is not 'C' an outdated language?

We have to fill our stomach every day 3 or 4 times so that our brain and body get enough energy to function. How about eating Vidyarthi Bhavan Dosa every day? What about Fridays when the eatery is closed? Why not buy Dosa batter from some nearby shop? Or do you prefer to make the batter yourself? Would you have time to do that? Would that depend on how deep your pockets are? Would you like to decrease your medical bills?

Every language has a philosophy. The language used by poets may not be suitable for conversation. Poets use ambiguity in meaning to their advantage, and some verses in Sanskrit have

more than one meaning. But that will not be suitable for writing a technical report. The goal of ‘C’ is efficiency. The safety is in the hands of the programmer. ‘C’ does very little apart from what the programmer has asked for.

Example: When we index outside the bounds of a list in Python, we get an “index error” at runtime. To support this feature, Python runtime should know the current size of a list and should also check whether the index is valid each time we index on a list. You are all very good programmers, and I am sure you never get an index error. You get what you deserve. If you are lucky, the program crashes. Otherwise, something subtle may happen, which later may lead to catastrophic failures.

- C gives importance to efficiency
- C is not very safe; you can make your program safe
- C is not very strongly typed; mixing of types may not result in errors
- C is the language of choice for all hardware related softwares
- C is the language of choice for software’s like operating system, compilers, linkers, loaders, device drivers.

Q8. Is ‘C’ not an old language?

Yes and No.

It was designed by Dennis Ritchie –We use ‘C’ like languages and Unix like operating systems both have his contribution – in 70s. But the language has evolved over a period. The latest ‘C’ was revised in 2011.

There is one more reason to learn ‘C’. ‘C’ is the second most popular language as of now according to TIOBE index ratings. <https://www.tiobe.com/tiobe-index/>.

Q9. What is TIOBE Index?

- **The Importance Of Being Earnest” - TIOBE.**
- TIOBE is an indicator of the popularity of programming languages.

- The TIOBE index is updated once a month.
- The ratings are based on the number of skilled engineers world-wide, courses and third party vendors.
- Popular search engines such as Google, Bing, Yahoo!, Wikipedia, Amazon, YouTube and Bing are used
- It is not about the best programming language or the language in which most lines of code have been written.

Mar 2022	Mar 2021	Change	Programming Language	Ratings	Change
1	3	▲	 Python	14.26%	+3.95%
2	1	▼	 C	13.06%	-2.27%
3	2	▼	 Java	11.19%	+0.74%
4	4		 C++	8.66%	+2.14%
5	5		 C#	5.92%	+0.95%
6	6		 Visual Basic	5.77%	+0.91%
7	7		 JavaScript	2.09%	-0.03%
8	8		 PHP	1.92%	-0.15%
9	9		 Assembly language	1.90%	-0.07%
10	10		 SQL	1.85%	-0.02%

Q10: What is the history of PLs?

- 1820-1850 England, Charles Babbage invented two mechanical Computational device i.e., Analytical Engine and Difference Engine
- In 1942, United States, ENIAC used electrical signals instead of physical motion
- In 1945, Von Newman developed two concepts: Shared program technique and Conditional control transfer
- In 1949, Short code appeared
- In 1951, Grace Hopper wrote first compiler, A-0

- Fortran: 1957, John Backus designed.
- Lisp, Algol- 1958
- Cobol: 1959
- Pascal: 1968, Niklaus Wirth
- **C: 1972, D Ritchie**
- C++: 1983, Bjarne Stroustrup, Compile time type checking, templates are used.
- Java: 1995, J. Gosling, Rich set of APIs and portable across platform through the use of JVM
- **Development of C**
 - **Martin Richards, around 60's developed BCPL [Basic Combined Programming Language]**
 - **Enhanced by Ken Thompson and Introduced B language.**
 - **C is originally developed between 1969 and 1973 at Bell Labs by Dennis Ritchie and Kernighan. Closely tied to the development of the Unix operating system**
- **Standardized by the ANSI [American National Standards Institute] since 1989 and subsequently by ISO [International Organization for Standardization].**
 - **ANSI C, C89, C99, C11, C17, or C2x**

Introduction to C

Let us discuss the **applications of C** before digging into C.

Operating Systems: A high-level programming language built in the C programming language was used to construct the first operating system, which was UNIX. Later on, the C programming language was used to write Microsoft Windows and several Android apps. Symbian OS is used in cellular phones.

GUI (Graphical User Interface): Since the beginning of time, Adobe Photoshop has been one of the most widely used picture editors. It was created entirely with the aid of the C programming language. Furthermore, C was used to develop Adobe Illustrator and Adobe Premiere.

Embedded Systems and mechatronic systems with hardware interfaces: Because it is directly related to the machine hardware, C programming is often regarded as the best choice for scripting programs and drivers for embedded systems, among other things.

Mozilla: Internet Browser Firefox Uses C++.

Bloomberg: Provides real time financial information to investors

Callas Software: Supports PDF creation, optimization, updation tools and plugins

Features of C Language

Simple and Efficient:

The basic syntax style of implementing C language is very simple and easy to learn. This makes the language easily comprehensible and enables a programmer to redesign or create a new application. C is usually used as an introductory language to introduce programming to school students because of this feature.

Fast:

It is a well-known fact that statically typed programming languages are faster than dynamic ones. C is a statically typed programming language, which gives it an edge over other dynamic language. Also, unlike Java and Python, which are interpreter-based, C is a compiler-based program. This makes the compilation and execution of codes faster. Another factor that makes C fast is the availability of only the essential and required features. Newer programming languages come with numerous features, which increase functionality but reduce efficiency and speed. Since C offers limited but essential features, the headache of processing these features reduces, resulting in increased speed.

Portability:

Another feature of the C language is portability. To put it simply, C programs are machine-independent which means that you can run the fraction of a code created in C on various machines with none or some machine-specific changes. Hence, it provides the functionality of using a single code on multiple systems depending on the requirement.

Extensibility:

You can easily (and quickly) extend a C program. This means that if a code is already written, you can add new features to it with a few alterations. Basically, it allows adding new features, functionalities, and operations to an existing C program.

Function-Rich Libraries:

C comes with an extensive set of libraries with several built-in functions that make the life of a programmer easy. Even a beginner can easily code using these built-in functions. You can also create your user-defined functions and add them to C libraries. The availability of such a vast scope of functions and operations allows a programmer to build a vast array of programs and applications.

Dynamic Memory Management:

One of the most significant features of C language is its support for dynamic memory management (DMA). It means that you can utilize and manage the size of the data structure in C during runtime. C also provides several predefined functions to work with memory allocation. For instance, you can use the `free ()` function to free up the allocated memory at any time. Similarly, there are other functions such as `malloc ()`, `calloc ()`, and `realloc ()` to perform operations on data structure and memory allocations.

Modularity with Structured Language:

C is a general-purpose structured language. This feature of C language allows you to break a code into different parts using functions which can be stored in the form of libraries for future use and reusability. Structuring the code using functions increases the visual appeal and makes the program more organized and less prone to errors.

Mid-Level Programming Language:

Although C was initially developed to do only low-level programming, it now also supports the features and functionalities of high-level programming, making it a mid-level language. And as a mid-level programming language, it provides the best of both worlds. For instance, C allows direct manipulation of hardware, which high-level programming languages do not offer.

Pointers:

With the use of pointers in C, you can directly interact with memory. As the name suggests, pointers point to a specific location in the memory and interact directly with it. Using the C pointers, you can operate with memory, arrays, functions, and structures.

Recursion:

C language provides the feature of recursion. Recursion means that you can create a function that can call itself multiple times until a given condition is true, just like the loops. Recursion in C programming provides the functionality of code reusability and backtracking.

We will start exploring C Constructs in the upcoming lecture notes by solving some of the interesting Problems!

Happy Coding!

Unit #: 1

Unit Name: Problem solving fundamentals

Topic: Introduction to GCC, Program Structure, C Programming Environment, Errors in C program

Course objectives:

The objective(s) of this course is to make students

CObj1: Acquire knowledge on how to solve relevant and logical problems using computing machine

CObj2: Map algorithmic solutions to relevant features of C programming language constructs

CObj3: Gain knowledge about C constructs and it's associated eco-system

CObj4: Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors

CObj5: Get insights about testing and debugging C Programs

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs

CO2: Understand, analyse and apply text processing and string manipulation methods using C Arrays, Pointers and functions

CO3: Understand prioritized scheduling and implement the same using C structures

CO4: Understand and apply sorting techniques using advanced C constructs

CO5: Understand and evaluate portable programming techniques using preprocessor directives and conditional compilation of C Programs

Introduction to GCC

The **original author of the GNU C Compiler is Richard Stallman**, the founder of the GNU Project. **GNU** stands for **GNU's not Unix**. The GNU Project was started in 1984 to create a **complete Unix-like operating system as free software**, in order to promote freedom and cooperation among computer users and programmers. The first release of gcc was made in 1987, as the first portable ANSI C optimizing compiler released as free software. A **major revision of the compiler came with the 2.0 series in 1992**, which added the ability to compile C++. The acronym gcc is now used to refer to the “**GNU Compiler Collection**”

GCC has been extended to support many additional languages, including **Fortran, ADA, Java and Objective-C**. Its development is guided by the gcc Steering Committee, a group composed of representatives from gcc user communities in industry, research and academia

Features of GCC:

- A portable compiler, it runs on most platforms available today, and can produce **output** for many types of **processors**
- Supports **microcontrollers, DSPs** and **64-bit** CPUs
- It is not only a native compiler, it can also cross-compile any program, producing executable files for a different system from the one used by **gcc** itself.
- Allows software to be compiled for embedded systems
- **Written in C with a strong focus on portability**, and can compile itself, so it can be adapted to new systems easily
- It has multiple language frontends, for parsing different languages
- It can compile or cross-compile programs in each language, for any architecture Example: Can compile an ADA program for a **microcontroller** or a C program for a **supercomputer**
- It has a **modular design**, allowing support for new languages and architectures to be added.
- It is **free software**, distributed under the GNU General Public License (GNU GPL), which means we have the freedom to use and to modify gcc, as with all GNU software.
- gcc users have the freedom to share any enhancements and also make use of enhancements to gcc developed by others.

Installation of gcc on different Operating systems

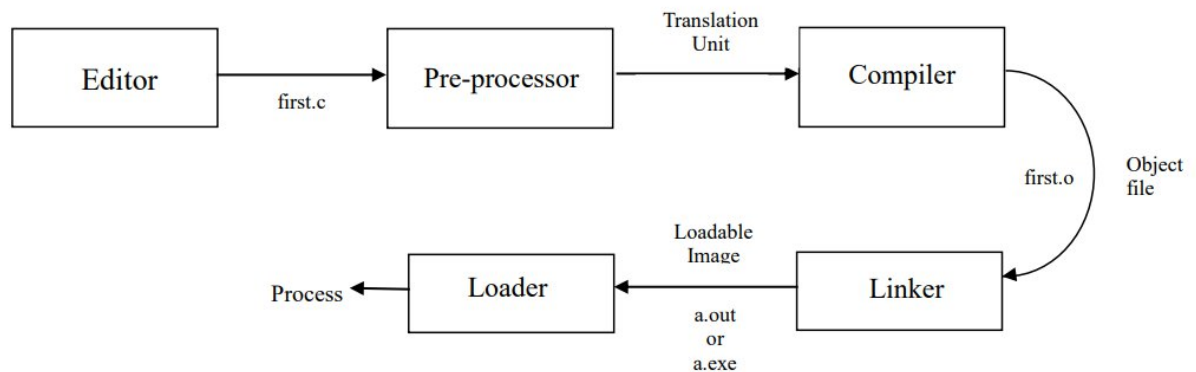
- Windows OS – Installation of gcc using Mingw.

<https://www.youtube.com/watch?v=sXW2VLrQ3Bs>

- Linux /MAC OS – gcc available by default

Program Development Life Cycle [PDLC]

Phases involved in PDLC are **Editing, Pre-processing, Compilation, Linking, Loading and execution.**



The stages for a C program to become an executable are the following:

1. Editing:

We enter the program into the computer. This is called editing. We save the **source program**. Let us create first.c

```
$ gedit first.c // press enter key
```

```
#include <stdio.h>
```

```
int main()
```

```
{    // This is a comment
```

```
    //Helps to understand the code Used for readability
```

```
    printf("Hello Friends");
```

```
    return 0;
```

```
// Save this code in first.c
```

2. Pre-Processing:

In this stage, the following tasks are done:

- a. Macro substitution–To be discussed in detail in Unit-5
- b. Comments are stripped off
- c. Expansion of the included files

The output is called a **translation unit or translation**.

To understand Pre-processing better, compile the above ‘first.c’ program using flag -E, which will print the pre-processed output to stdout.

```
$ gcc -E first.c// No file is created. Output of pre-processing on the standard output-terminal
```

```
.....  
# 846 "/usr/include/stdio.h" 3 4  
# 886 "/usr/include/stdio.h" 3 4  
extern void flockfile (FILE *__stream) __attribute__((__nothrow__));  
extern int ftrylockfile (FILE *__stream) __attribute__((__nothrow__));  
extern void funlockfile (FILE *__stream) __attribute__((__nothrow__));  
.....  
# 916 "/usr/include/stdio.h" 3 4  
# 2 "first.c"  
int main()  
{  
printf("HelloFriends");  
return0;  
}
```

In the above output, you can see that the source file is now filled with lots of information, but still at the end of it we can see the lines of code written by us. Some of the observations are as below.

- Comment that we wrote in our original code is not there. This proves that all the comments are stripped off.
- The ‘#include’ is missing and instead of that we see whole lot of code in its place. So it is safe to conclude that stdio.h has been expanded and literally included in our source file.

3. Compiling

Compilation refers to the process of converting a program from the textual source code into machine code, the sequence of 1's and 0's used to control the central processing unit (CPU) of the computer. Machine code is then stored in a file known as an executable file. C programs can be compiled from a single source file or from multiple source files, and may use system libraries and header files. Compiler processes statements written in a particular programming language and converts them into machine language or "code" that a computer's processor uses. The translation is **compiled**. The output is called an **object file**. There may be more than one translation. So, we may get multiple object files. When compiling with ‘-c’, the compiler automatically creates an object file whose name is the same as the source file, but with ‘.o’ instead of the original extension

```
gcc -c first.c
```

This command does pre-processing and compiling. Output of this command is first.o ->

Object file.

4. Linking

It is a computer program that takes one or more object files generated by a compiler and combine them into one, executable program. Computer programs are usually made up of multiple modules that span separate object files, each being a compiled computer program. This is the stage at which all the linking of function calls with their definitions are done. Till stage, gcc doesn't know about the definition of functions like printf(). Until the compiler knows exactly where all of these functions are implemented, it simply uses a place-holder for the function call. The definition of printf() is resolved and the actual address of the function printf() is plugged in. We put together all these object files along with the predefined library routines by **linking**. The output is called as

image or a loadable image with the name a.out [linux based] or a.exe[windows]

```
gcc first.o // Linking to itself
```

// If more than one object files are created after compilation, all must be mentioned with gcc and linked here to get one executable.

If a file with the same name[a.out/a.exe] as the executable file already exists in the current directory it will be overwritten. This can be avoided by using the gcc option -o. This is usually given as the last argument on the command line.

```
gcc first.o -o PESU // loadable image/Output file name is PESU
```

5.Executing the loadable image

Loader loads the image into the memory of the computer. This creates a **process** when command is issued to execute the code. We **execute or run** the process. We get some results. In ‘C’, we get what we deserve!

In windows,

a.exe and press enter OR

PESU and press enter

In Linux based systems,

./aout and press enter

./PESU and press enter

For interested students:

The following options of gcc are a good choice for finding problems in C and C++ programs.

```
gcc -ansi -pedantic -Wall -W -Wconversion -Wshadow -Wcast-qual -Wwrite-strings
```

Characteristics of a C program

- The language is **case sensitive**.
- The program is **immune to indentation**. ‘C’ follows the free format source code concept. The way of presentation of the program to the compiler has no effect on the logic of the program. However, we should properly indent the program so that the program becomes readable for humans. When you prepare a new dish, I am sure you will also consider how you present it.
- The program has a **well-defined entry point**. Every program has to have an entry point called main, which is the starting point of execution.

```
int main()           // execution begins here
{
    printf("Hello Friends");
    return 0;
}
```

Think: Why the main function has the return type int? Why not something else ? Can we have something else? Is it a good practice?

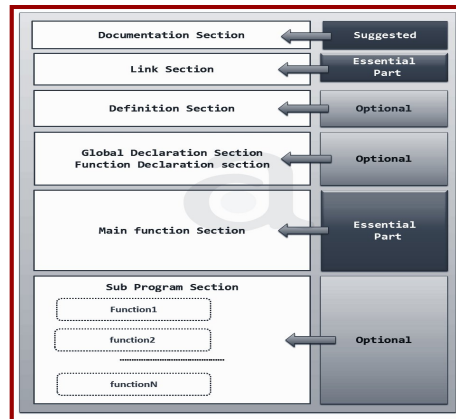
- A program in ‘C’ normally has one or more pre-processor directives. These start with a symbol #. One of the directives is “include”.
 - #include <stdio.h>

The above line asks the translator to find the file whose name is stdio.h at some location in our file system and read and expand it at that point in the ‘C’ code.

- A program in ‘C’ has to have a function called main. It is invoked(called) from some external agency – like our shell. It is the duty of this function to tell the caller whether it succeeded or failed. **So, by convention, main returns 0 on success and non-zero on failure.** Refer to coding example_1 for more details.
- A block contains a number of statements. Each statement ends in a semicolon. This symbol ; plays the role of statement terminator

Program structure

Consists of different sections of which few are optional and other few are mandatory sections. Refer to the below diagram.



Documentation Section: Optional but always good to have. Usually contains the information about the code in terms of comments

// Single line comment

/* multi line/ block

Comments */

Link Section: Provides instructions to the compiler to link functions from the system library such as using the #include directive. Basically Header files that are required to execute a C program are included in the Link section.

Definition Section: This is the optional section

Contains the definitions of functions before calling the functions

Global Declaration Section: This is the optional section

Contains global variables declaration and function declaration(no body, only prototype/signature)

Main function Section: Every C program must have one main function section. Contains two parts: **the declaration part and the executable part**. Declaration part declares all the variables used in the executable part. Executable part: There is at least one statement in the executable part. These two parts must appear between the opening and closing braces. The program execution begins at the opening brace and ends at the closing brace. The closing brace of the main function is the logical end of the program. All statements in the declaration and executable part end with a semicolon.

Sub program section: Optional section and contains user defined functions. We will discuss in Unit – 2.

Coding example_1: // Importance of return value in main

```
int main() // this code is saved as main.c
{
    printf("welcome to main function in C");
    return 0;
}
```

Execute the above code using below steps:

```
gcc -c main.c // press enter - main.o gets created if no error
gcc main.o // press enter - executable gets created if no error
a.exe // prints welcome to main function in C
```

If someone wants to know whether the recently executed process is executed successfully or not, this return value is very helpful.

After you get the output, use below commands to know what the return value of recently executed process.

```
echo %ERRORLEVEL% // in windows
echo $? // in linux based systems
```

Must return 0 to represent the success[by convention 0 is success]. Modify the code as below.

```
int main() // this code is saved as main.c
```



```
{  
    printf("welcome to main function in C");  
}
```

Repeat the same steps as mentioned in the first case, there is no way to get the success or failure info about the recently executed process. Results in some random value other than 0.

Coding example_2: // Compilation using different C Standards

__STDC_VERSION__: A macro defined in `stdio.h`

To understand better, use codes using **-E** option of GCC.

gcc -E

// file saved as `first.c`

```
int main()  
{  
    printf("C standard is %d\n",__STDC_VERSION__);  
    return 0;  
}
```

Preprocessing and Compilation using C99: **gcc -std=c99 -c first.c**

Linking remains the same: `gcc first.o`

Execute: `a.exe` OR `./a.out`

Output: **C standard is 199901**

Preprocessing and Compilation using C11: **gcc -std=c11 -c first.c**

Linking remains the same: `gcc first.o`

Execute: `a.exe` OR `./a.out`

Output: **C standard is 201112**

Preprocessing and Compilation using C89: **gcc -std=c89 -c first.c**

Results in Compile time Error as this macro is not available in C89 standard

Error(s) in C Code

Error is a mistake, the state or condition of being wrong in conduct or judgement, a measure of the estimated difference between the observed or calculated value of a quantity and its true value. Error is an illegal operation performed by the programmer which results in abnormal working of the program. Error is also known as bugs in the world of programming that may occur unwillingly which prevent the program to compile and run correctly as per the expectation of the programmer. We deal with four kinds of errors as of now.

Compile time Error

Link time Error

Run time Error

Logical Error

Compile time Error

Deviation from the rules of the Language or violating the rules of the language results in compile time Error. Whenever there is Compile Time Error, object file(.o file) will not be created. Hence linking is not possible. When the programmer does not follow the syntax of any programming language, then the compiler will throw the Syntax Error which are also compile time errors. Syntax Errors are easy to figure out because the compiler highlights the line of code that caused the error.

Coding Exampe_3:

```
#include<stdio.h>
int main()
{
    printf("hello friends);  //" missing at the end
    printf("%d",12/2)      // ; missing at the end of statement
return 0;
}
```

Link time Error

Errors encountered when the executable file of the code cannot be generated even though the code gets compiled successfully. This usually happens due to the mismatch between Function call and function definitions. When you compile the code, object file gets created. Sometimes with a warning. During linking, definition of Printf is not available in the included file. Hence Link time Error. No loadable object will be created due to this error. This Error is generated when a different object file is unable to link with the main object file.

Coding Exampe_4:

```
#include<stdio.h>

int main()
{
    Printf("hello friends"); // P is capital
    printf("%d",12/2);
    return 0;
}
```

Run time Error

Errors that occur during the execution (or running) of a program are called Runtime Errors. These errors occur after the program has been compiled and linked successfully. When the code is compiled, object file(.o file) will be created. Then during linking, executable will be created. When you run a loadable image, code terminates throwing an error. Generally occurs due to some illegal operation performed in the program. Examples of some illegal operations that may produce runtime errors are: Dividing a number by zero, trying to open a file which is not created and lack of free memory space.

Coding Exampe_5:

```
#include<stdio.h>

int main()
{
    printf("hello friends");
    printf("%d",12/0);           // Observe
    12/0printf("bye friends");
}
```

```
    return 0;  
}
```

Logical Error:

Even though the code seems error free, the output generated is different from the expected one. Logical error occurs when the code runs completely but the expected output is not same as the actual output.

Coding Example_6:

```
#include<stdio.h>  
  
int main ()  
{  
    printf("%d",12/21); //By Mistake entered 21  
                        //instead of 2  
    return 0;  
}
```

Warning(s) in C:

Warnings are **diagnostic messages that report constructions which are not inherently erroneous but which are risky or suggest there may have been an error or crash in future.** The compiler distinguishes between error messages, which prevent successful compilation, and warning messages which indicate possible problems but do not stop the program from compiling. It is very dangerous to develop a program without checking for compiler warnings. If there are any functions which are not used correctly they can cause the program to crash or produce incorrect results. Turning on the compiler warning option ‘-Wall’ for safety will catch many of the commonest errors which occur in c programming.

Why the warning must be enabled?

C and C++ compilers are notoriously bad at reporting some common programmer mistakes by default, such as:

- forgetting to initialise a variable
- forgetting to return a value from a function
- arguments in printf and scanf families not matching the format string
- a function is used without being declared beforehand (C only)

Most compilers understand options like **-Wall**, **-Wpedantic** and **-Wextra**. **-Wall** is essential and all the rest are recommended (note that, despite its name, **-Wall** only enables the most important warnings, not all of them). These options can be used separately or all together.

Error vs warning

Errors report problems that make it impossible to compile your program. GCC reports errors with the source file name and line number where the problem is apparent. Warnings report **other unusual conditions in your code that may indicate a problem, although compilation can proceed**.

Coding Example_7:

```
#include<stdio.h>

int main()
{
    int a;

    printf("enter the number\n");

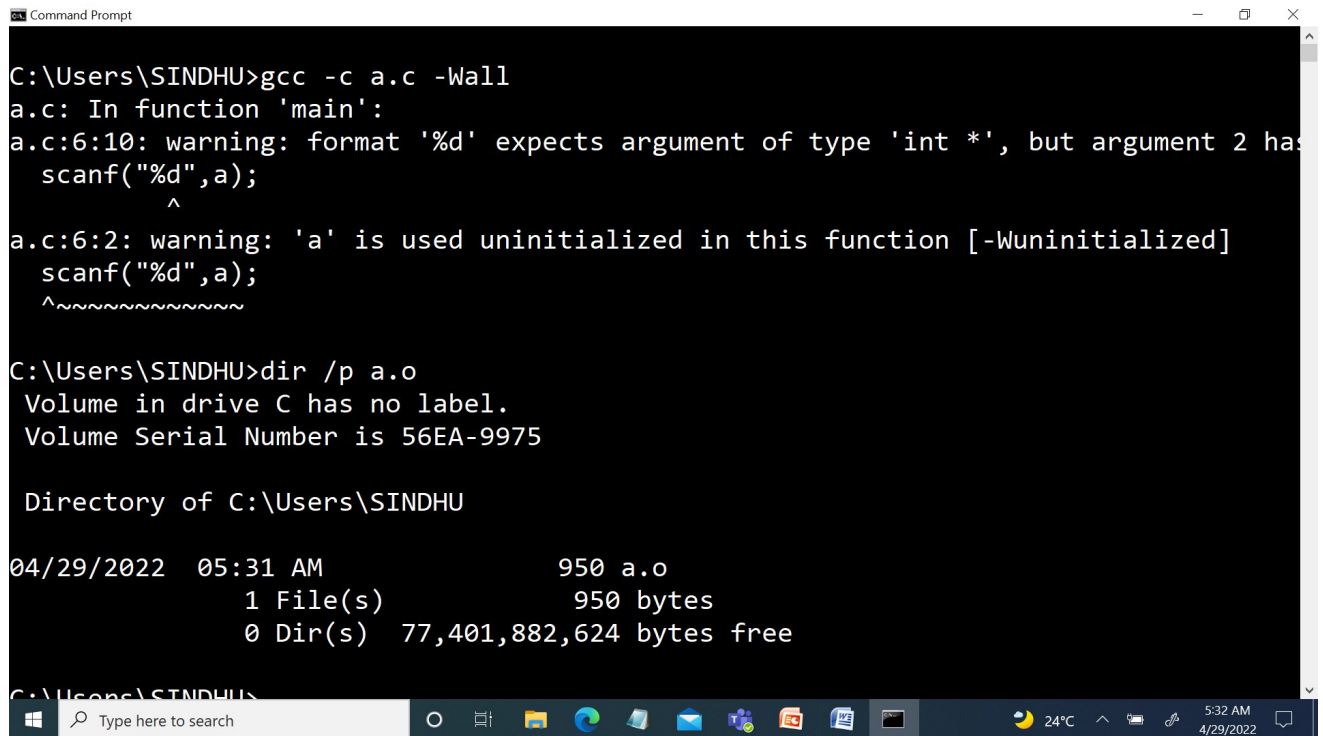
    scanf("%d",a);

    printf("you entered %d\n",a);

    return 0;
}
```

Without **-wall** option, there is no warning or error. But results in Runtime Error when the code is run using the executable.

The above code compilation using `-Wall` is as below. This creates the object file. But at runtime, it fails to execute



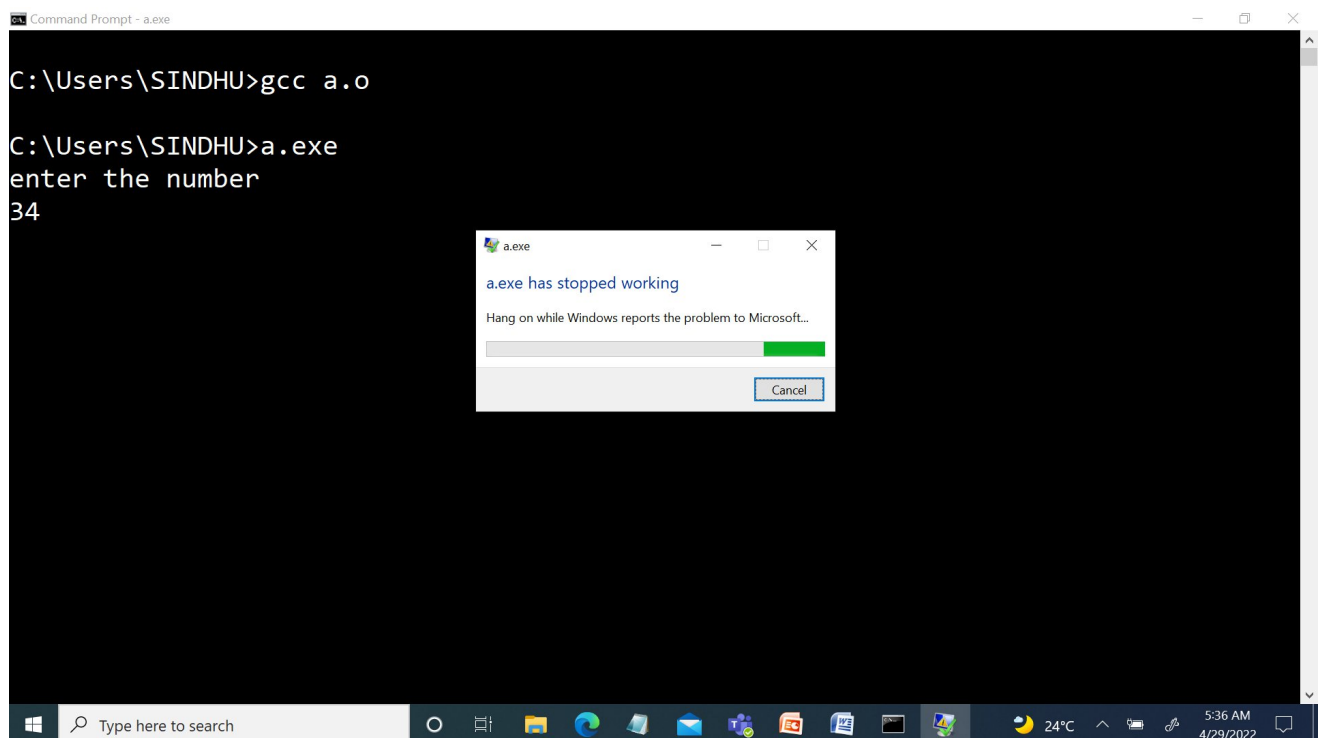
```
C:\Users\SINDHU>gcc -c a.c -Wall
a.c: In function 'main':
a.c:6:10: warning: format '%d' expects argument of type 'int *', but argument 2 has
scanf("%d",a);
      ^
a.c:6:2: warning: 'a' is used uninitialized in this function [-Wuninitialized]
scanf("%d",a);
      ^~~~~~

C:\Users\SINDHU>dir /p a.o
Volume in drive C has no label.
Volume Serial Number is 56EA-9975

Directory of C:\Users\SINDHU

04/29/2022  05:31 AM                950 a.o
              1 File(s)                950 bytes
              0 Dir(s)  77,401,882,624 bytes free

C:\Users\SINDHU>
```



```
C:\Users\SINDHU>gcc a.o

C:\Users\SINDHU>a.exe
enter the number
34
```

a.exe has stopped working

Hang on while Windows reports the problem to Microsoft...

Cancel

Happy Error Finding!!

Unit #: 1

Unit Name: Problem Solving Fundamentals

Topic: Basic constructs in C – Identifiers, Keywords, Variables and Data types, Range of values

Course objectives:

The objective(s) of this course is to make students

CObj1: Acquire knowledge on how to solve relevant and logical problems using computing machine

CObj2: Map algorithmic solutions to relevant features of C programming language constructs

CObj3: Gain knowledge about C constructs and it's associated eco-system

CObj4: Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors

CObj5: Get insights about testing and debugging C Programs

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs

CO2: Understand, analyse and apply text processing and string manipulation methods using C Arrays, Pointers and functions

CO3: Understand prioritized scheduling and implement the same using C structures

CO4: Understand and apply sorting techniques using advanced C constructs

CO5: Understand and evaluate portable programming techniques using preprocessor directives and conditional compilation of C Programs

Before we start with Problem Solving sessions, let us list few counting problems

1. Find the number of times a word is repeated in a given file
2. Count the number of different words that can be created using the given characters
Count the number of hosts in a network
3. Count the number of red pixels in a given image
4. What is the ratio of red:green:blue pixels in a given image
5. $A=\{a,b,c\}$ and $B=\{y,z\}$. How many ways are there to create a list of length 3 where the first and second value in the list come from A and the third value in the list comes from B.
6. For the sets $A=\{a,b,c\}$ and $B=\{x,y\}$ above, we want to count the number of lists of length 3 where the first two elements come from A, the third element comes from B, and where no two elements in the list are the same.
7. How many 7-digit numbers have alternating odd-even digits and no digit repeated.
8. ...

Problem to be solved at the end of Unit_#1: Count and display the number of characters, words and lines in a user input and in a given file.

Basic constructs in C

C program consists of various tokens and a **token can be any identifier -> a keyword, variable, constant, a string literal, or a symbol**. For example, the following C statement consists of five tokens.

```
printf("HelloFriends");
```

The individual tokens are –

```
printf  
(  
"HelloFriends"  
)  
;
```

Identifiers

An identifier is a **name used to identify a variable, function, or any other user-defined item**. An identifier **starts with a letter A to Z, a to z, or an underscore '_' followed by zero or more letters,underscores and digits (0 to 9)**. C does not allow few punctuation characters such as #, @ and % within identifiers. As C is a case-sensitive programming language, Manpower and manpower are two different identifiers in C.

Here are some examples of acceptable identifiers

```
_sum    stud_name  a_123  myname50  temp  j  
count123
```

Keywords

The keywords are identifiers which have special **meaning in C** and hence cannot be used as constants or variables. There are **32 keywords in C Language**.

Keywords in C Programming			
auto	break	case	char
const	continue	default	do
double	else	enum	extern
float	for	goto	if
int	long	register	return
short	signed	sizeof	static
struct	switch	typedef	union
unsigned	void	volatile	while

Variables

This is the most important concept in all languages. Variable is **a name given to a storage area that our programs can manipulate**. It has a **name, a value, a location and a type**. It also has something **called life, scope, qualifiers** etc. Variables are used to store information to be referenced and manipulated in a computer program. It provides a way of labeling data with a descriptive name, so our programs can be understood more clearly by the reader and ourselves. It is helpful to think of Variables as **containers that hold data or information**. **Purpose of Variables is to label and store data in memory, which then can be used throughout the program.**

Sometimes, the runtime makes variables with no names. These are called temporary variables. We cannot access them by name as we have no name in our vocabulary. A variable has a type. In 'C', type should be specified before a variable is ever used. We declare the variable with respect to type before using it. We cannot declare the variable more than once.

Examples: `int a; double b;`

Rules for naming a Variable aka User defined Identifier:

A Variable name **cannot be same as the keyword** in c Language. A Variable name **should not be same as the standard identifiers** in c Language. A Variable name can **only have letters (both uppercase and lowercase letters), digits and underscore**. The **first letter** of a Variable should be either a **letter or an underscore**. There is no rule on how long a variable name aka user defined identifier can be. Variable name may run into problems in some compilers, if the variable name is longer than 31 characters. C is a **strongly typed language**, which means that the variable type cannot be changed once it is declared.

The type of a variable can never change during the program execution. The type decides what sort of values this variable can take and what operations we can perform. Type is a compile time mechanism. **The size of a value of a type is implementation dependent and is fixed for a particular implementation.**

Variable is associated with three important terms: **Declaration, Definition and Initialization.**

- **Declaration of a variable** is for informing to the compiler the following information: **name of the variable, type of value it holds and the initial value if any it takes.** Declaration gives details about the properties of a variable.
- **Definition of a variable** says where the variable gets stored. i.e., memory for the variable is allocated during the definition of the variable. In C language, definition and declaration for a variable takes place at the same time. i.e. there is no difference between declaration and definition.

If we want to only declare variables and not to define it i.e. we do not want to allocate memory, then the following declaration can be used.

`extern int a; // We will discuss this in Unit - 5`

- **Initialization of a variable:** We can initialize a variable at the point of declaration. An uninitialized variable within a block has some undefined value. 'C' does not initialize any default value.

```
int c = 100;
```

```
int d; // undefined value !! variable can be assigned a value later in the code
```

```
int c = 200;
```

```
int c; // Declaration again throws an error
```

```
d = 300;
```

Note: Assignment is considered as an expression in 'C'.

Data Types

In programming, is a classification that specifies type of value and what type of mathematical, relational or logical operations can be applied to it without causing an error. It deals with the **amount of storage to be reserved for the specified variable.** Significance of data types are given below:

- 1) Memory allocation
- 2) Range of values allowed
- 3) Operations that are bound to this type
- 4) Type of data to be stored

Size of a type:

The size required to represent a value of that type. Can be found using an operator called `sizeof`. **The size of a type depends on the implementation.** We should never conclude that the size of an `int` is 4 bytes. Can you answer this question –How many pages a book has? What is the radius of Dosa we get in different eateries?

The **`sizeof` operator** is the most common operator in C. It is a **compile-time unary operator** and is used to compute the size of its operand. It returns the size of a variable/value/type. It can be applied to any data type, float type, pointer type variables. When `sizeof()` is used with the data types, it **simply returns the amount of memory allocated to that data type**. The output can be different on different machines like a 32-bit system can show different output while a 64-bit system can show different of same data types

The size of a type is decided on an implementation based on efficiency. C standards follow the below Rules.

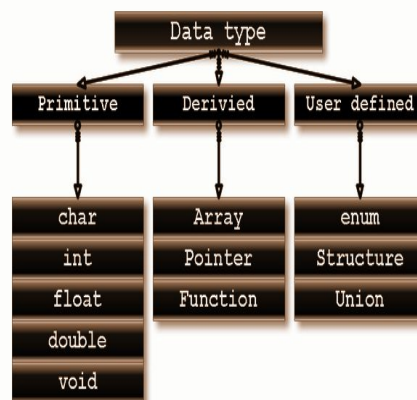
`sizeof(short int) <= sizeof(int) <= sizeof(long int) <= sizeof(long long int) <= sizeof(float) <= sizeof(double) <= sizeof(long double)`

Classification of Data types

Data types are categorized into **primary and non-primary (secondary) types**.

Primary ---> `int`, `float`, `double`, `char`, `void`

Secondary---> Derived and User-defined types



Data types can be extended using qualifiers:

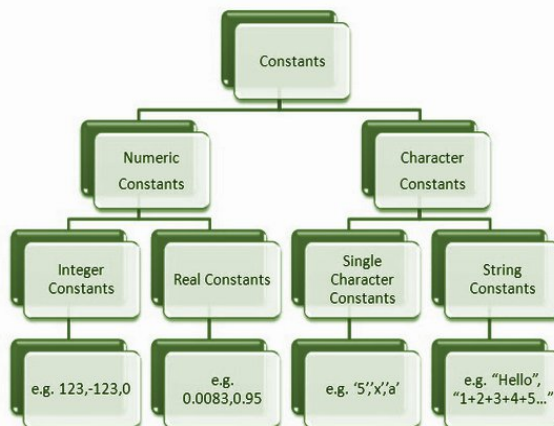
Size and Sign Qualifiers. We will discuss in detail in Unit - 5

Literals

Literals are the constant values assigned to the constant variables. There are four types of literals that exist in c programming:

1. **Integer literal**
2. **Float literal**
3. **Character literal**
4. **String literal – Will be discussed in Unit – 2**

Constant is basically a named memory location in a program that holds a single value throughout the execution of that program



Integer Literal: It is a numeric literal that represents only integer type values. Represents the value neither in fractional nor exponential part. Can be specified in the following three ways

Decimal number (base 10): It is defined by representing the digits between 0 to 9. Example: 1,3,65 etc

Octal number (base 8): It is defined as a number in which 0 is followed by digits such as 0,1,2,3,4,5,6,7. Example: 032, 044, 065, etc

Hexadecimal number (base 16): It is defined as a number in which 0x or 0X is followed by the hexadecimal digits (i.e., digits from 0 to 9, alphabetical characters from (a-f) or (A-F)) Example: 0XFE, 0xfe etc..

Float Literal: It is a literal that contains only **floating-point values or real numbers**. These real numbers contain the number of parts such as **integer part, real part, exponential part, and fractional part**. The floating-point literal must be specified either in **decimal or in exponential form**.

Decimal form: Must contain decimal **point, real part, or both**. If it does not contain either of these, then the compiler will throw an error. The decimal notation can be prefixed either by '+' or '-' symbol that specifies the positive and negative numbers. Example: +9.5, -18.738

Exponential form: The exponential form is useful when we want to represent the number, which is having a big magnitude. It contains **two parts, i.e., mantissa and exponent**. Example: the number is 3450000000000, and it can be expressed as 3.45e12 in an exponential form

Rules for creating an exponential notation: The mantissa can be specified either in decimal or fractional form. An exponent can be written in both uppercase and lowercase, i.e., e and E.

We can use both the signs, i.e., positive and negative, before the mantissa and exponent. Spaces are not allowed.

Character Literal: Contains a **single character enclosed within single quotes**. If we try to store more than one character in a character literal, then the warning of a multi-character character constant will be generated. A character literal can be represented in the following ways:

It can be represented by specifying a single character within single quotes.

Example: 'x', 'y', etc.

We can specify the escape sequence character within single quotes to represent a character literal.

Example, '\n', '\t', '\b'.

We can also use the ASCII in integer to represent a character literal.

Example: the ascii value of 65 is 'A'.

The octal and hexadecimal notation can be used as an escape sequence to represent a character literal.

Example, '\043', '\0x22'.

Think! What is the maximum and minimum integer value I can store in a variable?

The limits.h header file determines various properties of the various variable types.

The macros defined in this header, limits the values of various variable types like char, int and long. These limits specify that a variable cannot store any value beyond these limits. The values indicated in the table are implementation-specific and defined with the #define directive, but these values may not be any lower than what is given in the table.

Macro	Value	Description
CHAR_BIT	8	Defines the number of bits in a byte.
SCHAR_MIN	-128	Defines the minimum value for a signed char.
SCHAR_MAX	+127	Defines the maximum value for a signed char.
UCHAR_MAX	255	Defines the maximum value for an unsigned char.
CHAR_MIN	-128	Defines the minimum value for type char and its value will be equal to SCHAR_MIN if char represents negative values, otherwise zero.
CHAR_MAX	+127	Defines the value for type char and its value will be equal to SCHAR_MAX if char represents negative values, otherwise UCHAR_MAX.
MB_LEN_MAX	16	Defines the maximum number of bytes in a multi-byte character.
SHRT_MIN	-32768	Defines the minimum value for a short int.
SHRT_MAX	+32767	Defines the maximum value for a short int.
USHRT_MAX	65535	Defines the maximum value for an unsigned short int.
INT_MIN	-2147483648	Defines the minimum value for an int.
INT_MAX	+2147483647	Defines the maximum value for an int.
UINT_MAX	4294967295	Defines the maximum value for an unsigned int.
LONG_MIN	-9223372036854775808	Defines the minimum value for a long int.
LONG_MAX	+9223372036854775807	Defines the maximum value for a long int.

The **float.h** header file of the C Standard Library contains a **set of various platform-dependent constants** related to floating point values. These constants are proposed by ANSI C. They allow making more portable programs.

Coding Example_1:

```
#include<stdio.h>
#include<limits.h>
#include<float.h>
int main()
{
    int a = 8;
    char p = 'c';
    double d = 89.7;
    int e = 0113; // octal literal
    /*printf("%d\n",e);
    printf("%o\n",e);
```

```
printf("%X\n",e);
printf("%x\n",e);
*/
//int h = 0xRG; // error
int h = 0xA;
//printf("%d",h);
int i = 0b111;
//printf("%d",i);
//printf("%d %d %d %d\n",sizeof(int),sizeof(short int),sizeof(p),sizeof(long));
//int g = 787878787888888787;
int g = 2147483649;
/*printf("%d\n",INT_MAX);
printf("%d\n",INT_MIN);
printf("%d\n",INT_MIN);
printf("%d\n",INT_MIN);*/
printf("%d\n",CHAR_MAX);
printf("%g\n",DBL_MAX);
printf("%g\n",FLT_MAX);
printf("%d\n",g);
return 0;
}
```

Question for you to think!

- What is header file?
- What is Library file?
- Is there any difference between the above two?

Unit #: 1

Unit Name: Problem Solving Fundamentals

Topic: Simple Input functions in C

Course objectives:

The objective(s) of this course is to make students

CObj1: Acquire knowledge on how to solve relevant and logical problems using computing machine

CObj2: Map algorithmic solutions to relevant features of C programming language constructs

CObj3: Gain knowledge about C constructs and it's associated eco-system

CObj4: Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors

CObj5: Get insights about testing and debugging C Programs

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs

CO2: Understand, analyse and apply text processing and string manipulation methods using C Arrays, Pointers and functions

CO3: Understand prioritized scheduling and implement the same using C structures

CO4: Understand and apply sorting techniques using advanced C constructs

CO5: Understand and evaluate portable programming techniques using preprocessor directives and conditional compilation of C Programs

Input and Output functions in C

Input and output functions are available in the c language to perform the most common tasks. In every c program, three **basic functions take place namely accepting of data as input, the processing of data, and the generation of output.**

When a programmer says input, it would mean that they are feeding some data in the

program. Programmer can give this input from the command line or in the form of any file. The c programming language comes with a set of various built-in functions for reading the input and then feeding it to the available program as per our requirements.

When a programmer says output, they mean displaying some data and information on the printer, the screen, or any other file. The c programming language comes with various built-in functions for generating the output of the data on any screen or printer, and also redirecting the output in the form of binary files or text file.

Simple Input functions in C

Two Types: Formatted Input function and Unformatted Input function

Formatted Input function – scanf

scanf takes a format string as the first argument. The input should match this string.

int scanf(const char *format, ...);

where

int (integer) is the return type

format is a string that contains the type specifier(s).

"..." (ellipsis) indicates that the function accepts a variable number of arguments; each argument must be a memory address where the converted result is written to. On success, the function writes the result into the arguments passed.

- `scanf( "%d%d", &var1,&var2 );` // & -address operator is compulsory in scanf for all primary types

When the user types, 23 11, 23 is written to var1 and 11 is to var2.

- `scanf("%d,%d", &a, &b);`
scanf has a comma between two format specifiers, the input should have a comma between a pair of integers.
- `scanf("%d%d\n", &a, &b);`

It is not a good practice to have any kind of escape sequence in scanf. In the above code, it expects two integers. Ex: 20 30. Then if you press enter key, scanf does not terminate. You need to give some character other than the white space.

The function returns the following value:

>0 - The number of items converted and assigned successfully.

0 - No item was assigned.

<0 - Read error encountered or end-of-file (EOF) reached before any assignment was made.

- `n = scanf("%d",&a);` // If user enters 20, a becomes 20 and 1 is returned by the function.
- `n = scanf("%d,%d",&a,&b);` // If user enters 20 30, a becomes 20, value of b is undefined and 1 is returned by the function

Unformatted input function

A character in programming refers to a single symbol. A character in 'C' is like a very small integer having one of the 256 possible values. It occupies a single byte. We code English alphabets A as 65, B as 66, Z as 90, a as 97 and so on in a coding scheme called ASCII.

To read a character from the keyboard, we could use `scanf("%c", x);`

We could also use `x = getchar();`

We prefer the second as it is developed only for handling a single char and therefore more efficient even though it is not generic.

The unformatted input functions further have two categories: **Character functions and string functions**

Character functions: `getchar( ch)`, `getche()`, `getch()` where `ch` is the character variable

String functions: `gets` – This will be discussed in Unit - 2

Coding Example_1: To read two characters and display them.

```
int main()
{
    char ch = 'p'; // ch is a variable of type char and it can store only one character at a time.
```

```
//Value for a character variable must be within single quote. //
printf("Ch is %c\n",ch);// p
//char ch1 = 'pqrs';// TERRIBLE
CODEprintf("Ch is %c\n",ch);// s
/*
char x; char y;

scanf("%c", &x);
scanf("%c", &y);
printf("x:%c y:%c\n", x, y);
*/
x = getchar(); y = getchar(); putchar(x);
putchar(y); printf("%d", y);

return 0;
}
// If I enter P<newline>, x becomes P and y becomes newline
// scanf and printf : generic; not simple
// getchar and putchar : not generic; read /write a char; simple
// If I enter p<space>q, q will not be stored in y. Only space is stored. This can be avoided using
fflush(stdin) function between two getchar function calls. This function clears the key board
buffer.

// fflush(stdin) –windows
//_fpurge(stdin) – Linux based. Include <stdio_ext.h> in Linux based
```

Note: While using scanf for reading character input, care should be taken to handle White Spaces and Special Characters handling buffering problem

Unit #: 1

Unit Name: Problem Solving Fundamentals

Topic: Operators in C

Course objectives:

The objective(s) of this course is to make students

CObj1: Acquire knowledge on how to solve relevant and logical problems using computing machine

CObj2: Map algorithmic solutions to relevant features of C programming language constructs

CObj3: Gain knowledge about C constructs and it's associated eco-system

CObj4: Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors

CObj5: Get insights about testing and debugging C Programs

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs

CO2: Understand, analyse and apply text processing and string manipulation methods using C Arrays, Pointers and functions

CO3: Understand prioritized scheduling and implement the same using C structures

CO4: Understand and apply sorting techniques using advanced C constructs

CO5: Understand and evaluate portable programming techniques using preprocessor directives and conditional compilation of C Programs

Operator

An operator is a symbol that tells the compiler to perform specific mathematical or logical functions. Operator is a symbol used for calculations or evaluations

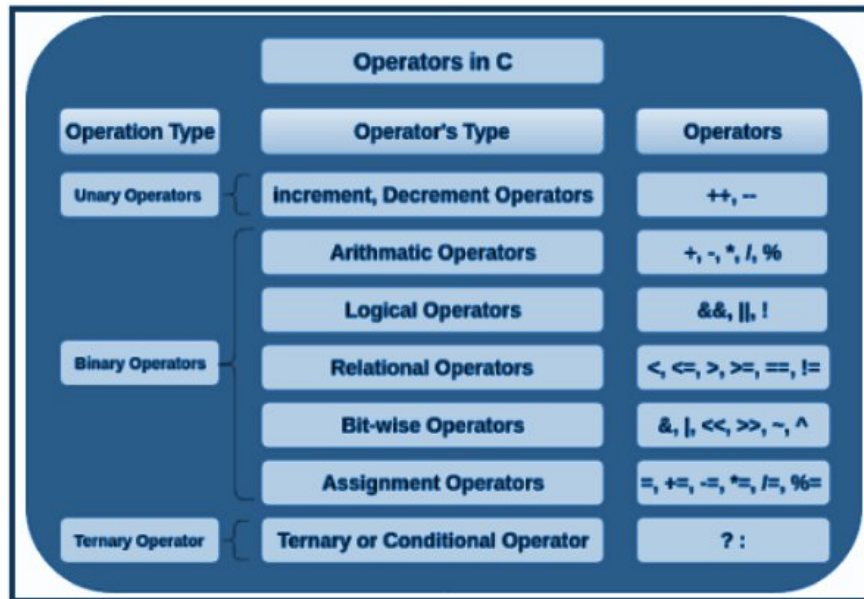
Classification of Operators:

Based on the number of operands.

- 1) Unary +, -, ++, --, size of, &, *
- 2) Binary +, -, *, /, %
- 3) Ternary ?:

Based on the operation on operands.

- 1) Arithmetic Operators
+, -, *, /, %
- 2) Increment and Decrement Operators
++, --
- 3) Relational
==, !=, >=, >, <=, <
- 4) Logical
&&, ||, !
- 5) Bitwise
&, |, ^, >>, <<
- 6) Address Operator and Dereferencing Operator
&, *
- 7) Assignment =
- 8) Short-hand
+=, -=, *=, /=, %=
- 9) Misc operators
sizeof



Arithmetic Operators in c

These operators are used to perform arithmetic/mathematical operation and give us results accordingly.

+ (Plus) – Give sum as a result. Example: 1+2=3

– (Minus) – Give the difference as a result. Example: -1-2=-3

* (Asterisk) – Used for multiplication and given the product as a result. Example: 2*2=4

/ (Forward slash) – Used for division and give quotient as a result. Example: 10/2 = 5

% (modulus) – Gives the remainder as a result. Example: 10 % 11 = 10; 17 %3 = 2

+ and –can also be used as unary operators –in such case, the operator is prefixed –appears to the left of the operand.

If the operands are int for /, the result is int quotient obtained truncation of the result. **The operator**

% is defined on integral types only.

If the operands are mixed types(int and double) in operations + -* /, the result is of type double.

There is no exponentiation operator in C.

Coding Example_1: Arithmetic operators in an expression

```
#include <stdio.h>

int main()
{
    printf("%d%d%d%d",2+3,4-2,4*2,5/2);    // 5,2,8,2

    printf("%d %f %f %f\n", 25 / 4, 25.0 / 4.0, 25.0 / 4, 25 / 4.0);

    // 6 6.25 6.25 6.25

    // mixed mode

    // 25.0 / 4 ==> 25.0 / (double)4.0

    return 0;
}
```

Increment and Decrement Operators

There are two types of **Increment (++)** and **Decrement (--)** Operators.

1. Pre -increment and Pre-decrement operators: First increment or decrement the value and then use it in the expression.
2. Post -increment and post-decrement operators: First use the value in the expression and then increment or decrement.

When ++ and -- are used in stand-alone expression, pre and post increment and decrement doesn't make difference.

Coding Example_1: Demonstration of pre and post increment/decrement in stand-alone expressions

```
#include<stdio.h>

int main()
```



```
{    int i=5;

    int j=5;

    print("Beginning i value is %d\n",i); //5

    print("Beginning j value is %d\n",i);

    ++i;// i++;

    printf("Later i value is %d\n",i);

    j--;// --j;

    printf("Later j value is %d\n",j);

    return 0;

}
```

Coding Example_2: Code which demonstrates the usage of ++ and -- in an expression.

```
#include<stdio.h>
```

```
int main()
```

```
{    int a=34; int b;

    printf("Beginning a value is %d\n",a);// 34

    b=a++; // b is 34 and a is incremented to 35

    //b=++a;// First increment and then use the value in the expression

    // b is 35 and a is 35

    printf("b is %d and a is %d\n",b,a);

    // same works for pre and post decrement operators

    return 0;

}
```

Relational Operators

Used to check the **relationship between the two operands**. If the relation is true, it returns 1. If the relation is false, it returns the value 0. Relational operators are used in **decision-making and loops constructs**. Programming language like C which doesn't support Boolean data type return result as 1 or 0.

Greater than (>) – Returns true (1) when the left operand value is greater than the right operand value. Example: $15 > 13$ is evaluated to 1.

Less than (<): Returns true(1) when the left operand value is less than the right operand value else false(0). Example: $15 < 13$ is evaluated to 0.

Greater than or equal to (>=): Return true(1) when the left operand value is greater than or equal to the right operand value. Example: $15 \geq 13$ is evaluated to 1.

Less than or equal to (<=): Return true(1) when the left operand value is less than or equal to the right operand. Example: $15 \leq 13$ is evaluated to 0.

Equal to (==): Returns true when left operand value is equal to right operand value. Example: $5 == 5$ is evaluated to 1.

Not Equal to (!=): Return true when left operand value is not equal to right operand. Example: $2 != 3$ is evaluated to 0

The result of relational operation is 1 or 0. 1 stands for true and 0 for false. **The relational operators should not be cascaded – not logically supported in C.**

Coding Example_3: Demo of cascading of operators in C

```
#include<stdio.h>

int main()
{
    int a=23;    int b=45;
    //printf("%d",5==5 && 5==5); //True results in 1
    printf("%d", 5 == 5 == 5); // results in 0
    return 0;
}
```

Bit-wise Operators

Used for performing bitwise operations on **bit patterns or binary numerals that involve the manipulation of individual bits. Bitwise operators always evaluate both operands.** Bitwise operators work on bits and **perform bit-by-bit operations.**

& (AND): Example: $a \& b$

| (OR): Example: $a | b$

^ (Exclusive OR (XOR)): Example: $a \wedge b$

~ (One's Complement (NOT)): Example: $\sim a$

>> (Shift Right): Example: $a \gg 1$

<< (Shift Left): Example: $a \ll 1$

Few inputs about carrying out arithmetic operation using bit wise operators

* multiply variable n by 2

$n \ll 1$

* check whether n is odd

$n \& 1$

* swap two variables a and b

$a = a \wedge b$

$b = a \wedge b$

$a = a \wedge b$

* check whether ith bit in variable n is 1

$n \& (1 \ll i)$: 0 implies not set and non 0 implies set

* set the ith bit in n

$n = n | 1 \ll i$

* clear the ith bit in n

$n = n \& \sim(1 \ll i)$

Coding Example_4: // Demo of bitwise operators

```
int main()
{
    int a=10;
    int b=8;
    printf("%d",a&b);    //8
    printf("%d",a|b);    //10
    printf("%d",a<<2);  //40
    printf("%d",a>>2);  //2
    printf("%d",~(10)); // -11
    return 0;
}
```

Assignment Operators

Used to assign a new value to a variable **Assignment operators** can also be used for **logical operations**, **bitwise logical operations** or **operations** on **integral** operands and **Boolean** operands.

= **Simple assignment operator**: Assigns values from right side operands to left side operands.

Example: $C = A + B$ will assign the value of $A + B$ to C

+= **Add AND assignment operator**: It adds the right operand to the left operand and assigns the result to the left operand.

Example: $C += A$ is equivalent to $C = C + A$

-= **Subtract AND assignment operator**: It subtracts the right operand from the left operand and assigns the result to the left operand.

Example: $C -= A$ is equivalent to $C = C - A$

*= **Multiply AND assignment operator**: It multiplies the right operand with the left operand and assigns the result to the left operand.

Example: $C *= A$ is equivalent to $C = C * A$

/= **Divide AND assignment operator**: It divides the left operand with the right operand and assigns the result to the left operand.

Example: $C /= A$ is equivalent to $C = C / A$

`%=`: Modulus AND assignment operator: It takes modulus using two operands and assigns the result to the left operand.

Example: `C %= A` is equivalent to `C = C % A`

Ternary/Conditional operator

`?:` Conditional Operator and it requires three operands. `E1 ? E2 : E3` where `E1`, `E2` and `E3` are expressions. The expression `E1` is first evaluated. If it is true, then the value of the expression is `E2` else it is `E3`.

Coding Example_5: Demo of `?:`

```
int main()
{
    int a= 10; int b = 20;
    printf("%d\n",(a>b)?a:b);           //20
    (a>b)? printf("%d",a):printf("%d",b); //20
    printf("%d",(a>b)?a:b);           //20
    return 0;
}
```

Logical Operators

Logical Operators are operators that determine the **relation between 2 or more operands** and **return a specific output as a result of that relation**.

NOT (!) – Used to negate a Logical statement.

AND (&&) and OR (||) – Used to combine simple relational statements into more complex Expressions. In ‘C’, 0 is false and any non-zero value is true. C follows **short circuit evaluation**. Evaluation takes place left to right and evaluation stops as soon as the truth or falsehood of the expression is determined.

Note: Some of the operators support Sequence Point Operation.

Sequence Point Operation

How do we write safe and proper code in ‘C’ if operators have side effects and the variable values are not defined in such cases? The language ‘C’ also **specifies points in the code beyond which all these effects will definitely be complete. These are called sequence points.**

It defines **any point in the execution of a program which guarantees or ensures that all the side effects the previous evaluation of the program’s code are done or successfully performed.** It ensures that **none of the alterations or side effects of the subsequent evaluations is yet to be performed at all.** The term “side effects” is nothing but **the changes done by any sort of function or expression where the state of something gets changed.**

There are some of the basic sequence points available in C Language.

- Logical AND - &&
- Logical OR - ||
- Conditional Operator - ?:
- Comma Operator - ,
- Expression of while loop
-

```
// int a = 10;
```

```
// to support short ckt evaluation, && becomes a sequence point
```

```
// expression before &&, || will be completely evaluated.
```

```
//a++ == 10 && a == 11 // ok; will be true 1
```

a + a++ is undefined as the value of left a is not defined.

a++ == 10 && a == 11 will be true.

The value a++ is 10 and a becomes 11 before the evaluation moves across the sequence point &&.

Coding Example_6: //

```
#include<stdio.h>
```

```
int main()
```

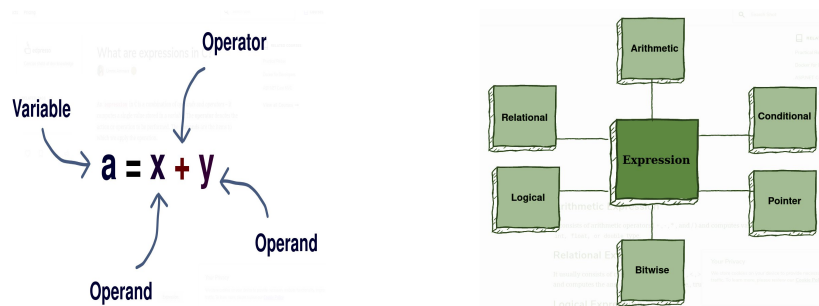
```
{
```

```
    int a = 11;
```

```
printf("%d",a++==12 || a==12); // 1
printf("\n%d",a); // 12
return 0;
}
```

Expression in C

An expression in C is a **combination of operands and operators**. Expressions compute a single value stored in a variable. The **operator denotes the action or operation** to be performed. The **operands are the items to which we apply the operation**.



An expression can be defined depending on **the position and number of its operator and operands**.

Infix Expression: Operator is used between the operands

$a = x + y$

Postfix Expression: Operator is used after the operands

$xy+$

Prefix Expression: Operator is used before the operands

$+xy$

Unary Expression: One operator and one operand

$++x$

Binary Expression: One operator and two operands

$x+y$

Arithmetic Expression: It consists of arithmetic operators ($+$, $-$, $*$, and $/$) and computes values of int, float, or double type.

Relational Expression: It consists of comparison operators ($>$, $<$, $>=$, $<=$, $==$, and $!=$) and

computes the answer in true (1) or false (0)

Logical Expression: It consists of logical operators (&&, ||, and !) and combines relational expressions to compute answers in true (1) or false (0)

Conditional Expression: It consists of conditional statements that return true(1) if the condition is met and else false(0)

Pointer Expression: It consists of an ampersand (&) operator and returns address values.

Bitwise Expression: It consists of bitwise operators (>>, <<, ~, &, |, and ^) and performs operations at the bit level.

Evaluation of an expression involves two parts:

1. Evaluation of operands:

This is **fetching the operands from the memory of the computer to the registers of the CPU. All the evaluations happen within the CPU. This order is not defined.** The idea is to allow the compiler to optimize fetching of the operands. The compiler may generate code such a way that if an operand is used more than once, it may get fetched only once.

2. Evaluation of operators:

This follows the **rules of precedence** and if more than one operator has the same level of precedence, follows the **rules of association**.

Table of operator Precedence and Associativity is shown below and Link for reference:
<http://web.cse.ohio-state.edu/~babic.1/COperatorPrecedenceTable.pdf>

Does the first rule have any effect on our programs? Yes. It affects our programs if the expressions we use are not pure and have side effects.

All assignments have side effects.

Let us consider an example.

```
a = 10;
```

```
b = a * a++;
```


The value of `a++` is the old value of `a` and therefore 10. What is the value of the leftmost `a`? Is the value evaluated before incrementing or after incrementing? It depends on the compiler and the compiler options. So value of leftmost `a` is undefined. So, value of `b` is undefined.

```
// 10 * 10 => 100
```

```
// 11 * 10 => 110
```

```
//undefined; depends on the order of evaluation of operands
```

```
// bad code
```

C Operator Precedence Table

C operators are listed in order of *precedence* (highest to lowest). Their *associativity* indicates in what order operators of equal precedence in an expression are applied.

Operator	Description	Associativity
() [] . -> ++ --	Parentheses: grouping or function call Brackets (array subscript) Member selection via object name Member selection via pointer Postfix increment/decrement	left-to-right
++ -- + - ! ~ (type) * & sizeof	Prefix increment/decrement Unary plus/minus Logical negation/bitwise complement Cast (convert value to temporary value of <i>type</i>) Dereference Address (of operand) Determine size in bytes on this implementation	right-to-left
* / %	Multiplication/division/modulus	left-to-right
+ -	Addition/subtraction	left-to-right
<< >>	Bitwise shift left, Bitwise shift right	left-to-right
< <= > >=	Relational less than/less than or equal to Relational greater than/greater than or equal to	left-to-right
== !=	Relational is equal to/is not equal to	left-to-right
&	Bitwise AND	left-to-right
^	Bitwise exclusive OR	left-to-right
	Bitwise inclusive OR	left-to-right
&&	Logical AND	left-to-right
	Logical OR	left-to-right
? :	Ternary conditional	right-to-left
= += -= *= /= %= &= ^= = <<= >>=	Assignment Addition/subtraction assignment Multiplication/division assignment Modulus/bitwise AND assignment Bitwise exclusive/inclusive OR assignment Bitwise shift left/right assignment	right-to-left
,	Comma (separate expressions)	left-to-right

Unit #: 1

Unit Name: Problem Solving Fundamentals

Topic: Control Structures

Course objectives:

The objective(s) of this course is to make students

CObj1: Acquire knowledge on how to solve relevant and logical problems using computing machine

CObj2: Map algorithmic solutions to relevant features of C programming language constructs

CObj3: Gain knowledge about C constructs and it's associated eco-system

CObj4: Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors

CObj5: Get insights about testing and debugging C Programs

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs

CO2: Understand, analyse and apply text processing and string manipulation methods using C Arrays, Pointers and functions

CO3: Understand prioritized scheduling and implement the same using C structures

CO4: Understand and apply sorting techniques using advanced C constructs

CO5: Understand and evaluate portable programming techniques using preprocessor directives and conditional compilation of C Programs

Introduction

A program is nothing but the execution of sequence of one or more instructions. It is most of the time desirable to alter the sequence of the statements in the program depending upon certain circumstances or as per the logic. Involves decision making to see whether a particular condition has been met or otherwise and direct the computer to execute certain statements accordingly. Based on logic, it becomes necessary to **alter the flow of a program, test the logical conditions and control the flow of execution** as per the selection of these conditions that can be placed in the program using decision-making statements

Control Structures

C programming language supports a few looping and a few selection structures. It may also support some unconventional control structures.

1) Looping structures:

- while statement
- for statement
- do ... while statement

2) Selection structures/ Decision Making/ Conditional structures:

- Simple if Statement
- if – else Statement
- Nested if-else statement
- else if Ladder
- switch statement

3) Unconditional structures:

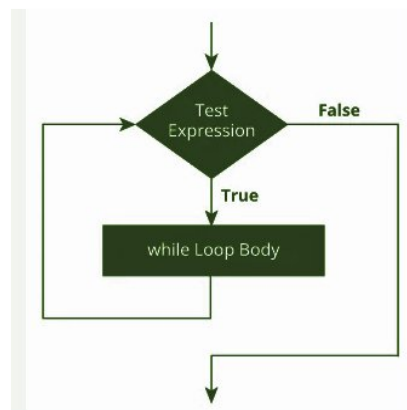
- goto statement
- break Statement
- continue Statement
- return Statement

Looping

Let us start the discussion with the while statement. The while statement starts with the **keyword while** and is **followed by an expression in parentheses** and is followed by a statement or a block.

```
while(Expression)
{
    Statement(s); // block
}
```

The expression of the while is evaluated. It could be non-zero or zero. If it is non-zero, the body of the while is executed and again the control comes back to the expression of the while. If the expression of the while is false, the loop is exited.



The loop **evaluates** the **Test Expression** inside the **parentheses()**. If Test Expression is TRUE, statements inside the body of while loop are executed. For looping to continue Test Expression has to be evaluated again as TRUE; terminated if the evaluation results in FALSE.

It is a good programming practice to always use a block even if only one statement is in the body of the loop. **The body of the while loop is executed 0 or more times** as the condition to be checked is at the top.

Coding Example_1: Display all the numbers from n to 0.

Version 1: Results in **infinite** loop as n remains to be 5 → is always true

```
int n = 5;
while(n
```

```
printf("%d ", n);  
n= n -1;//control will never execute this statement
```

Version 2: Displays 5 4 3 2 1. But the program layout does not indicate the logic structure of the program. Always indent the code to indicate the logic structure

```
int n = 5; while(n){  
    printf("%d ", n);  
    n = n -1;}
```

Version 3: Indent the code. Always use the block.

```
int n = 5;  
while(n)  
{  
    printf("%d ", n); // output : 5 4 3 2 1. This is nice.  
    n = n -1;  
}
```

Version 4: Observe the value of `n--` is the old value of `n`. When `n` is 1, the value of `n` becomes 0 but the decrement operator returns 1.

```
int n = 5;  
while(n)  
{  
    printf("%d ", n--); // 5 4 3 2 1  
}
```

Version 5: Pre-decrement operator returns the changed value.

```
int n = 5;  
while(n)  
{  
    printf("%d ", --n); // 4 3 2 1 0  
}
```

Version 6: The expression of while acts like a sequence point.

```
int n = 5;
while(n--)
{
    printf("%d ", n); // 4 3 2 1 0
}
```

Observe that the old value of *n* is used to check whether the condition is true or false and by the time, the body of the loop is entered, *n* would have the decremented value.

Version 7:

```
int n = 5;
while(--n)
{
    printf("%d ", n); // 4 3 2 1
}
```

Compare the last two versions. In the first case, the loop is executed *n* times and in the second the loop is executed *n* – 1 times. In the first case, the value of *n* is -1 and in the second case, it is 0.

Coding Example_2: Find the sum of numbers from 1 to *n*.

```
int n; // read n from the user
int i = 1; // initialization
int sum = 0;
while(i <= n) // condition
{
    sum += i++; //equivalent to sum = sum + i
                // modification
}
```

There is clearly some **initialization**, some **processing** and some **modification** at the end of the loop. In such cases, we may want to use the **for statement**.

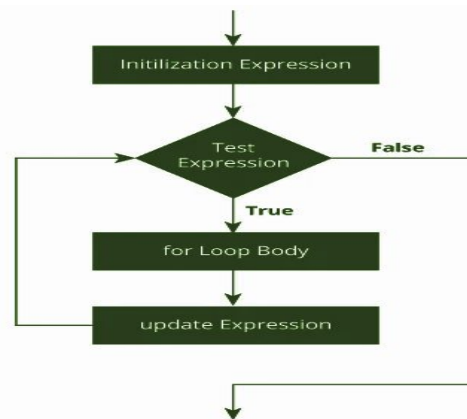
```
// <for stmt>:: for(e1; e2; e3) <block>|<stmt>
```

```
// e1, e2, e3 : expressions
```

```
// e1 : initialization
```

```
// e2 : condition
```

```
// e3 : modification
```



For loop in Detail:

e1 is executed once. e2 is checked for condition. If it is non-zero, execute the body of the loop and then execute e3. Again, check for e2 and thus the process continues till e2 results in Zero Value. Once e2 is Zero, come out of the loop and execute the statement outside the body of the loop. **for loop is also called as a pre-tested loop. The general structure of for loop syntax in C is**

```
for (initial value; condition; increment or decrement )
{
    statements;
}
```

Note: The semantics of for statement is same as the while statement.

Few questions to Answer:

- Can you have initialization outside the for loop and modification inside the body of the loop?
- Can you skip condition in for loop?
- Can you have only ;; in for loop?

Coding Example_2: What this code does? Why it is wrong?

```
int sum = 0;
// wrong code
for(int i = 1; i <= n; sum += i)
{
    ++i;
}
```

Observe closely. You will find that the summation starts from 2 and not from 1. We will also end up adding $n + 1$. Check the way a for statement executes.

```
int sum = 0;
for(int i = 1; i <= n; ++i)
{
    sum += i;
}
```

‘C’ for statement is same as the ‘C’ while statement –we say they are isomorphic. It is a question of choice and style while writing programs in ‘C’.

Coding Example_3: Find what the digital root of a given number

The idea is to add the digits of a given number. If that sum exceeds a single digit, repeat the operation until the sum becomes a single digit. This is useful in parity checking.

```
for(s = 0; n; n /= 10)
{
    s += n % 10;
}
```

This loop finds the sum of digits of n . The result is in s and n would have become 0. If s exceeds 9, we would like to copy s to n , repeat this operation.

Can we put this code under a `while(s > 9) { <this code> }`?

The answer is a NO as the value of s is not initialized until we enter the inner loop? Shall we initialize to 0? Then the loop itself is not entered. How about making s 10? Looks very unnatural.

Can you realize here that the sum becomes available only after doing the summation once?

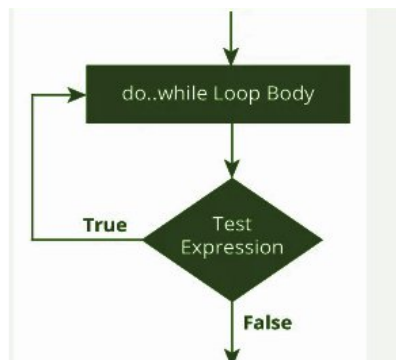
We require in this case, a bottom tested looping structure which executes the code at least once

Here is the solution using do – while.

```
do
{
    for(s=0; n; n/2)
    {
        s+=n%10
    }
    n=s;
}while(s>9);
```

do –while loop:

The body of do...while loop is executed at least once. For loop to continue, the test expression is evaluated for TRUE; terminated if the evaluation results in FALSE.

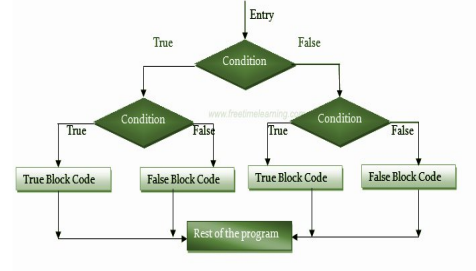
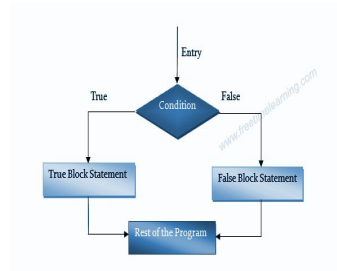
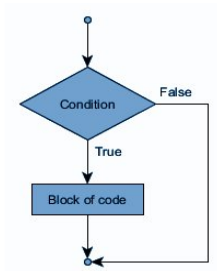


Selection statements

If the expression of if is true we execute the block following if, otherwise execute the else part if any. It is always preferred to use a block even if there is a single statement in the if and else portions.

```
//<if stat> ::= if (<expr>) <stat>|<block>
```

```
//<if stat> ::= if (<expr>) <stat>|<block> else <stat>|<block>
```



Coding Example_4: Classify triangles given the 3 sides as equilateral, isosceles or scalene. This is one possible solution.

```
int count = 0;
scanf("%d%d%d", &a, &b, &c);
if(a == b) ++count;
if(b == c) ++count;
if(c == a) ++count;
if(count == 0) printf("scalene\n");
if(count == 3) printf("equi\n");
if(count == 1) printf("iso\n");
```

Compare every pair of sides and increment the count each time –initialized to 0 on start. Can the count be 2?

Observe a few points here.

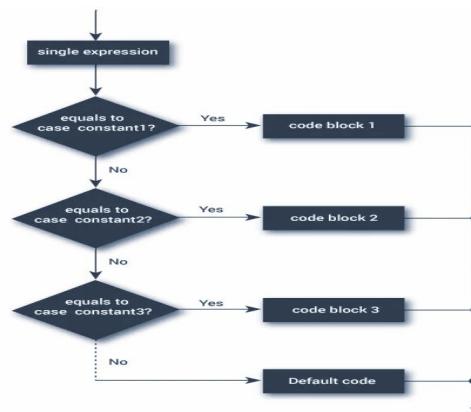
- We are comparing integers (integral values)
- We are comparing a variable (an expression) with constants
- We are comparing for equality (no > or <)
- In all comparisons, the same variable is used.

In such cases, we may use switch statement.

```
switch(count)
{
    case 0: printf("scalene\n"); break;
    case 3: printf("equi\n"); break;
    case 1: printf("iso\n"); break;
}
```

The value of count is compared with case clauses. This comparison decides the entry into the switch and not exit from there. To avoid the code fall through, we use break statement. The rule of 'C' is **"break if switch"**.

If we do not use the break statement, all statements after the matching label are also executed. These switch statements are more efficient compared to normal if statements. But can you include else part of if in switch case? -- Yes using default. But the default clause inside the switch statement is optional.



Coding Example_5: Inclusion of default in switch case to have the else facility

```
printf("enter three sides of a triangle\n");
```

```
int a,b,c;
```

```
scanf("%d%d%d",&a,&b,&c);
```

```
int count = 0;
```

```
if(a == b) count++;
```

```
if(b == c) count++;
```

```
if(c == a) count++;
```

```
switch(count)
```

```
{
```

```
    case 1:printf("iso");break;
```

```
    case 3:printf("equi");break;
```

```
        case 0:printf("scalene");break;

        default:printf("not a valid count");break;

    }

    printf("switch ended");
```

Few points to think!!

- Is it compulsory to have break for default?
- Can we have default at the beginning of the switch case?

Coding Example_6: The count value cannot be anything other than 0,1 or 2. So change in the default case as below.

```
switch(count)
{
    case 1: printf("iso");break;
    case 3: printf("equi");break;
    default: printf("scalene");break;
}

printf("switch ended");
```

Coding Example_7: Find whether the entered character is a vowel or not.

```
#include<stdio.h>

int main()
{
    printf("enter the character\n");
    char ch;
    ch = getchar();
    //scanf("%c",&ch);

    // solution using if
```

```
/*if(ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u')
{
    printf("you entered an vowel\n");
}
else{
    printf("you did not enter the vowel\n");
}*/
```

// Solution using switch case

```
switch(ch)
{
    //case 'a' || 'A': printf("hello 1"); break; // output of expression will be the case label
    case 'a':
    case 'A':
    case 'e':
    case 'E':
    case 'i':
    case 'o':
    case 'u': printf("you entered an vowel\n");break;
    default: printf("you did not enter the vowel\n");
}
// if(ch == 'a' || 'e' ) // Logical error
return 0;
}
```

Nested control structures

We may have loops and selections nested.

Coding Example_8: Generate all Armstrong numbers between 0 and 999.

The number obtained by raising each of the digits to the power of its length and adding together the sum of the terms obtained. If this sum is same as the given number, the number is same to be Armstrong.

Example: The three digit Armstrong number is 153. The sum of cubes of digits is same as the number. The program shows how not to write programs. We have a loop on hundredth digit, a loop on tenth digit and a loop on unit digit. We cube the digits, find the sum, form the number using these digits and compare the sum with the number formed.

```
#include<stdio.h>

int main()
{
    int h,t,u,hc,tc,uc;
    int number;
    for(h = 0; h<10;h++)
    {
        for(t = 0; t< 10;t++)
        {
            for(u = 0;u< 10;u++)
            {
                hc = h*h*h;
                tc = t*t*t;
                uc = u*u*u;
                int sum = hc+tc+uc;
                number = h*100 + t*10 + u*1;
                if(number == sum)
                    printf("%d\n",number);
            }
        }
    }
    return 0;
}
```

The above code is generating only few Armstrong numbers. Not printing all. Change the implementation to print all Armstrong numbers between 0 and 999. Think about adding ifs to check and count the number the digits

Also, think how many multiplications do we do for cubing? Should we repeatedly evaluate cube of t in the inner loop when it is not changing. Rearrange the code. Put the statements in the right blocks. You will be able to reduce the number of multiplications for cubing to approximately 1/3rd of what it is now.

Happy Coding!!

Unit #: 1**Unit Name: Problem Solving Fundamentals****Topic: Problems on basic constructs in C and control structures****Course objectives:**

The objective(s) of this course is to make students

CObj1: Acquire knowledge on how to solve relevant and logical problems using computing machine

CObj2: Map algorithmic solutions to relevant features of C programming language constructs

CObj3: Gain knowledge about C constructs and it's associated eco-system

CObj4: Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors

CObj5: Get insights about testing and debugging C Programs

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs

CO2: Understand, analyse and apply text processing and string manipulation methods using C Arrays, Pointers and functions

CO3: Understand prioritized scheduling and implement the same using C structures

CO4: Understand and apply sorting techniques using advanced C constructs

CO5: Understand and evaluate portable programming techniques using preprocessor directives and conditional compilation of C Programs

List of problems

1. You are transporting some boxes through a tunnel whose height is only 41 feet. Given the length, width, and height of the box, calculate the volume of those boxes and check if they pass through the tunnel.

Ex: Input = 5, 5, 5 Output = 125

Input = 1, 2, 40 Output = 80

Input = 10, 5, 41 Output = Can't Pass!

2. Given the number of rows, print a hollow diamond using star symbol: Ex: Input = 5

```

      *
     * *
    *  *
   *   *
  *    *
 *     *
*      *
 *     *
  *    *
   *   *
    *  *
     * *
      *
```

3. Check whether the given number is divisible by the sum of its digits. Display appropriate message
4. Write a C program to find the eligibility of admission for a professional course based on the following criteria: Eligibility Criteria : Marks in Maths ≥ 65 and Marks in Physics ≥ 50 and Marks in Chemistry ≥ 55 and Total in all three subjects ≥ 190 or Total in Maths and Physics ≥ 140
5. Construct a C program to reverse the bits of an integer.
6. Construct a menu-based calculator to perform arithmetic operations (Add, Subtract, Multiply) on complex numbers.
7. Given a number n, find the count of those numbers whose product of digits is equal to n and does not contain 1. Ex: input: 4 output: 2 ($2 \times 2 = 4$, so count is 2)
8. You are given a number N. Your task is to find if there exist two distinct natural numbers such that their bitwise XOR is N and bitwise AND is 0. If such numbers exist, then print 1. Otherwise, print 0. Ex: Input = 0 Output = 0 Input = 5 Output = 1 (Possible numbers are 4 and 1)
9. Using Switch case, implement a converter to obtain an integer from a hexadecimal byte

10. Given a number, check whether the nth bit is set or not. N value received it from the user

A few solutions

1. You are transporting some boxes through a tunnel whose height is only 41 feet. Given the length, width, and height of the box, calculate the volume of those boxes and check if they pass through the tunnel.

Ex: Input = 5, 5, 5 Output = 125

Input = 1, 2, 40 Output = 80

Input = 10, 5, 41 Output = Can't Pass!

Solution:

```
#include<stdio.h>

int main()
{
    int h, w, l;
    printf("Enter the length, width, height: ");
    scanf("%d %d %d", &l, &w, &h);
    if(h >= 41)
    {
        printf("Can't pass!");
    }
    else
    {
        int volume = l*w*h;
        printf("%d", volume);
    }
    return 0;
}
```

2. Given the number of rows, print a hollow diamond using star symbol: Ex: Input = 5

```

      *
     * *
    * *
   * *
  * *
 * *
* *
 * *
  * *
   * *
    * *
     * *
      *
```

Solution:

```
#include<stdio.h>

int main()
{
    int n;
    printf("Enter number of rows: ");
    scanf("%d",&n);
    for(int i=1; i<= n; i++)
    {
        for(int j=i; j<= n; j++)
            printf(" ");
        for(int k = 1; k <= 2*i-1; k++)
        {
            if(k == 1 || k == (2*i-1))
                printf("*");
            else
                printf(" ");
        }
        printf("\n");
    }
    for(int i = n-1; i>=1; i--)
    {
        for(int j = n; j>=i; j--)
            printf(" ");
```

```
for(int k = 1; k <= 2*i-1; k++)
{
    if(k == 1 || k == 2*i-1)
        printf("*");
    else
        printf(" ");
}
printf("\n");
}
return 0;
}
```

3. Check whether the given number is divisible by the sum of its digits. Display appropriate message

Solution:

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int n;
    printf("Enter the number:");
    scanf("%d", &n);
    int temp = n;
    //finding the sum of digits of the number
    if(n<=0)
        printf("invalid");
    else
    {
        int sum = 0;
        while(n)
        {
```

```
int r = n % 10;
sum = sum + r;
n = n / 10;
}
//check if sum of digits divides the number
if(temp%sum == 0)
    printf("Divisible");
else
    printf("Not Divisible");
}
return 0;
}
```

4. Write a C program to find the eligibility of admission for a professional course based on the following criteria: Eligibility Criteria : Marks in Maths ≥ 65 and Marks in Physics ≥ 50 and Marks in Chemistry ≥ 55 and Total in all three subjects ≥ 190 or Total in Maths and Physics ≥ 140

Solution:

```
#include <stdio.h>
int main()
{
    int p,c,m,total,math_phy;
    printf("Input the marks obtained in Physics, Chemistry, Math :");
    scanf("%d %d %d",&p, &c, &m);
    total = m + p + c;
    math_phy = m + p;
    printf("Total marks of Maths, Physics and Chemistry : %d\n",m+p+c);
    printf("Total marks of Maths and Physics : %d\n",m+p);
    if (m $\geq$ 65)
        if(p $\geq$ 50)
            if(c $\geq$ 55)
                if((total) $\geq$ 190||(math_phy) $\geq$ 140)
```

```
        printf("The candidate is eligible for admission.\n");
    else
        printf("The candidate is not eligible.\n");
    else
        printf("The candidate is not eligible.\n");
    else
        printf("The candidate is not eligible.\n");
    else
        printf("The candidate is not eligible.\n");
    return 0;
}
```

5. Construct a C program to reverse the bits of an integer.

Input: 3 output: 3

Input: 6 output: 3

Input: 2 output: 1

Solution:

```
#include<stdio.h>

int main()
{
    int n;
    printf("Enter the number:");
    scanf("%d", &n);
    int rev_bits = 0;
    while(n > 0)
    {
        rev_bits = rev_bits << 1;
        //if the bit is 1, then we XOR with 1
        if(n & 1 == 1)
            rev_bits = rev_bits ^ 1;
        //right shift n
    }
}
```

```
        n = n >> 1;
    }
    printf("%d", rev_bits);
    return 0;
}
```

6. Construct a menu-based calculator to perform arithmetic operations (Add, Subtract, Multiply) on complex numbers.

Solution:

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int choice, a, b, c, d;
    do
    {
        printf("1. Addition\n");
        printf("2. Subtraction\n");
        printf("3. Multiplication\n");
        printf("4. Divide\n");
        printf("Enter your choice\n");
        scanf("%d", &choice);
        if (choice > 4 || choice < 1)
            exit(0);          // terminates the program
        printf("Enter a and b where a + ib is the first complex number.");
        scanf("%d %d", &a, &b);
        printf("Enter c and d where c + id is the second complex number.");
        scanf("%d %d", &c, &d);
        /*
        if (choice == 1)
        {
```



```
int real = a+c;
int img = b+d;
if (img >= 0)
    printf("Sum = %d + %di\n", real, img);
else
    printf("Sum = %d%di\n", real, img);
}
else if (choice == 2)
{
    int real = a-c;
    int img = b-d;

    if (img >= 0)
        printf("Difference = %d + %di", real, img);
    else
        printf("Difference = %d %di", real, img);
}
else if (choice == 3)
{
    int real = a*c - b*d;
    int img = b*c + a*d;

    if (img >= 0)
        printf("Product = %d + %di", real, img);
    else
        printf("Product = %d %di", real, img);
}
else if (choice == 4)
{
    if (c == 0 && d == 0)
        printf("Division by 0 + 0i isn't allowed.");
}
```

```
else
{
    int x = a*c + b*d;
    int y = b*c - a*d;
    int z = c*c + d*d;

    if (x%z == 0 && y%z == 0)
    {
        if (y/z >= 0)
            printf("Division of the complex numbers = %d + %di", x/z, y/z);
        else
            printf("Division of the complex numbers = %d %di", x/z, y/z);
    }
    else if (x%z == 0 && y%z != 0)
    {
        if (y/z >= 0)
            printf("Division of two complex numbers = %d + %d/%di", x/z, y, z);
        else
            printf("Division of two complex numbers = %d %d/%di", x/z, y, z);
    }
    else if (x%z != 0 && y%z == 0)
    {
        if (y/z >= 0)
            printf("Division = %d/%d + %di", x, z, y/z);
        else
            printf("Division = %d %d/%di", x, z, y/z);
    }
    else
    {
        if (y/z >= 0)
            printf("Division = %d/%d + %d/%di", x, z, y, z);
```

```
        else
            printf("Division = %d/%d %d/%di", x, z, y, z);
        }
    }
}*/
int real, img;
switch(choice)
{
case 1:
    real = a+c;
    img = b+d;
    if (img >= 0)
        printf("Sum = %d + %di\n", real, img);
    else
        printf("Sum = %d%i\n", real, img);
    break;

case 2:
    real = a-c;
    img = b-d;
    if (img >= 0)
        printf("Difference = %d + %di\n", real, img);
    else
        printf("Difference = %d %di\n", real, img);
    break;

case 3:
    real = a*c - b*d;
    img = b*c + a*d;
    if (img >= 0)
        printf("Product = %d + %di\n", real, img);
```

```
else
    printf("Product = %d %di\n", real, img);
break;
case 4:
    if (c == 0 && d == 0)
        printf("Division by 0 + 0i isn't allowed.");
    else
    {
        int x = a*c + b*d;
        int y = b*c - a*d;
        int z = c*c + d*d;
        if (x%z == 0 && y%z == 0)
        {
            if (y/z >= 0)
                printf("Division of the complex numbers = %d + %di", x/z, y/z);
            else
                printf("Division of the complex numbers = %d %di", x/z, y/z);
        }
        else if (x%z == 0 && y%z != 0)
        {
            if (y/z >= 0)
                printf("Division of two complex numbers = %d + %d/%di", x/z, y, z);
            else
                printf("Division of two complex numbers = %d %d/%di", x/z, y, z);
        }
        else if (x%z != 0 && y%z == 0)
        {
            if (y/z >= 0)
                printf("Division = %d/%d + %di", x, z, y/z);
            else
                printf("Division = %d %d/%di", x, z, y/z);
        }
    }
}
```

```
    }  
    else  
    {  
        if (y/z >= 0)  
            printf("Division = %d/%d + %d/%di",x, z, y, z);  
        else  
            printf("Division = %d/%d %d/%di", x, z, y, z);  
    }  
}  
break; Breaks come out of current loop  
}  
} while(choice<=4);  
return 0;  
}
```

HAPPY CODING!

Solve other problems. If any issues, please contact sindhurpai@pes.edu

Unit #: 1**Unit Name: Problem solving Fundamentals****Topic: Single Character input and output – Problem solving****Course objectives:**

The objective(s) of this course is to make students

CObj1: Acquire knowledge on how to solve relevant and logical problems using computing machine

CObj2: Map algorithmic solutions to relevant features of C programming language constructs

CObj3: Gain knowledge about C constructs and it's associated eco-system

CObj4: Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors

CObj5: Get insights about testing and debugging C Programs

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs

CO2: Understand, analyse and apply text processing and string manipulation methods using C Arrays, Pointers and functions

CO3: Understand prioritized scheduling and implement the same using C structures

CO4: Understand and apply sorting techniques using advanced C constructs

CO5: Understand and evaluate portable programming techniques using preprocessor directives and conditional compilation of C Programs

Counting Problem

We are required to **find the number of characters, number of words and number of lines in an user input and in a given file**. We want to simulate a program in Unix command called `wc` – word count .

We have not learnt yet how to play with files in our programs. It is a bit too early in this course. We do know how to read from the keyboard. **Can the operating system open a file for me and make it logically available on the keyboard? It can. This concept is called input redirection.**

input redirection:

```
$ executable < filename
```

It is similarly possible to collect the output of the program which would appear on the screen in a file using output redirection.

output redirection:

```
$ executable > filename
```

To solve this problem stated, we should know

- a) how to read and display a character?–Discussed in Character I/O
- b) how to read a line?–Discussed in Character I/O
- c) how to make out when we reach the end of file?
- d) how to break a given sequence of characters into words and lines?

Few points to think before we start with the solution.

- It is always preferred to read at only one place.
- If you read input at two places, code could work. But what if we reach EOF in the inner loop?
- It is always preferred to have a while with an if over while within a while.
- Can we use a boolean variable to indicate whether we are in a word or not.
- If we are in a word and we reach a space, the word ends and we count. If we are not in a word and we encounter a space, we ignore it. We set `in_word` when we encounter a non-white-space.

Coding Example_1: Reading a character and displaying a character

```
#include<stdio.h>

int main()
{
    char ch = getchar();
    putchar(ch);
    return 0;
}
```

Coding Example_2: Reading a line and displaying a line

```
#include<stdio.h>

int main()
{
    char ch;
    while((ch = getchar()) != '\n')
    {
        putchar(ch);
    }
    return 0;
}
```

Our hurdle is to read the whole file. How do we know we have reached the end of file. 'C' handles this in a very interesting way. When the end of file is reached –which the operating system can make out –a particular value gets is returned by getchar. This value is decided by 'C' and not by the operating system. This value is associated with the name EOF. So, we keep reading until getchar returns EOF.

Coding Example_3: Reading characters till EOF and displaying it

```
#include<stdio.h>

int main()
{
    // printf("%d",EOF); // EOF is a macro which has the value -1 in stdio.h
    char ch;
    while((ch = getchar()) != EOF)
```



```
{  
    putchar(ch);  
}  
return 0;  
}
```

Coding Example_4: Solution to find the number of characters, number of words and number of lines in an user input and in a given file

Every character read is considered for character count. Word count is incremented if it encounters a space or a tab(\t) or new line(\n). Line count is incremented when it encounters a new line(\n)

```
#include<stdio.h>
```

```
int main()
```

```
{  
    char ch;  
    int nl = 0;  
    int nw = 0;  
    int nc = 0;  
    while((ch = getchar()) != EOF)  
    {  
        nc++;  
        if(ch == '\n')  
            nl++;  
  
        if(ch == ' ' || ch == '\t' || ch == '\n')  
        {  
            nw++;  
        }  
    }  
    printf("words = %d\nlines = %d\ncharacters=%d\n",nw,nl,nc);  
    return 0;  
}
```

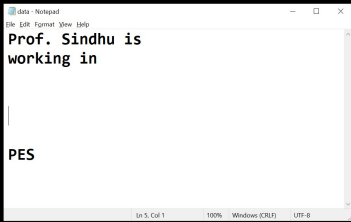
```
Command Prompt
C:\Users\SINDHU\Second_Sem\Day12>gcc 12_solution_counting.c -c
C:\Users\SINDHU\Second_Sem\Day12>gcc 12_solution_counting.c
C:\Users\SINDHU\Second_Sem\Day12>a.exe
Prof. Sindhu is
working in    PES
words = 6
lines = 2
characters=31
C:\Users\SINDHU\Second_Sem\Day12>
```

If the data file contains the same content, how do we give this data file as an input to this executable? Using redirection operator (<) as below.

```
Command Prompt
C:\Users\SINDHU\Second_Sem\Day12>a.exe < data.txt
words = 6
lines = 2
characters=31
C:\Users\SINDHU\Second_Sem\Day12>
```

The problem with the above is if the data contains multiple spaces and multiple new lines, the respective counts will be incremented.

```
Command Prompt
C:\Users\SINDHU\Second_Sem\Day12>a.exe < data.txt
words = 10
lines = 7
characters=35
C:\Users\SINDHU\Second_Sem\Day12>
```



Coding Example_5: The above issue can be avoided by using a boolean variable to indicate whether we are in a word or not. If we are in a word and we reach a space, the word ends and we count. If we are not in a word and we encounter a space, we ignore it. We set inword when we encounter a non-white-space.

```
#include<stdio.h>

int main()
{
    int inword = 0;
    while((ch = getchar()) != EOF)
    {
        putchar(ch);
        nc++;
        if(ch == '\n')
            nl++;

        if(inword && (ch == ' ' || ch == '\n' || ch == '\t'))
        {
            nw++;
            inword = 0;
        }
        else if(!(ch == ' ' || ch == '\n' || ch == '\t'))
        {
            inword = 1;
        }
    }

    printf("words = %d\nlines = %d\ncharacters=%d\n",nw,nl,nc);
    return 0;
}
```

```
Command Prompt
C:\Users\SINDHU\Second_Sem\Day12>gcc 12_solution_counting.c -c
C:\Users\SINDHU\Second_Sem\Day12>gcc 12_solution_counting.c -c
C:\Users\SINDHU\Second_Sem\Day12>gcc 12_solution_counting.o
C:\Users\SINDHU\Second_Sem\Day12>a
sindhu r      pai

is

from PES
words = 6
lines = 5
characters=29
C:\Users\SINDHU\Second_Sem\Day12>
```

```
Command Prompt
C:\Users\SINDHU\Second_Sem\Day12>a < data.txt
words = 6
lines = 5
characters=29
C:\Users\SINDHU\Second_Sem\Day12>
```

```
data - Notepad
File Edit Format View Help
sindhu r      pai

is

from PES
Ln 6, Col 1 100% Windows (CP1252) UTF-8
```

The above code solves the issue of word count with respect to multiple spaces and multiple lines. Include the logic for handling the line count when there is no data/content in the line with multiple new line characters.

This completes Unit – 1..!

Unit #: 1

Unit Name: Problem Solving Fundamentals

Topic: Language Specifications - Behaviors

Course objectives:

The objective(s) of this course is to make students

CObj1: Acquire knowledge on how to solve relevant and logical problems using computing machine

CObj2: Map algorithmic solutions to relevant features of C programming language constructs

CObj3: Gain knowledge about C constructs and it's associated eco-system

CObj4: Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors

CObj5: Get insights about testing and debugging C Programs

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs

CO2: Understand, analyse and apply text processing and string manipulation methods using C Arrays, Pointers and functions

CO3: Understand prioritized scheduling and implement the same using C structures

CO4: Understand and apply sorting techniques using advanced C constructs

CO5: Understand and evaluate portable programming techniques using preprocessor directives and conditional compilation of C Programs

Language Specifications

A Programming language specification is **a document in human language describing how a system is supposed to work**. It is a documentation that defines a programming language so that **users and implementers can agree on what programs in that language mean**. In case of a programming language, **it describes how the compiler or interpreter is to react to various inputs - which are valid/invalid, and what happens when they are executed**. Typically **detailed and formal, and primarily used by implementers referring to them in case of ambiguity**

A reference implementation such as **"this specific compiler"** is **not a specification**. It also defines a function from inputs to outputs, but it cannot be read by humans, we can only experiment with it.

Language Specification take several forms:

An explicit definition of the syntax and semantics of the language.

A description of the behavior of a "translator" for the language

"Model implementation" is a program that implements all requirements from a corresponding specification

Standardization of a Language

Type of standards

de Jure: Legally recognized practices, regardless of whether the practice exists in reality.

de Facto: Describes situations that exist in reality, even if not legally recognized

Language may have one or more implementations which acts as deFacto Language may be implemented and then specified, or vice versa or together.

- Languages can exist and be popular for decades without a specification - Perl
- After 20 years of usage, specification for PHP in 2014
- ALGOL 68 : First (and possibly one of the last) major language for which a full formal definition was made before it was implemented

Behaviors defined by C standards

There are four specific behaviors defined.

Locale-specific behavior - The **behavior** that are **dependent** on the **implementation** of the **C library**, and are not defined by **gcc** itself

Unspecified behavior - **Order of evaluation** of **arguments** in **printf** function

Implementation - defined behavior – **Size of each type**

Undefined Behaviour – **We will discuss in detail**

Undefined Behavior

The result of executing a program whose behavior is prescribed to be unpredictable in the language specification to which the computer code adheres. It is the **name of a list of conditions that the program must not meet**. Standard imposes no requirements: May fail to compile, may crash, may generate incorrect results, may fortunately do what exactly programmer intended

Examples: Memory access outside of array bounds, Signed integer overflow

Coding Example_1: Demo of C standard behavior

```
#include<stdio.h>

int main() {
    //printf("%d\n",5/0); // Undefined behavior float a = 23.5;
    //int b = a/0;
    printf("a is %d\n",a);// Undefined behavior
    //printf("b is %d\n",b);
    //printf("value is %s\n","50"); // proper output
    //printf("value is %s\n",50);
    //printf("%u %u %u\n",sizeof(int),sizeof(short int),sizeof(float));
    //      Implementation defined behavior

    int d = 10;
    printf("%d %d %d\n", d++, d++ - --d,--d); // Unspecified behavior

    return 0;
}
```

Unit #: 2

Unit Name: Counting, Sorting and Searching

Topic: Arrays, Initialization and Traversal

Course objectives:

The objective(s) of this course is to make students

CObj1: Acquire knowledge on how to solve relevant and logical problems using computing machine

CObj2: Map algorithmic solutions to relevant features of C programming language constructs

CObj3: Gain knowledge about C constructs and it's associated eco-system

CObj4: Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors

CObj5: Get insights about testing and debugging C Programs

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs

CO2: Understand, analyze and apply text processing and string manipulation methods using C Arrays, Pointers and functions

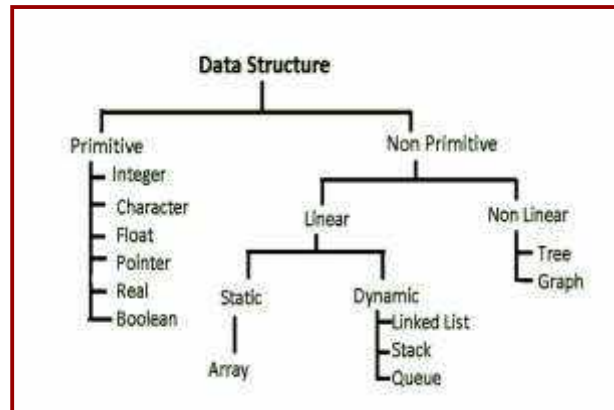
CO3: Understand prioritized scheduling and implement the same using C structures

CO4: Understand and apply sorting techniques using advanced C constructs

CO5: Understand and evaluate portable programming techniques using preprocessor directives and conditional compilation of C Programs

Data Structures

Data structures are **used** to **store data** in an **organized** and **efficient** manner so that the **retrieval is efficient**. Classification of data structures in C is as below.



When solving problems, it is important to visualize the data related to the problem. Sometimes the data consist of just a single number (as we discussed in unit-1, primitive types). At other times, the data may be a coordinate in a plane that can be represented as a pair of numbers, with one number representing the x-coordinate and the other number representing the y-coordinate. There are also times when we want to work with a set of similar data values, but we do not want to give each value a separate name. For example, we want to read marks of 1000 students and perform several computations. To store marks of thousand students, we do not want to use 1000 locations with 1000 names. To store group of values using a single identifier, we use a data structure called an Array.

An array is a linear data structure, which is a finite collection of similar data items stored in successive or consecutive memory locations. Arrays are used to store multiple values in a single variable, instead of declaring separate variables for each value. It can store a fixed-size sequential collection of elements of the same type. An array is used to store a collection of data, but it is often more useful to think of an array as a collection of variables of the same type.

To create an array, define the data type and specify the name of the array followed by square brackets [] with size mentioned in it. **Example: int s[100];**

Characteristics/Properties of arrays:

- Non-primary data or secondary data type
- Memory allocation is contiguous in nature
- Elements need not be unique.
- Demands same /homogenous types of elements
- Random access of elements in array is possible
- Elements are accessed using index/subscript
- Index or subscript starts from 0
- Memory is allocated at compile time.
- Size of the array is fixed at compile time. Returns the number of bytes occupied by the array.
- Cannot change the size at runtime.
- Arrays are assignment incompatible.
- Accessing elements of the array outside the bound will have undefined behaviour at runtime.

Types of Array

Broadly classified into two categories as

Category 1:

1. Fixed Length Array: Size of the array is fixed at compile time
2. Variable Length Array - Not supported by older few C standards. Better to use dynamic memory allocation functions than variable length array.

Category 2:

1. One Dimensional Array
2. Multi-Dimensional Array

One dimensional Array

- One dimensional array is also called as linear array(also called as 1-D array).
- The one dimensional array stores the data elements in a single row or column.

Array Declaration:

Syntax: `Data_type Array_name[Size];` // Declaration syntax

- `Data_type` : Specifies the type of the element that will be contained in the array
- `Size`: Indicates the maximum number of elements that can be stored inside the array
- `Array_name`: Identifier to identify a variable.
- Subscripts in array can be integer constant or integer variable or expression that yields integer
- C performs no bound checking – care should be taken to ensure that the array indices are within the declared limits

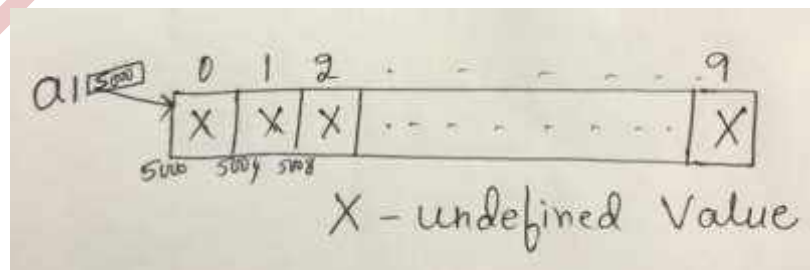
Examples:

- `float average [6];` // Can contain 6 floating point elements, 0 to 5 are valid array indices
- `char name[20];` // Can contain 20 char elements, 0 to 19 are valid array indices
- `double x[15];` // Can contain 15 elements of type double, 0 to 14 are valid array indices.

Consider `int a1[10];`

Declaration allocates number of bytes in a contiguous manner based on the size specified. All these memory locations are filled with undefined values. If the starting address is 0x5000 and if the size of integer is 4 bytes, refer the diagrams below to understand this declaration clearly. Number of bytes allocated = size specified*size of integer i.e, 10*4.

`printf("size of array is %d\n", sizeof(a1));` // 40 bytes



Address of the first element is called the Base address of the array. Address of i^{th} element of the array can be found using formula: **Address of i^{th} element = Base address + (size of each element * i)**

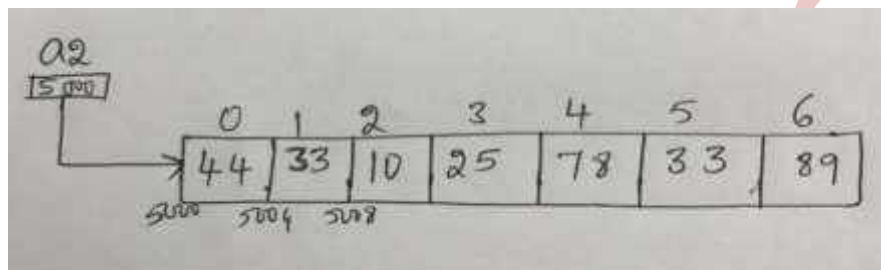
Array Initialization:

- After an array is declared it must be initialized.
- An Uninitialized array will contain undefined values/junk values.
- An array can be initialized at either compile time or at runtime

Consider `int a2[ ] = {44,33,10,25,78,33,89};`

Above initialization does not specify the size of the array. Size of the array is based on the number of elements stored in it and the size of type of elements stored. So, the above array occupies $7 \times 4 = 28$ bytes of memory in a contiguous manner.

`printf("size of array is %d\n", sizeof(a2));` //28 bytes



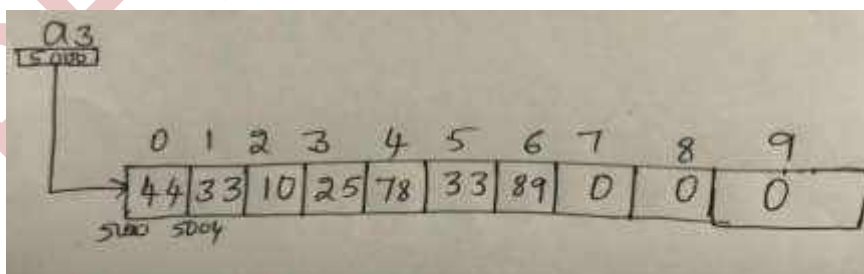
Partial Initialization:

Consider `int a3[10] = {44,33,10,25,78,33,89};`

Size of the array is specified. But only few elements are stored. Now, the size of the array is $10 \times 4 = 40$ bytes of memory in a contiguous manner. **All uninitialized memory locations in the array are filled with default value 0.**

`printf("size of array is %d\n", sizeof(a3));` //40 bytes

`printf("%p %p\n", a3, &a3);` // Must be different. But shows same address



Compile time Array initialization:

Compile time initialization of array elements is same as ordinary variable initialization. The general form of initialization of array is,

```
Syntax: data-type array-name[size] = { list of values };  
int marks[4]={ 67, 87, 56, 77 }; // integer array initialization  
float area[5]={ 23.4, 6.8, 5.5 }; // float array initialization  
intarr[] = {2, 3, 4};  
int marks[4]={67,87,56,77,5} //undefined behavior
```

Variable length array:

Variable length arrays are also known as runtime sized or variable sized arrays. The size of such arrays is defined at run-time.

Example:

```
void fun(int n)  
{  
    int arr[n];  
    // .....  
}  
int main()  
{  
    fun(6);  
}
```

The variable length arrays are always unsafe. This dynamically creates stack memory based on data size, which can cause overflow of stack frame.

Designated initializers (C99):

- It is often the case that relatively few elements of an array need to be initialized explicitly; the other elements can be given default values:

```
int a[15] = {0,0,29,0,0,0,0,0,0,7,0,0,0,48};
```

- So, we want element 2 to be 29, 9 to be 7 and 14 to be 48 and the other values to be zeros. For large arrays, writing an initializer in this fashion is tedious.

- C99's designated initializers

```
int a[15] = {[2] = 29, [9] = 7, [14] = 48};
```

- Each number in the brackets is said to be a designator.
- Besides being shorter and easier to read, designated initializers have another advantage: the order in which the elements are listed no longer matters. The previous expression can be written as:

```
int a[15] = {[14] = 48, [9] = 7, [2] = 29};
```

Designators must be constant expressions:

- If the array being initialized has length n, each designator must be between 0 and n-1.
- if the length of the array is omitted, a designator can be any non-negative integer.
- In that case, the compiler will deduce the length of the array by looking at the largest designator.

```
int b[] = {[2] = 6, [23]=87};
```

- Because 23 has appeared as a designator, the compiler will decide the length of this array to be 24.
- An initializer can use both the older technique and the later technique.

```
int c[10] = {5, 1, 9, [4] = 3, 7, 2, [8]=6};
```

Runtime initialization: Using a loop and input function in c

Example:

```
inta[5];
for(int i=0;i<5;i++)
{
    scanf("%d",&a[i]);
}
```

Traversal of the Array:

Accessing each element of the array is known as traversing the array.

Consider `int a1[10];`

- How do you access the 5th element of the array? **a1[4]**
- How do you display each element of the array?

```
int i;
```

```
for(i = 0; i<10;i++)
```

```
    printf("%d\t",a1[i]);//Using the index operator [ ].
```

```
//Above code prints undefined values as a1 is just declared.
```

Consider `int a2[ ] = {12,22,44,14,77,911};`

```
int i;
```

```
for(i = 0; i<10;i++)
```

```
    printf("%d\t",a2[i]);//Using the index operator [ ].
```

Above code prints initialized values when `i` is between 0 to 5. When `i` becomes 6, `a2[6]` is outside bound as the size of `a2` is fixed at compile time and it is 6×4 (size of `int` is implementation specific) = 24 bytes

Anytime accessing elements outside the array bound is an undefined behavior.

Coding Example_1: Let us consider the program to read 10 elements from the user and display those 10 elements.

```
#include<stdio.h>
```

```
int main()
```

```
{    int arr[100];
```

```
    int n;
```

```
    printf("enter the number of elements\n");
```

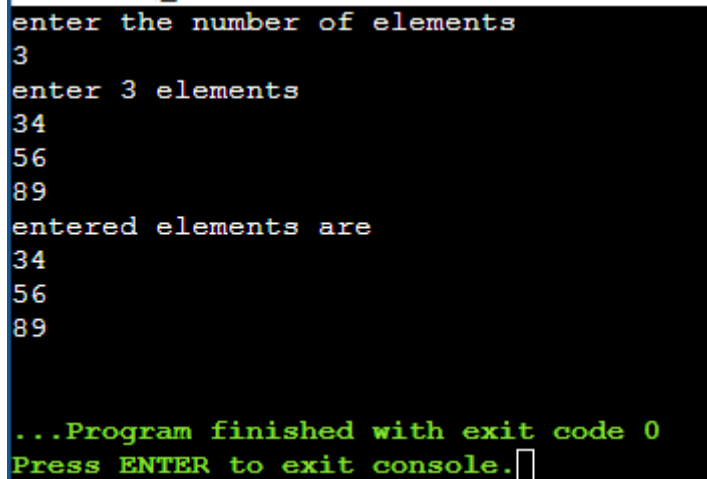
```
    scanf("%d",&n);
```

```
    printf("enter %d elements\n",n);
```

```
    int i = 0;
```

```
    while(i<n)
```

```
{    scanf("%d",&arr[i]);
    i++;
}
printf("entered elements are\n");
i = 0;
while(i<n)
{    printf("%d\n",arr[i]);
    i++;
}
return 0;
}
```

Output:

```
enter the number of elements
3
enter 3 elements
34
56
89
entered elements are
34
56
89

...Program finished with exit code 0
Press ENTER to exit console.
```

Coding Example_2: Let us consider the program to find the sum of the elements of an array.

```
int arr[] = {-1,4,3,1,7};
int i;
int sum = 0;
int n = sizeof(arr)/sizeof(arr[0]);
for(i = 0; i<n; i++)
{    sum += arr[i];
}
printf("sum of elements of the array is %d\n", sum);
```


Think about writing a function to find the sum of elements of the array. Few questions to think about above codes?

- Should I use while only? for and while are same in C except for syntax.
- Can I use arr[n] while declaration? Variable length array
- Can we write read and display user defined functions to do the same? Yes. To do this, we should understand functions and how to pass array to a function.

Happy Coding using Arrays!!

Unit #: 2

Unit Name: Counting, Sorting and Searching

Topic: Multi-dimensional Array

Course objectives:

The objective(s) of this course is to make students

CObj1: Acquire knowledge on how to solve relevant and logical problems using computing machine

CObj2: Map algorithmic solutions to relevant features of C programming language constructs

CObj3: Gain knowledge about C constructs and it's associated eco-system

CObj4: Appreciate and gain knowledge about the issues with C Standards and its respective behaviors

CObj5: Get insights about testing and debugging C Programs

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs

CO2: Understand, analyse and apply text processing and string manipulation methods using C Arrays, Pointers and functions

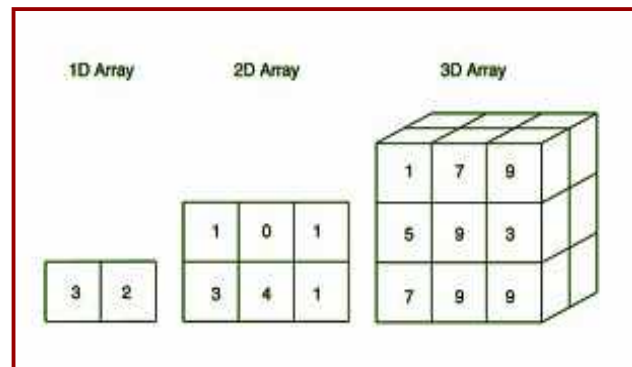
CO3: Understand prioritized scheduling and implement the same using C structures

CO4: Understand and apply sorting techniques using advanced C constructs

CO5: Understand and evaluate portable programming techniques using preprocessor directives and conditional compilation of C Programs

Introduction

A multi-dimensional array is an array with more than one level or dimension. It might be 2-Dimensional and 3-Dimensional and so on.



1D array may be used to store single dimension data or information. When we want to manipulate the arrays by rearranging the elements by using functions like reshape, squeeze and so on, which may be very challenging to visualize using a 1D array. Then, 2D array can come into play we may have to store complex data which has rows and columns. Likewise, we may want to store three dimensions of data as well. Then we go for 3D array and so on.

Consider the example of storing marks of 5 students in some particular subject A. We can define,
`int A_marks[ ] = {90,95,99,100,89};`

Consider the example of storing marks of 5 students in 3 different subjects A, B and C. We can say,

Version-1:

```
int A_marks[ ] = {90,95,99,100,89};
```

```
int B_marks[ ] = {95,90, 91,99, 94};
```

```
int C_marks[ ] = {91,92,93,95,100};
```

Version-2:

```
int student_1_marks[ ] = {90, 95, 91};
```

```
int student_2_marks[ ] = {95, 90, 92};
```

```
int student_3_marks[ ] = {99, 91, 93};
```

```
int student_4_marks[ ] = {100, 99, 95};
```

```
int student_5_marks[ ] = {89, 94, 100};
```

Now, rather than storing marks this way in One - Dimensional Array, we can use Two - Dimensional array as we have **two dimensions involved** in this particular example: student and subject marks

Two – Dimensional Array

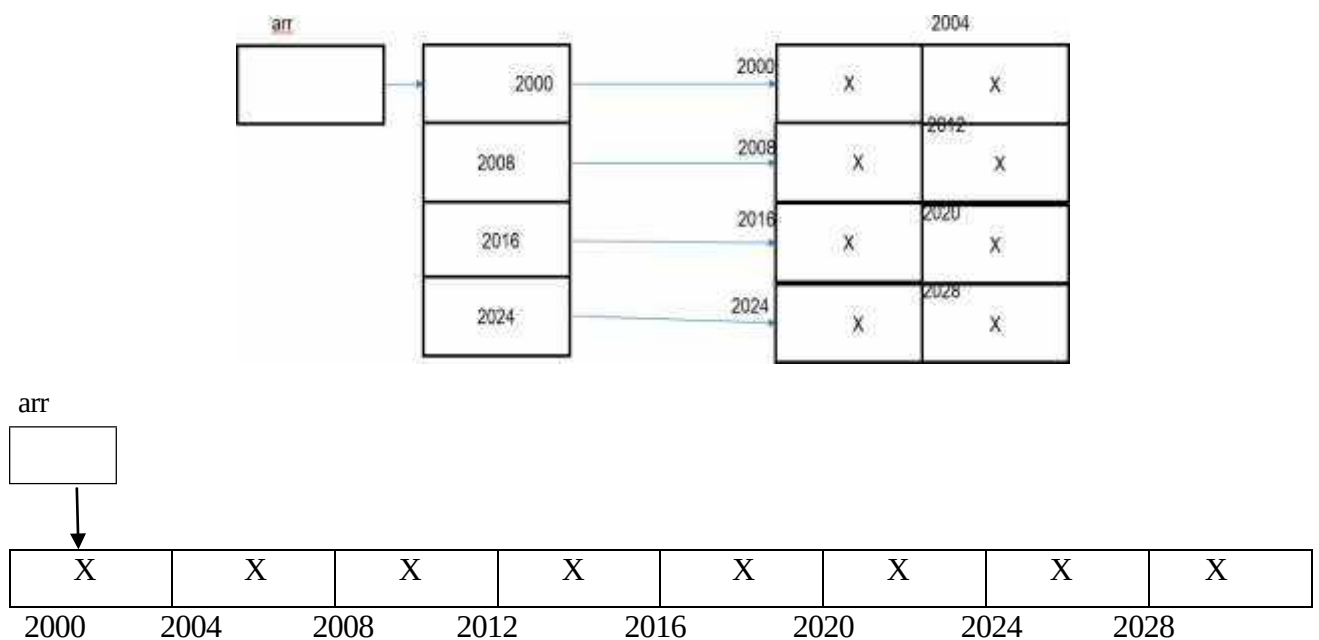
It is treated as an array of arrays. Every element of the array must be of same type as arrays are homogeneous. Hence, every element of the array must be an array itself in 2D array.

Let us consider the above example of storing 5 students marks in three different subjects. `int marks[5][3] = { {90, 95, 91}, {95, 90, 92}, {99, 91, 93}, {100, 99, 95}, {89, 94, 100} };`

Declaration of a 2D Array

```
data_type array_name[size_1][size_2];
```

```
int arr[4][2]; // Allocate 8 contiguous memory locations
```



If the size of the integer is 4 bytes, 8 contiguous locations allocated

Initialization of a 2D Array: Compiler knows the array size based the array elements and allocates memory

```
data_type array_name[size_1][size_2] = {elements separated by comma};
```

```
int arr[][] = {11,22,33,44,55,66}; // Error. Column size is compulsory
```

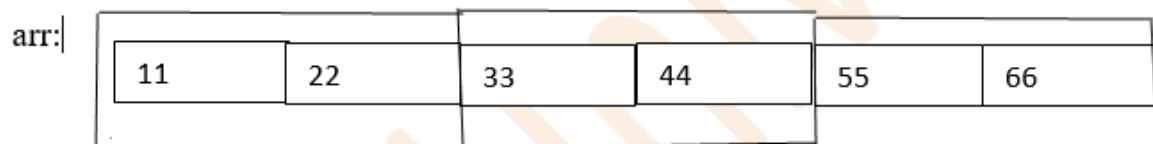
Without knowing the number of columns in each row – it is impossible to locate an element in 2D array.

```
int arr[3][2] = {{11,22},{33,44},{55,66}};
```

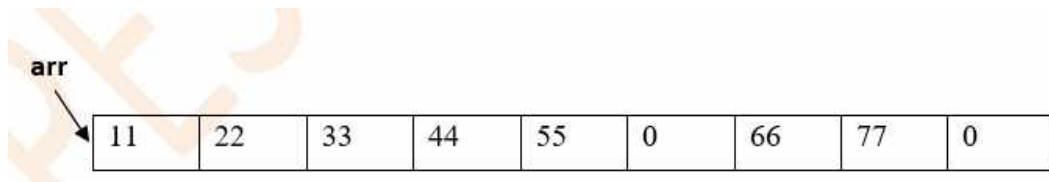
```
int arr[3][2] = {11,22,33,44,55,66};
```

// Allocate 6 contiguous memory locations and assign the values

```
int arr[][2] = {11,22,33,44,55,66};
```



```
int arr[][3] = {{11,22,33},{44,55},{66,77}}; //Partial initialization
```

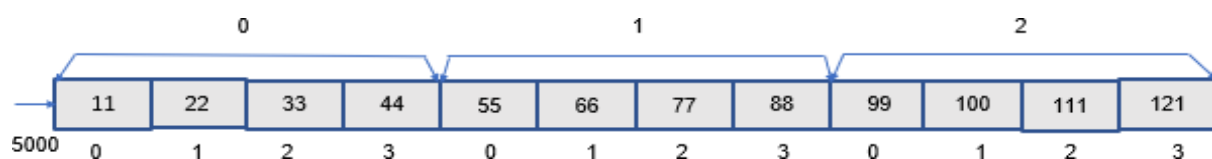


Internal Representation of a 2D Array

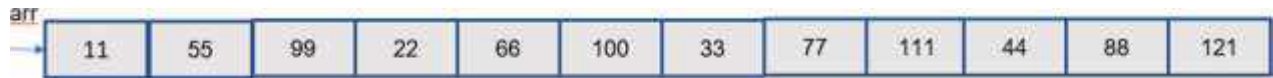
As two - dimensional array itself is an array, elements are stored in contiguous memory locations. And since elements are stored in contiguous memory locations, there are two possibilities.

```
int arr[3][4] = {{11, 22, 33, 44}, {55, 66, 77, 88}, {99, 100, 111, 121}};
```

Row major Ordering: All elements of one row are stored followed by all elements of the next row and so on.



Column Major Ordering: All elements of one column are stored followed by all elements of the next column and so on.



Generally, systems support Row Major Ordering.

Address of an element in a 2D Array

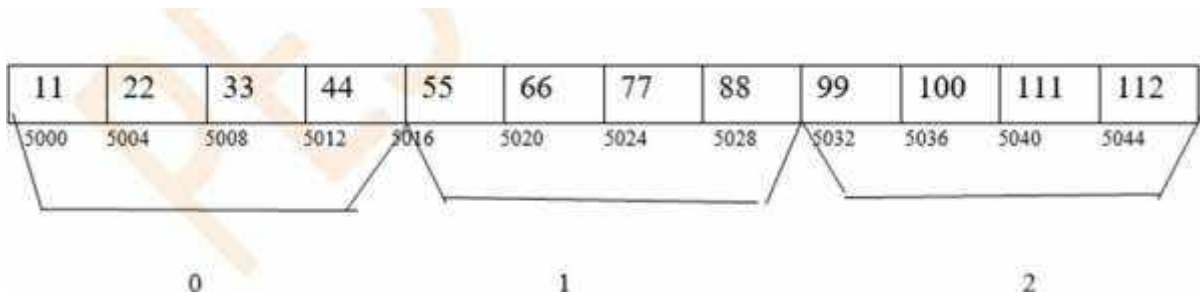
Address of $A[i][j]$ = Base_Address + $((i * \text{No. of columns in every row}) + j) * \text{size of every element}$;

Consider, `int arr[3][4] = {{11, 22, 33, 44}, {55, 66, 77, 88}, {99, 100, 111, 121}};`

If the size of integer is 4 bytes in the system and the base address is 5000, then address of `arr[1][2]` can be found by $5000 + ((1 * 4) + 2) * 4 = 5000 + 6 * 4 = 5000 + 24 = 5024$.

Consider, `int arr[3][4] = {11, 22, 33, 44, 55, 66, 77, 88, 99, 100, 111, 121};`

If the size of integer is 4 bytes in the system and the base address is 5000, then address of `arr[1][5]` can be found by $5000 + ((1 * 4) + 5) * 4 = 5000 + 9 * 4 = 5000 + 36 = 5036$.



Think about finding the address of `a[0][5]` and `a[1][3]`. Is this same as `a[5][0]` and `a[3][1]` respectively?

Coding Example_1: Reading and displaying a 2D Array

```
// code to read n*m elements
int a[100][100];
int n; int m;
printf("enter row num and column numbers\n");
scanf("%d %d",&n,&m);
printf("Enter %d elements",n*m);
for(int i = 0;i<n;i++) // n – row index      // m – column index
{
    for(int j= 0;j<m;j++)
    {
        scanf("%d",&a[i][j]);
    }
}

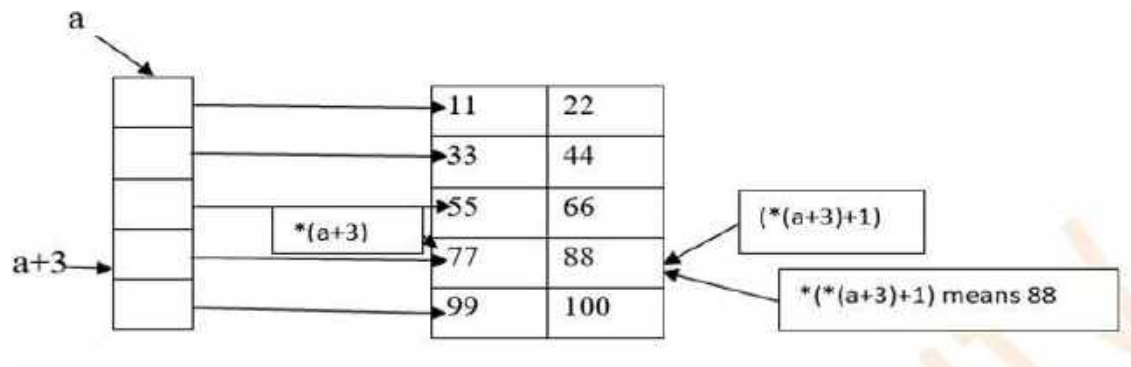
// code to display n*m elements
printf("entered elements are\n");
for(int i = 0;i<n;i++)
{
    for(int j= 0;j<m;j++)
    {
        printf("%d\t",a[i][j]);
    }
    printf("\n");
}
```

Two-Dimensional Array and Pointers

Consider `int a[5][2] = {{11,22}, {33,44}, {55,66}, {77,88}, {99,100}};`

```
printf("Using array, accessing the elements of the array");
printf("%d\n",a[3][2]);      // 99  array notation
printf("%d\n",a[3][1]);      // 88  array notation
printf("%d\n",*(*(a+3)+1)); // 88   // pointer notation
```

Array name is a pointer to a row. The expression `a + 3` is a pointer to the third row. `*(a + 3)` becomes a pointer to the zeroth element of that row. Then `+1` becomes a pointer to the next element in that row. To get the element in that location, dereference the pointer.



In General, `(a + i)` points to *i*th 1-D array

```
//int *p1 = a;// incompatible type. Warning during compilation
```

```
// If above statement is fine, what happens when p1[7] is accessed and p1[2][1] is accessed
```

```
int (*p)[2] = a;// p is a pointer to an array of 2 integers
```

Since subscript (`[ ]`) have higher precedence than indirection (`*`), it is necessary to have parentheses for indirection operator and pointer name. Here the type of `p` is 'pointer to an array of 2 integers'.

```
printf("using pointer accessing the elements of the array")
printf("%d\n",p[3][1]);      // 88  array notation
printf("%d\n",*(*(p+3)+1)); // 88   pointer notation
```

Note : The pointer that points to the 0th element of array and the pointer that points to the whole array are totally different.

Coding Example_2: Illustration of the size of pointer to an array of items.

```
int arr[4][3]={33,44,11,55,88,22,33,66,99,11,80,9};

//assigning array to a pointer
int *p=arr;           //gives warning
//printf("%d",p[7]); //looks fine, not a good idea.
//printf("\n%d\n",p[2][1]);      //error..Column size not available to p.
//create a pointer to an array of integers
int (*p)[3]=arr;
printf("%d\n",p[2][1]);
printf("%d\n",*(*(p+2)+1));
printf("%d\n",sizeof(p));    //size of pointer
printf("%d %d\n",sizeof(*p),sizeof(arr));    //size of the array pointing to 0th row and size of the
entire 2D array
```

Passing 2D Array to a function

Coding Example_3: Read and display 2D array using functions. Client code is as below.

```
void read(int (*a1)[],int,int);
void display( int (*a1)[],int,int);
int main()
{
    int a[100][100];
    int m1,n1;
    printf("enter the order of a\n");
    scanf("%d%d",&m1,&n1); // user entered 3 5
    printf("enter the elements of a\n");
    read(a,m1,n1);
    printf("elements of A are\n");
    display(a,m1,n1);
    return 0;
}
```

Consider the following cases while passing 2D array to functions

Case1: Declaration and definition of the function with no column size

```
void read(int[][],int m,int n);    //Compiletime error
void display(int[][],int m,int n);
```

Case 2: Declaration and definition of the function with column size not same as what is given in the declaration of the array in the client code

```
void read(int[][3],int m,int n);    // warning but code works
void display(int[][3],int m,int n);
```

Case 3: Declaration and definition of the function with column size same as what is given in the declaration of the array in the client code

```
void read(int[][100],int m,int n);    // fine.
void display(int[][100],int m,int n);

// But can we say void read(int[][n],int m,int n ) ? Think !!!!
```

Case 4: If the order of parameters is changed as below in declaration and definition of the function, we need to change the client code. Changing the client code is not a good programmer's habit. Because interface cannot be changed. Implementation can change. So refer to case 5.

```
void read(int m,int n,int a[][n]);
void display(int m,int n,int a[][n]);
```

Case 5: Forward declaration

```
void read(int n;int[][n],int m,int n);
void display(int n; int[][n],int m,int n);
```

```
void read(int n;int a[][n],int m,int n)
{
    int i,j;
    for(i = 0;i<m;i++)
```

```
        for(j = 0;j < n;j++)
        {
            printf("enter the element");
            scanf("%d",&a[i][j]);
        }
    }

void display(int n; int a[][n],int m,int n)
{
    int i,j;
    for(i = 0;i<m;i++)
    {
        for(j = 0;j < n;j++)
        {
            printf("%d\t",a[i][j]);
        }
        printf("\n");
    }
}
```

Case 6: Array degenerates to a pointer at run time. So think about using pointer to an array as a parameter.

```
void read(int n; int(*)[n],int m,int n);
void display(int n; int(*)[n],int m,int n);

void read(int n; int (*a)[n],int m,int n)
{
    int i,j;
    for(i = 0;i<m;i++)
        for(j = 0;j < n;j++)
        {
            printf("enter the element");
```

```
scanf("%d",&a[i][j]);  
    }  
}
```

```
void display(int n; int (*a)[n],int m,int n)  
{  
    int i,j;  
    for(i = 0;i<m;i++)  
    {  
        for(j = 0;j < n;j++)  
        {  
            printf("%d\t",a[i][j]);  
        }  
        printf("\n");  
    }  
}
```

Happy Coding!

Unit #: 2

Unit Name: Counting, Sorting and Searching

Topic: Programs on Arrays and Strings

Course objectives:

The objective(s) of this course is to make students

CObj1: Acquire knowledge on how to solve relevant and logical problems using computing machine

CObj2: Map algorithmic solutions to relevant features of C programming language constructs

CObj3: Gain knowledge about C constructs and it's associated eco-system

CObj4: Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors

CObj5: Get insights about testing and debugging C Programs

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs

CO2: Understand, analyse and apply text processing and string manipulation methods using C Arrays, Pointers and functions

CO3: Understand prioritized scheduling and implement the same using C structures

CO4: Understand and apply sorting techniques using advanced C constructs

CO5: Understand and evaluate portable programming techniques using preprocessor directives and conditional compilation of C Programs

Team – PSWC,

Jan - May, 2022

Dept. of CSE,

PES University

List of Problems

1. Program to find the count of given character in a given string.
2. WAP to input a string and print the Square of all the Numbers in a String. Take care of special cases. Input: wha4ts12ap1p and Output: 16 1 4 1
3. Write a function to find the position of First occurrence of the input character in a given string. Use this function to find the positions of all occurrences of the input character in the string
4. Write a function which takes two string arguments and find the Cartesian product of passed strings.
Input: str1: abc str2: ab and Output: aa, ab, ba, bb, ca, cb
5. WAP to find the sum of all substrings of a string representing a number.
Input : num = "1234"
Output : 1670 Sum = $1+2 + 3 + 4 + 12 + 23 + 34 + 123 + 234 + 1234 = 1670$
6. Program to find the addition of pair of elements in a given array
input: 2 3 4 5 6 7 output: 5 9 13
input: 2 3 4 5 6 output: 5 9 6
7. Write a function to reverse a string using recursion and test the function
8. Write a program to delete the characters from string1 which are present in string2.
input: string1: whatsapp string2: wat output: hspp
9. Write a program to rotate the array based on the rotation value entered by the user
input: 12345 rotation_val: 2 output: 34512
10. Write a program to find if two strings are anagrams.
Anagrams: Same set of letters with different arrangements. Number of times a letter present in one string must be equal to the number of times the letter is present on another string
11. Given a number n, WAP to find all 1 to n bit numbers with no consecutive 1s in their binary representation.
Input : n = 4 Output : 1 2 4 5 8 9 1 These are numbers with 1 to 4 bits and no consecutive ones in binary representation.
Input : n = 3 Output : 1 2 4 5

A few Solutions

1. Program to find the count of given character in a given string.

Solution:

```
int main()
{
    char str1[100]; printf("enter the string\n");
    scanf("%[^\\n]s",str1);    // To receive multiple words and store multiple words under one name
    printf("enter the character\n");
    fflush(stdin);
    char ch = getchar();
    int count = find_frequency(str1, ch);
    printf("%c is present in %s, %d times\n",ch, str1,count);
    return 0;
}

int find_frequency(char *s, char ch)
{
    int count = 0;
    for(; *s ; s++)           // exit the loop when *s is '\\0'
    {
        if (*s == ch) count++;
    }
    return count;
}
```

2. WAP to input a string and print the Square of all the Numbers in a String. Take care of special cases. Input: wha4ts12ap1p and Output: 16 1 4 1

Solution:

```
#include<stdio.h>

int sqr(int r);

void extract_num_square(char *c);

int main()
{
    char ch[400];
    scanf("%s",ch); //wha4ts12ap1p
    extract_num_square(ch);
    return 0;
}

int sqr(int r)
{
    return r*r;
}

void extract_num_square(char *c)
{
    while(*c != '\0')
    {
        if((*c >= '1' && *c <= '9'))
        {
            printf("%c--",(*c));
            printf("%d\n",sqr(*c-'0'));
        }
        c++;
    }
}
```


3. Write a function to find the position of First occurrence of the input character in a given string. Use this function to find the positions of all occurrences of the input character in the string.

Solution:

```
#include<stdio.h>

int find_first(char* ,char);

int main()
{
    char text[100];
    printf("enter the text\n");
    scanf("%[^\n]s",text);      // Till \n, receive input and store in one char array variable
    fflush(stdin);
    printf("enter the character which u want to find\n");
    char ch = getchar();
    int res = find_first(text,ch);
    if(res == -1)
        printf("not present in the text\n");
    else
        printf("available at %d position\n",res);
    return 0;
}
```

Different versions of find_first function are as below.

Version 1:

```
int find_first(char *a,char c)
{
    int p = -1;
    int i;
    int check = 0;
    for(i = 0;i<strlen(a);++i)
    {
        if(a[i]==c && check == 0)
        {
```

```
        p = i; check = 1;
    }
}
return p;
}
```

Version 2:

```
int find_first(char *a, char c)
{
    int i = 0;
    while((a[i]) && a[i] != ch )
    {
        i++;
    }
    return a[i]? i: -1;
}
```

Version 3:

```
int find_first(char *a,char ch)
{
    int i = 0;
    while(*a && *a != ch )
    {
        a++;i++;
    }
    return (*a) ? i : -1;
}
```

Use this solution to find the count of given character in a string.

Think: Can you use recursive function for this?

Part 2: Using this function to find all occurrences of a character.

```
int main()
{
    char str[100];
    printf("enter the string\n");
    scanf("%s",str);
    fflush(stdin);
    char ch;
    printf("enter the character\n");
    ch = getchar();
    //find all occurrences
    int pos = -1;
    int i;
    while( (i = find_first(str + pos + 1, ch)) != -1)
    {
        pos = pos + i + 1;
        printf("%d ", pos);
    }
    printf("\n");
    return 0;
}
```

6. Program to find the addition of pair of elements in a given array

input: 2 3 4 5 6 7 output: 5 9 13

input: 2 3 4 5 6 output: 5 9 6

Solution:

```
#include<stdio.h>

void pair_addition(int *a, int n);

int main()
{
    int a1[] = {2,5,6,7,8,1};
    int a2[] = {2,5,6,7,8};
    int n1 = sizeof(a1)/sizeof(*a1);
```

```
    int n2 = sizeof(a2)/sizeof(*a2);  
    pair_addition(a1,n1);  
    pair_addition(a2,n2);  
    return 0;  
}  
void pair_addition(int *a, int n)  
{  
    int i;  
    for(i = 0; i < n-1; i += 2)  
    {  
        int sum = a[i]+a[i+1];  
        printf("%d\t",sum);  
    }  
    if(n%2)  
        printf("%d",a[i]);  
    printf("\n");  
}
```

Happy Coding!!

Unit #: 2

Unit Name: Counting, Sorting and Searching

Topic: Pointers

Course objectives:

The objective(s) of this course is to make students

CObj1: Acquire knowledge on how to solve relevant and logical problems using computing machine

CObj2: Map algorithmic solutions to relevant features of C programming language constructs

CObj3: Gain knowledge about C constructs and it's associated eco-system

CObj4: Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors

CObj5: Get insights about testing and debugging C Programs

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs

CO2: Understand, analyze and apply text processing and string manipulation methods using C Arrays, Pointers and functions

CO3: Understand prioritized scheduling and implement the same using C structures

CO4: Understand and apply sorting techniques using advanced C constructs

CO5: Understand and evaluate portable programming techniques using preprocessor directives and conditional compilation of C Programs

Pointers

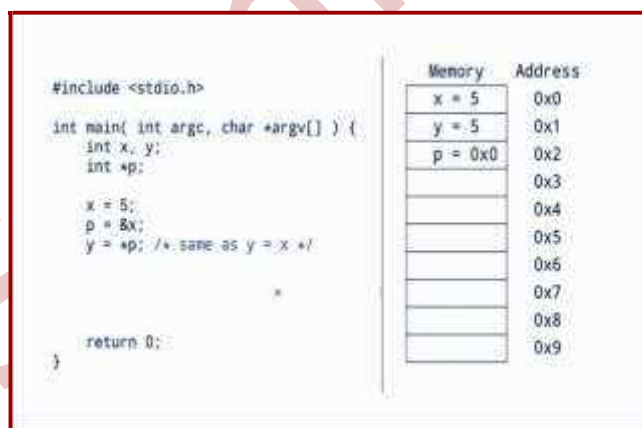
- Pointer is a variable which contains the address. This address is the location of another object in the memory
- Pointers can be used to access and manipulate data stored in memory.
- Pointer of particular type can point to address any value of that particular type.
- Size of pointer of any type is same /constant in that system.
- Not all pointers actually contain an address

Example: NULL pointer // Value of NULL pointer is 0.

Pointer can have three kinds of contents in it

1. The address of an object, which can be de referenced.
2. A NULL pointer
3. Undefined value // If p is a pointer to integer, then – int *p;

Note: A **pointer** is a **variable** that **stores** the **memory address** of **another variable** as **its value**. The **address of the variable we are working with is assigned to the pointer**



The image shows a C program snippet on the left and a memory diagram on the right. The code defines variables x, y, and p, assigns values to x and y, and then assigns the address of x to p. The memory diagram shows the memory layout with addresses 0x0 to 0x9. Variable x is at 0x0 (value 5), y is at 0x1 (value 5), and p is at 0x2 (value 0x0).

```
#include <stdio.h>

int main( int argc, char *argv[] ) {
    int x, y;
    int *p;

    x = 5;
    y = 5;
    p = &x; /* same as y = x */

    return 0;
}
```

Memory	Address
x = 5	0x0
y = 5	0x1
p = 0x0	0x2
	0x3
	0x4
	0x5
	0x6
	0x7
	0x8
	0x9

We can get the value of the variable the pointer currently points to, by dereferencing pointer by using the * operator. When used in declaration like int* ptr, it creates a pointer variable. When not used in declaration, it act as a dereference operator w.r.t pointers

Pointer Declaration:

Syntax: Data-type *name;

Example: `int *p;` // Compiler assumes that any address that it holds points to an integer type.

`p = ∑` // Memory address of sum variable is stored into p.

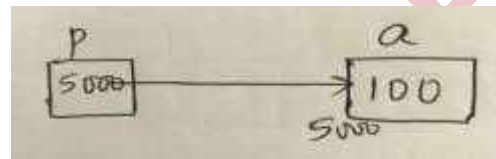
Coding Example_1:

`int *p;` // p can point to anything where integer is stored. `int*` is the type. Not just `int`.

`int a = 100;`

`p = &a;`

`printf("a is %d and *p is %d", a, *p);`



Coding Example_2: Pointer pointing to an array

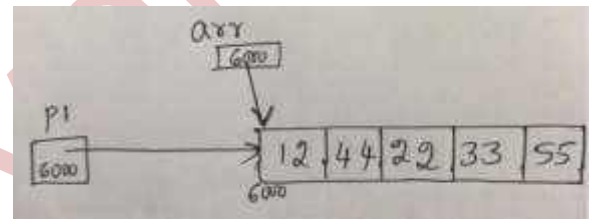
Now, `int arr[] = {12,44,22,33,55};`

`int *p1 = arr;` // same as `int *p1;`

`p1 = arr;` // same as `int *p1;` `p1 = &arr[0];`

`int arr2[10];`

`arra2 = arr;` // **Arrays are assignment incompatible. Compile time Error**



Pointer Arithmetic:

Below arithmetic operations are allowed on pointers

- Add an int to a pointer
- Subtract an int from a pointer
- Difference of two pointers when they point to the same array.

Integer is not same as pointer. We get warning when we try to compile the code where integer is stored in variable of `int*` type.

Coding Example_3:

`int arr[] = {12,33,44};`

`int *p2 = arr;`

`printf("before increment %p %d\n", p2, *p2);` // 12

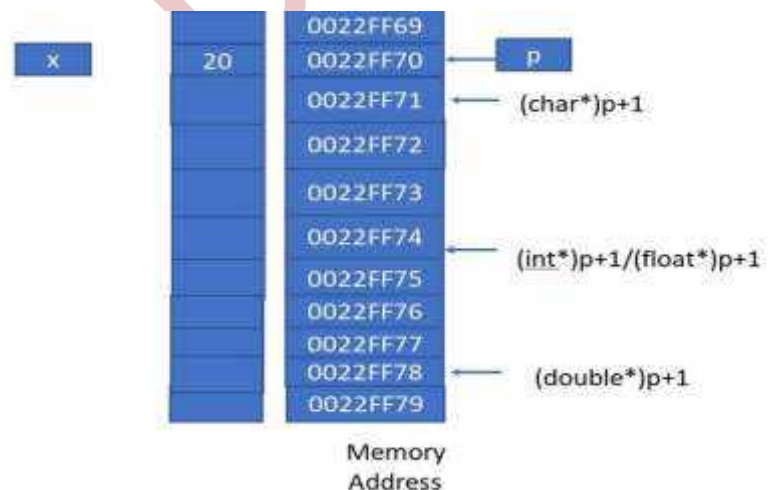
```
p2++; //same as p2 = p2+1
// This means 5000+sizeof(every element)*1 if 5000 is the base address
//increment the pointer by 1. p2 is now pointing to next location.
printf("after increment %p %d\n",p2, *p2); // 33
```

Coding Example_4: Example on Pointer Arithmetic :

```
int *p, x = 20;
p = &x;
printf("p = %p\n", p);
printf("p+1 = %p\n", (int*)p+1);
printf("p+1 = %p\n", (char*)p+1);
printf("p+1 = %p\n", (float*)p+1);
printf("p+1 = %p\n", (double*)p+1);
```

Sample output:

```
p = 0022FF70
p+1 = 0022FF74
p+1 = 0022FF71
p+1 = 0022FF74
p+1 = 0022FF78
```



Coding Example_5:

```
int main()
{
    int *p;
    int a = 10;
    p = &a;
    printf("%d\n",(*p)+1);    // 11 ,p is not changed
    printf("before *p++ %p\n",p);    //address of p
    printf("%d\n",*p++);    // same as *p and then p++ i.e 10
    printf("after *p++ %p\n",p);
    //address incremented by the size of type of value stored in it
    return(0);
}
```

Coding Example_6:

```
int main()
{
    int *p;
    int a = 10;
    p = &a;
    printf("%d\n",*p); //10
    printf("%d\n",(*p)++); // 10 value of p is used and then value of p is incremented
    printf("%d\n",*p); // 11
    return 0;
}
```

Array Traversal using pointers:

Coding Example_7: Try all these versions after writing the diagrams properly.

```
int arr[] = {12,44,22,33,55};  
int *p3 = arr;  
int i;
```

Version 1: Index operator can be applied on pointer. Array notation

```
for(i = 0; i < 5; i++)  
    printf("%d\t", p3[i]); // 12 44 22 33 55  
// every iteration added i to p3 .p3 not modified
```

Version 2: Using pointer notation

```
for(i = 0; i < 5; i++)  
    printf("%d\t", *(p3+i)); // 12 44 22 33 55  
// every iteration i value is added to p3 and content at that address is printed.  
// p3 not modified
```

Version 3:

```
for(i = 0; i < 5; i++)  
    printf("%d\t", *p3++); // 12 44 22 33 55  
// Use p3, then increment, every iteration p3 is incremented.
```

Version 4: undefined behavior if you try to access outside bound

```
for(i = 0; i < 5; i++)  
    printf("%d\t", *++p3); // 44 22 33 55 undefined value  
// every iteration p3 is incremented.
```

Version 5:

```
for(i = 0; i < 5; i++)  
    printf("%d\t", (*p3)++); // 12 13 14 15 16  
// every iteration value at p3 is used and then incremented.
```

Version 6:

```
for(i = 0;i<5;i++,p3++)  
    printf("%d \t",*p3); // 12 44 22 33 55  
// every iteration value at p3 is used and then p3 is incremented.
```

Version 7: p3 and arr has same base address of the array stored in it. But array is a constant pointer. It cannot point to anything in the world.

```
for(i = 0;i<5;i++)  
    printf("%d \t", *arr++); // Compile Time Error
```

Arrays and Pointers:

An array during compile time is an actual array but degenerates to a constant pointer during run time. Size of the array returns the number of bytes occupied by the array. But the size of pointer is always constant in that particular system.

Coding Example_8:

```
int *p1;  
float *f1 ;  
char *c1;  
printf("%d%d%d ",sizeof(p1),sizeof(f1),sizeof(c1)); // Same value for all  
int a[] = {22,11,44,5};  
int *p = a;  
a++; // Error constant pointer  
p++; // Fine  
p[1] = 222; // allowed  
a[1] = 222 ; // Fine
```

Note: If variable i is used in loop for the traversal, a[i], *(a+i), p[i], *(p+i), i[a], i[p] are all same.

Differences between array and pointer:

1. The sizeof operator:
 - sizeof(array) returns the amount of memory used by all elements in array
 - sizeof(pointer) only returns the amount of memory used by the pointer variable itself

2. The & operator:

- &array is an alias for &array[0] and returns the address of the first element in array
- &pointer returns the address of pointer

3. String literal initialization of a character array – Will be discussed in detail in next lecture

- char array[] = "abc" sets the first four elements in array to 'a', 'b', 'c', and '\0'
- char *pointer = "abc" sets pointer to the address of the "abc" string (which may be stored in read-only memory and thus unchangeable)
- Pointer variable can be assigned a value whereas array variable cannot be.

4. Pointer variable can be assigned a value whereas array variable cannot be.

```
int a[10];  
int *p;  
p=a; //allowed  
a=p; //not allowed
```

5. An arithmetic operation on pointer variable is allowed.

```
int a[10];  
int *p;  
p++; /*allowed*/  
a++; /*not allowed*/
```

Happy Coding using Arrays and Pointers!!

Unit #: 2

Unit Name: Counting, Sorting and Searching

Topic: Functions in C

Course objectives:

The objective(s) of this course is to make students

CObj1: Acquire knowledge on how to solve relevant and logical problems using computing machine

CObj2: Map algorithmic solutions to relevant features of C programming language constructs

CObj3: Gain knowledge about C constructs and it's associated eco-system

CObj4: Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors

CObj5: Get insights about testing and debugging C Programs

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs

CO2: Understand, analyse and apply text processing and string manipulation methods using C Arrays, Pointers and functions

CO3: Understand prioritized scheduling and implement the same using C structures

CO4: Understand and apply sorting techniques using advanced C constructs

CO5: Understand and evaluate portable programming techniques using preprocessor directives and conditional compilation of C Programs

Introduction

In case we want to repeat the set of tasks or instructions, we may have two options:

- Use the same set of statements every time we want to perform the task
- Create a function to perform that task, and just call it every time when we need to perform that task, is usually a good practice and a good programmer always uses functions while writing code in C

Functions **break large computing tasks into smaller ones** and enable people to build on what others have done instead of starting from scratch. In programming, **Function is a subprogram to carry out a specific task**. A function is a self-contained block of code that performs a particular task. Once the function is designed, it can be **treated as a black box**. The inner details of operation are invisible to rest of the program. Functions are useful because of following reasons:

- To **improve the readability** of code.
- **Improves the reusability** of the code, same function can be used in any program rather than writing the same code from scratch.
- **Debugging of the code would be easier** if we use functions, as errors are easily traced.
- **Reduces the size of the code**, redundant set of statements are replaced by function calls.

Types of functions

C functions can be broadly classified into two categories:

Library Functions

Functions which are defined by C library. Examples include printf(), scanf(), strcat() etc. We just need to include appropriate header files to use these functions. These are already declared and defined in C libraries.

User-defined Functions

Functions which are defined by the developer at the time of writing program. Developer can make changes in the implementation as and when he/she wants. It reduces the complexity of a big program and optimizes the code, supporting Modular Programming Paradigm

In order to make use of user defined functions, we need to understand three key terms associated with functions: **Function Definition, Function call and Function Declaration.**

Function Definition

Each function definition has the form

```
return_type function_name(parameters) // parameters optional  
{  
    // declarations and statements  
}
```

A function definition should have a return type. The return type must match with what that function returns at the end of the function execution. If the function is not designed to return anything, then the type must be mentioned as void. Like variables, function name is also an identifier and hence must follow the rules of an identifier. Parameters list is optional. If more than one parameter, it must be comma separated. Each parameter is declared with its type and parameters receive the data sent during the function call. If function has parameters or not, but the parentheses is must. Block of statements inside the function or body of the function must be inside { and }. There is no indentation requirement as far as the syntax of C is considered. For readability purpose, it is a good practice to indent the body of the function.

Function definitions can occur in any order in the source file and the source program can be split into multiple files. One must also notice that only one value can be returned from the function, when called by name. There can also be side effects while using functions in c

```
int sum(int a, int b)  
{  
    return a+b;  
}  
  
int decrement(int y)  
{  
    return y-1;  
}
```

```
void disp_hello()    // This function doesn't return anything
{
    printf("Hello Friends\n");
}

double use_pow(int x)
{
    return pow(x,3);    // using pow() function is not good practice.
}

int fun1(int a, int)    // Error in function definition.
{ }

int fun2(int a, b)    // Invalid Again.
{ }
```

Function Call

Function-name(list of arguments); // semicolon compulsory and Arguments depends on the number of parameters in the function definition

A function can be called by using **function name followed by list of arguments** (if any) enclosed in parentheses. The function which calls other function is known to be Caller and the function which is getting called is known to be the Callee. **The arguments must match the parameters in the function definition in it's type, order and number. Multiple arguments must be separated by comma. Arguments can be any expression in C. Need not be always just variables. A function call is an expression. When a function is called, Activation record is created. Activation record is another name for Stack Frame.** It is composed of:

- Local variables of the callee
- Return address to the caller
- Location to store return
- value Parameters of the
- callee Temporary variables

The order in which the arguments are evaluated in a function call is not defined and is determined by the calling convention(out of the scope of this notes) used by the compiler. It is left to the compiler Writer. **Always arguments are copied to the corresponding parameters.** Then the control is transferred to the called function. Body of the function gets executed. When all the statements are executed, callee returns to the caller. OR when there is return statement, the expression of return is evaluated and then callee returns to the caller. If the return type is void, function must not have return statement inside the function.

Coding Example_1:

```
#include<stdio.h>

int main()
{
    int x = 100;
    int y = 10;
    int answer = sum(x,y);
    printf("sum is %d\n",answer);
    answer = decrement(x);
    printf("decremented value is %d\n",answer);
    disp_hello();
    double ans = use_pow(x);
    printf("ans is %lf\n",ans);
    answer = sum(x+6,y);
    printf("answer is %d\n", answer);
    printf("power : %lf\n", use_power(5));
    return 0;
}
```

Function Declaration/ Prototype

All functions must be declared before they are invoked. The function declaration is as follows.

return_type Function_name (parameters list); // semicolon compulsory

A function may or may not accept any argument. A function may or may not return any value

directly when called by name. There are different type of functions based on the arguments and return type.

- Function without arguments and without return value
- Function without arguments and with return value
- Function with arguments and without return value
- Function with arguments and with return value

The parameters list must be separated by commas. The parameter names do not need to be the same in declaration and the function definition. The types must match the type of parameters in the function definition in number and order. Use of identifiers in the declaration is optional. When the declared types do not match with the types in the function definition, compiler will produce an error.

Parameter Passing in C

Parameter passing is always by Value in C. Argument is copied to the corresponding parameter. The parameter is not copied back to the argument. It is possible to copy back the parameter to argument only if the argument is l- value. Argument is not affected if we change the parameter inside a function.

Types of parameters: Actual Parameter/Arguments and Formal Parameters

Actual parameters are values or variables containing valid values that are passed to a function when it is invoked or called. **Formal parameters** are the variables defined by the function that receives values when the function is called.

Coding Example_2:

```
void fun1(int a1); // declaration
void main()
{
    int a1 = 100;
    printf("before function call a1 is %d\n", a1); // a1 is 100
    fun1(a1);      // call
```

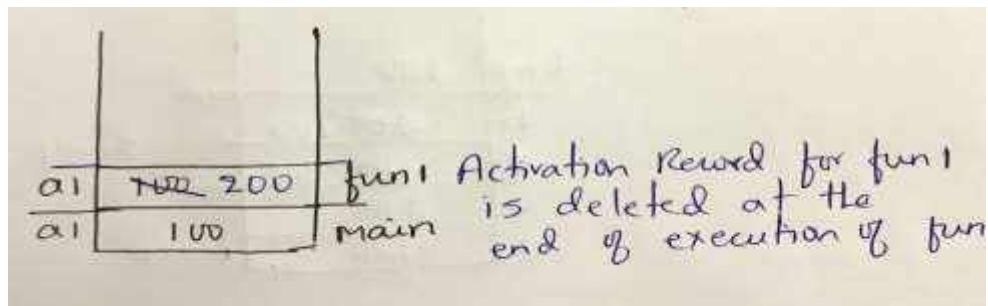
```

printf("after function call a1 is %d\n", a1); // a1 is 100
return 0;
}

void fun1(int a1)
{
    printf("a1 in fun1 before changing %d\n", a1); //100 a1 = 200;
    printf("a1 in fun1 after changing %d\n", a1); //200
}

```

a1 has not changed in main function. The parameter a1 in fun1 is a copy of a1 from main function. Refer to the below diagram to understand activation record creation and deletion to know this program output in detail.



Think about this Question: Is it impossible to change the argument by calling a function? Yes – by passing the l-value

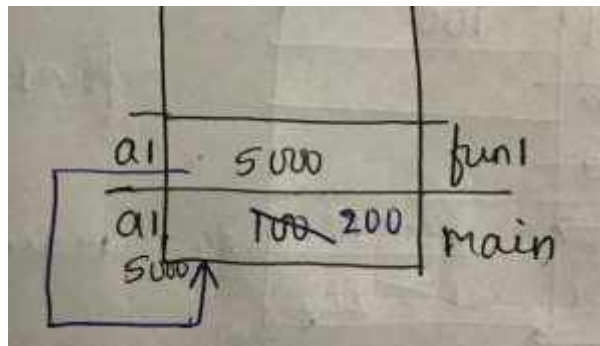
Coding Example_3:

```

void fun1(int *a1);
int main()
{
    int a1 = 100;
    printf("before function call a1 is %d\n", a1); // 100
    fun1(&a1); // call
    printf("after function call a1 is %d\n", a1); // 200
}

```

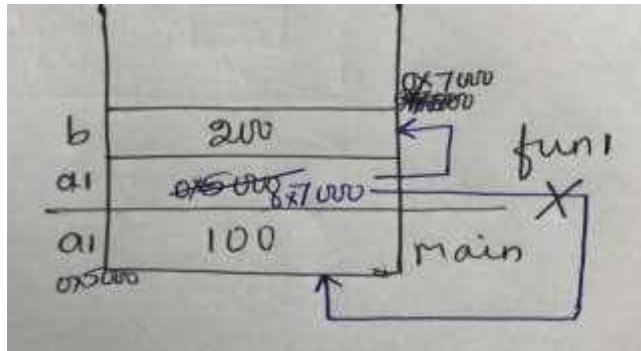
```
void fun1(int *a1)
{
    printf("**a1 in fun1 before changing %d\n", *a1);    //100
    *a1 = 200;
    printf("**a1 in fun1 after changing %d\n", *a1);      //200
}
```



Coding Example_3:

```
void fun1(int *a1);
int main()
{
    int a1 = 100;
    printf("before function call a1 is %d\n", a1); // 100
    fun1(&a1);    // call
    printf("after function call a1 is %d\n", a1); // 100
}

void fun1(int *a1)
{
    int b = 200;
    printf("**a1 in fun1 before changing %d\n", *a1);    //100
    a1 = &b;
    printf("**a1 in fun1 after changing %d\n", *a1);      //200
}
```



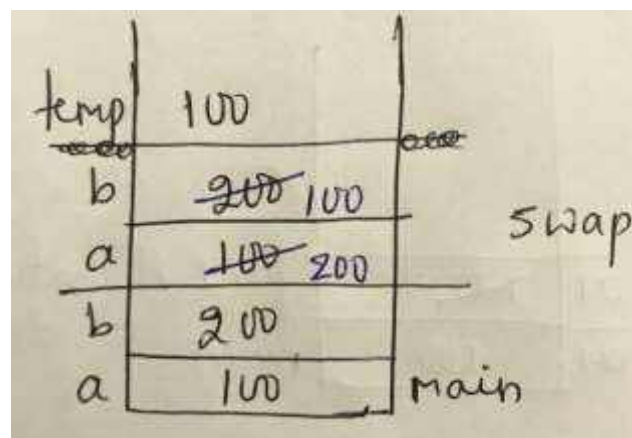
Shall we write the function to swap two numbers and test this function?

Coding Example_4:

Version – 1: Is this right version?

```
void swap(int a, int b)
{
    int temp = a; a = b; b = temp;
}

int main()
{
    int a = 100; int b = 200;
    printf("before call a is %d and b is %d\n", a, b);    // a is 100 and b is 200
    swap(a, b);
    printf("after call a is %d and b is %d\n", a, b);    // a is 100 and b is 200
    return 0;
}
```

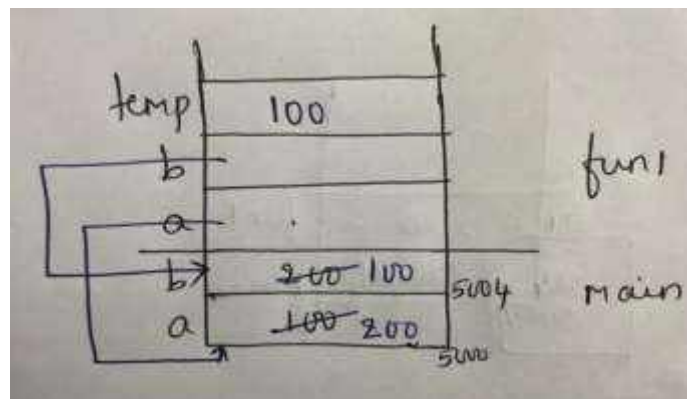


Version 2:

```
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main()
{
    int a = 100;
    int b = 200;

    printf("before call a is %d and b is %d\n", a, b);    // a is 100 and b is 200
    swap(&a, &b);
    printf("after call a is %d and b is %d\n", a, b);    // a is 100 and b is 200
    return 0;
}
```



return keyword in C

Function must do what it is supposed to do. It should not do something extra. For Example, refer to the below code.

```
int add(int a, int b)
{
    return a+b;
}
```

The function `add()` is used to add two numbers. It should not print the sum inside the function. We cannot use this kind of functions repeatedly if it does something extra than what is required. Here comes the usage of `return` keyword inside a function. **Syntax: `return expression`;**

In 'C', function returns a single value. The expression of return is evaluated and copied to the temporary location by the called function. This temporary location does not have any name. If the return type of the function is not same as the expression of return in the function definition, the expression is cast to the return type before copying to the temporary location. The calling function will pick up the value from this temporary location and use it.

Coding Example_5:

```
void f1(int);
void f2(int*);
void f3(int);
void f4(int*); i
nt* f5(int* );
int* f6();
int main()
{   int x = 100;
    f1(x);
    printf("x is %d\n", x); // 100
    double y = 6.5;
    f1(y); // observe what happens when double value is passed as argument to integer
parameter?
    printf("y is %lf\n", y); //      6.500000
    int *p = &x; // pointer variable
    f2(p);
    printf("x is %d and *p is %d\n", x, *p);      // 100 100
    f3(*p);
    printf("x is %d and *p is %d\n", x, *p);      // 100 100
    f4(p);
    printf("x is %d and *p is %d\n", x, *p, p);
```

```

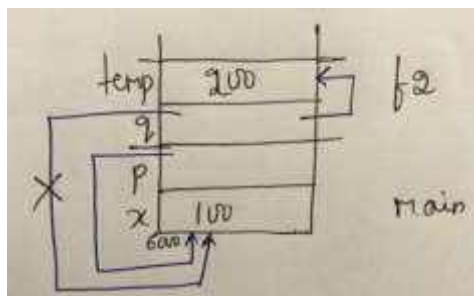
int z= 10;
p =f 5(&z);
printf("z is %d and %d\n", *p, z);    // 10    10
p = f6();
printf("*p is %d \n", *p);
return 0;
}

```

```

void f1(int x)
{
    x = 20;
}
void f2(int* q)
{
    int temp = 200;
    q = &temp;
}

```

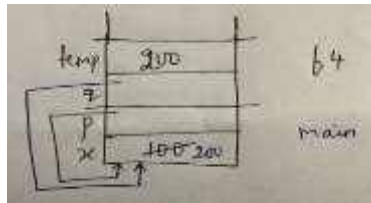


```

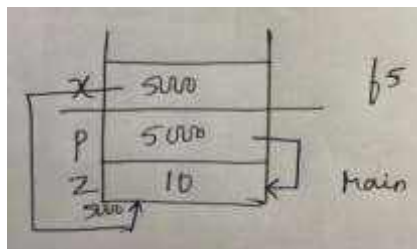
void f3(int t)
{
    t = 200;
}
void f4(int* q)
{
    int temp = 200;
    *q = temp;
}

```


}

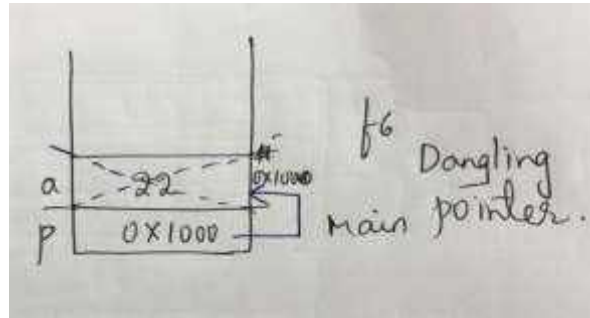


```
int* f5(int* x)
{
    return x;
}
```



When there is function call to f6, Activation Record gets created for that and deleted when the function returns. p points to a variable which is not available after the function execution. This problem is known as Dangling Pointer. The pointer is available. But the location it is not available which is pointed by pointer. Applying dereferencing operator(*) on a dangling pointer is always undefined behaviour.

```
int* f6()
{
    int a = 22;    // We should never return a pointer to a local variable
    return &a;     //      Compile time Error
}
```



When there is function call to f6, Activation Record gets created for that and deleted when the function returns. p points to a variable which is not available after the function execution. This problem is known as Dangling Pointer. The pointer is available. But the location it is not available which is pointed by pointer. Applying dereferencing operator(*) on a dangling pointer is always undefined behaviour.

void data type in c

void is an **empty data type that has no value**. We mostly use void data type in functions when we don't want to return any value to the calling function. We also use void data type in pointer like "void *p", which means, pointer "p" is not pointing to any non void data types . void * acts as generic pointer. We will be using void *, when we are not sure on the data type that this pointer will point to. We can use void * to refer either integer data or char data. void * should not be dereferenced without explicit type casting. We use void in functions as "int function_name (void)", which means, the function does not accept any argument

Unit #: 2

Unit Name: Counting, Sorting and Searching

Topic: Array and functions

Course objectives:

The objective(s) of this course is to make students

CObj1: Acquire knowledge on how to solve relevant and logical problems using computing machine

CObj2: Map algorithmic solutions to relevant features of C programming language constructs

CObj3: Gain knowledge about C constructs and it's associated eco-system

CObj4: Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors

CObj5: Get insights about testing and debugging C Programs

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs

CO2: Understand, analyse and apply text processing and string manipulation methods using C Arrays, Pointers and functions

CO3: Understand prioritized scheduling and implement the same using C structures

CO4: Understand and apply sorting techniques using advanced C constructs

CO5: Understand and evaluate portable programming techniques using preprocessor directives and conditional compilation of C Programs

Team – PSWC,

Jan - May, 2022

Dept. of CSE,

PES University

Array and functions

When array is passed as an argument to a function, arguments are copied to parameters of the function and parameters are always pointers. **Array degenerates to a pointer at runtime.** All the operations that are valid for pointer will be applicable for array too in the body of the function. **Function call happens always at run time.**

Coding Example_1: Using functions read n elements into an array and display it. Also include a function to find the sum of all the elements in the array

Version 1: n is a global variable

```
#include<stdio.h>
void read_array(int[]);
int n; // global variable
void display_array(int[]);
int find_sum(int[]);
int main()
{
    int a[100];
    printf("how many elements u want to add\n");
    scanf("%d",&n);
    printf("enter %d elements\n",n);
    read_array(a);
    printf("entered elements are\n");
    display_array(a);
    printf("\nsum is %d\n",find_sum(a));
    return 0;
}
void read_array(int a[])
{
    for(int i= 0; i<n;i++)
    {
```

```
        scanf("%d",&a[i]);
    }
}
void display_array(int a[])
{
    for(int i= 0; i<n;i++)
    {
        printf("%d\t",a[i]);
    }
}
int find_sum(int a[])
{
    int sum = 0;
    for(int i= 0; i<n;i++)
    {
        sum = sum+a[i];
    }
    return sum;
}
```

Version 2: n is local to a main function. Can other functions access n then? It throws Compile time Error

Version 3: As the array becomes pointer at runtime, finding the size of the passed argument to any function is same as finding the size of the pointer.

```
#include<stdio.h>
void read_array(int[]);
int n; // global variable
void display_array(int[]);
int find_sum(int[]);
int main()
{
```

```
int a[100];
printf("how many elements u want to add\n");
scanf("%d",&n);
printf("sizeof a is %d\n",sizeof(a)); // in my system, 400 bytes
printf("enter %d elements\n",n);
read_array(a);
...
return 0;
}
void read_array(int a[])
{
    printf("inside read_array sizeof a is %d\n",sizeof(a));
    //4 bytes or constant value for any pointer
    for(int i= 0; i<n;i++)
    {
        scanf("%d",&a[i]);
    }
}
```

Version 4: As n is local to main function, send this to functions as an argument

```
#include<stdio.h>
void read_array(int[],int);
void display_array(int[],int);
int find_sum(int[],int);
void increment(int a[],int n);
int main()
{
    int a[100];
    int n;
    printf("how many elements u want to add\n");
    scanf("%d",&n);
```

```
printf("enter %d elements\n",n);
printf("sizeof a is %d\n",sizeof(a));
read_array(a,n);
printf("entered elements are\n");
display_array(a,n);
printf("\nsum is %d\n",find_sum(a,n));
increment(a,n);
printf("array with updated elements are\n");
display_array(a,n);
return 0;
}
void read_array(int a[],int n)
{
    for(int i= 0; i<n;i++)
    {
        scanf("%d",&a[i]);
    }
}
void display_array(int a[],int n)
{
    for(int i= 0; i<n;i++)
    {
        printf("%d\t",a[i]);
    }
}
int find_sum(int a[],int n)
{
    int sum = 0;
    for(int i= 0; i<n;i++)
    {
        sum = sum+a[i];
    }
}
```

```
    }  
    return sum;  
}
```

Version 5: Using the pointer in the parameter makes more sense as array become pointer at runtime during function call.

```
#include<stdio.h>  
void read_array(int*,int);  
void display_array(int*,int);  
int find_sum(int*,int); // observe this  
int main()  
{  
    ...  
    void read_array(int *a,int n) // this too  
    {  
        ...  
    }  
    void display_array(int *a,int n)  
    {  
        ...  
    }  
    int find_sum(int *a,int n)  
    {  
        ...  
    }
```

Version 6: The display and find_sum functions should not be allowed to make any changes to the array sent in the argument. But it is possible now.

```
#include<stdio.h>  
void read_array(int*,int);  
void display_array(int*,int);  
int find_sum(int*,int);  
int main()  
{  
    ...  
    void read_array(int *a,int n)  
    {  
        ...  
    }  
  
    void display_array(int *a,int n)
```



```
{    a[4] = 8989;    // allowed    }  
int find_sum(int *a,int n)  
{    ...    }
```

Version 7: How to avoid this problem stated in version 6. Make it as const. Create pointer to constant integer.

```
#include<stdio.h>  
void read_array(int*,int);  
void display_array(const int*,int);  
int find_sum(const int*,int);  
int main()  
{    ...    }  
void read_array(int *a,int n)  
{  
void display_array(const int *a,int n)  
{    a[4] = 8989;    } //compiletime Error  
int find_sum(const int *a,int n)  
{    ...    }
```

Unit #: 2

Unit Name: Counting, Sorting and Searching

Topic: Implementations of Matrix operations

Course objectives:

The objective(s) of this course is to make students

CObj1: Acquire knowledge on how to solve relevant and logical problems using computing machine

CObj2: Map algorithmic solutions to relevant features of C programming language constructs

CObj3: Gain knowledge about C constructs and its associated eco-system

CObj4: Appreciate and gain knowledge about the issues with C Standards and its respective behaviors

CObj5: Get insights about testing and debugging C Programs

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs

CO2: Understand, analyse and apply text processing and string manipulation methods using C Arrays, Pointers and functions

CO3: Understand prioritized scheduling and implement the same using C structures

CO4: Understand and apply sorting techniques using advanced C constructs

CO5: Understand and evaluate portable programming techniques using preprocessor directives and conditional compilation of C Programs

Let us write separate functions to **add, subtract and multiply two matrices**. Display appropriate message when two matrices are incompatible for these operations. Add the implementations of other operations such as finding the transpose of a matrix, identity matrix or not etc

Coding Example_1:

```
void read(int (*a1)[],int,int);
void display( int (*a1)[],int,int);
void multiply(int n1; int n2; int (*a)[n1],int (*b)[n2],int m1,int n1,int n2,int (*c)[n2]);
void subtract(int n; int (*a)[n],int (*b)[n],int m, int n, int (*c)[n]);
void add(int n; int (*a)[n],int (*b)[n],int m, int n, int (*c)[n]);
int main()
{
    int a[100][100];
    int b[100][100];
    int c[100][100];
    int m1,n1;
    int m2,n2;
    printf("enter the order of a\n");
    scanf("%d%d",&m1,&n1);
    printf("enter the elements of a\n");
    read(a,m1,n1);
    printf("enter the order of b\n");
    scanf("%d%d",&m2,&n2);
    printf("enter the elements of b\n");
    read(b,m2,n2);
    if (m1 == m2 && n1 == n2)
    {
        printf("elements of A are\n");
        display(a,m1,n1);
        printf("elements of B are\n");
```

```
        display(b,m2,n2);
        printf("Addition of two matrices\n");
        add(a,b,m1,n1,c);
        display(c,m2,n2);
        printf("Subtraction of two matrices\n");
        subtract(a,b,m1,n1,c);
        display(c,m2,n2);
    }
    else

    {   printf("Two matrices are not compatible for addition and subtraction\n");   }
    if (n1==m2)
    {   printf("Multiplication of two matrices\n");
        multiply(a,b,m1,n1,n2,c);
        display(c,m1,n2);
    }
    else
    {   printf("Two matrices are not compatible for multiplication too\n");   }
    return 0;
}

void add(int n; int (*a)[n],int (*b)[n],int m, int n, int (*c)[n])
{
    for(int i = 0;i<m;i++)
    {
        for(int j= 0;j<n;j++)
        {
            c[i][j] = a[i][j]+b[i][j];
        }
    }
}
```

```
void subtract(int n; int (*a)[n],int (*b)[n],int m, int n, int (*c)[n])
{
    for(int i = 0;i<m;i++)
    {
        for(int j= 0;j<n;j++)
        {
            c[i][j] = a[i][j] - b[i][j];
        }
    }
}

void multiply(int n1; int n2; int (*a)[n1],int (*b)[n2],int m1,int n1,int n2,int (*c)[n2])
{
    for(int i = 0;i<m1;i++)
    {
        for(int j= 0;j<n2;j++)
        {
            c[i][j] = 0;
        }
    }
    for(int i=0;i<m1;i++)
    {
        for(int j=0;j<n2;j++)
        {
            int sum=0;
            for(int k=0;k<n1;k++)
            {
                sum=sum+a[i][k]*b[k][j];
            }
            c[i][j]=sum;
        }
    }
}
```

Think about sending only a and b matrices to add, subtract and multiply functions and c is the result of operation and is the returned value from these functions. Is it correct to implement this way?

Coding Example_2: Let us write a program to take n names from the user and print it. Each name can have maximum of 20 characters.

```
#include<stdio.h>

int main()
{
    // Think about the commented code? Is it correct?
    /*
    char name1[20]; //Can store one name with maximum of 20 characters including '\0'
    // So need n different variables to store n names
    // But can we store n names under the same variable name???? --- Yes. Use 2D array
    */
    char names[200][20];
    int n;
    printf("How many names you want to store?\n");
    scanf("%d", &n);
    int i;
    printf("enter %d names\n", n); for(i=0;i<n;i++)
    {
        scanf("%[^\\n]s",names[i]);
    }
    printf("U entered below names\n"); for(int i=0;i<n;i++)
    {
        printf("%s\n",names[i]);
    }
    return 0;
}
```

Happy Coding!

Unit #: 2

Unit Name: Counting , Sorting and Searching

Topic: Storage Classes

Course objectives:

The objective(s) of this course is to make students

CObj1: Acquire knowledge on how to solve relevant and logical problems using computing machine

CObj2: Map algorithmic solutions to relevant features of C programming language constructs

CObj3: Gain knowledge about C constructs and it's associated eco-system

CObj4: Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors

CObj5: Get insights about testing and debugging C Programs

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs

CO2: Understand, analyse and apply text processing and string manipulation methods using C Arrays, Pointers and functions

CO3: Understand prioritized scheduling and implement the same using C structures

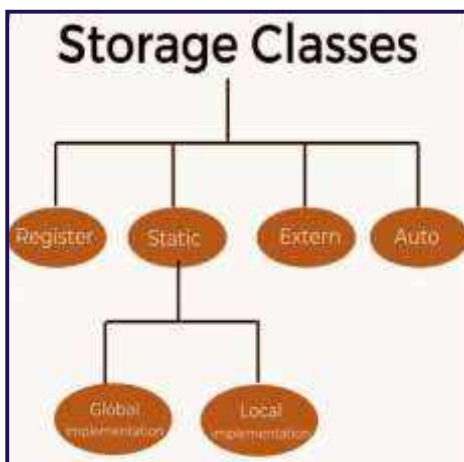
CO4: Understand and apply sorting techniques using advanced C constructs

CO5: Understand and evaluate portable programming techniques using preprocessor directives and conditional compilation of C Programs

Storage classes

Introduction

Storage Classes are used to describe the features of a variable/function. These features basically include the **scope(visibility)** and **life-time** which help us to trace the existence of a particular variable during the runtime of a program. The following storage classes are most often used in C programming. Storage classes in C are used to determine the **lifetime, visibility, memory location, and initial value of a variable**.



Automatic Variables

A variable declared inside a function without any storage class specification is by default an automatic variable. They are created when a function is called and are destroyed automatically when the function execution is completed. Automatic variables can also be called local variables because they **are local to a function**. By default, they are assigned to undefined values.

Coding Example_1:

```

#include<stdio.h>

int main()
{
    int i=90;      // by default auto because defined inside a function
    auto float j=67.5;
    printf("%d %f\n",i,j);
}
  
```



```
int f1()
{
    int a; // by default auto because declared inside a function f1()
           // Not accessible outside this function.
}
```

External variables

The extern keyword is used before a variable **to inform the compiler that the variable is declared somewhere else**. The extern declaration does not allocate storage for variables. All functions are of type extern. **The default initial value of external integral type is 0 otherwise null. We can only initialize the extern variable globally, i.e., we cannot initialize the external variable within any block or method.**

Coding Example_2:

```
int main()
{
    extern int i;    // Information saying declaration exist somewhere else and make it available during linking
                   // if u comment this line, it throws an error: i undeclared
    printf("%d\n",i);
}
int i=23;
```

Note:

An external variable can be declared many times but can be initialized at only once.

The extern modifier is most commonly used when there are two or more files sharing the same global variables or functions

Coding Example_3: Two files share the same global variable

Sample.c

```
#include <stdio.h>

int count ;

extern void write_extern();

int main() {
    count = 5;
    write_extern();
    return 0;
}
```

Sample1.c

```
#include <stdio.h>

extern int count;

void write_extern(void)
{
    printf("count is %d\n", count);
}
```

Here, extern is being used to declare count in the second file, where as it has its definition in the first file, main.c.

// please check the execution steps below

```
gcc Sample.c Sample1.c -c
```

```
gcc Sample.o Sample1.o
```

Output - count is 5

Static variables

A static variable tells the compiler to persist the variable until the end of program. Instead of creating and destroying a variable every time when it comes into and goes out of scope, static is initialized only once and remains into existence till the end of program. A static variable can either be local or global depending upon the place of declaration.

Scope of local static variable remains inside the function in which it is defined but the life time of local static variable is throughout that program file. Global static variables remain restricted to scope of file in each they are declared and life time is also restricted to that file only. **All static variables are assigned 0 (zero) as default value.**

Coding Example_4: Demo for Life time of Static variable

```
#include<stdio.h>

int* f1();

int main()
{
    int *res = f1();
    printf("Res is %p %d\n",res,*res);    //same address in all three outputs. Then 11
    res = f1();
    printf("Res is %p %d\n",res,*res);    // 12
    res = f1();
    printf("Res is %p %d\n",res,*res);    // 13
    return 0;
}

int* f1()
{
    static int a= 10;// memory is shared. This line is ignored after first function call
    a++;
    return &a;    // valid because life time of static variable is through out the file execution
}
```

Coding Example_5: Understand the difference between local and global static variable.

```
#include<stdio.h>

void f1();

static int j=20;    // global static variable: cannot be used outside this program file
int k=23;           //global variable: can be used anywhere by linking with this file.
                   //By default has extern with it.

int main( )
```

```
{  
    f1();           // 0 20  
    f1();           // 0 21  
    f1();           // 0 22  
    return 0;  
}  
void f1()  
{  
    int i=0;  
    printf("%d %d\n",i,j);  
    i++;  
    j++;  
}
```

Register variables

Registers are faster than memory to access, so the variables which are most frequently used in a C program can be put in registers using register keyword. The keyword register hints to compiler that a given variable can be put in a register. It's compiler's choice to put it in a register or not.

Register is an integral part of the processor. It is a very fast memory of computer mainly used to execute the programs and other main operation quite efficiently. They are used to quickly store, accept, transfer, and operate on data based on the instructions that will be immediately used by the CPU. Main operations are fetch, decode and execute. Numerous fast multiple ported memory cells are the atomic part of any register. Generally, compilers themselves do optimizations and put the variables in register. If a free register is not available, these are then stored in the memory only. If & operator is used with a register variable then compiler may give an error or warning (depending upon the compiler used), because when a variable is a register, it may be stored in a register instead of memory and accessing address of a register is invalid. **The access time of the register variables is faster than the automatic variables.**

Mostly used cpu registers are Accumulator, Flag Register, Address Register (AR), Data Register (DR), Program Counter (PC), Instruction Register (IR), Stack Control Register (SCR), Memory Buffer Register (MBR) and Index register (IR)

Coding Example_6:

```
#include<stdio.h>
int main()
{
    register int i = 10;
    int* a = &i; // error
    printf("%d", *a);
    getchar();
    return 0;
}
```

Coding Example_7: We can store pointers into the register, i.e., a register can store the address of a variable.

```
#include<stdio.h>
int main()
{
    int i = 10;
    register int* a = &i; // fine
    printf("%d", *a);
    getchar();
    return 0;
}
```

Note: Static variables cannot be stored into the register since we cannot use more than one storage specifier for the same variable

Global variables

The variables declared outside any function are called global variables. They are not limited to any function. **Any function can access and modify global variables.** Global variables are automatically initialized to 0 at the time of declaration.

Coding Example_8:

```
#include<stdio.h>

int i = 100;

int j;

int main()
{
    printf("%d\n",i);           // 100
    i=90;

    printf("%d\n",i);           // 90

    f1();

    printf("%d\n",i);           // 40
    printf("%d\n",j);           // 0 by default, global variable is 0 if just declared
    return 0;
}

int f1()
{
    i = 40;    // Any function can modify the global variable.
}
```

Think about it!

What if we have `int i = 200;` in a function `f1()` ?

Happy Thinking and Coding!!



Unit #: 2

Unit Name: Counting , Sorting and Searching

Topic: Callback

Course objectives:

The objective(s) of this course is to make students

COBj1: Acquire knowledge on how to solve relevant and logical problems using computing machine

COBj2: Map algorithmic solutions to relevant features of C programming language constructs

COBj3: Gain knowledge about C constructs and its associated eco-system

COBj4: Appreciate and gain knowledge about the issues with C Standards and its respective behaviors

COBj5: Get insights about testing and debugging C Programs

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs

CO2: Understand, analyse and apply text processing and string manipulation methods using C Arrays, Pointers and functions

CO3: Understand prioritized scheduling and implement the same using C structures

CO4: Understand and apply sorting techniques using advanced C constructs

CO5: Understand and evaluate portable programming techniques using preprocessor directives and conditional compilation of C Programs

Introduction

In computer programming, a **callback** is also known as a "**call-after function**". The callback execution may be immediate or it might happen at a later point in time. Programming languages support call backs in different ways, often implementing them with subroutines, lambda expressions, blocks, or function pointers. In C, a **callback function is a function that is called through a function pointer/pointer to a function**. A callback function has a specific action which is bound to a specific circumstance. It is an important element of GUI in C programming.

Let us understand the use of **pointer to a function or a function pointer**.

Function Pointer points to code, not data. The function pointer stores the start of executable code. The function pointers points to functions and store its address.

Syntax: `int (*ptrFunc) ();`

The **ptrFunc** is a pointer to a function that takes no arguments and returns an integer. If parentheses are not given around a function pointer then the compiler will assume that the ptrFunc is a normal function name, which takes nothing and returns a pointer to an integer. Function pointers are not used for allocating or deallocating memory, instead used in reducing code redundancy.

Consider the below cases:

Case 1: `int *a1(int, int, int);` // a1 is a function which takes three int arguments and returns a pointer to int.

Case 2: `int (*p)(int, int, int);` //p is a pointer to a function which takes three int as parameters and returns an int.

We can assign a function name to p as long as the function has the same signature which means the function takes three integer parameters and returns an integer. The function name acts like a pointer to a function.


```
int a2(int,int,int);  
p = a2;  
int y = p(2,3,4); // This statement is the same as calling a2.
```

Why is the callback function required?

Let us answer a few questions to understand the importance of callback function.

How to write a generic library for sorting algorithms such as bubble sort, shell sort, shake sort, quick sort?

How to create a notification event to set a timer in your application?

How to have one common method to develop libraries and event handlers for many programming languages?

How redirect page action is performed for the user based on one click action?

How to extend the features of one function using another one?

A callback is any executable code that is passed as an argument to other code, which is used to call (execute) the argument at a given time. In simple language, if a function name is passed to another function as an argument to call it, then it will be called as a Callback function.

Coding Example_1: Consider the below simple example to understand callback

```
int what(int x, int y, int z, int (*op)(int, int,int))  
{  
    return op(x, y, z);  
}
```

Observe the third argument op. It is a pointer to a function which takes three int arguments and returns an int. The return statement in turn calls the function with three arguments. This function does not know which function is getting called. It depends on the value of op. Compiler knows the type and not the value. The value of a variable is a runtime mechanism.

```
#include<stdio.h>

int what(int x, int y, int z, int (*op)(int, int,int))
{
    return op(x, y, z);
}

int add(int x, int y, int z)
{
    return x + y + z;
}

int mul(int x, int y, int z)
{
    return x * y * z;
}

int main() {
    int (*p)(int, int, int);

    p = add; // p = &add; Both are equivalent
    int res = p(2,3,4); // (*p)(2, 3, 4); //Both are equivalent
    printf("result is %d\n", res);

    p = mul;
    res = p(2,3,4);
    printf("result is %d\n", res);
    // function name can be directly passed as argument to function with callback
    printf("sum is %d\n", what(1, 3, 4, add));
    printf("product is %d\n", what(5, 3, 4, mul));
    return 0;
}
```

Happy Calling Back!!

Unit #: 2

Unit Name: Counting ,Sorting and Searching

Topic: Recursion

Course objectives:

The objective(s) of this course is to make students

CObj1: Acquire knowledge on how to solve relevant and logical problems using computing machine

CObj2: Map algorithmic solutions to relevant features of C programming language constructs

CObj3: Gain knowledge about C constructs and it's associated eco-system

CObj4: Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors

CObj5: Get insights about testing and debugging C Programs

Course outcomes:

At the end of the course, the student will be able to

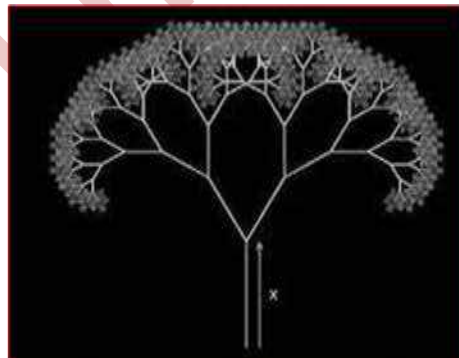
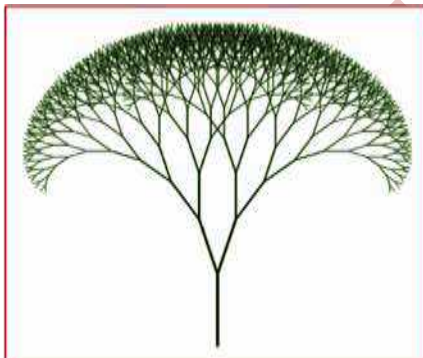
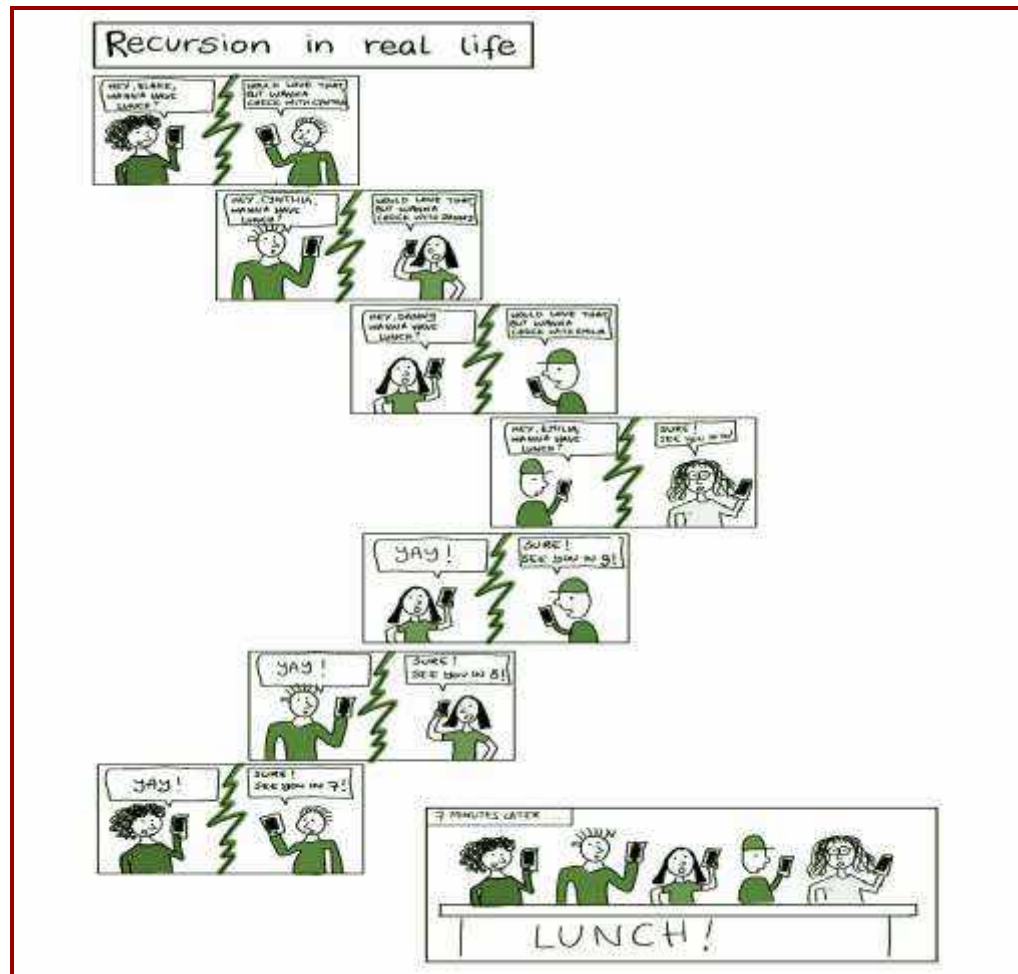
CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs

CO2: Understand, analyse and apply text processing and string manipulation methods using C Arrays, Pointers and functions

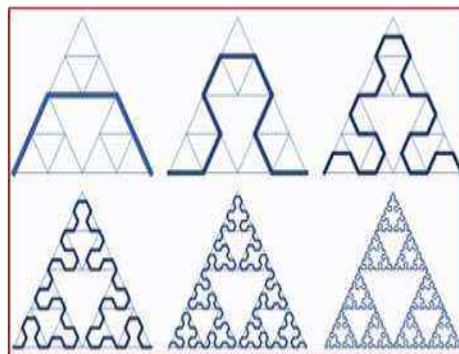
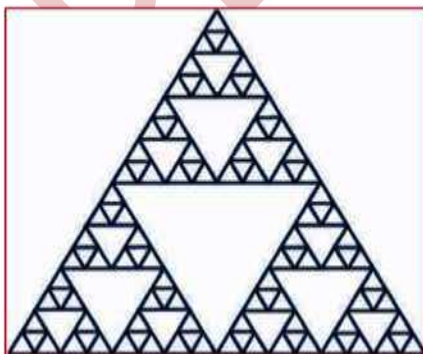
CO3: Understand prioritized scheduling and implement the same using C structures

CO4: Understand and apply sorting techniques using advanced C constructs

CO5: Understand and evaluate portable programming techniques using preprocessor directives and conditional compilation of C Programs



Fractals



The Sierpinski Triangle

Recursion

A function may call itself directly or indirectly. **The function calling itself with the termination/stop/base condition is known as recursion.** Recursion is used to solve various problems by dividing it into smaller problems. We can **opt if and recursion instead of looping statements.** In recursion, we try to **express the solution to a problem in terms of the problem itself, but of a smaller size.**

Coding Example_1:

```
int main()
{
    printf("Hello everyone\n");
    main();
    return 0;
}
```

Execution starts from main() function. Hello everyone gets printed. Then call to main() function. Again, Hello everyone gets printed and these repeats. We have not defined any condition for the program to exit, results in Infinite Recursion. In order **to prevent infinite recursive calls, we need to define proper base condition in a recursive function.**

Let us write Iterative and Recursive functions to find the Factorial of a given number.

Coding Example_2:

```
int fact(int n);
int main()
{
    int n = 6;
    printf("factorial of %d is %d\n",fact(n));
    return 0;
}
```

Iterative Implementation:

```
int fact(int n)
{
    int result = 1;
    int i;
```

```
for (i = 1; i<=n; i++)  
{  
    result = result * i;  
}  
  
return result;  
  
}
```

Recursive Implementation:

Logic: If n is 0 or n is 1, result is 1 Else, result is $n*(n-1)!$

```
int fact(int n)  
{  
    if (n==0)  
        return 1;  
  
    else  
    {  
        return n*fact(n-1);  
    }  
}
```

Pictorial representation: When n is 5

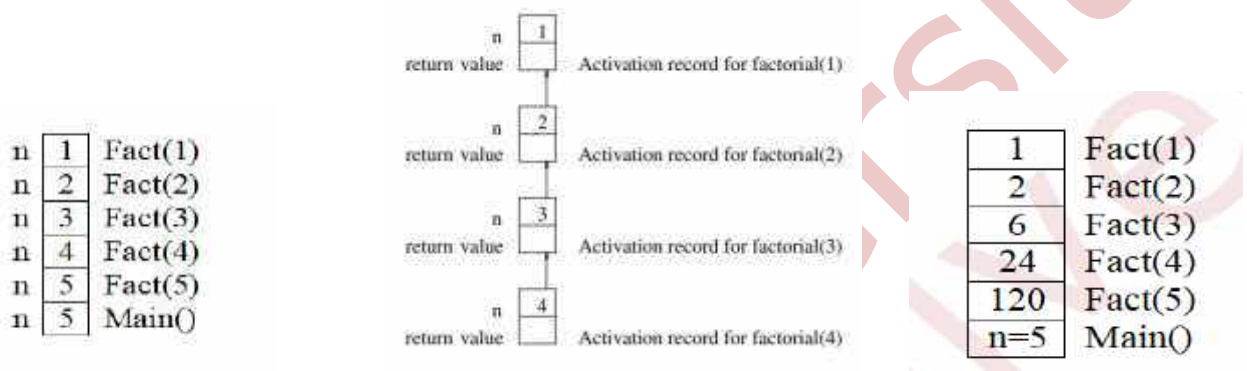
```
=factorial(5)  
=5*factorial(4)  
=5*4*factorial(3)  
=5*4*3*factorial(2)  
=5*4*3*2*factorial(1)  
=5*4*3*2*1*factorial(0)  
=120
```

Output: When n is 6

```
factorial of 6 is 720  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Activation Record

- Each function call generates an instance of that function containing memory for each parameter, memory for each local variable and memory for return values. This chunk of memory is called as an **activation record**.
- To support recursive function calls, the run-time system treats memory as a stack of activation record.
- For example computing the factorial (n) requires allocation of 'n' activation records on the stack.



Let us try to solve below examples to understand recursive calls better.

Coding Example_3: What this code does? Write activation frame for each call and then decide

```
#include<stdio.h>
int what1(int n)
{
    if(n==0) return 0;
    else return (n%2)+10*what1(n/2);
}
int main()
{
    printf("%d",what1(8));
    return 0;
}
```

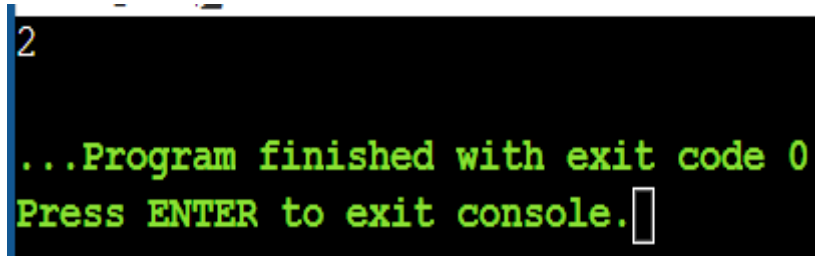
Output: This code prints the binary representation of a given number

```
1000
...Program finished with exit code 0
Press ENTER to exit console.
```

Coding Example_4: What this code does?

```
#include<stdio.h>
int what2_v1(int n);
int what2_v2(int n);
int main()
{
    int n = 10;
    printf("%d",what2_v1(n));
    //printf("%d",what2_v2(n));
    return 0;
}
int what2_v2(int n)
{
    if(!n) return 0;
    else if (!(n%2)) return what2_v2(n/2);
    else
        return 1+what2_v2(n/2);
}
int what2_v1(int n)
{
    if(n==0) return 0;
    else return (n&1)+ what2_v1(n>>1);
}
```


Output: Finds the number of 1s in the binary representation of the number



```
2
...Program finished with exit code 0
Press ENTER to exit console.
```

Coding Example_4: *What this code does?*

```
#include <stdio.h>

int what3_v1(int,int);

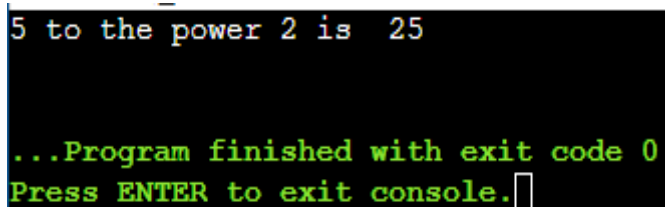
int main()
{   int a=5,b=2;
    printf("%d to the power %d is  %d\n",a,b,what3_v1(a,b));
    //printf("%d to the power %d is  %d\n",a,b,what3_v2(a,b));
    return 0;
}

int what3_v1(int b,int p)
{   int result=1;
    if(p==0)
        return result;
    else
        result=b*(what3_v1(b,p-1));
}

int what3_v2(int b, int p)
{
    if(!p)
        return 1;
    else if(!(p % 2))
        return what3_v2(b*b, p/2);
    else
```

```
return b*what3_v2(b*b,p/2);  
}
```

Output: Finds a to the power b



```
5 to the power 2 is 25  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Coding Example_5: Last example for today!!

```
#include <stdio.h>  
int what4(int);  
int main()  
{  
    int a=5;  
    printf("the sum of numbers upto %d is %d\n",a,what4(a));  
    return 0;  
}  
int what4(int n)  
{  
    if (n>0)  
        return n + what4(n - 1);  
    else if (n==0)  
        return 0;  
    else  
        return -1;  
}
```

Output: If the given number is n, Finds $n+(n-1)+(n-2)+ \dots +0$ If n is -ve, returns -1.

```
the sum of numbers upto 5 is 15  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Few problems for you to Code:

- Recursive function to reverse a string
- Recursive function to print from 1 to n in reverse order
- Find the addition, subtraction and multiplication of two numbers using recursion. Write separate recursive functions to perform these.
- Find all combinations of words formed from Mobile Keypad.
- Find the given string is palindrome or not using Recursion.
- Find all permutations of a given string using Recursion

Unit #: 2

Unit Name: Counting, Sorting and Searching

Topic: Searching

Course objectives:

The objective(s) of this course is to make students

CObj1: Acquire knowledge on how to solve relevant and logical problems using computing machine

CObj2: Map algorithmic solutions to relevant features of C programming language constructs

CObj3: Gain knowledge about C constructs and it's associated eco-system

CObj4: Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors

CObj5: Get insights about testing and debugging C Programs

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs

CO2: Understand, analyse and apply text processing and string manipulation methods using C Arrays, Pointers and functions

CO3: Understand prioritized scheduling and implement the same using C structures

CO4: Understand and apply sorting techniques using advanced C constructs

CO5: Understand and evaluate portable programming techniques using preprocessor directives and conditional compilation of C Programs

Introduction

Even a single day does not pass without we searching for something in our day to day life. Might be car keys, books, pen, mobile charger and what not. Same is the life of a computer. There is so much data stored in it and whenever user asks for some data, computer has to search it's memory to look for the data and make it available to the user. And the computer has it's own techniques to search through it's memory in a faster way.

Why Searching?

We often need to find one particular item of data amongst many hundreds, thousands, millions or more. For example, if one wants to find someone's phone number in your phone, or a particular business's address from address book.

A searching algorithm can be used to find the data without spending much time. What if you want to write a program to search a given integer in an array of integers? Two popular algorithms available:

Linear Search and Binary Search

Linear Search

A linear search, also known as a sequential search, is a method of finding an element within a collection. It checks each element of the list sequentially until a match is found or the whole list has been searched. It returns the position of the element in the array, else it return -1. Linear Search is applied on unsorted or unordered collection of elements.

Coding Example_1: Implementation of Linear Search

```
#include<stdio.h>
int linear_search(int *a, int n, int key);
int main()
{
    int a[100] = {22,55,77,99,25,90,100};
    int n = 7;

    int key = 100;
```

```
int res=linear_search(a,n,key);
if(res==-1)
    printf("%d not found\n",key);
else
    printf("%d found at pos %d\n",key,res);

key=177;
res=linear_search(a,n,key);
if(res==-1)
    printf("%d not found\n",key);
else
    printf("%d found at pos %d\n",key,res);
}

int linear_search(int *a, int n, int key)
{
    for(int i = 0; i < n; i++)
    {
        if(a[i]==key)
        {
            return i;
        }
    }
    return -1;
}
```

For practice: To avoid multiple return statements, linear search may also be coded as:

```
int linear_search(int *a, int n, int key)
{
    int i;
    int found = 0;
    int pos = -1;
    for(i = 0; i < n; i++)
    {
        if(a[i]==key && found == 0)
        {
            found = 1;
            pos = i;
        }
    }
    return pos;
}
```

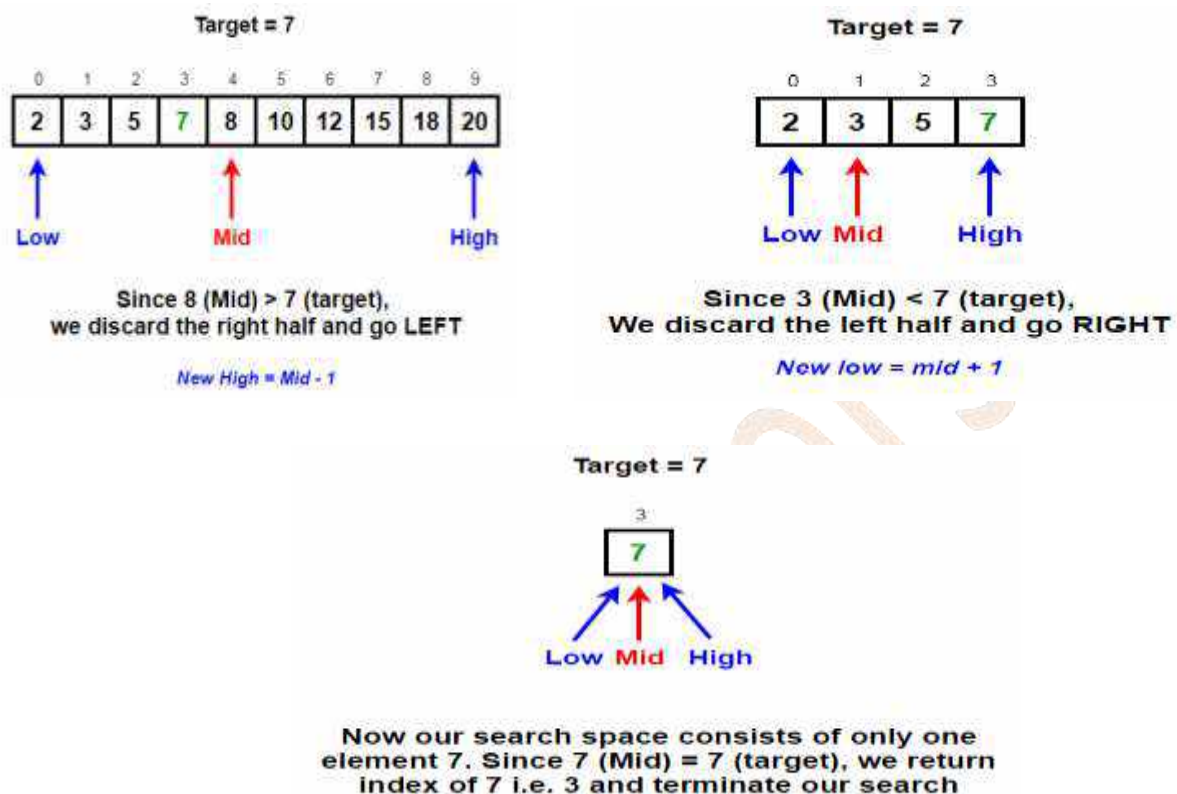
Binary Search

Binary Search is used to search an element in **a sorted array**. The following steps are applied to search an element.

1. Start by comparing the element to be searched with the element in the middle of the array.
2. If we get a match, we return the index of the middle element.
3. If we do not get a match, we check whether the element to be searched is lesser or greater in value than the middle element.
4. If the element/number to be searched is greater in value than the middle number, then we pick the elements on the right side of the middle element(as the array is sorted, hence on the right, we will have all the numbers greater than the middle number), and start again from the step 1.
5. If the element/number to be searched is lesser in value than the middle number, then we pick the elements on the left side of the middle element, and start again from the step 1.

Binary Search Representation

Example: Take an input array by name `int arr = {2, 3, 5, 7, 8, 10, 12, 15, 18, 20}`; and **target or key** to be searched is 7.



Note: Between 2,3,5,7 and 7, write diagrams for missing two iterations. Drawing these two iterations might help you to understand better.

Coding Example_2: Implementation of Binary Search (iterative & recursive) on an array of integers

`/* Binary search: works on sorted collection of elements.`

`Time required to access each element in the collection must be same - constant time`

`*/`

`#include<stdio.h>`

`//Iterative solution`

`int binarySearchIter(int a[],int low,int high,int key)`

`{`


```
while(low<=high)
{
    int mid = (low+high)/2;
    if(a[mid]==key)
        return mid;
    else if(key<a[mid])
        high = mid-1;
    else
        low = mid+1;
}
return -1;
}

// Recursive solution
int binarySearchRec(int a[],int low,int high,int key)
{
    if(low > high) // base condition
        return -1;
    else
    {
        int mid = (low+high)/2;
        if(a[mid]==key)
        {
            return mid;
        }
        else if(key<a[mid])
            return binarySearchRec(a,low,mid-1,key);
        else
            return binarySearchRec(a,mid+1,high,key);
    }
}
```



Happy Searching!!



Unit #: 2

Unit Name: Counting, Sorting and Searching

Topic: Sorting

Course objectives:

The objective(s) of this course is to make students

CObj1: Acquire knowledge on how to solve relevant and logical problems using computing machine.

CObj2: Map algorithmic solutions to relevant features of C programming language constructs.

CObj3: Gain knowledge about C constructs and its associated eco-system.

CObj4: Appreciate and gain knowledge about the issues with C Standards and its respective behaviors.

CObj5: Get insights about testing and debugging C Programs.

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs.

CO2: Understand, analyse and apply text processing and string manipulation methods using C Arrays, Pointers and functions.

CO3: Understand prioritized scheduling and implement the same using C structures.

CO4: Understand and apply sorting techniques using advanced C constructs.

CO5: Understand and evaluate portable programming techniques using pre-processor directives and conditional compilation of C Programs.

Introduction

There are so many things in our real life that we need to search for, like a particular record in the database, roll numbers in the merit list, a particular telephone number in a telephone directory, a particular page in a book etc. The concept of sorting came into existence, making it easier for everyone to arrange data in order to make the searching process easy and efficient. **Arranging the data in ascending or descending order is known as sorting.**

Why Sorting?

Think about searching for something in a sorted drawer and unsorted drawer. If the large data set is sorted based on any of the fields, then it becomes easy to search for a particular data in that set. Sometimes in real world applications, it is necessary to arrange the data in a sorted order to make the searching process easy and efficient.

Example: Candidates selection process for an Interview- Arranging candidates for the interview involves sorting process (sorting based on qualification, Experience, skills or age etc).

Sorting Algorithms

Sorting algorithm specifies the way to arrange data in a particular order. Most common orders are in numerical or lexicographical order. Depending on the data set, Sorting algorithms exhibit different time and space complexity.

Following are samples of sorting algorithms.

1. Bubble Sort
2. Insertion Sort
3. Quick Sort
4. Merge Sort
5. Radix Sort

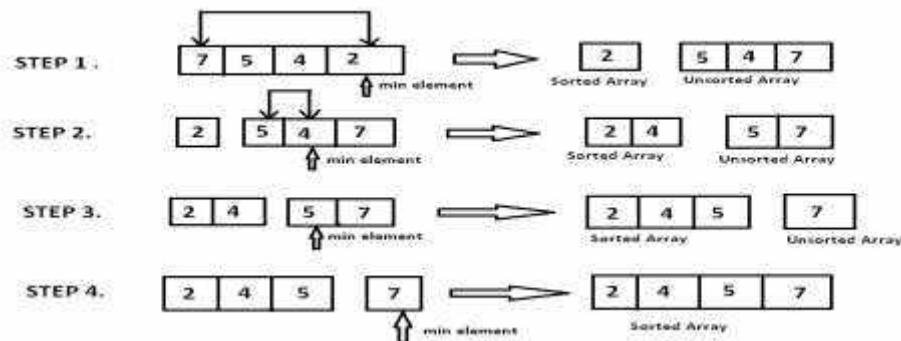
6. Selection Sort -- Dealt in detail

7. Heap Sort

Selection sort

- The method of sorting is faster as the movement of data is minimized.
- Used to arrange data in ascending order, the smallest number is selected and placed at the beginning of the collection.
- When the next pass takes place the second minimum number is selected and placed in the next position.
- The process is repeated until all the elements are placed in the correct order.
- With each pass the unsorted elements are reduced by 1 as one element is sorted and placed in its final position.

Pictorial Representation



Selection Sort Algorithm

The selection sort algorithm sorts an array by repeatedly finding the minimum element (considering ascending order) from the unsorted part and putting it at the beginning. It is **an in-place comparison** sorting algorithm. The algorithm **maintains two sub-arrays in a given array**.

- 1) **The sub-array which is already sorted.**
- 2) **Remaining sub-array which is unsorted.**

Initially, the sorted part of the array is empty and unsorted part is the given array. Sorted part is placed at the left, while the unsorted part is placed at the right.

For sorting in ascending order, in every iteration, the minimum element from the unsorted sub-array is picked and moved to the sorted subarray.

For sorting in descending order, in every iteration, the maximum element from the unsorted sub-array is picked and moved to the sorted sub-array.

Coding Example_1: Selection sort on array of integers

```
#include<stdio.h>
void sort(int *a, int n);
int main()
{
    int a[] = {22,19,14,25,99,81};
    int n = sizeof(a)/sizeof(*a);

    printf("before sorting\n");
    for(int i = 0; i < n; i++)
    {
        printf("%d\t",a[i]);
    }

    sort(a,n);

    printf("\nafter sorting\n");
    for(int i = 0; i < n; i++)
    {
        printf("%d\t",a[i]);
    }

    return 0;
}
```

```
void sort(int *a, int n)
{
    int pos;
    for(int i = 0; i < n-1; i++)
    {
        pos = i;
        for(int j = i+1; j <= n-1; j++) // or j < n
        {
            if(a[pos] > a[j])
            {
                pos = j;
            }
        }
        if(pos != i)
        {
            int t = a[pos];
            a[pos] = a[i];
            a[i] = t;
            //swap(&a[pos], &a[i]);
        }
    }
}
```

Coding Exampe_2: Sort the elements of the array given by the user

```
#include<stdio.h>

void read(int a[],int n);
void disp(int a[],int n);
void sort(int a[],int n);
void swap(int *x,int *y);
```

```
int main()
{
    int a[100];
    int n;
    printf("enter the number of elements u want to sort\n");
    scanf("%d",&n);

    printf("enter %d elements\n",n);
    read(a,n);

    printf("before sorting\n");
    disp(a,n);

    sort(a,n);

    printf("after sorting\n");
    disp(a,n);

    return 0;
}

void read(int a[],int n)
{
    for(int i = 0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
}
```



```
void disp(int a[],int n)
```

```
{
    for(int i = 0;i<n;i++)
    {
        printf("%d ",a[i]);
    }
    printf("\n");
}
```

```
void swap(int *x,int *y)
```

```
{
    int temp = *x;
    *x = *y;
    *y = temp;
}
```

```
void sort(int a[],int n)
```

```
{
    int i,pos,j;
    for(i = 0;i<n-1;i++)
    {
        pos = i;
        for(j = i+1;j<n;j++)
        {
            if(a[pos]>a[j])
                pos = j;
        }
        if(pos != i) // think about this – Is it really required?
            swap(&a[pos],&a[i]);
    }
}
```



Problem Solving with C – UE25CS151B



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE25CS151B**

***Lecture #1
String in C***

**By,
Prof. Sindhu R Pai,
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Poojari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: Text Processing and User-Defined Types****Topic: String in C**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Let us answer a few questions before starting with Unit 3.

What is Text Processing?

The term text processing refers to the **Analysis, Manipulation, and Generation of text**. Text usually **refers to all the alphanumeric characters specified on the keyboard**. It plays a crucial role in systems where working with natural language or textual information is important — such as in **Search engines, Chat bots, Spell Checkers in Word processors, Text Analytics to analyze reviews/feed backs and AI models like ChatGPT!**

Key tasks of Text Processing Units

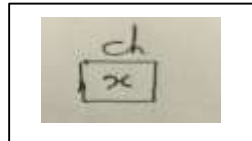
1. **Tokenization** - Breaking text into smaller parts like words, sentences, or characters.
 - Example: "I love pizza." → ["I", "love", "pizza", "."]
2. **Normalization** - Making text uniform by converting to lowercase, removing punctuation, etc.
 - Example: "Hello!" → "hello"
3. **Stop word Removal** - Removing common words like "is", "the", "and" that may not be useful in the application.
4. **Stemming/Lemmatization** - Reducing words to their root forms.
 - Example: "running", "ran" → "run"
5. **Part-of-Speech Tagging (POS)** - Identifying the grammar role of each word (noun, verb, adverb, adjective, etc.).
6. **Named Entity Recognition (NER)** - Detecting names of people, places, dates, etc.
 - Example: "Barack Obama was born in Hawaii." → Recognize "Barack Obama" as a person and "Hawaii" as a location.
7. **Parsing or Syntax Analysis** - Understanding the grammatical structure of sentences.

Text processing in C means working with and manipulating strings. C doesn't have a built-in "string" type like some other languages. So it uses **character arrays and functions** from the standard libraries to handle text.

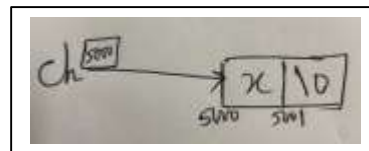
Introduction to Strings in C

String in C is an array of characters terminated with a special character '\0' or NULL. ASCII value of NULL character is 0. Each character occupies 1 byte of memory. There is NO data type available to represent a string in C.

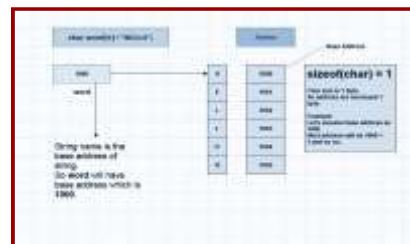
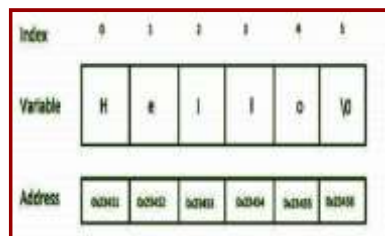
`char ch = 'x' ; // character constant in single quote.`



`char ch[ ] = "x" ; // String constant. Always in double quotes terminated with '\0'. Always occupies 1 byte more when specified between “ and ” in initialization.`



Note: When the compiler encounters a sequence of characters enclosed in the double quotes, it appends a null character '\0' at the end by default. The NULL character is crucial. Without it, C won't know where the string ends.



Definition of a String

`char variable_name[size]; // Size of the array must be specified compulsorily.`

`char a1[10]; // Memory locations are filled with undefined values/garbage value`

`printf("sizeof a1 is %d",sizeof(a1)); // 10`



Initialization of a string

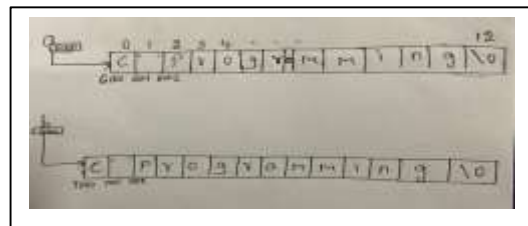
If the size is not specified, the compiler counts the number of elements in the array and allocates those many bytes to an array.

If the size is specified, it allocates those many bytes and unused memory locations are initialized with default value '\0'. This is **partial initialization**.

If the string is hard coded, it is **programmer's responsibility to end the string with '\0' character**.

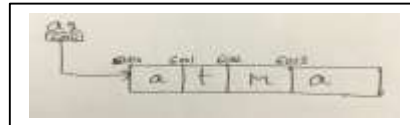
```
char a[ ] = { 'C', ' ', 'P', 'r', 'o', 'g', 'r', 'a', 'm', 'm', 'i', 'n', 'g', '\0' };
char b[ ] = "C Programming";
```

C standard gives shorthand notation to write initializer's list. **b is a shorthand available only for storing strings in C.**

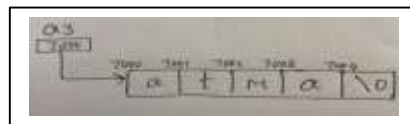


Let us consider below code segments.

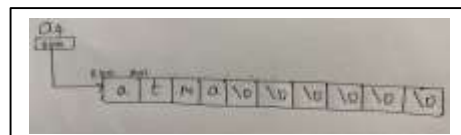
```
char a2[ ] = {'a','t','m','a'}; // Initialization
printf("sizeof a2 is %d", sizeof(a2)); // 4 but cannot assure about a2 while printing
```



```
char a3[ ] = "atma" ; // Initialization
printf("sizeof a3 is %d", sizeof(a3)); // 5 sure about a3 while printing
```

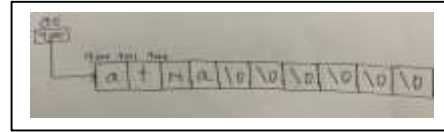


```
char a4[10] = {'a','t','m','a'} // Partial Initialization
printf("sizeof a4 is %d", sizeof(a4)); // 10 sure about a4 while printing
```



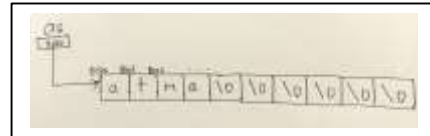
```
char a5[10] = "atma";
```

```
printf("sizeof a5 is %d",sizeof(a5));// 10 sure about a5 while printing
```



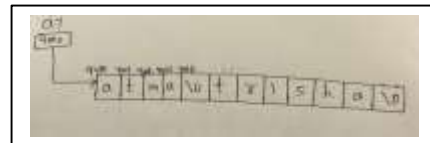
```
char a6[10] = {'a','t','m','a', '\0'};
```

```
printf("sizeof a6 is %d",sizeof(a6));// 10 sure about a6 while printing
```



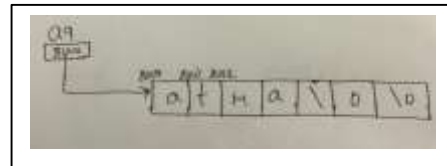
```
char a7[] = {'a', 't', 'm', 'a', '\0', 't', 'r', 'i', 's', 'h', 'a', '\0'};
```

```
printf("sizeof a7 is %d",sizeof(a7)); // 12 a7 will be printed only till first '\0'
```



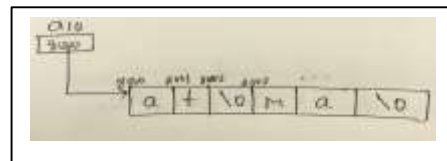
```
char a9[] = "atma\\0" ;
```

```
printf("sizeof a9 is %d",sizeof(a9));// 7 sure about a9 while printing
```



```
char a10[] = "at\\0ma" ;
```

```
printf("sizeof a10 is %d",sizeof(a10));//6 a10 will be printed only till first '\0'
```



Read and Display Strings in C

As usual, the formatted functions, **scanf** and **printf** are used to read and display a string. **%s is the format specifier for string.**

Consider

```
char str1[] = {'a', 't', 'm', 'a', 't', 'r', 'i', 's', 'h', 'a', '\0'};
```


One way of printing all the characters is using putchar/printf in a loop.

```
int i;  
for (i = 0; i<sizeof(str1); i++)  
    printf("%c",str1[i]);    //each element of string is a character. So %c.
```

Think about using while(str1[i] != '\0') rather than sizeof

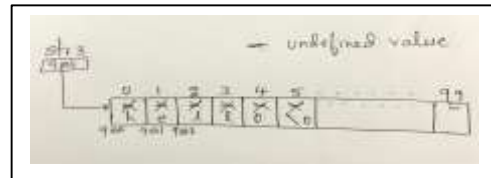
```
while(str1[i] != '\0')  
{  
    printf("%c",str1[i]);  
    i++;  
}
```

Also taking each character from the user using getchar() looks like a tedious task and also it doesn't serve the purpose if you want to store all characters in a string variable and play with it. You can only store one character at a time in a char variable.

So, use %s format specifier with scanf. **scanf with %s will introduce '\0' character at the end of the string. printf with %s requires the address and will display the characters until '\0' character is encountered.**

Coding Example_1: Input a string from the user and print that string.

```
char str3[100] ;  
printf("Enter the string");  
scanf("%s", str3);    // User entered Hello Strings and pressed enter key  
printf("%s\n",str3);    // Hello . Reason: scanf terminates when white space is  
encountered in the user input
```



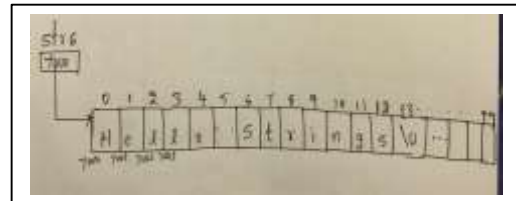
Coding Example_2: Read two strings from the user separated by a white space.

```
char str4[100] ;  
char str5[100] ;  
printf("Enter the string");  
scanf("%s %s", str4, str5);    // User entered Hello Strings and pressed enter key  
printf("%s %s",str4, str5);    // Hello Strings
```

If you want to store the entire input from the user until user presses new line in one character array variable, use `[\n]` with `%s`. Till `\n` encountered, everything has to be stored in one variable.

Coding Example_3: to store the entire input from the user until user presses new line in one string variable

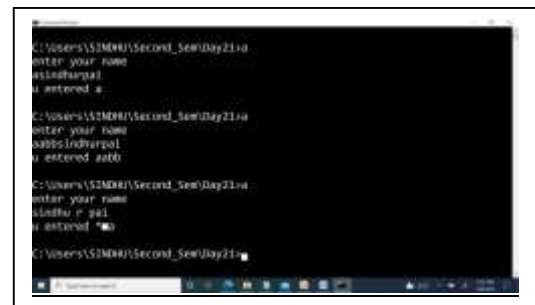
```
char str6[100];
printf("Enter the string");
scanf("%[^\n]s", str6); // User entered Hello Strings and pressed enter key
printf("%s", str6);      // Hello Strings
```



If you want few restricted characters only to be allowed in user input, how do you handle it?

Coding Example_4:

```
char str6[100];
printf("enter your name\n");
scanf("%[abcd]s", str6); // [ ] character class, starting with either a or b or c or d.
//When it encounters other characters, scanf terminates
printf("u entered %s\n", str6);
```



Coding Example_5: How to negate the above character class

```
char str7[100];
printf("enter your name\n");
scanf("%[^abcd]s", str6); // neither a nor b nor c nor d
//When it encounters any of these characters, scanf terminates
printf("u entered %s\n", str6);
```

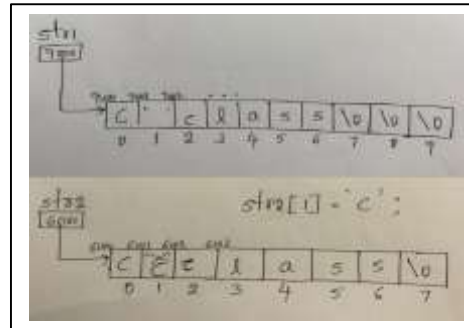


Pointers v/s String

As array and pointers are related, strings and pointers go hand in hand. Let us consider a few examples.

Coding Example_6: Modifying the string pointed by character array.

```
char str1[10] = "C class" ;
printf("%s", str1); // C class
char str2[ ] = "C class" ;
printf("%s\n", str2); // C class
str2[1] = 'C' ;
printf("%s\n", str2); // CCclass
```



Coding Example_7: Modifying the string pointed by character pointer

```
char *p1 = "pesu"; //p1 is stored at stack. "pesu" is stored at code segment of memory. It is
read only. // This statement assigns to p1 variable a pointer to the character array
printf("size is %d\n", sizeof(p1)); // size of pointer
printf("p1 is %s", p1); //pesu // p1 is an address. %s prints until \0 is
encountered
p++; // Pointer may be modified to point elsewhere.
printf("p1 is %s", p1); //esu
p1[1] = 'S'; // No compile time Error
printf("p1 is %s", p1); //Behaviour is undefined if you try to modify the read only location
```

Coding Example_8: Array is a constant pointer.

```
char p2[] = "pesu"; // Stored in the Stack segment of memory
printf("size is %d\n", sizeof(p2)); // 5 = number of elements in array+1 for '\0'
printf("p2 is %s", p2); //pesu
p2[1] = 'E'; //Individual characters within the array may be modified.
printf("p2 is %s", p2); //pEsu
p2++; // Compile time Error // Array always refers to the same storage.
```

Happy Coding using Strings!!



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE25CS151B**

***Lecture #3
String Manipulation Functions in C***

**By,
Prof. Sindhu R Pai,
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Poojari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: Text Processing and User-Defined Types****Topic: String Manipulation Functions in C**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and its respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Introduction

We know that in C, **strings** are represented as arrays of characters ending with a **null character** (`'\0'`). Since C **does not have a built-in string type** like some other languages such as Python and Javascript, string manipulation is done using **standard library functions** provided in the **string.h** header file. These functions help perform **common operations** like:

- Copying strings
- Concatenating strings
- Comparing strings
- Finding the length of a string
- Searching within strings

The `<string.h>` provides **around 25 standard string and memory functions**, although the **exact count may vary slightly** depending on the compiler and platform. The below table lists **most commonly used string related functions in from string.h**, including:

- What each function does
- What it returns on success
- What it returns on failure (if applicable)

Function Name	Description	Return values
<code>strlen(str)</code>	Returns the number of characters in <code>str</code> (excluding <code>'\0'</code>)	Length of the string (<code>size_t</code>) on success; undefined behavior if <code>str</code> is <code>NULL</code>
<code>strcpy(dest, src)</code>	Copies <code>src</code> into <code>dest</code> including the null terminator	Pointer to <code>dest</code> on success; undefined behavior if <code>src</code> or <code>dest</code> is <code>NULL</code> or overlap
<code>strncpy(dest, src, n)</code>	Copies up to <code>n</code> characters from <code>src</code> to <code>dest</code>	Pointer to <code>dest</code> ; may not null-terminate if <code>src</code> is longer than <code>n</code> ; undefined if <code>NULL</code>
<code>strcat(dest, src)</code>	Appends <code>src</code> to the end of <code>dest</code>	Pointer to <code>dest</code> on success; undefined behavior if strings overlap or <code>dest</code> is too small
<code>strncat(dest, src, n)</code>	Appends up to <code>n</code> characters of <code>src</code> to <code>dest</code>	Pointer to <code>dest</code> ; undefined behavior if space is insufficient
<code>strcmp(str1, str2)</code>	Compares <code>str1</code> and <code>str2</code>	0 if equal, <code><</code> if <code>str1 < str2</code> , <code>></code> if <code>str1 > str2</code> ; undefined if <code>NULL</code>
<code>strncmp(str1, str2, n)</code>	Compares up to <code>n</code> characters of two strings	Same as <code>strcmp</code> ; undefined if either string is <code>NULL</code>
<code>strchr(str, ch)</code>	Finds first occurrence of <code>ch</code> in <code>str</code>	Pointer to first match on success; <code>NULL</code> if not found
<code>strrchr(str, ch)</code>	Finds last occurrence of <code>ch</code> in <code>str</code>	Pointer to last match on success; <code>NULL</code> if not found
<code>strstr(haystack, needle)</code>	Finds first occurrence of <code>needle</code> in <code>haystack</code>	Pointer to match on success; <code>NULL</code> if not found
<code>strtok(str, delim)</code>	Splits <code>str</code> into tokens using <code>delim</code>	Pointer to next token on success; <code>NULL</code> if no more tokens

Note:

- Most functions **return a pointer** to allow chaining and efficiency.
- Many functions exhibit **undefined behavior** if passed **NULL pointers** or **non-null-terminated strings**.
- strtok() maintains internal state, so it is **not thread-safe** — use strtok_r() in multi-threaded programs.

Coding Example_1: Demo of all above functions with simple examples

```
#include <stdio.h>
#include <string.h>

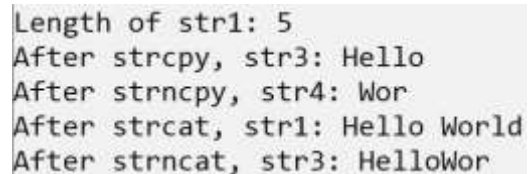
int main() {
    char str1[100] = "Hello";
    char str2[] = "World";
    char str3[100];
    char str4[100];

    // strlen
    printf("Length of str1: %lu\n", strlen(str1)); // Output: 5

    // strcpy    // Compiletime Error
    str3 = str1; // we are trying to equate two addresses. Array Assignment incompatible
    strcpy(str3, str1);
    printf("After strcpy, str3: %s\n", str3);

    // strncpy
    strncpy(str4, str2, 3);
    str4[3] = '\0'; // manually null-terminate
    printf("After strncpy, str4: %s\n", str4);

    // strcat
    strcat(str1, " ");   strcat(str1, str2);
    printf("After strcat, str1: %s\n", str1); // Make sure str1 is big enough to hold str1 and str2 both.
```



```
Length of str1: 5
After strcpy, str3: Hello
After strncpy, str4: Wor
After strcat, str1: Hello World
After strncat, str3: HelloWor
```

```
// strncat
strncat(str3, str2, 3);
printf("After strncat, str3: %s\n", str3); // Output: HelloWor

// strcmp
printf("strcmp(\"abc\", \"abd\"): %d\n", strcmp("abc", "abd")); // < 0

// strncmp
printf("strncmp(\"abc\", \"abd\", 2): %d\n", strncmp("abc", "abd", 2)); // 0

// strchr
char *ptr = strchr(str1, 'W');
if (ptr) {
    printf("First 'W' in str1 at index: %ld\n", ptr - str1); // Output: 6
}

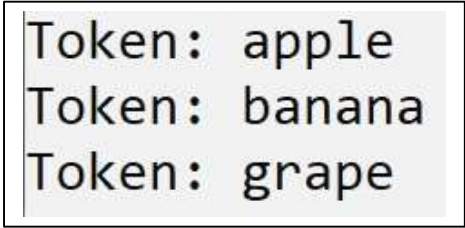
// strrchr
char testStr[ ] = "banana";
char *last_a = strrchr(testStr, 'a');
if (last_a) {
    printf("Last 'a' in %s at index: %ld\n", testStr, last_a - testStr); // Output: 5
}

// strstr
char *found = strstr(str1, "World");
if (found) {
    printf("\"World\" found at position: %ld\n", found - str1); // Output: 6
}
```

```
strcmp("abc", "abd"): -1
strncmp("abc", "abd", 2): 0
First 'W' in str1 at index: 6
Last 'a' in banana at index: 5
"World" found at position: 6
```



```
// strtok  
char fruits[ ] = "apple,banana,grape";  
char *token = strtok(fruits, ",");  
while (token != NULL) {  
    printf("Token: %s\n", token);  
    token = strtok(NULL, ",");  
}  
return 0;  
}
```



```
Token: apple  
Token: banana  
Token: grape
```

Now let us try writing the user defined functions to understand few operations in detail.

Coding Example_2: User defined function to find the string length. Client code is given.

```
char mystr[ ] = "pes";  
printf("Length is %d\n", my_strlen(mystr));
```

Different versions of my_strlen() implementation are as below.

Version 1: Iterate through every character of the string using a variable. Stop iteration when NULL character is encountered and return the value of that variable.

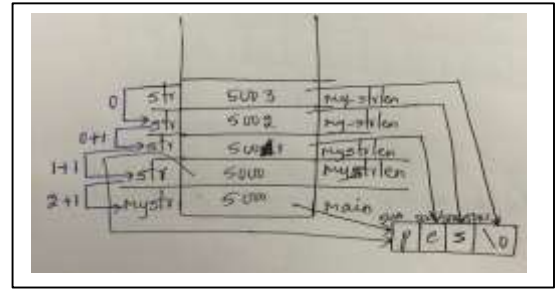
```
int my_strlen(char str[])  
{  
    int i = 0;  
    while(str[i] != '\0')    {    ++i;    }  
    return i;  
}
```

Version 2: Run the loop till *s becomes '\0' character. Increment the pointer and the counter when *str is not NULL.

```
int my_strlen(char *str)  
{  
    int i = 0;  
    while(*str){  
        // str[i] != '\0' // *(str+i) != '\0' ---> all these are same  
        i++;    str++;    }  
    return i;  
}
```

Version 3: Using recursion and pre-increment

```
int my_strlen(char *str)
{
    if (!(*str))    return 0;
    else            return 1+my_strlen(++str);
}
```

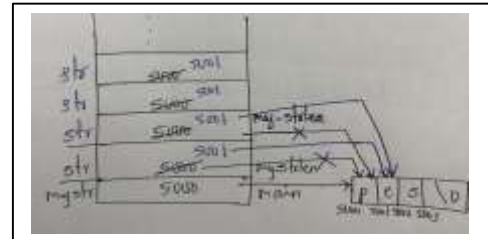


Version 4: Using recursion and no change in the pointer in the function call

```
int my_strlen(char *str)
{
    if (!(*str))    return 0;
    else return 1+my_strlen(str+1);
}
```

Version 5: Using recursion and post-increment

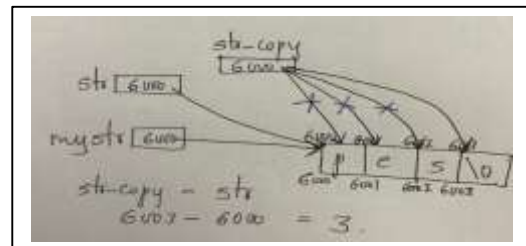
```
int my_strlen(char *str)
{
    if (!(*str)) return 0;
    else return 1+my_strlen(str++);
}
```



Version 6: Using Pointer arithmetic

Use a local pointer which points to the first character of the string. Keep incrementing this till '\0' is found. Then subtract this from the pointer specified in the parameter which points to the beginning of the string. This finds the length of the string.

```
int my_strlen(char *str)
{
    char *str_copy = str;
    while(*str_copy){ str_copy++; }
    return str_copy - str;
}
```



Coding Example_3: User defined function to copy the given string. Client code is given.

```
char mystr1[ ] = "pes university";  
char mystr2[100];  
printf("%s\n",mystr1);  
my_strcpy(mystr2, mystr1);  
printf("%s\n",mystr2);
```

Version 1: Implementation of my_strcpy () by passing arrays to function and using array notation

```
void my_strcpy(char *b, char *a)  
{  
    // copy a to b  
    int i =0;  
    while(a[i] != '\0')  
    {  
        b[i]=a[i];  
        i++;  
    }  
    b[i] = '\0';    // append '\0' at the end  
}
```

Version 2: Implementation of my_strcpy () by passing arrays to function and using pointer notation

```
void my_strcpy(char *b, char *a)  
{    // copy a to b  
    while(*a != '\0')  
    {    *b = *a; b++; a++;    }  
    *b = '\0';    // append '\0' at the end  
}
```

Version 3: Implementation of my_strcpy () using pointer notation

```
void my_strcpy(char *b, char *a)  
{  
    while((*b = *a) != '\0')  
    {        b++; a++;    }  
}
```

Version 4: Implementation of my_strcpy() using pointer notation

```
void my_strcpy(char *b, char *a)
{
    while(*b++ = *a++);    // same as for(;*b++ = *a++;);
}
```

Coding Example_4: User defined function to compare the given two strings. Client code is given.

```
char str1[100]; char str2[100];
printf("enter the first string\n"); scanf("%s",str1);
printf("enter the second string\n"); scanf("%s",str2);
int res = my_strcmp(str1,str2);
printf("result is %d\n",res);
if(!res)
    printf("%s and %s are equal\n",str1,str2);
else if(res > 0)
    printf("%s is higher than %s\n",str1,str2);
else
    printf("%s is lower than %s\n",str1,str2);
```

Version 1: my_strcmp() returns < 0 if a < b, 0 if a == b , > 0 if a > b

```
int my_strcmp(char *a, char *b)
{
    int i;
    for(i = 0; b[i] != '\0' && a[i] != '\0' && b[i] == a[i]; i++);
    return a[i] - b[i];
}
```

Version 2: Using pointer notation

```
int my_strcmp(char *a, char *b)
{
    for(;*b && *a && *b == *a; a++,b++);
    return *a - *b;
}
```

Coding Example_5: User defined function to find the address and the position of character in a given string. Client code is given.

```
char str1[100];
printf("enter the string\n");
scanf("%s",str1);
printf("enter the character\n");
char ch = getchar();
char *p = my_strchr(str, ch);
printf("present in this address %d",p);
if(p)
    printf("present in %d position\n", p - a);
else
    printf("character not present\n");
```

Version 1:

```
char* mystrchr(char *a, char c)
{
    char *p = NULL;
    char *s = a;
    while(*s != '\0' && p==NULL)
    {
        if(*s == c)
            p = s;
        s++;
    }
    return p;
}
```

Version 2:

```
char *mystrchr(char *a,char c)
{
    while(*a && *a != c)
    {
        a++;
    }
    if (!(*a)) return NULL;
    else return a;
    // Can replace if else with return !(*a)? NULL: a;
}
```

Coding Example_6: User defined function to concatenate the given two strings. Client code is given.

```
char str1[100];
char str2[100];
printf("enter first string\n"); scanf("%s",str1);
printf("enter second string\n"); scanf("%s",str2);
my_strcat(str1,str2);
printf("str1 is %s and str2 is %s\n",str1,str2);
```

Version 1:Using Array notation on the pointer

```
void my_strcat(char *a, char *b)
{
    int i = 0;
    while(a[i] != '\0') { i++; } // while can be replaced with strlen()
    int j;
    for(j = 0; b[j] != '\0'; j++, i++)
        a[i] = b[j];
    a[i] = '\0';
}
```

Version 2:Using Pointer notation on the pointer

```
void my_strcat(char *a, char *b)
{
    while(*a){ a++; } // increment a till NULL.
    while(*b) { *a = *b; a++; b++; }
    // Once NULL, copy each character from b to a and increment a and b both till b becomes '\0'
    *a = '\0'; // assign '\0' character to a's end
}
```

While these functions are efficient, they require **careful attention to memory management, null terminators, and buffer sizes to prevent common pitfalls such as buffer overflows, memory corruption, and undefined behavior.** In this regard, typical mistakes that occur during string manipulation and best practices to avoid them are explored in the next Lecture notes.

Happy Coding using String Functions!!



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE25CS151B**

***Lecture #4
Errors and Best Practices associated with String
Manipulation Functions***

**By,
Prof. Sindhu R Pai,
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Poojari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: Text Processing and User-Defined Types****Topic: Errors and best Practices associated with String Manipulation Functions**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Introduction

String manipulation functions, while powerful and efficient, can lead to serious issues if not used carefully. This lecture notes highlights common errors developers encounter—such as **buffer overflows, improper null termination, and memory corruption**—and provides insights into how these problems arise and **how they can be effectively avoided through best practices**.

Buffer Overflow:

A buffer is a **temporary, fixed-size block of memory** used to store data—such as characters in a string while it is being transferred or processed. In programming, buffers help manage data efficiently, especially during input/output operations or string manipulations. **Buffer overflow occurs when writing beyond the bounds of the destination buffer.**

Coding Example_1:

```
char dest[5];  
strcpy(dest, "Hello, World!"); // Overflows dest buffer
```

The above problem can be avoided by ensuring that the **destination buffer is large enough and use safer alternatives like strncpy.**

```
char dest[20];  
strncpy(dest, "Hello, World!", sizeof(dest) - 1);  
dest[sizeof(dest) - 1] = '\0'; // Ensure null termination
```

Missing Null Terminator:

Some functions do not automatically null-terminate strings if the size limit is reached.

Coding Example_2:

```
char dest[5];  
strncpy(dest, "Hello", 5); // Copies 5 characters, but no space for '\0'  
printf("%s\n", dest); // May cause undefined behavior
```

The above problem can be avoided by ensuring that the **destination buffer is large enough and use safer alternatives like strncpy.**

```
char dest[6]; // observe this
snprintf(dest, sizeof(dest), "%s", "Hello");
// snprintf automatically ensures null termination as long as the buffer size is respected.
printf("%s\n", dest);
```

Coding Example_3: Using strlen on a string that is not null-terminated.

```
char arr[5] = {'H', 'e', 'l', 'l', 'o'}; // No '\0'
printf("%zu", strlen(arr)); // Unsafe
```

The above problem can be avoided by ensuring that the **data is within double quotes and memory is large enough to hold the data.**

```
char arr[6] = "Hello"; // Automatically null-terminated
printf("%zu", strlen(arr)); // 5
```

Using Uninitialized Pointers:

An uninitialized pointer is a pointer variable that has been declared but not assigned any valid memory address before being used. **Performing string operations on uninitialized pointers can lead to segmentation faults.**

Coding Example_4:

```
char *str; // Uninitialized pointer
strcpy(str, "Hello");
// str doesn't point to valid memory. So copying a string into it is dangerous
```

The above problem can be avoided **by initializing the pointers before use—either by assigning them to a valid variable's address or by dynamically allocating memory.**

```
char *str = malloc(20); // Allocate memory
strcpy(str, "Hello"); // Safe
printf("%s\n", str);
free(str); // Best practice related to DMA functions
```

When working with string manipulation functions in C, even small mistakes can lead to serious issues such as **crashes, memory corruption, or security vulnerabilities**. To ensure safe and reliable code, it's essential to **follow certain best practices**. The following tips help prevent common pitfalls and promote robust string handling.

1. **Allocate Sufficient Buffer Size**
 - Always include space for the **null terminator** (`\0`) when defining string buffers.
2. **Use Safer Alternatives**
 - Prefer `strncpy`, `strncat`, and `snprintf` over `strcpy`, `strcat`, and `sprintf`.
3. **Manually Add Null Terminator if Needed**
 - Functions like `strncpy` may not null-terminate—do it manually.
4. **Check Buffer Sizes Before Copying**
 - Prevent buffer overflows by validating destination size before any copy/append.
5. **Initialize Pointers Properly**
 - Don't use pointers before assigning them a valid memory address.
6. **Free Allocated Memory**
 - Use `free()` to avoid memory leaks when memory is allocated with `malloc`.
7. **Validate Strings Before Using `strlen`**
 - Ensure the string is null-terminated before calling `strlen` or similar functions.
8. **Sanitize and Validate Input**
 - Always check the size and validity of input strings, especially from user input.

To further strengthen the safety of your code, avoid using functions like `strcpy` or `memcpy` when the source and destination memory areas may overlap—opt for **`memmove`, which is designed to handle such cases safely**. Additionally, make use of compiler warnings (such as `-Wall` and `-Wextra`) and static analysis tools to catch potential issues early in the development cycle. These proactive measures help in identifying risky patterns and maintaining the overall reliability and security of your string operations.

Avoid hidden bugs and ensure smooth execution of string manipulation functions!!!



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE25CS151B**

***Lecture #6
Command Line Arguments***

**By,
Prof. Sindhu R Pai,
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Poojari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: Text Processing and User-Defined Types****Topic: Command Line Arguments**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Introduction

Command line arguments are values or inputs passed **to a program directly from the terminal or command prompt when the program is run**. They allow users to customize the program's behavior without changing the code or without waiting for user input during execution. Hence, the program input can be dynamic and flexible, accepting data without hard coding it. These are **widely used in scripting, automation, and software tools** where inputs need to be customized without modifying the code.

Structure of main() with CLA

The arguments are provided to the program after the name of the executable in command-line shell of Operating Systems and handled using two special parameters in the main() function:

int main (int argc, char *argv[])

argc (Argument Count) tells how many arguments were passed. It always includes the program name as the first argument. So, argc is **at least 1**.

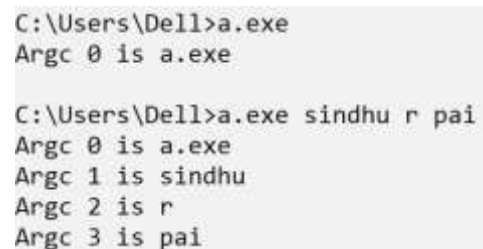
argv (Argument Vector) is an array of character pointers listing all arguments, starting with the program name itself.

- argv[0] is the name of the program.
- argv[1], argv[2], ..., argv[argc-1] are the actual command line arguments.

NOTE: All the arguments which are passed in command line are accessed as **string inside the program and must be converted to desired type using different functions.**

Coding Example_1: Display all the command line arguments passed

```
#include<stdio.h>
int main (int argc, char *argv[])
{
    int i;
    for (i = 0; i < argc; i++)
        printf ("Argc %d is %s\n", i, argv[i]);
    return 0;
}
```



```
C:\Users\Dell>a.exe
Argc 0 is a.exe

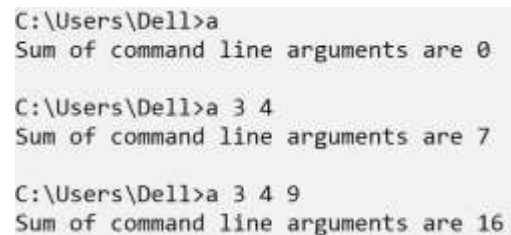
C:\Users\Dell>a.exe sindhu r pai
Argc 0 is a.exe
Argc 1 is sindhu
Argc 2 is r
Argc 3 is pai
```

Coding Example_2: Program to add three integers passed in the command line

```
#include<stdio.h>
#include<string.h>
int main(int argc,char *argv[]) //conventionally argc and argv, Can use any variable
{
    printf("count of arguments are %d\n",argc); // argc =4
    printf("argv[0] is %s\n",argv[0]);
    printf("argv[1] is %s\n",argv[1]);
    printf("argv[2] is %s\n",argv[2]);
    printf("argv[3] is %s\n",argv[3]);
    printf("Sum is is %d\n",atoi(argv[1])+ atoi(argv[2])+ atoi(argv[3]));
    // The C library function atoi(str) converts the string argument str to an integer
    //(type int). If no valid conversion could be performed, it returns 0
    return 0;
}
```

Coding Example_3: Find the sum of all numbers provided in the command line.

```
#include<stdio.h>
#include<stdlib.h>
int main(int argc,char *argv[])
{
    int i;
    int sum = 0;
    for(i = 0; i < argc; i++)
        sum = sum + atoi(argv[i]);
    printf("Sum of command line arguments are %d\n",sum);
    return 0;
}
```



```
C:\Users\Dell>a
Sum of command line arguments are 0

C:\Users\Dell>a 3 4
Sum of command line arguments are 7

C:\Users\Dell>a 3 4 9
Sum of command line arguments are 16
```

Real-world use cases of Command Line Arguments (CLA)

Command Line Arguments in C are widely used in real-world applications like file processors, data converters, and automation scripts, enabling users to pass inputs dynamically at runtime without modifying the code.

- Programs that read from or write to files often take file names as command line arguments.

```
./filecopy input.txt output.txt // Executable name is filecopy.
```

- Perform arithmetic based on operands and operators passed via CLA.

```
./calc 12 + 8
```

- Convert data or files from one format to another.

```
./convert file.csv file.json
```

- Search for a keyword or pattern in a file.

```
./search "error" log.txt
```

- Automate tasks like backups or process monitoring using file paths.

```
./backup /home/user/docs/
```

- Accept runtime parameters like speed, size, or time to control simulation.

```
./simulate 60 300 //Runs simulation with speed = 60 and duration = 300 units.
```

- Sort data in files by a specific field or apply filters.

```
./sort students.csv age
```

- Accept credentials or access keys as arguments (often in secure environments)

```
./login admin mypass123
```

- Pass file names and operation types (compress/decompress).

```
./zipper compress data.txt
```

- Customize game levels or character selection at startup.

```
./game start level3 warrior
```

- Input a date and get formatted output or day of the week.

```
./calendar 2025-04-11
```

Happy coding with Command Line Arguments!



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE25CS151B**

***Lecture #8
Dynamic Memory Management in C***

**By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Poojari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: Text Processing and User-Defined Types****Topic: Dynamic Memory Management in C**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and its respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Introduction

Dynamic memory management refers to the **ability to allocate, reallocate, and free memory during the execution of a program**. Unlike static memory allocation, where the size and structure of memory are fixed at compile-time, dynamic memory **allows programs to be more flexible and efficient, adapting to varying data sizes or user inputs at runtime**. This is particularly useful for working with data structures such as **linked lists, trees, dynamic arrays, and graphs**, where the amount of memory required cannot be determined in advance.

Few points to think before we proceed:

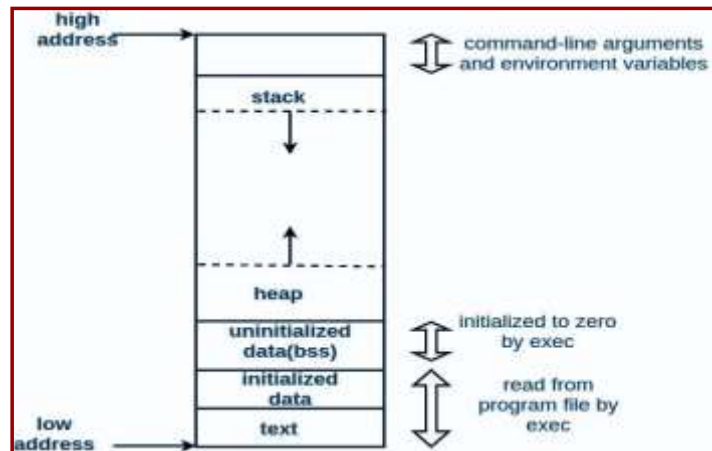
- In case of an array, memory is allocated before the execution time. Is there over utilization or under utilization of memory? - Yes
- Can we take the size of the array at runtime and request to allocate memory for the array at runtime? – This is Variable length Array(VLA).

It is not a good idea to use this as there is no functionality in VLA to check the non availability of the space. If the size is large, **results in code crash without any intimation**.

- Can we avoid this by allocating memory whenever and how much ever we require using some of the functions? – Yes, Use DMM functions

Memory Layout of C Program

The Operating System does the memory management job for C Program. It helps in the **allocation and deallocation of memory blocks** either during **compile-time** or during the **run-time**. When the **memory is allocated during compile-time**, it is **stored** in the **Static Memory** and it is **known** as **Static Memory Allocation**, and when the **memory is allocated during run-time**, it is stored in the **Dynamic Memory** and it is known as **Dynamic Memory Allocation**. When a C program is executed, its memory is divided into several distinct segments, each with a specific purpose. Understanding this layout is essential for effective use of memory, especially when working with pointers and dynamic memory. Refer to the diagram in the next page for the pictorial representation of Memory Layout.



Text segment

It contains **machine code** of the **compiled program**. Usually, it is sharable so that only a single copy needs to be in memory for frequently executed programs, such as **text editors**, **C compiler**, **shells**, and so on. The text segment of an **executable object file** is usually **read-only segment** that prevents a program from being accidentally modified.

Initialized Data Segment

Stores all **global, static, constant, and external variables** - declared with `extern` keyword that are initialized beforehand. It is **not read-only**, since the values of the variables can be changed at run time. This segment can be further classified into **initialized read-only area** and **initialized read-write area**.

```
#include <stdio.h>
const char Entity[]="PES University"; /* global variable stored in Initialized Data Segment in read-only area*/
char Faculty[]="Nitin V Pujari"; /* global variable stored in Initialized Data Segment in read-write area*/
int main()
{
    static int Value = 11; /* static variable stored in Initialized Data Segment*/
    return 0;
}
```

Uninitialized Data Segment(bss)

Data in this segment is initialized to arithmetic 0 before execution of the program. Uninitialized data starts at the end of the data segment and contains all **global variables and static variables that are initialized to 0** or **do not have explicit initialization in source code**.

```
#include <stdio.h>
char Entity; /* Uninitialized variable stored in bss*/
char Faculty; /* Uninitialized variable stored in bss*/
int main()
{
    static int Value; /* Uninitialized static variable stored in bss */
    return 0;
}
```

Heap Segment

It is the segment where dynamic memory allocation usually takes place. When some more memory need to be allocated using malloc and calloc function, heap grows upward. The Heap area is shared by all shared libraries and dynamically loaded modules in a process.

```
#include <stdio.h>

int main()
{
    int *Value=(int*)malloc(sizeof(int)); /* memory allocating in heap segment */
    return 0;
}
```

Stack Segment

Is used to **store all local variables and is used for passing arguments to the functions** along with the return address of the instruction which is to be executed after the function call is completed. Local variables have a scope to the block where they are defined in, they are created when control enters into the block. All recursive function calls are added to stack.

Note: The stack and heap are traditionally located at opposite ends of the process's virtual address space

C does **not provide any operator for dynamic memory management**. Instead, it relies on library functions, known as **memory management functions**, to allocate and release memory during program execution. These functions are declared in the **stdlib.h** header file.

1. malloc():

It **allocates the specified number of bytes and returns a void pointer to the first byte** of the allocated memory block. Does not initialize the memory and contains **garbage values**.

Prototype: **void *malloc(size_t size);**

If the allocation is successful, it returns a valid memory address. If it fails (e.g., due to insufficient memory), it returns NULL.

Coding Exampe_1: Dynamic allocation for an integer

```
int main() {  
    int* p = (int*)malloc(sizeof(int)); //dynamic allocation for one integer  
    //address of that location is returned in p  
    *p = 10;  
    printf("p = %d", *p);  
}
```



```
C:\Users\De11>a  
p = 10
```

Coding Exampe_2: Dynamic allocation to n integers and storing n integers

```
#include<stdio.h>  
#include<stdlib.h>  
int main() {  
    int *x; int n, i;  
    printf("Enter the number of elements\n");  
    scanf("%d",&n);  
    x = (int*)malloc(n * sizeof(int)); //memory will be allocated for 5 integer numbers  
    if (x == NULL) //to check if memory is allocated or not by malloc  
        printf("Memory not allocated.\n");  
    else  
    {
```

```

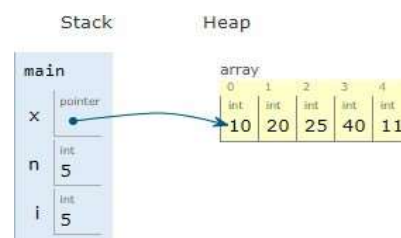
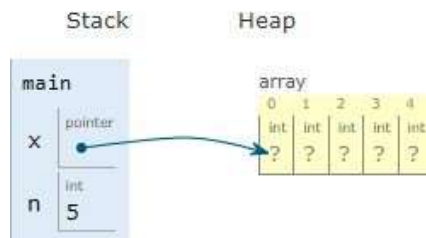
printf("Memory successfully allocated using malloc.\n");
printf("Enter the integer values:\n");
for (i = 0; i < n; i++) // To read inputs from user
{
    scanf("%d",&x[i]);
}
printf("Output: "); // Printing
for (i = 0; i < n; i++)
    printf("%d\t", *(x+i));
}
return 0;
}

```

```

C:\Users\De11>a
Enter the number of elements
5
Memory successfully allocated using malloc.
Enter the integer values:
10
20
25
40
11
Output: 10    20    25    40    11

```



2. calloc ():

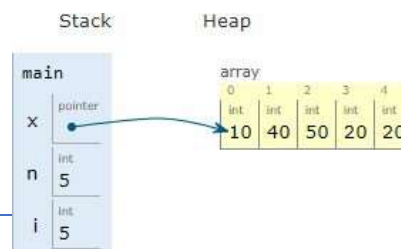
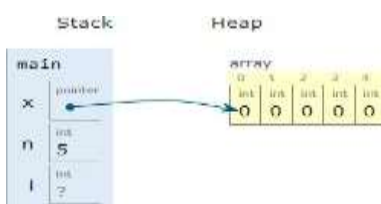
It allocates space for elements, **initialize them to zero** and then return a void pointer to the memory.

Prototype: **void *calloc(size_t nmemb, size_t size);**

Allocates memory for an array of nmemb elements of size bytes each and returns a pointer to the allocated memory. If calloc fails, it returns NULL. NULL may also be returned by a successful call to calloc() with nmemb or size equal to zero.

In the above Coding Example_2, if malloc is replaces with calloc as below,

```
x = (int*)calloc(n,sizeof(int));
```



3. realloc():

This function is used **to resize a previously allocated memory block** (via malloc() or calloc()). It adjusts the size of the memory block **without losing existing data** (up to the new size).

Prototype: **void *realloc(void *ptr, size_t size);**

It returns a pointer to the newly allocated memory block if the reallocation is successful. This pointer may be the same as the original or a new one, depending on whether the existing block could be resized in place. If the function fails to allocate the requested memory, it returns NULL, and the original memory block remains unchanged. To prevent memory leaks, it is recommended to assign the return value to a temporary pointer before updating the original one.

Note:

If ptr is NULL, then the call is equivalent to malloc(size), for all values of size.

If size is equal to zero, and ptr is not NULL, then the call is equivalent to free(ptr).

Unless ptr is NULL, it must have been returned by an earlier call to malloc(), calloc() or realloc(). If the area pointed to was moved, a free(ptr) is done. The realloc() function returns a pointer to the newly allocated memory, which is suitably aligned for any kind of variable and may be different from ptr, or NULL if the request fails. If size was equal to 0, either NULL or a pointer suitable to be passed to free() is returned. **If realloc() fails the original block is left untouched, and it is not freed or moved.**

Coding Exampe_3: Demo of realloc if we modify the number of memory blocks allocated using DMA function

```
#include<stdio.h>
#include<stdlib.h>
int new_size;
int main()
{
    int* p = (int*)malloc(3*sizeof(int));
    if(p == NULL)
        printf("memory not allocated\n");
    else
    {
```



```

    int i;
    printf("initial address is %p\n",p);
    printf("enter three elements\n");
    for(i = 0;i<3;i++)
    {
        scanf("%d",&p[i]);
    }
    printf("entered elements are\n");
    for(i = 0;i<3;i++)
    {
        printf("%d\t",p[i]);
    }
}

printf("\nenter the new size: ");
scanf("%d",&new_size);
p = (int*)realloc(p,new_size*sizeof(int));
if(p==NULL)
    printf("not allocated\n");
else
{
    int i;
    printf("new address is %p\n",p);
    for(i = 0;i<new_size;i++)
    {
        printf("%d\t",p[i]);
    }
}
return 0;
}

```

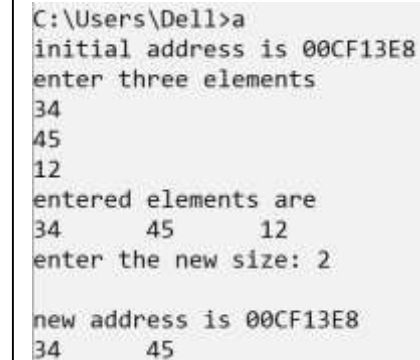


```

C:\Users\Dell>a
initial address is 006813E8
enter three elements
33
22
15
entered elements are
33    22    15
enter the new size: 10
new address is 006813E8
33    22    15    1547322173    158994953    3194    6819384 682879
2     977485170    1852397404

```

New size > initial size



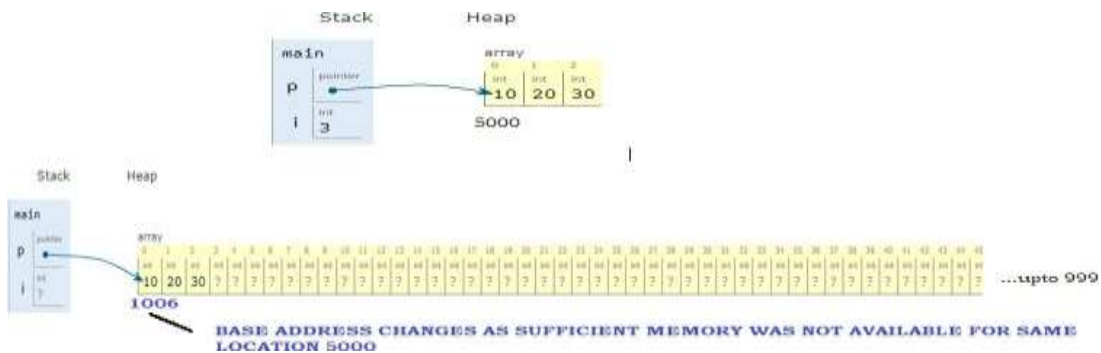
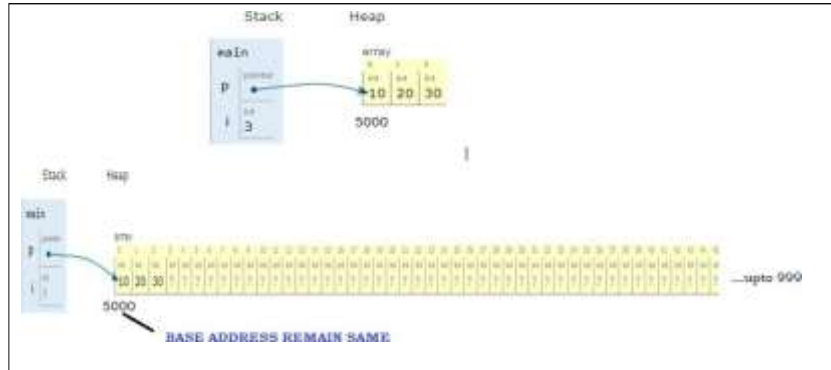
```

C:\Users\Dell>a
initial address is 00CF13E8
enter three elements
34
45
12
entered elements are
34    45    12
enter the new size: 2
new address is 00CF13E8
34    45

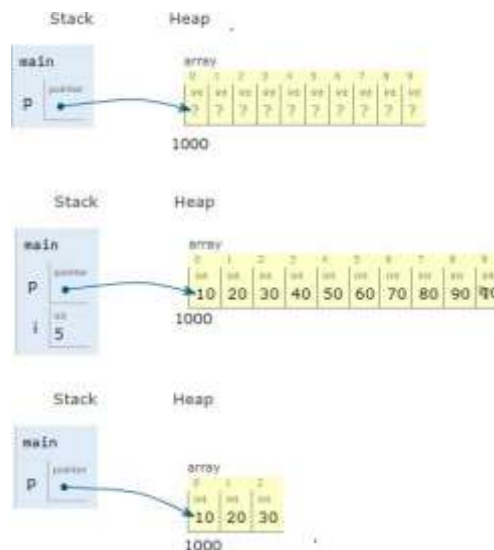
```

New size <= initial size

In Coding Example_3, when new size is greater than 3, here are the pictorial representations.



In Coding Example_3, when new size is less than 3, here is the pictorial representation.

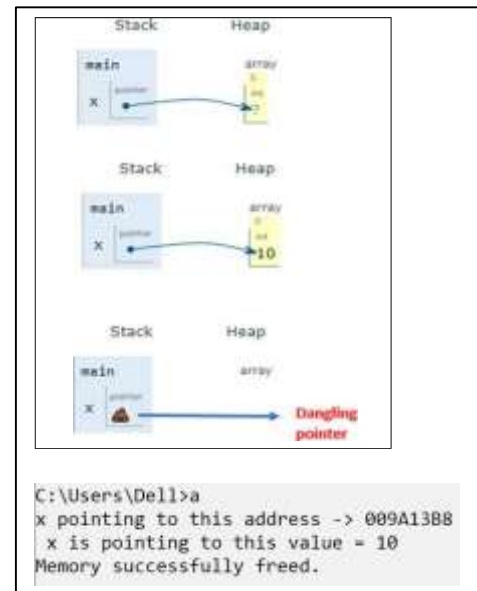


4. free()

It is used to **deallocate memory** that was previously allocated using malloc(), calloc(), or realloc(). Helps preventing **memory leaks** by returning the memory back to the system once it is no longer needed.

Coding Exampe_5: Usage of free

```
#include<stdio.h>
#include<stdlib.h>
int main(){
    int *x;
    x = (int*)malloc(sizeof(int));
    *x = 10;
    printf("x pointing to this address -> %p\n", x);
    printf("x is pointing to this value = %d", *x);
    free(x); // x is dangling pointer after this statement
    printf("\nMemory successfully freed.\n");
    return 0;
}
```



How does free function knows how much memory must be returned/released?

On allocation of Memory using memory management functions, it stores somewhere in memory the # of bytes allocated. This is known as **book keeping information**. We cannot access this info. But implementation has access to it. The function free() finds this size given the pointer returned by allocation functions and de-allocates the required amount of memory. Observing the diagrams, we can understand that once the allocated memory block is deallocated using the function, the pointer still holds the old address and becomes a **dangling pointer**.

Keep coding smartly with Dynamic Memory Allocation!



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE25CS151B**

***Lecture #9
Errors and Best practices associated with Dynamic
Memory Management Functions***

**By,
Prof. Sindhu R Pai,
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Poojari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: Text Processing and User-Defined Types****Topic: Errors and Best practices associated with Dynamic Memory Management Functions**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Introduction

Dynamic Memory Allocation in C provides flexibility and efficient memory use, but also introduces potential pitfalls that can lead to serious runtime issues if not handled carefully. From **memory leaks to undefined behavior**, these errors are often subtle but critical. **Understanding these common mistakes/errors is essential for writing safe and reliable C programs.**

Improper handling of functions like malloc(), calloc(), realloc(), and free() can lead to serious bugs.

Below are some typical errors described with code snippets.

1. **Dangling Pointer**
2. **NULL Pointer**
3. **Memory Leak**
4. **Double free error**
5. Uninitialized Pointer Use
6. Memory Overrun
7. Invalid free
8. Not checking return values of functions

Dangling Pointer

A pointer which points to a location that doesn't exist is known as dangling pointer. It can happen anywhere in the memory segment. Solution is to assign the pointer to NULL. Situations that results in dangling pointer are as below.

- Freeing the memory results in dangling pointer
- Using new pointer variable to store return address in realloc results in dangling pointer

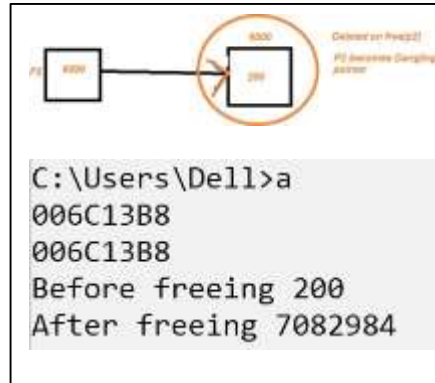
Coding Example_1: Freeing the memory results in dangling pointer

```
#include<stdio.h>
#include<stdlib.h>
int main(){
    int *p2 = (int*)malloc(sizeof(int));    // conversion from void* to int*
    printf("%p\n",p2);
    *p2 = 200;
    printf("%p\n", p2);
```

```

printf("Before freeing %d\n", *p2);    // 200
free(p2); // p2 becomes dangling after this
printf("After freeing %d\n", *p2);    // undefined behavior
return 0;
}

```

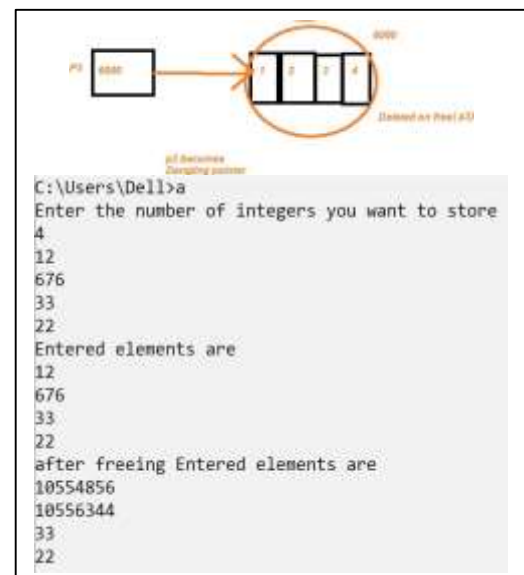


Coding Example_2: Freeing the memory results in dangling pointer

```

int main()
{
    printf("Enter the number of integers you want to store\n");
    int n; int i;
    scanf("%d", &n); // user entered 4
    int *p3 = (int*)malloc(n*sizeof(int)); // initially all values undefined values
    printf("Enter %d elements\n", n);
    for(i = 0; i < n; i++)
        scanf("%d", &p3[i]); // (p3+i) // user entered 1 2 3 4
    printf("Entered elements are\n");
    for(i = 0; i < n; i++)
        printf("%d\n", p3[i]); // *(p3+i)
    free(p3);
    printf("After freeing Entered elements are\n");
    for(i = 0; i < n; i++)
        printf("%d\n", p3[i]); // *(p3+i)
    return 0;
}

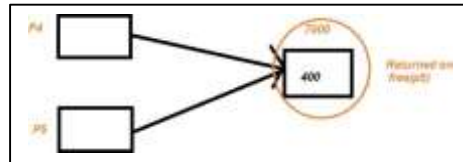
```



Coding Example_3:

```
int *p4 = (int*)malloc(sizeof(int));
*p4 = 400;
printf(" %d\n", *p4);
int *p5 = p4;
free(p5);

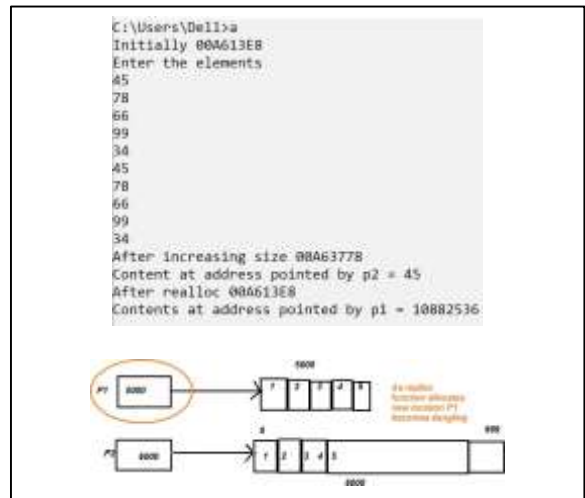
// value of the pointer has the book keeping info available
//Both p4 and p5 becomes dangling pointer
```



Coding Example_4: Using new pointer variable to store return address in realloc results in dangling pointer.

```
int main()
{
    int *p1 = (int *) calloc(5, sizeof(int));
    printf("Initially %p\n", p1);
    printf("Enter the elements\n");
    for(int i = 0; i < 5; i++)
        scanf("%d", &p1[i]);
    for(int i = 0; i < 5; i++)
        printf("%d\n", p1[i]);

    int *p2 = (int*) realloc(p1, 1000*sizeof(int)); // p1 becomes dangling if new locations allotted
    printf("After increasing size %p\n", p2); // same address as p1 if same location can be extended
    //else Different address
    printf("Content at address pointed by p2 = %d\n", *p2);
    printf("After realloc %p\n", p1);
    printf("Contents at address pointed by p1 = %d\n", *p1); return 0;
}
```

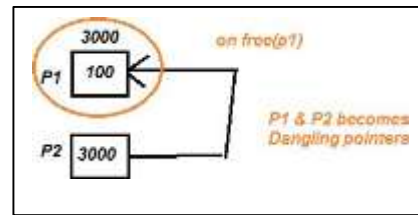


NULL Pointer

It is always a good practice to assign the pointer to NULL once after the usage of free on the pointer to avoid dangling pointers. This results in NULL Pointer. **Dereferencing the NULL pointer results in guaranteed crash.**

Coding Example_5:

```
int *p1 = (int*)malloc(sizeof(int));
*p1 = 100;    printf(" %d\n", *p1); // 100
int *p2 = p1;
printf(" %d\n", *p2); // 100
free(p1); // p1 and p2 both becomes dangling pointer.
p1 = NULL; p2 = NULL; // safe programming
printf(" %d\n", *p1); // Guaranteed crash
printf(" %d\n", *p2);
```



Garbage/Memory Leak

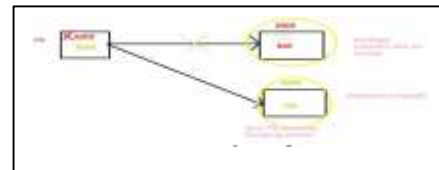
Garbage is a **location which has no name and hence no access**. If the same pointer is allocated memory more than once using the DMA functions, initially allocated spaces becomes garbage. **Garbage in turn results in memory leak. Memory leak can happen only in Heap region.**

Coding Example_6:

```
#include<stdio.h>
#include<stdlib.h>
int main()
{   int *p6 = (int*)malloc(sizeof(int));
    *p6 = 600;
    printf(" %d\n", *p6); // 600
    p6 = (int*)malloc(sizeof(int));

    // changing p6 loses the pointer to the location allocated by the previous malloc
    // So memory allocated by previous malloc has no access. Hence becomes garbage

    *p6 = 700;
    printf(" %d\n", *p6);
    free(p6);
    return 0;
}
```



Double free error: DO NOT TRY THIS

- If free() is used on a memory that is already freed before.
- Leads to undefined behavior.
- Might corrupt the state of the memory manager that can cause existing blocks of memory to get corrupted or future allocations to fail.
- Can cause the program to crash or alter the execution flow.

Coding Example_7:

```
int* p = (int*)malloc(sizeof(int));  
*p = 10;  
printf("p = %d", *p);  
free(p);  
free(p);
```

The above code might lead to undefined behavior. So, it is a good practice to make the pointer NULL after the usage of free. By chance, if the pointer is freed again, it doesn't do anything with NULL.

```
free(p);  
p = NULL; //Function free does nothing with a NULL pointer.  
free(p);
```

NOTE:

- Dereferencing the dangling pointer results in undefined behavior.
- Dereferencing the NULL Pointer results in guaranteed crash.

Coding Example_8: Uninitialized Pointer use - Using a pointer without assigning valid memory.

```
int *ptr;      *ptr = 5; // Undefined behavior
```

Coding Example_9: Memory overrun - Writing outside the bounds of allocated memory.

```
int *ptr = malloc(3 * sizeof(int));    ptr[3] = 10; // Index out of bounds!
```

Coding Example_10: Invalid free - Trying to free memory that was not dynamically allocated.

```
int a = 5;      int *ptr = &a; free(ptr); // Invalid: memory not allocated via malloc
```

Best Practices/Guidelines to be followed while using the Dynamic Memory Management functions.

- **Always check for NULL:** After calling malloc, calloc, or realloc, verify if the returned pointer is **not NULL** before using it.

```
int *arr = malloc(n * sizeof(int));  
if (arr == NULL) {  
    // Handle memory allocation failure  
}
```
- **Initialize memory when needed:** Use calloc if you need zero-initialized memory instead of malloc + manual initialization.
- **Avoid memory leaks:** Always free any dynamically allocated memory once it is no longer needed.
- **Avoid dangling pointers:** After freeing memory, **set the pointer to NULL** to avoid accidental access.
- **Be cautious with realloc:** Store the result of realloc in a **temporary pointer**, check for NULL, and then assign.
- **Use correct size calculations:** Always multiply the number of elements with sizeof(datatype) during allocation. To allocate 10 locations to store integers,

```
int *p = (int*) malloc(10) ; // Wrong  
int *p = (int*) malloc(10*sizeof(int)); // Perfect
```
- **Match allocation and deallocation:** Every malloc/calloc/realloc must be matched with one free. Use consistent naming to track ownership.

Happy managing memory smartly with DMM!!



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE25CS151B**

***Lecture #10
User Defined Type – Structures in C***

**By,
Prof. Sindhu R Pai,
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Poojari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: Text Processing and User-Defined Types****Topic: User Defined Type – Structures in C**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Introduction

A structure in C is a **user-defined data type** that allows **grouping variables of different types under a single name**. It helps in **creating complex and meaningful data types** by combining related data elements. Using structures, you can conveniently refer to a **collection of diverse but related items through one identifier**. This makes them **especially useful when multiple pieces of data need to be organized together**—such as storing records from a database or maintaining contact details in an address book.

Why Do We Need Structures in C?

Let's understand the need for structures through the following program. Suppose we want to store the **roll number, total marks, and names of 20 students**.

Coding Example_1: Here the approach is to use separate arrays for each data item

```
char names[20][15];
int marks[20];
int roll_num[20];
for(int i = 0; i < 20; i++) {
    printf("Enter Roll Number, Marks, and Name:\n");
    scanf("%d", &roll_num[i]);
    scanf("%d", &marks[i]);
    scanf(" %[^\n]s", names[i]); // Note the space before % to consume newline
}
```

Although we have stored the data, there is **no built-in connection between a student's roll number, marks, and name**. We're simply assuming that the i-th entry in all three arrays belongs to the same student. This **implicit relationship is error-prone and hard to manage**. To **explicitly group related information** and ensure that all details belong to the same student record, we use structures. They allow us to **bind related variables of different types together**, making our code more **logical, readable, and maintainable**. This **grouping of related attributes into one logical entity** is what we call a structure. When such a type is created, it **also determines the binary layout in memory, including the order of fields and the total size of any variable declared using that type**.

Key Characteristics of Structures in C

- Structures are **user-defined** (non-primitive/secondary) data types.
- They consist of **one or more named components**, commonly referred to as **data members**.
- The data members can be of **homogeneous** (same) or **heterogeneous** (different) types.
- The **memory size** of a structure is typically **at least** the total of its members' sizes, but it may vary depending on **compiler-specific alignment and padding rules**. The **offsets are determined at compile time**.
- **Structure members do not occupy memory** on their own; memory is allocated only when a **structure variable** is declared.
- **Compatible structures** (i.e., with the same layout and member types) can be **assigned directly** to each other.

NOTE: To define a new type/entity, use the **keyword – struct**

The format for declaring a structure is as below:

```
struct Tag
{
    Data_type member1;
    data_type member2;
    .....
    data_type membern;
};    // Don't forget semicolon here
```

where Tag is the name of the entire structure and Members are the elements within the struct.

Example: User defined type Student entity is created.

```
struct Student
{
    int roll_no;
    char name[20];
    int marks;
}; //No memory allocation for declaration/description of the structure.
```

Accessing members of a structure

All the data members inside a structure are accessible to the functions defined outside the structure. To access the data members in a function including main, we need to create a structure variable. We can **access the members of a structure only when we create instance variables of that type**. If struct Student is the type, we can create the instance variable as:

struct Student s1; // s1 is the instance variable of type struct Student. **Now memory gets allocated for data members**

struct Student* s2; // s2 is the instance variable of type struct Student*. Now memory gets allocated for pointer. **s2 is pointer to structure**

Two operators used.

1. Dot operator (.)

Any member of a structure can be accessed using the structure variable as:

structure_variable_name.member_name

Suppose, s1 is the structure variable name and we want to access roll_no member of s1. Then, it can be accessed as: **s1.roll_no**

2. Arrow operator (->)

Any member of a structure can be accessed using the pointer to a structure as:

pointer_variable->member_name

Suppose, s2 is the pointer to structure variable and we want to access roll_no member of s2. Then, it can be accessed as: **s2->roll_no**

Coding Example_2: Basic Declaration, Assignment and Display. Use this structure for all programming examples listed below.

```
struct student // Template declaration: defines what a 'student' looks like
{
    int roll_no;      // Member 1: roll number
    char name[20];    // Member 2: name (character array)
    int marks;      // Member 3: marks
};
```

Note: Declaration does not allocate memory. Only defines the blueprint of the type.


```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    struct student s1; // Declare a structure variable of type 'student'
```

```
    printf("Before assigning, Student Details:\n");
```

```
    printf("Roll Number: %d\n", s1.roll_no);
```

```
    printf("Name: %s\n", s1.name);
```

```
    printf("Marks: %d\n", s1.marks);
```

```
    // Assigning values to members
```

```
    s1.roll_no = 101;
```

```
    s1.marks = 87;
```

```
    strcpy(s1.name, "Rahul"); // Use string handling function to assign name
```

```
    // Displaying values
```

```
    printf("After assigning Student Details:\n");
```

```
    printf("Roll Number: %d\n", s1.roll_no);
```

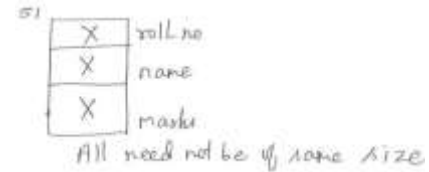
```
    printf("Name: %s\n", s1.name);
```

```
    printf("Marks: %d\n", s1.marks);
```

```
    return 0;
```

```
}
```

Before assigning Student details



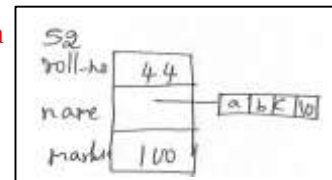
```
C:\Users\Dell>a
Before assigning, Student Details:
Roll Number: 4200928
Name: P a
Marks: 4194432
After assigning Student Details:
Roll Number: 101
Name: Rahul
Marks: 87
```

Coding Example_3: Initialization of the structure while defining it

```
    struct student s2 = {44, "abc", 100}; // Initialization
```

```
    printf("%d %d %s", s2.marks, s2.roll_no, s2.name);
```

```
    // 100 44 abc
```

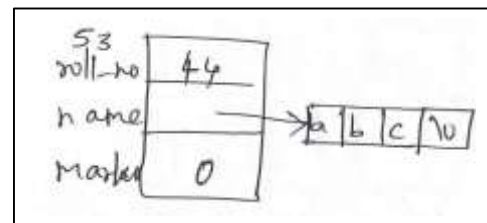


Partial Initialization of structures

If you initialize the given structure partially, **only the specified members get values**, and the rest are **automatically initialized to zero** (or null for pointers/arrays).

Coding Example_4: Partial Initialization of Structures

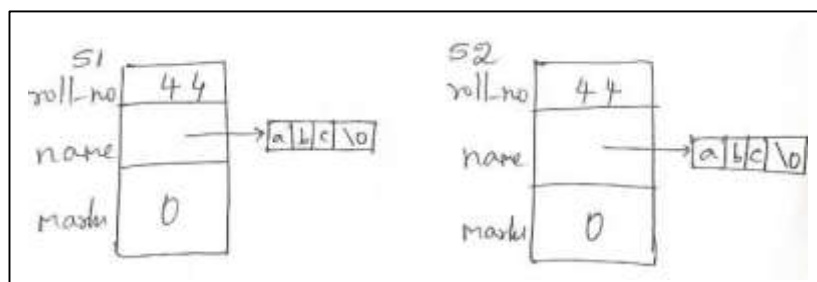
```
struct student s3 = {44, "abc"};      // Partial Initialization
// Explicitly few members are initialized. others are initialized to default value
printf("%d %d %s", s3.marks, s3.roll_no, s3.name); // 0 44 abc
```



Designated initializers allow you to **specify values for specific members by name, in any order**. Very useful in large structures with many fields. It requires **C99 or later C standard**. **Not supported** in old C standards like C89 unless enabled with compiler extensions (e.g., `-std=c99` in GCC). Specify the name of a field to initialize with `‘.member_name =’` or `‘member_name:’` before the element value. Others are initialized to default value.

Coding Example_5: Designated initializers in C

```
struct student s1 = {.name = "abc", .roll_no = 44};
struct student s2 = {name : "abc", roll_no : 44};
printf("%d %d %s\n", s1.roll_no, s1.marks, s1.name);      // 44 0 abc
printf("%d %d %s", s2.roll_no, s2.marks, s2.name); // 44 0 abc
```



Memory Allocation for Structure Variable

Every data type in C has alignment requirement (infact it is mandated by processor architecture, not by language). A processor will have processing word length as that of data bus size. On a 32 bit machine, the processing word size will be 4 bytes.

More details, Refer to below links.

<https://www.geeksforgeeks.org/structure-member-alignment-padding-and-data-packing/>

<https://www.geeksforgeeks.org/data-structure-alignment/>

NOTE:

- What is the minimum number of bytes allocated for the structure variable in each of these cases? – Sum of the sizes of all the data members. Size of data members is implementation specific.
- What is the maximum number of bytes allocated for the structure variable in each of these cases? **Implementation Specific.**

Coding Example_6: Below programs are run on Windows7 machine and 32 bit GCC compiler

```
struct test{    int i;    char j;    };
struct test1{   char j;    int i;    };
struct test2{    char k;    char j;    int i;    };
struct test3{   int i;    char k;    int j;    };
int main()
{
    printf("size of the structure is %lu\n",sizeof(struct test));    //8bytes    4+4
    struct test t;
    printf("size of the structure is %lu\n",sizeof(t));        // 8bytes    4+4
    printf("size of the structure is %lu\n",sizeof(struct test1)); //8bytes    4+4
    printf("size of the    structure is %lu\n",sizeof(struct test2)); //8bytes    4+4
    printf("size of the structure is %lu\n",sizeof(struct test3)); //12bytes    4+4+4
    return 0;
}
```

#pragma in Struct Memory Management

#pragma is a **preprocessor directive** used to provide special instructions to the compiler. Its behavior is **compiler-specific**. This directive can be used with structures to **control memory alignment and padding**. By default, compilers may insert padding between structure members to align data in memory for efficient access. However, in some cases—like memory-mapped hardware, binary file structures, or network packets—you need precise control over the layout of the structure. **The #pragma pack() directive allows you to specify byte alignment to ensure that the compiler does not add unnecessary padding, or aligns members in a specified way.** The **#pragma pack(n)** directive in C is used to control **alignment** by specifying a **byte boundary (n)**. This tells the compiler to align structure members on addresses that are multiples of n. The value **n must be a power of 2: 1, 2, 4, 8, 16, 32...** (up to the compiler's supported maximum).

Coding Example_7:

Version_1: Without #pragma, normally, the compiler adds padding between char a and int b to align b on a 4-byte boundary.

```
#include <stdio.h>

struct NormalStruct {
    char a;    // 1 byte          // 3 bytes padding
    int b;     // 4 bytes (usually aligned at 4-byte boundary)
};

int main() {
    struct NormalStruct ns;
    printf("Size of NormalStruct: %zu bytes\n", sizeof(ns)); // 8 bytes
    return 0;
}
```

Version_2: With #pragma pack(1) -> Disables padding, so the structure becomes tightly packed.

```
#include <stdio.h>

#pragma pack(1)

struct PackedStruct {
    char a;    // 1 byte
```

```
int b;    // 4 bytes (no padding!)
};
#include<stdio.h>
#pragma pack()
int main() {
    struct PackedStruct ps;
    printf("Size of PackedStruct: %lu bytes\n", sizeof(ps)); // 5 bytes
    return 0;
}
```

Comparison of Structure variables

In C, you **cannot directly compare two structure variables using == or !=** like you do with primitive types. **Always prefer field-by-field comparison for accuracy and safety.** `memcmp()` should be used with caution due to padding or non-deterministic behavior.

Coding Example_8: Field-by-field comparison of data members of structure variables

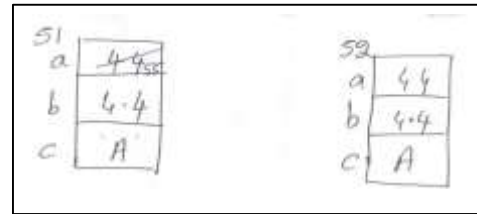
```
int main() {
    struct Student s1 = {101, "sindhu", 85};
    struct Student s2 = {101, "sindhu", 85};
    /*if (s1 == s2) { // Error: invalid operands to binary ==
        // ...
    } */
    if (s1.roll_no == s2.roll_no && strcmp(s1.name, s2.name) == 0 && s1.marks == s2.marks)
    {
        printf("The structures are equal.\n");
    }
    else
    {
        printf("The structures are different.\n");
    }
    return 0;
}
```

Member - wise copy

Structures of **same type are assignment compatible**. When you assign one structure variable to another structure variable of same type, member- wise copy happens. **Both of them do not point to the same memory location**. The **compiler generates the code for member – wise copy**.

Coding Example_9:

```
struct testing {      int a; float b; char c;  };  
int main(){  
    struct testing s1 = {44,4.4, A'};  
    struct testing s2 = s1;  
    s1.a = 55;  
    printf("%d\n",s1.a);  
    printf("%d\n",s2.a);  
    return 0;  
}
```



Few points to think:

- Structures can be assigned even if the structure contains an array. Is it True?
- If the structure contains pointer, how member-wise copy happens?

Usage of typedef

The typedef is a keyword which is used to **give a type, a new name**. By convention, uppercase letters are used for these definitions to remind the programmer that the type name is really a symbolic abbreviation, but we can use lowercase as well. Also, used to **assign alternative names to existing data types**. It is used to provide an interface for the client. Once a new name is created using typedef, the client may use this without knowing the underlying type. It is limited to giving symbolic names to types only. The typedef interpretation is performed by compiler.

I want to store the age of a person. Best way to define a variable is `int age = 80;`

Alternatively, you can also say, **typedef int age;** //another name for int is age.

Then **age a = 80;**

Can we say `age age = 80;` // Think about this.

If we know how to declare a variable, then prefixing the declaration with typedef makes that a new name.

```
int b[10];      // b is an array variable to store 10 integers
typedef int c[10];  // c is a name for an array of 10 integers
```

NOTE: typedef is mostly used with user defined data types when names of the data types become slightly longer and complicated to use in programs.

Coding Example_10:

Version 1:

```
struct player
{
    int id; char name[20]; };
typedef struct player player_t;    // player_t is a new name to struct player.
int main()
{
    player_t p1;    // p1 is a variable
}
```

Version 2: You can include typedef when defining a new type itself.

```
typedef struct player
{
    int id; char name[20]; } player_t;    // player_t is a new name to struct player.
int main()
{
    player_t p1;    // p1 is a variable
}
```

Nested Structures in C

A nested structure is a **structure that contains another structure as a member**. This allows grouping related data hierarchically, improving organization and code clarity for complex data. **Nested structures** can be used in a few different **styles**, depending on how and where the inner structure is defined. Broadly, there are **3 common types** of nested structures.

A. Named Structure inside another (Standard Nested Structure)

One named structure is used as a member of another. Advantage is to reuse the structure in multiple outer structures.

Coding Example_11:

```
struct Date {  int day, month, year;  };
struct Student {
    int roll_no;
    struct Date dob; // nested named structure
};
int main()
{
    struct Student s1;
    s1.roll_no = 91;
    s1.dob.day = 31;
    s1.dob.month = 1;
    s1.dob. year = 2025;                // Output: 91 --- 31/1/2025
    printf("%d --- %d/%d/%d", s1.roll_no, s1.dob.day, s1.dob.month, s1.dob.year);
    return 0;
}
```

B. Anonymous (Unnamed) Structure inside Another

The **inner structure is defined directly inside the outer structure without a name**. Good for one-time use. But not reusable outside the structure.

Coding Example_12:

```
struct Student {
    int roll_no;
    struct {
        int day, month, year;
    } dob; // anonymous structure
};
int main()
{
    struct Student s1;
```

```
s1.roll_no = 91;
s1.dob.day = 31;
s1.dob.month = 1;
s1.dob.year = 2025;           // Output: 91 --- 31/1/2025
printf("%d --- %d/%d/%d", s1.roll_no, s1.dob.day, s1.dob.month, s1.dob.year);
return 0;
}
```

C. Structure within Structure as Pointer (Nested via Pointer)

Instead of embedding the structure directly, a pointer to a structure is used. **This allows dynamic memory allocation for the nested part.** Very useful in memory-optimized or linked data structures.

Coding Example_13:

```
struct Date {
    int day, month, year; };

struct Student {
    int roll_no; struct Date *dob; // pointer to nested structure
};

int main()
{
    struct Student s1;
    s1.roll_no = 91;
    s1.dob = malloc(sizeof(struct Date));
    (s1.dob)->day = 31;
    (s1.dob)->month = 1;
    (s1.dob)->year = 2025;           //Output: 91 --- 31/1/2025
    printf("%d --- %d/%d/%d", s1.roll_no, (s1.dob)->day, (s1.dob)->month, (s1.dob)->year);
    return 0;
}
```

Happy Coding with Structures!!!



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE25CS151B**

***Lecture #11
Structures and Functions***

**By,
Prof. Sindhu R Pai,
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: Text Processing and User-Defined Types****Topic: Structures and Functions**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Introduction

Structures are used to group different types of data into a single logical unit, while **functions** allow breaking the program into smaller and reusable blocks of code. When we combine these two, we get a powerful way to **pass, manipulate, and return complex data** efficiently. Also, we can build modular, organized, and reusable code using structures and Functions together.

Passing a structure to a function

Functions typically receive data through arguments. When dealing with multiple related data items, structures offer a clean way to group them. To operate on such grouped data, we can **pass entire structure or it's address to functions**. Depending on the use case, structures can be passed to a function which **copies the entire structure or it's address only**. Also, function can return a structure to the calling function.

Consider the Player structure.

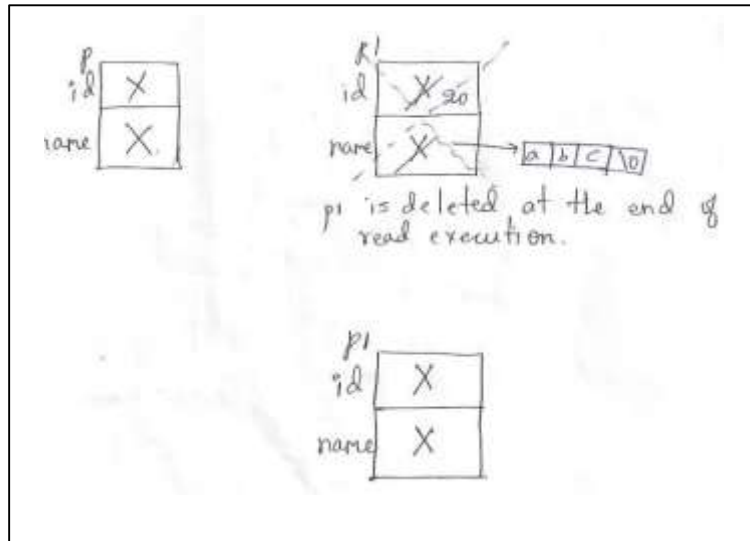
```
struct Player {  
    int id;  
    char name[20];  
};
```

Coding Example_1: Function to read and display the player details. Does this code work accordingly? Think!!

```
int main(){  
    struct Player p;  
    printf("Enter id and name\n");  
    read(p);    disp(p);  
    return 0;  
}  
  
void read(struct Player p1){  
    scanf("%d ", &p1.id);    // user enters 20 and abc  
    scanf("%s", p1.name);  
}
```

```
void disp(struct Player p1){  
    printf("%d\n",p1.id);           // undefined values get printed  
    printf("%s",p1.name);  
}
```

Parameter passing is always by value in C. The argument p is passed to the parameter p1 in read function. This is pass by value and as the structure is involved, it is member-wise copy. Whatever the user enters, it is stored in p1, not p. p1 is a parameter and it is lost once after the completion of read() execution. The same p is passed to display function. Again pass by value and member-wise copy. So the output is undefined values only. **Pictorial representation is as below.**



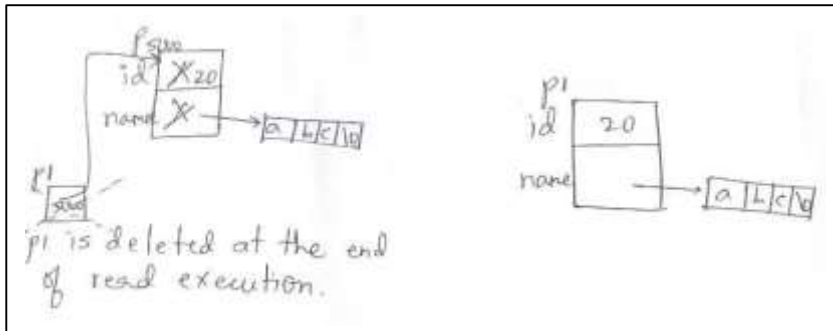
If you want to modify the structure inside a function, pass a pointer to a structure(l-value) as an argument.

Coding Example_2:

```
int main( ){  
    struct Player p;  
    printf("Enter id and name\n");  
    read(&p);    disp(p); // Think, can we also pass &p to disp(). Is there any harm?  
    return 0;  
}
```

```
void read(struct Player *p1) {  
    scanf("%d ", &(p1->id));    // or &((*p1).id)    // user enters 20  
    scanf("%s", p1->name);    // abc entered  
}  
  
void disp(struct Player p1) {  
    printf("%d ", p1.id);    // actual values will be printed.  
    printf("%s", p1.name);  
}
```

Parameter passing is always by value in C. The argument `&p` is passed to the parameter `p1` (pointer to structure) in `read` function. The parameter `p1` is pointing the `p` structure. Whatever the user enters, it is stored in `p`, through `p1`. `p1` is a parameter and it is lost once after the completion of `read( )` execution. The **updated `p` is passed to display function. Again pass by value and member-wise copy of `p` to `p1` in the `display( )`.** So the output is updated values in `p`. **Pictorial representation is as below.**



Assume the struct `Player` contains many data members. Can we just use structure variable in the parameter of `disp()`? If we do so, this is considered as a bad practice. As every member of argument is copied to every member of parameter (member-wise copy), it requires more space and more time. Also, when we are very sure that we do not want to make any changes to the argument, **parameter of `disp()` can be a pointer to constant structure.**

Coding Example_3:

```
int main( ){  
    struct Player p;    printf("Enter id and name\n");  
    read(&p);    disp(&p);    return 0;  
}
```

// read() remains the same as previous

void disp(const struct Player* p1)

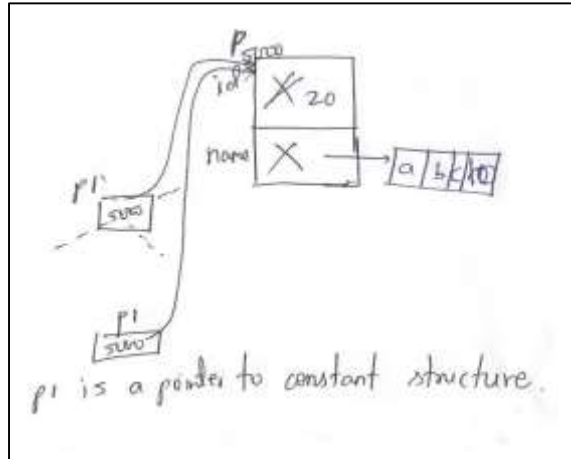
{

// p1.id = 34; // throws error as p1 is a pointer to constant structure

printf("%d ",p1->id); // actual values will be printed.

printf("%s",p1->name);

}



Coding Example_4: Function returning the structure to the calling function.

struct Player modify(struct Player);

int main()

{

struct Player p1={20,"sachin"};

printf("before change %s",p1.name);

p1=modify(p1);

printf("after change %s",p1.name);

return 0;

}

struct Player modify(struct Player p)

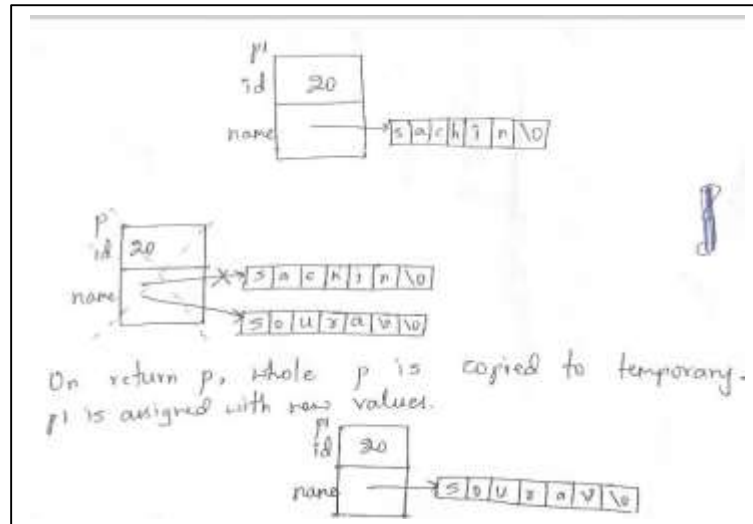
{

strcpy(p.name, "Sourav");

return p;

}

When the function `modify()` returns a changed parameter, it is copied to a temporary. Then in the client code, the temporary is assigned to `p1`.



Think about it!!

- Can we make `const struct Player p` as the parameter for read?
- Can we say `struct player *p` as the parameter without changing the client code?
- If we change the client code, can we use `struct player *p` as the parameter?
- Can we return pointer to structure?
- Can we use the `const` qualifier with the return type of the `read()` ?

Keep exploring Structures and Functions together!!



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE25CS151B**

***Lecture #13
Array of Structures***

**By,
Prof. Sindhu R Pai,
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: Text Processing and User-Defined Types****Topic: Array of Structures**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Introduction

We know that the structures are used to group different types of variables under one name. But what if we want to store multiple records of such structured data? That's where **Array of structures** comes into play. **An array of structures allows us to maintain a collection of records—each record being a structure.** This is particularly useful in programs dealing with multiple entities of the same type, such as **collection of students, collection of employees, collection of products, collection of books, collection of items, collection songs, collection of movies, collection of images, collection of subjects, collection of files, collection of devices etc.**

An **array of structures** is a collection of **structure variables stored in contiguous memory locations**, allowing you to manage **multiple records of the same type** efficiently.

Consider the Student structure.

```
struct Student {  
    int id;  
    char name[20];  
    int marks;  
};
```

The new data type called **struct student** is shown as above.

- We create a structure variable s1 as:
struct Student s1;
- If the user wishes to store details of two students then two variables should be created as:
struct Student s1, s2;
- To **store the details of 100 students**, we would be required to use 100 different structure variables from s1 to s100, which is definitely not very convenient. A better approach would be to **use an Array of structures.**

Defining an Array of structure variable – Two Approaches

1. **At the time of structure Declaration:** Immediately after defining the structure by appending them to the closing brace.

```
struct student {
    int roll_no;   char name[100];   int marks;
} s[100]; // Structure variable 's' declared as an array of 100 elements
```

2. **After the Structure Declaration is complete:**

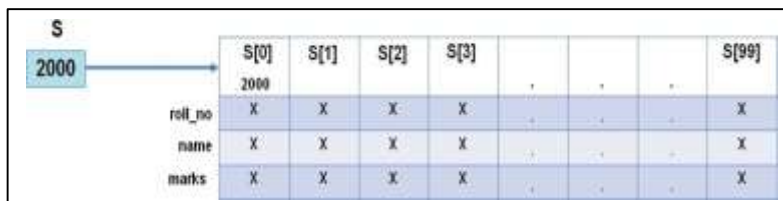
- Inside the main() function:

```
struct student s[100];
```

- Globally using the struct keyword: (Less preferred approach)

```
struct student {
    int roll_no;   char name[100];   int marks;
};
struct student s[100];
```

NOTE: In an array of structures, all elements are stored in contiguous memory locations, just like arrays of primitive data types.



Initialization of Array of Structures

Compile-time Initialization: Here, user has to enter the details within the program.

Case 1: `struct student S1[] = { {1, "John", 60}, {2,"Jack", 40}, {3, "Jill", 77} };`

// size is decided by the compiler **based on how many students details are stored**

Case 2: `struct student S2[3] = { {1, "John", 60}, {2,"Jack", 40}, {3, "Jill", 77} };`

// size is specified and initialized values are exactly matching with the size specified.

Case 3: `struct student S3[3] = {1, "John", 60, 2,"Jack", 40, 3, "Jill", 77};`

//initialization can also be done this way

Case 4: `struct student S4[5] = { {1, "John", 60} };`

//Partial initialization. Default values are stored in remaining memory locations

Runtime Initialization: Runtime is the period of time when a program is running and user is allowed to enter the input values to the variables of the structures.

Point to note:

Flexible Array Member(FAM) is a feature introduced in C99, which allows a structure to contain an array without a fixed size as its last member. This enables the structure to accommodate a variable amount of data. We can declare an array without a dimension and whose size is flexible in nature. Such an array inside the structure should **preferably be declared as the last member of structure** and its size is variable meaning can be changed be at runtime. Hence, must use **Dynamic Memory Management functions** to create enough space. The structure must contain at least one more named member in addition to the FAM. Example structure:

```
struct student {  
    int roll_no;  
    char name[100]  
    int marks[]; // Flexible Array Member  
};
```

Coding Example_1: Without initializing any value to member of structure i.e Number

```
#include <stdio.h>  
  
struct book { int Number; };  
  
int main(){  
    int n; int i;  
    struct book r[10];  
    printf("Enter how many books you want to store\n");  
    scanf("%d",&n);  
    for(i=0;i<n;i++){  
        printf ("book number is: %d", r[i].Number);  
    }  
    return 0; }
```

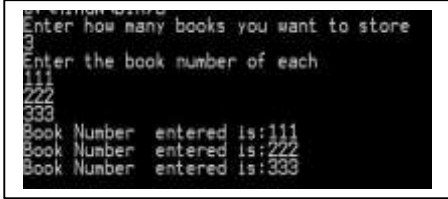


Coding Example_2: Initializing the structure member

```
#include <stdio.h>

struct book{   int Number;   };

int main(){
    int n; int i;
    struct book r[10];
    printf("Enter how many books you want to store\n");
    scanf("%d",&n); // assumption n <=10
    printf("Enter the book number of each\n");
    for( i=0;i<n;i++){
        fflush(stdin);
        scanf("%d",&r[i].Number);
    }
    for(i=0;i<n;i++)
        printf("Book Number entered is:%d\n",r[i].Number);
    return 0;
}
```



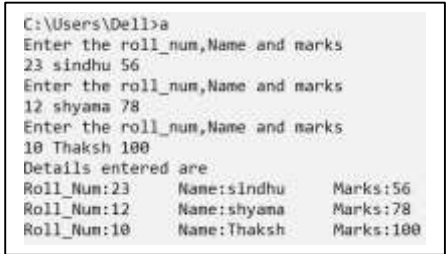
```
Enter how many books you want to store
3
Enter the book number of each
111
222
333
Book Number entered is:111
Book Number entered is:222
Book Number entered is:333
```

Coding Example_3: Read and display the details of n=3 students

```
#include<stdio.h>

struct student {   int roll_no; char name[30]; int marks;   };

int main(){
    struct student s[100];
    int i;
    for( i=0;i<3;i++) {
        printf("Enter the roll_num, Name and marks\n");
        scanf("%d%s%d",&s[i].roll_no,s[i].name,&s[i].marks); //for Name variable no &
    }
    printf("Details entered are\n");
    for(i=0;i<3;i++) {
        printf("Roll_Num:%d\t",s[i].roll_no); pr intf("Name:%s\t",s[i].name);
        printf("Marks:%d\n",s[i].marks); }
    return 0; }
```



```
C:\Users\Dell>a
Enter the roll_num,Name and marks
23 sindhu 56
Enter the roll_num,Name and marks
12 shyama 78
Enter the roll_num,Name and marks
10 Thaksh 100
Details entered are
Roll_Num:23      Name:sindhu      Marks:56
Roll_Num:12      Name:shyama     Marks:78
Roll_Num:10      Name:Thaksh     Marks:100
```

Pointer to an Array of Structures

Pointer is used to access the array of structure variables efficiently. Suppose we have to store record of 5 students, then using array of structure it can be easily done as:

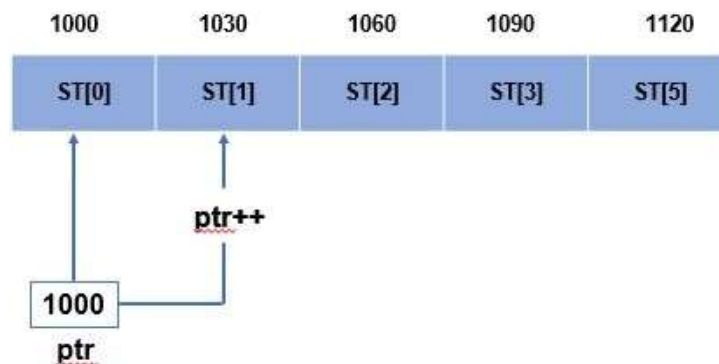
```
struct student
{
    int roll_no;
    char name[30];
    int marks;
}ST[5];
```

The sizeof operator when applied on structure student will take at least 30 bytes in memory. (Please note this is implementation specific). Suppose, base address of the array ST is 1000, then pictorial representation of the same in memory will be:



Now, if the pointer ptr is initialized to ST as: **struct student *ptr = ST;**

It means that the pointer ptr is of type struct student and is holding the address pointed by the array of structure ST (which is 1000).



Suppose if we perform ptr++, the ptr gets updated and now it will start pointing to next array record starting from 1030 (ptr++ → ptr+1 → 1000 + 30 = 1030), which is the record of next student.

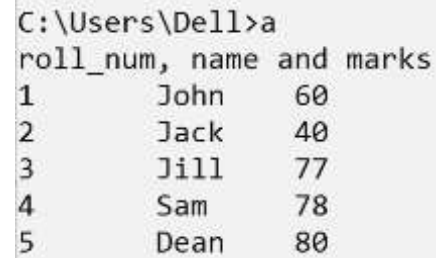
Coding Example_4:

```
struct Student ST[] = {{1, "John", 60}, {2, "Jack", 40}, {3, "Jill", 77}, {4, "Sam", 78 }, {5, "Dean", 80}};

struct Student *ptr = ST;    //&ST is illogical
printf("\n%d\n", *ptr);      //prints 1
printf("%d \t %s \t %d\n", ptr->Roll_no, (ptr+1)->Name, (ptr+2)->Marks); //prints 1 Jack 77
ptr++;                      // ptr now points to ST[1]
printf("\n%d\n", *ptr);      //prints 2
```

Coding Example_5: How we can use the pointer ptr to print all the details of all students.

```
struct student *ptr = ST;
int i;
printf("roll_num, name and marks\n");
for(i = 0; i < (sizeof(ST)/sizeof(ST[0])) ; i++)
{
    printf("%d\t", (ptr+i)->roll_no); // ptr[i].roll_no
    printf("%s\t", (ptr+i)->name);
    printf("%d\n", (ptr+i)->marks);
}
```



roll_num	name	marks
1	John	60
2	Jack	40
3	Jill	77
4	Sam	78
5	Dean	80

Passing an Array of structure to a function

Structures allow grouping of related data under a single name. When multiple records of such structured data (like multiple students, employees, products, etc.) need to be managed, an array of structures is used. To process this data efficiently, we often need to pass the entire array of structures to a function. This approach promotes modularity and avoids repetitive code. **Passing an array of structures to a function allows the function to access and modify the original data directly.**

Consider,

// Structure definition

```
struct Student { int roll_no; char name[50]; float marks; };
```


Coding Example_6: Read and display the details of n=3 students using two functions – read() and display()

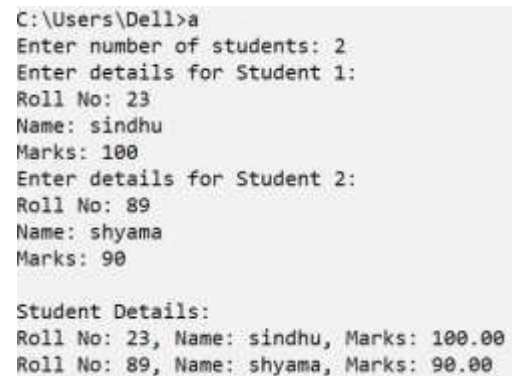
```
// Function to read input for all students
```

```
void read(struct Student *s, int n) {  
    for(int i = 0; i < n; i++) {  
        printf("Enter details for Student %d:\n", i + 1);  
        printf("Roll No: ");  
        scanf("%d", &s[i].roll_no);  
        printf("Name: ");  
        scanf(" %[^\\n]", s[i].name); // to read full name with spaces  
        printf("Marks: ");  
        scanf("%f", &s[i].marks);  
    }  
}
```

```
// Function to display all students
```

```
void display(struct Student *s, int n) {  
    printf("\nStudent Details:\n");  
    for(int i = 0; i < n; i++) {  
        printf("Roll No: %d, Name: %s, Marks: %.2f\n", s[i].roll_no, s[i].name, s[i].marks);  
    }  
}
```

```
int main() {  
    struct Student s[1000];  
    int n;  
    printf("Enter number of students: ");  
    scanf("%d", &n);  
    read(s, n); // Reading data - This array degenerates to a pointer at runtime  
    display(s, n); // Displaying data - Here too, an array degenerates to a pointer at runtime  
    return 0;  
}
```



```
C:\Users\Dell>a  
Enter number of students: 2  
Enter details for Student 1:  
Roll No: 23  
Name: sindhu  
Marks: 100  
Enter details for Student 2:  
Roll No: 89  
Name: shyama  
Marks: 90  
  
Student Details:  
Roll No: 23, Name: sindhu, Marks: 100.00  
Roll No: 89, Name: shyama, Marks: 90.00
```

Coding Example_7: The Book entity contains these data members: **id, title, author, price, year of publication**. Write separate functions to **read details of n books and display it**. Also include functions to do the following.

- **Fetch all the details of books published in the year entered by the user.**
- **Fetch all the details of books whose author name is entered by the user. Display appropriate message if no data found.**

```
typedef struct Book {
    int id;          char title[100];      char author[100];    int price;    int year;
}book_t;

#include<stdio.h>
#include<string.h>
// Function prototypes
void read_details(book_t *b,int n);
void display_details(book_t *b, int n);
int fetch_books_year(book_t *b,int n, int year, book_t*);
int fetch_books_author(book_t *b,int n, char *author, book_t*);

// Client Code
int main()
{
    book_t b[100];
    book_t b_year[100];
    book_t b_author[100];
    printf("How many books details you want to enter?");
    int n;          scanf("%d",&n);
    printf("enter the details of %d books\n",n); read_details(b,n);
    int count;
    printf("enter the year of publication to find list of books in that year");
    int year;       scanf("%d",&year);
    count = fetch_books_year(b,n, year,b_year);
```

```
        if(count){
            printf("List of books with %d as year of publication\n",year);
            display_details(b_year, count);
        }
        else
            printf("books published in %d is not available in the dataset\n",year);
        printf("\n");
        printf("enter the author name to find the list of books by that author\n");
        char author[100];    scanf("%s",author);
        count = fetch_books_author(b,n, author,b_author); if(count)
        if (count){
            printf("List of books by %s\n",author);
            display_details(b_author, count);
        }
        else
            printf("books by %s is not available in the dataset\n",author);
        return 0;
    }

// Function to read book details
void read_details(book_t *b,int n){
    int i;
    for(i = 0; i< n;i++)
    {
        printf("enter id, title, author, price and year of publication for book %d\n",i+1);
        scanf("%d%s%s%d%d",&b[i].id, b[i].title, b[i].author, &b[i].price, &b[i].year);
        // &((b+i)->id) is also valid in scanf. Using the pointer notation
    }
}
```

// Function to display book details

```
void display_details(book_t *b, int n){
    int i;
    for(i = 0; i < n; i++)
    {
        printf("\n-->%d\t%s\t%s\t%d\t%d\n", b[i].id, b[i].title, b[i].author, b[i].price,
b[i].year); // (b+i)->id is a valid pointer notation to print id
    }
}
```

// Function to fetch count of books with the given year

```
int fetch_books_year(book_t *b, int n, int year, book_t *b_year)
{
    int i; int count = 0;
    for(i = 0; i < n; i++)
    {
        if (b[i].year == year)        { count++; b_year[count-1] = b[i]; }
    }
    return count;
}
```

// Function to fetch count of books by the given Author

```
int fetch_books_author(book_t *b, int n, char *author, book_t *b_author) {
    int i; int count = 0;
    for(i = 0; i < n; i++)
    {
        if (!(strcmp(b[i].author, author)))
        {
            count++; b_author[count-1] = b[i];
        }
    }
    return count;
}
```



```
C:\Users\Delila
How many books details you want to enter??
enter the details of 3 books
enter id, title, author, price and year of publication for book 1
123 abc AUTH001 3000 2024
enter id, title, author, price and year of publication for book 2
456 pqr AUTH002 4000 2021
enter id, title, author, price and year of publication for book 3
897 xyz AUTH003 5000 2025
enter the year of publication to find list of books in that year2025
List of books with 2025 as year of publication
-->897 xyz AUTH003 5000 2025

enter the author name to find the list of books by that author
AUTH001
List of books by AUTH001
-->123 abc AUTH001 3000 2024
```

Keep Coding using Array of Structures!!!



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE25CS151B**

***Lecture #15
Sorting and Searching on Array of Structures***

**By,
Prof. Sindhu R Pai,
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: Text Processing and User-Defined Types****Topic: Sorting and Searching on Array of Structures**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and its respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Introduction

To efficiently manage and retrieve data from the array structures, **Sorting** and **Searching** operations become essential. **Sorting arranges the structures based on a specific field while searching allows us to find a particular entry based on a given criterion.** This note explores how to perform sorting and searching on arrays of structures using Selection Sort and Binary Search respectively.

Let us explore Sorting on the Array of structures.

Given the student template, use selection sort to sort the given set of student details.

```
struct student {  
    char name[50];  
    int roll_no;  
    float marks;  
};
```

Coding Example_1: Reading the details of students and sorting in an ascending order based on the name field. Display all the details of students using a function.

```
#include <stdio.h>  
#include <string.h>  
  
// Function to read student details  
void readStudents(struct student s[], int n) {  
    for (int i = 0; i < n; i++) {  
        printf("Enter details for student %d:\n", i + 1);  
        printf("Name: ");  
        scanf(" %[\n]", s[i].name); // reads string with spaces  
        printf("Roll No: ");  
        scanf("%d", &s[i].roll_no);  
        printf("Marks: ");  
        scanf("%f", &s[i].marks);  
    }  
}
```

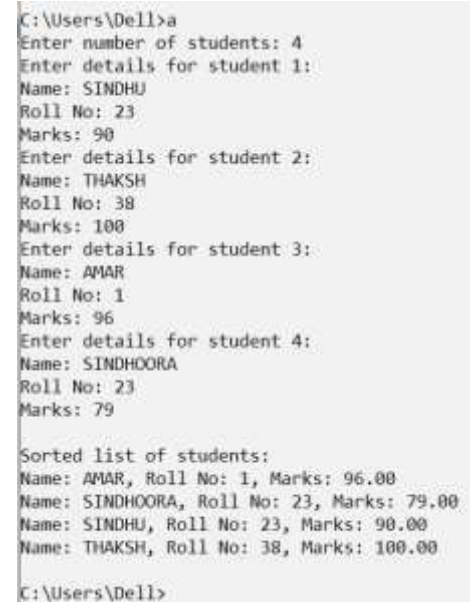
// Function to display student details

```
void displayStudents(struct student s[], int n) {  
    printf("\nSorted list of students:\n");  
    for (int i = 0; i < n; i++) {  
        printf("Name: %s, Roll No: %d, Marks: %.2f\n", s[i].name, s[i].roll_no, s[i].marks);  
    }  
}
```

// Function to sort students by name using Selection Sort

```
void sortByName(struct student s[], int n) {  
    for (int i = 0; i < n - 1; i++) {  
        int minIndex = i;  
        for (int j = i + 1; j < n; j++) {  
            if (strcmp(s[j].name, s[minIndex].name) < 0) {  
                minIndex = j;  
            }  
        }  
        // Swap if needed  
        if (minIndex != i) {  
            struct student temp = s[i];  
            s[i] = s[minIndex];  
            s[minIndex] = temp;  
        }  
    }  
}
```

```
int main() {  
    int n;    printf("Enter number of students: ");    scanf("%d", &n);  
    struct student s[1000];  
    readStudents(s, n);  
    sortByName(s, n);  
    displayStudents(s, n);  
    return 0;  
}
```



```
C:\Users\Dell>a  
Enter number of students: 4  
Enter details for student 1:  
Name: SINDHU  
Roll No: 23  
Marks: 90  
Enter details for student 2:  
Name: THAKSH  
Roll No: 38  
Marks: 100  
Enter details for student 3:  
Name: AMAR  
Roll No: 1  
Marks: 96  
Enter details for student 4:  
Name: SINDHOORA  
Roll No: 23  
Marks: 79  
  
Sorted list of students:  
Name: AMAR, Roll No: 1, Marks: 96.00  
Name: SINDHOORA, Roll No: 23, Marks: 79.00  
Name: SINDHU, Roll No: 23, Marks: 90.00  
Name: THAKSH, Roll No: 38, Marks: 100.00  
  
C:\Users\Dell>
```


If the requirement is to sort the Student data based on the roll_no, what changes to be done?

Coding Example_2: Instead of comparing names, compare roll numbers in the sort function.

`void sortByRollNo(struct student s[], int n) { // function name better to change and call accordingly.`

```
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (s[j].roll_no < s[minIndex].roll_no) { // Only change
                minIndex = j;
            }
        }
        // Swap if needed
        if (minIndex != i) {
            struct student temp = s[i];
            s[i] = s[minIndex];
            s[minIndex] = temp;
        }
    }
}
```

```
C:\Users\Dell>a
Enter number of students: 4
Enter details for student 1:
Name: sindhu
Roll No: 23
Marks: 67
Enter details for student 2:
Name: thaksh
Roll No: 12
Marks: 99
Enter details for student 3:
Name: rama
Roll No: 10
Marks: 23.7
Enter details for student 4:
Name: bheema
Roll No: 90
Marks: 10.4

Sorted list of students:
Name: rama, Roll No: 10, Marks: 23.70
Name: thaksh, Roll No: 12, Marks: 99.00
Name: sindhu, Roll No: 23, Marks: 67.00
Name: bheema, Roll No: 90, Marks: 10.40
```

Now, let us explore the Binary Search algorithm on an array of Student structures. I hope you remember that the **Binary Search can be applied on only sorted collection of items**. So, if we have to search for particular student, the array must be sorted first based on one field within the User defined Type.

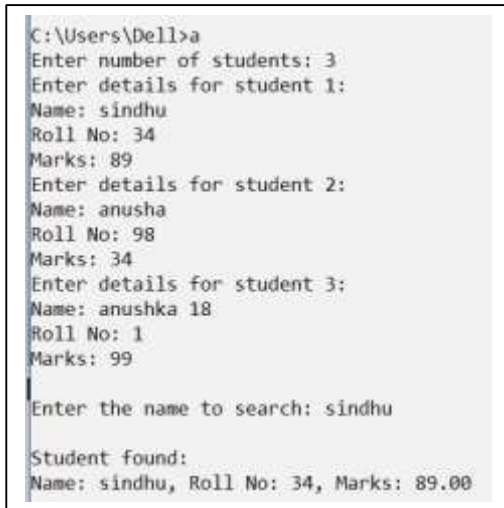
Coding Example_3: Given the name of the student to search, performing Binary Search on an array of student structures

```
int main() {
    int n;
    printf("Enter number of students: ");
    scanf("%d", &n);
    struct student s[1000];
    readStudents(s, n);
}
```

```
sortByName(s, n);
char key[50];
printf("\nEnter the name to search: ");
scanf(" %[^\\n]", key);
int index = binarySearchByName(s, n, key);
if (index != -1) {
    printf("\nStudent found:\\n");
    displayStudent(s[index]);
} else {
    printf("\nStudent with name \"%s\" not found.\\n", key);
}
return 0;
}
```

// Binary Search function

```
int binarySearchByName(struct student s[], int n, char key[]) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        int cmp = strcmp(key, s[mid].name);
        if (cmp == 0)
            return mid;
        else if (cmp < 0)
            high = mid - 1;
        else
            low = mid + 1;
    }
    return -1;
}
```



```
C:\Users\Dell>a
Enter number of students: 3
Enter details for student 1:
Name: sindhu
Roll No: 34
Marks: 89
Enter details for student 2:
Name: anusha
Roll No: 98
Marks: 34
Enter details for student 3:
Name: anushka 18
Roll No: 1
Marks: 99
Enter the name to search: sindhu
Student found:
Name: sindhu, Roll No: 34, Marks: 89.00
```

// Function to display a student

```
void displayStudent(struct student s) {
    printf("Name: %s, Roll No: %d, Marks: %.2f\\n", s.name, s.roll_no, s.marks);
}
```

Happy Sorting and Searching on Array of Structures!!



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE25CS151B**

***Lecture #16
Bit Fields in C***

**By,
Prof. Sindhu R Pai,
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: Text Processing and User-Defined Types****Topic: Bit fields in C**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and its respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Introduction

In C programming, Bit field is a data structure that **allows the programmer to allocate memory to structures and unions in bits** in order to utilize computer memory in an efficient manner. This is especially useful when memory conservation is critical, such as in **embedded systems or when dealing with hardware registers**. Instead of allocating a full int (typically 4 bytes) to every variable, **bit fields let you store data with as few bits as needed**. In programming terminology, a bit field is of great significance due to following reasons:

- You can efficiently pack multiple variables into a single word.
- Controlling the memory layout at the bit level is possible.
- You can represent flags or values that need only a few bits (e.g., 0–7 using 3 bits).

The idea is to use memory efficiently when we know that the value of a field or group of fields will never exceed a limit or is within a small range. **The variables defined with a predefined width are called bit fields**. A bit field can hold more than a single bit.

Syntax: `struct structure_name {
 data_type member_name : number_of_bits;
 };`

data_type: Typically int, unsigned int, or signed int. Using char or short may lead to implementation-defined behavior.

member_name: Name of the bit-field.

number_of_bits: The exact number of bits to allocate for that member (a positive integer).

Example: `struct Flags {
 unsigned int isVisible : 1;
 unsigned int isEnabled : 1;
 unsigned int errorCode : 3;
 };`

- isVisible and isEnabled use 1 bit each (ideal for true/false flags).
- errorCode uses 3 bits (can represent values from 0 to 7).
- This structure will use only 5 bits (plus padding as needed by the compiler). Max 1 byte, instead of 12 or more bytes if defined without bit fields.

Characteristics/Properties of Bit Fields:

- **Used Within Structures or Unions Only:** Bit fields can only be defined as members of a struct. You cannot declare a bit field outside.
- **Specify Memory in Bits:** Bit fields allow you to allocate a specific number of bits (not bytes) to a structure member.
- **Typically int, signed int, or unsigned int:** Although other integer types can be used, int, signed int, and unsigned int are most common and portable. Usage of char or short is compiler-dependent.
- **Maximum Width = Size of Base Type:** The number of bits assigned to a bit field must not exceed the size (in bits) of the underlying data type
- **Automatically Packed (with Padding Possible):** The compiler packs bit fields into as few memory units as possible, but may introduce padding to maintain alignment. The order in which bit fields are packed (left-to-right or right-to-left) is compiler-specific and can affect portability.
- **Unnamed Bit Fields for Padding:** Anonymous bit fields (without a name) can be used to add padding or align other fields:
- **No Address Access:** You cannot take the address of a bit-field member using &, since it may not have a distinct memory address.
- **Cannot Be Arrays or Pointers:** Bit field members cannot be declared as arrays or pointers.
- **Storage Class Restrictions:** Bit fields cannot be declared with the static or extern storage class within a structure definition. These qualifiers apply to the structure variable, not to individual bit-field members.

Coding Example_1: All these examples are run on Windows 32 bit machine

```
#include<stdio.h>
struct Status
{
    unsigned int bin1:1; // 1 bit is allocated for bin1. only two digits can be stored 0 & 1.
};
```

```
int main()
{
    printf("Size of structure is %lu\n",sizeof(struct Status));
    struct Status s;
    //scanf("%d",&s.bin1);    // Error
    s.bin1 = 0;    printf("%d\n",s.bin1);
    s.bin1 = 1;    printf("%d",s.bin1);
    return 0;
}
```

```
C:\Users\Dell>a
Size of structure is 4
0
1
```

The maximum value that can be stored in bit fields depends on whether the bit field has enough memory to hold the value. If it can't fit, results in **compile time warning**. **If you need a variable to store a value from 0 to 7, then you can define a bit field with a width of 3 bits.**

Coding Example_2: The age variable is going to use only 3 bits to store the value. If you try to store the value with more than 3 bits, it doesn't allow.

```
#include <stdio.h>
#include <string.h>
struct
{
    unsigned int age : 3;    } Age;
int main( )
{
    Age.age = 4;
    printf( "Sizeof( Age ) : %d\n", sizeof(Age) );
    printf( "Age.age : %d\n", Age.age );
    Age.age = 7;
    printf( "Age.age : %d\n", Age.age );
    Age.age = 8;
    printf( "Age.age : %d\n", Age.age );
    return 0;
}
```

```
C:\Users\Dell>gcc -c check.c
check.c: In function 'main':
check.c:12:14: warning: large integer implicitly truncated to unsigned type [-Woverflow]
    Age.age = 8;
               ^
```

```
C:\Users\Dell>a
Sizeof( Age ) : 4
Age.age : 4
Age.age : 7
Age.age : 0
```

Coding Example_3: Assigning signed value to an unsigned variable for which the bit fields are specified.

```
#include<stdio.h>

struct Status
{
    unsigned int bin1:4;    // 4 bits is allocated for bin1. 0 to 15, any number can be used.
    unsigned int bin2:2;    // 2 bits allocated for bin2. 0 to 3, any number can be used
};

int main()
{
    printf("Size of structure is %lu\n",sizeof(struct Status));    // again 4 bytes
    struct Status s1;
    s1.bin1=-7;    // it is unsigned. but sign is given while assigning
    s1.bin2=2;
    printf("%d %d",s1.bin1,s1.bin2);    //
    return 0;
}
```

```
C:\Users\De11>a
Size of structure is 4
9 2
```

Coding Example_4: Structure having two members with signed and unsigned variable for which bit fields are specified.

```
#include<stdio.h>

struct Status
{
    int bin1:4; // one bit is used for representing the sign. Another 3 bits for data
    unsigned int bin2:2; // 2 bits allocated for bin2. 0 to 3, any number can be used
};

int main()
{
    printf("Size of structure is %lu\n",sizeof(struct Status)); // again 4 bytes
    struct Status s1; s1.bin1=-7; s1.bin2=2;
    printf("%d %d",s1.bin1,s1.bin2);
    return 0;
}
```

```
Size of structure is 4
-7 2
```


Unnamed Bit Fields

Bit-field **members declared without an identifier** (i.e., no variable name) are unnamed bit fields. They are primarily used for:

Padding: To align the next member on a specific boundary.

Skipping bits: When certain bits are reserved or not used.

```
struct example {  
    unsigned int a : 3;  
    unsigned int : 2; // unnamed 2-bit field (padding)  
    unsigned int b : 4;  
}; // a uses 3 bits, 2 bits are skipped (no name, so not accessible) and b uses 4 bits.
```

Bit fields with a length of 0 must be unnamed. Unnamed bit fields cannot be referenced or initialized. A zero-width bit field can cause the next field to be aligned on the next container boundary where the container is the same size as the underlying type of the bit field.

Points to note:

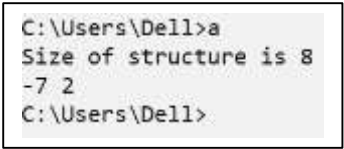
- You **cannot access unnamed bit fields** — they don't have a name.
- Their only purpose is **alignment or spacing** within the memory layout.
- They help in **matching memory layouts** with hardware registers or protocols.
- The bit width **must be specified** even for unnamed fields.

Coding Example_5:

struct Status

```
{    int bin1:4;    // one bit is used for representing the sign. Another 3 bits for dat  
    //int i:0;// you cannot allocate 0 bits for any member inside the structure. Error  
    int :0;// A special unnamed bit field of size 0 is used to force alignment on  
    // next boundary. observe no member variable. only size is 0 bit  
    unsigned int bin2:2; // 2 bits allocated for bin2. 0 to 3, any number can be used  
};
```

```
int main()
{
    // observe the size of the structure. Now 8 bytes
    printf("Size of structure is %lu\n",sizeof(struct Status));
    struct Status s1;
    s1.bin1=-7; s1.bin2=2;
    printf("%d %d",s1.bin1,s1.bin2);
    return 0;
}
```



```
C:\Users\Dell>a
Size of structure is 8
-7 2
C:\Users\Dell>
```

Invalid Usage of Bit Fields in C

Coding Example_6:

```
#include <stdio.h>

struct InvalidBitFields {
    int a[5] : 3;        // ✗Error: Bit field cannot be an array
    int *ptr : 4;         // ✗Error: Bit field cannot be a pointer
    static int b : 2;     // ✗Error: Bit field cannot have static storage class
    unsigned int data : 40; // ✗Error: Width exceeds the size of base type (usually 32 bits)
};

int main() {
    struct InvalidBitFields ib;
    ib.data = 100;
    printf("Invalid Bit Fields: %u\n", ib.data);
    return 0;
}
```

Bit fields offer a **smart way to manage memory efficiently by packing data tightly**, especially useful in **systems with limited resources**. Whether it's setting flags or designing hardware-friendly structures, bit fields help you write compact, efficient C code.

Use bit fields — save space, code efficiently!!



Unit #: 3

Name: Text Processing and user defined types

Topic: Unions in C

Course objectives:

The objective(s) of this course is to make students

CObj1: Acquire knowledge on how to solve relevant and logical problems using computing machine

CObj2: Map algorithmic solutions to relevant features of C programming language constructs

CObj3: Gain knowledge about C constructs and it's associated eco-system

CObj4: Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors

CObj5: Get insights about testing and debugging C Programs

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs

CO2: Understand, analyse and apply text processing and string manipulation methods using C Arrays, Pointers and functions

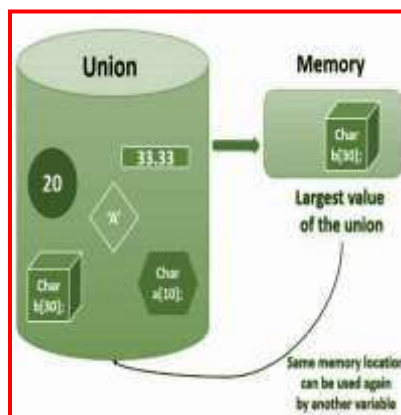
CO3: Understand prioritized scheduling and implement the same using C structures

CO4: Understand and apply sorting techniques using advanced C constructs

CO5: Understand and evaluate portable programming techniques using preprocessor directives and conditional compilation of C Programs

Introduction

A Union is a user-defined datatype in the C programming language. It is a collection of variables of different datatypes in the same memory location. We can define a union with many members, but at a given point of time, only one member can contain a value. Union unlike structures, share the same memory location. Using Union in C will save Memory Space in a given context.



Both structure and union are the custom data types that store different types of data together as a single entity. The members of structure and union can be entities of any type, such as other structures, unions, or arrays. Both structures or unions can be passed by value to a function and also return the value by functions, the argument will need to have the same type as the function parameter.

Note: To access members, we use the '.' operator or '->' in a given context.

C unions are used to save memory. To better understand a union, think of it as a chunk of memory that is used to store variables of different types. When we want to assign a new value to a field, the existing data is replaced with new data. C **unions allow data members which are mutually exclusive to share the same memory**. This is quite important when memory is valuable, such as in embedded systems. Unions are mostly used in embedded programming where direct access to memory is needed.

Defining a union:

You can define a union with many members, but **only one member can contain a value at any given time. The memory occupied by a union will be large enough to hold the largest member of the union. So, the size of a union is the size of the biggest component. At a given point in time, only one can exist. All the fields overlap and they have the same offset: 0.**

The format for defining new type using union is as below:

```
union Tag // union keyword is used
{
    data_type member1;
    data_type member2;
    data_type member n;
}; // ; is compulsory
```

Example:

```
union car
{
    char name[10]; float price;
};
```

Now, a variable of car type can store a floating-point number or a string, i.e., the same memory location is used to store multiple types of data. In the above example, the data type will occupy 10 bytes of memory space. Because this is the maximum space that can be occupied by a character array.

Coding Example_1: Displays the total memory size occupied by the union

```
#include <stdio.h>

union car //Assumptions
{
    char name[10]; // 1 byte for char
    float price; // 4 bytes for float
};
```



```
int main( )
{
    union car c;

    printf( "Memory size occupied by data in bytes : %d\n", sizeof(car));

    return 0;
}
```

Output: Memory size occupied by data in bytes : 10

Accessing Union Members using unionvariable:

To access any member of a union, we use the member access operator (.). The member access operator is coded as a period between the union variable name and the union member that we wish to access.

Coding Example_2:

```
#include <stdio.h>
#include <string.h>
union car
{
    char name[10];
    float price;
};
int main( )
{
    union car c;
    strcpy( c.name, " Benz");
    c.price=4000000.00;
    printf( "car name: %s\n", c.name);
    printf( "car price: %f\n", c.price);
    return 0;
}
```



Output: When the above code is compiled and executed, the value of name gets corrupted because the final value assigned to the variable has occupied the memory location, and this is the reason that the value of price member is printed well.

Coding Example_3: Prints all the members of union

```
#include <stdio.h>
#include<string.h>
union car
{
    char name[10];
    float price;
};
int main( )
{
    union car c;
    strcpy( c.name, "Benz");
    printf( "car name: %s\n", c.name);
    c.price=4000000.00;
    printf( "car price: %f\n",c.price);
    return 0;
}
```

Output:

```
Car name: Benz
Car price:4000000.00
```

Coding Example_4: Size of union and accessing the union members

```
#include<stdio.h>
union X
{
    int i;
    int j;
    double k;
};
// ; compulsory
```



```
int main()
{
    printf("sizeof X is %lu\n",sizeof(union X));
    union X x1;
    x1.i = 23;
    x1.j = 23;
    x1.i = 49;
    printf("%d %d\n",x1.i, x1.j);
    return 0;
}
```

Output:

```
sizeof X is 8           //system specific
49  49
```

Coding Example_5: Size of union and accessing the union members

```
#include<stdio.h>
union Z
{
    int a;  int b[3];
};
int main()
{
    union Z z;
    z.a = 23;
    printf("size of Z is %d\n",sizeof(z));
    printf("%d %d %d",z.b[0], z.b[1],z.b[2]);
    return 0;
}
```

Output:

```
size of Z is 12
23 3698688 4194432 // 23  junk value  junk value
```




Coding Example_6: All the fields overlap and they have the same offset: 0 in union

```
#include<stdio.h>
#include<stddef.h>          // offsetof function is from this header file
union A
{
    int x;  int y;  int z;
};
struct B
{
    int x;  int y;  int z;
};
int main()
{
    printf("%lu\n",offsetof(union A,y));// 0
    printf("%lu\n",offsetof(struct B,y)); // 4
    printf("%lu\n",offsetof(struct B,z)); // 8
    // assumption int occupies four bytes
}
```

Structure v/s Union:

Structure:

1. The keyword struct is used to define a
2. When a variable is associated with a structure, memory is allocated to each member of the structure. The size of the structure is greater than or equal to the sum of its members.
3. Each member within a structure is assigned unique storage area of location
4. In Structure, the address of each member will be in ascending order. This indicates that memory for each member will start at different offset values.
5. In Structure, altering the value of a member will not affect other members of the structure.
6. All members can be accessed at a time.

Union:

1. The keyword union is used to define a union.
2. When a variable is associated with a union, memory is allocated by considering the size of the



largest So, the size of a union is equal to the size of its largest member.

3. Memory allocated for union is shared by individual members of
4. For unions, the address is same for all the members of a union. This indicates that every member begins at the same offset.
5. Altering the value of any of the member will alter other member values.
6. In Unions, only one member can be accessed at a time.

Similarities between structures and unions:

1. Both are user-defined data types used to store data of different types as a single unit.
2. Their members can be objects of any type, including other structures, unions, or arrays. A member can also consist of a bit of a field.
3. Both structures and unions support only assignment = and sizeof operators. The two structures or unions in the assignment must have the same members and member types.
4. A structure or a union can be passed by value to functions and returned by value by functions. The argument must be of the same type as the function parameter. A structure or union is passed by value just like a scalar variable as a corresponding parameter.
5. ". "operator is used for accessing members.

We know that the fields of a union will share memory, so in the main program we ask the user which data he/she would like to store, and depending on the user's choice, the appropriate field will be used. By this way, we can use the memory efficiently.

Coding Example_7: Usage of anonymous union inside a structure

```
#include<stdio.h>

struct student
{
    union
    {
        //anonymous union (unnamed union)
        char name[10];
```



```
        int roll;
    };
    int mark;
};
int main()
{
    struct student stud;
    char choice;
    printf("\n You can enter your name or roll number ");
    printf("\n Do you want to enter the name (y or n): ");
    scanf("%c",&choice);
    if(choice=='y' || choice=='Y')
    {
        printf("\n Enter name: ");
        scanf("%s",stud.name);
        printf("\n Name:%s",stud.name);
    }
    else
    {
        printf("\n Enter roll number");
        scanf("%d",&stud.roll);
        printf("\n Roll:%d",stud.roll);
    }
    printf("\n Enter marks");
    scanf("%d",&stud.mark);
    printf("\n Marks:%d",stud.mark);
    return 0;
}
```

Output:

You can enter your name or roll number

Do you want to enter the name (y or n) : y

Enter name: john



Name:john

Enter marks: 45

Marks:45

Coding Example_8: Usage of anonymous union inside a union

```
#include<stdlib.h>
#include<string.h>
#include<stdio.h>
union test
{
    int i;
    union
    {
        char a[20];
        float k;
    };
};

int main()
{
    printf("%lu",sizeof(union test));    // 20
    union test t;
    t.i=78;    // One member at a time from union
    printf("\ni is %d\n",t.i);
    strcpy(t.a,"sindhu");
    printf("a is %s\n",t.a);
    t.k=45.5;
    printf("k is %f",t.k);
    return 0;
}
```



Output:

20

i is 78

a is sindhu

k is 45.500000

Coding Example_9: Structure inside Union

```
#include<stdio.h>
```

```
struct student
```

```
{
```

```
    char name[30];
```

```
    int rollno;
```

```
    float percentage;
```

```
};
```

```
union details
```

```
{
```

```
    struct student s1;
```

```
};
```

```
int main()
```

```
{
```

```
    union details set;
```

```
    printf("Enter details:");
```

```
    printf("\nEnter name : ");
```

```
    scanf("%s", set.s1.name);
```

```
    printf("\nEnter roll no : ");
```

```
    scanf("%d", &set.s1.rollno);
```

```
    printf("\nEnter percentage :");
```

```
    scanf("%f", &set.s1.percentage);
```

```
    printf("\nThe student details are : \n");
```

```
    printf("\name : %s", set.s1.name);
```

```
printf("\nRollno : %d", set.s1.rollno);  
printf("\nPercentage : %f", set.s1.percentage);  
return 0;  
}
```

Output:

Enter details:

Enter name : sindhu

Enter roll no : 123

Enter percentage :67

The student details are :

Name : sindhu

Rollno : 123

Percentage : 67.000000

Happy Coding using Unions and Structures!!



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE25CS151B**

***Lecture #18
Enumeration in C***

**By,
Prof. Sindhu R Pai,
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: Text Processing and User-Defined Types****Topic: Enumeration in C**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Introduction

An **enumeration** (or **enum**) is a user-defined data type that **consists of a set of named constant values**. It helps make a program more readable by **replacing numeric constants with meaningful names**. These names enhance the program's maintainability. It is **easy to remember names** rather than the numbers. **We all know google.com. We don't remember the IP address of website google.com.** Typically, enums are used to **represent a collection of fixed sets of related constants**, such as **days of the week, directions, or states in a process**. Also, used in **state machines and flag management**.

The **keyword 'enum'** is used to declare new enumeration types in C. In Essence, Enumerated types provide a symbolic name to represent one state out of a list of states. The names are symbols for **integer constants, which won't be stored anywhere in program's memory**. Enums are used to replace #define chains.

Enumeration data type consists of named integer constants as a list. **It start with 0 (zero) by default and value is incremented by 1 for the sequential identifiers in the list.**

Syntax:

```
enum identifier { enumerator-list }; // semicolon compulsory & identifier optional
```

Examples:

```
enum month { Jan, Feb, Mar }; // Jan, Feb and Mar constants will be assigned to 0, 1 and 2 respectively by default
```

```
enum Direction { NORTH, SOUTH, EAST, WEST }; // 0, 1, 2, 3 by default
```

```
enum month { Jan = 1, Feb, Mar }; // Feb and Mar will be assigned with 2 and 3 respectively by default
```

Note: It doesn't make sense to take the addresses of the constants within the enumeration.

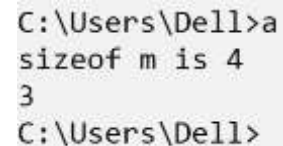
Characteristics of Enums in C

1. **Underlying type is int:** By default, enum constants are assigned integer values starting from 0, incremented by 1. Enum constants are automatically assigned values if no value specified.
2. **Values can be explicitly assigned:** You can manually assign specific integer values to enum constants.
3. **Same value can be assigned to multiple names.**
4. **Scoped within the same block or globally:** **Enum names must be unique** in their scope.
5. **No type safety in C:** Enum variables are essentially int, so you can assign any integer to an enum variable, even if it's not a valid constant defined within the enum.

Coding Example_1: Enum constants are automatically assigned values if no value specified.

```
#include<stdio.h>
enum months
{
    jan, feb, mar, apr, may, jun, jul, aug, sep, oct, nov, dec
    // symbol jan gets the value 0, all others previous+1
    // all constants are separated by comma(.).
    // no memory allocated for these constants. Available in this file anywhere directly.
};

int main()
{
    enum months m;      // memory is allocated for m.
    printf("sizeof m is %lu\n",sizeof(m));
    m=apr;              // same as the size of integer
    printf("%d",m);
    return 0;
}
```



```
C:\Users\Dell>a
sizeof m is 4
3
C:\Users\Dell>
```

Coding Example_2: We can assign values to some of the symbol names in any order. All unassigned names get value as, value of previous name plus one.

```
#include <stdio.h>
enum day {sunday = 1, monday, tuesday = 5, wednesday, thursday = 10, friday, saturday};
int main()
{
    //printf("enter the value for monday\n");
    //scanf("%d",&monday);    // error: not legal
```

```
// Memory not allocated for constants in enum. So ampersand(&) can't be used with those.
//Also value already assigned to monday that can't be changed
printf("%d %d %d %d %d %d %d", sunday, monday, tuesday, wednesday, thursday,
friday, saturday);
return 0;
}
```

```
C:\Users\De11>a
1 2 5 6 10 11 12
C:\Users\De11>
```

Coding Example_3: Only integer constants are allowed. Below code results in Error if abc=3.5 is uncommented.

```
#include<stdio.h>
enum examples
{
    abc = 3.5, def, // Value is not an integer constant
    pqr = "Hello", lmn; // Value is not an integer constant
};
int main()
{
    enum examples e1;
    return 0;
}
```

```
C:\Users\De11>gcc -c check.c
check.c:3:13: error: enumerator value for 'abc' is not an integer constant
{
  abc = 3.5, def, // Value is not an integer constant
  ^
check.c:4:10: error: enumerator value for 'pqr' is not an integer constant
pqr = "Hello", lmn; // Value is not an integer constant
     ^~~~~~
check.c:4:22: error: expected ';', ',' or '}' before ';' token
pqr = "Hello", lmn; // Value is not an integer constant
                   ^
```

Coding Example_4: Valid arithmetic operators are +, - * and / and %

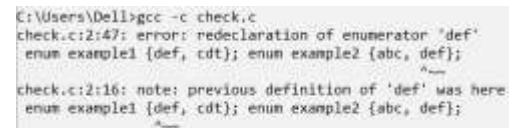
```
#include<stdio.h>
enum months{
    jan,feb,mar,apr,may,jun,july,aug,sep,oct,nov,dec
};
int main()
{
    printf("%d\n",mar-jan); // valid
    printf("%d\n",mar*jan); // valid
    printf("%d\n",mar&&feb); // valid
    //mar++; //error : mar is a constant
    //printf("after incrementing %d\n",mar); // error
    enum months m=feb;
    m=(enum months)(m + jan); // m can be changed. m is a variable of type enum months.
    // But all constants in enum cannot be changed its value
    printf("m, after incrementing %d\n",m);
    for(enum months i=jan;i<=dec;i++) // loop to iterate through all constants in enum
    {
        printf("%d\n", i);
    }
}
```

Coding Example_5: Enumerated Types are Not Strings .Two enum symbols/names can have same value.

```
#include <stdio.h>
enum State {Working = 1, Failed = 0, Freezed = 0};
int main()
{
    printf("%d, %d, %d", Working, Failed, Freezed); // associated values are printed
    return 0;
}
```

Coding Example_6: All enum constants must be unique in their scope.

```
#include<stdio.h>
enum example1 {def, cdt};
enum example2 {abc, def};
int main()
{
    enum example1 e; return 0;
}
```



Coding Example_7: It is not possible to change the constants.

```
#include<stdio.h> // Uncommenting the statement feb=10, results in error.
enum months { jan,feb=11,mar,apr,may=23,jun,july };
int main()
{
    enum months m;
    //feb=10; // error
    printf("%d",feb); // no error // we can access it directly
    return 0;
}
```

Coding Example_8: Storing the symbol of one enum in another enum variable. It is allowed.

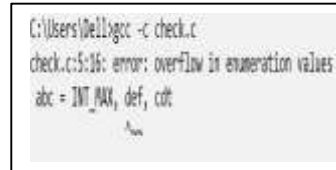
```
#include<stdio.h>
enum nation{ India=1, Nepal };
enum States{ Delhi, Katmandu };
int main()
{
    enum States n=Delhi; enum States n1=India;
    // Value 1 is stored in n1 which is of type enum States
    enum nation n2=Katmandu;
    printf("%d\n",n); printf("%d\n",n1); printf("%d",n2);
    return 0;
}
```



Coding Example_9: The value assigned to enum names must be some integral constant, i.e., the value must be in range from minimum possible integer value to maximum possible integer value

```
#include<limits.h>
#include<stdio.h>
enum examples
{
    abc = INT_MAX, def, cdt
    // INT_MAX is defined in limits.h
    // def must get INT_MAX+1 value which is an error
    // delete def and cdt and compile and run again
};

int main()
{
    enum examples e; e = abc;    printf("%d",e);
    return 0;
}
```

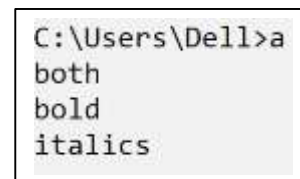


Coding Example_10: Enumerations are actually used to restrict the values to a set of integers within the range of values. But outside the range, if any value is used it is not an error in C standards like C11, C99 and C89. Other languages(ex: Java) doesn't support any value outside the range.

Code to demonstrate why enum support values outside range values in C.

```
#include<stdio.h>
enum design_flags
{
    bold =1, italics=4, underline=8    }df;
int main()
{
    df=bold;
    df=italics|bold; // to set bold and italics. Now df contains 5 which is not defined in enum
    // if you write again df=italics, 5 is replaced with 4. So text is no more bold. It is only italics
    if(df==(bold|italics))
        printf("both\n");
    else if(df==bold)
        printf("bold it is\n");
    else if(df==italics)
        printf("italics it is\n");

    if(df&bold)    // whether the text is bold
        printf("bold\n");
    if(df & italics) // whether the text is italics
        printf("italics\n");
}
```



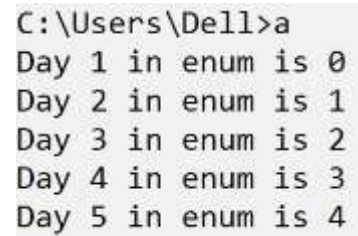
```
    if(df & underline)    // whether the text is underlined
        printf("underline\n");
    return 0;
}
```

Array of Enums

C allows you to store **multiple values of an enumeration type in a single collection**, just like an array of integers. Since enums are internally treated as integers, you can easily declare an array using the enum type and assign it values using the defined enum constants. It combines the **readability of named constants (from enums) with the power of indexed access (from arrays)**, allowing you to **handle a fixed set of meaningful values in a structured manner**.

Coding Example_11: Demo of array of enums

```
#include <stdio.h>
enum Day { MON, TUE, WED, THU, FRI, SAT, SUN };
int main() {
    enum Day workWeek[5] = { MON, TUE, WED, THU, FRI };
    for (int i = 0; i < 5; i++) {
        printf("Day %d in enum is %d\n", i + 1, workWeek[i]);
    }
    return 0;
}
```

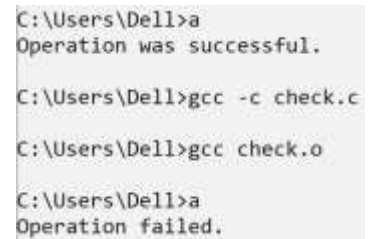


Passing Enum to a Function

Since enums are internally treated as int in C, passing them to functions is straightforward — just like passing an integer.

Coding Example_12: Demo of passing enum to a function

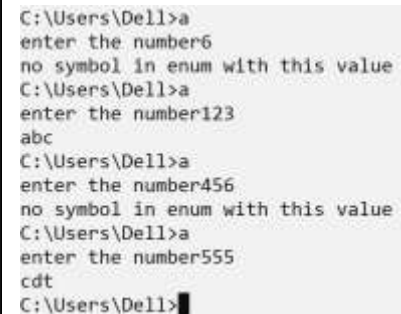
```
#include <stdio.h>
enum Status { SUCCESS = 1, FAILURE = 0 };
void printStatus(enum Status s) {
    if (s == SUCCESS)    printf("Operation was successful.\n");
    else    printf("Operation failed.\n");
}
int main() {
    enum Status currentStatus = SUCCESS;    //enum Status currentStatus = FAILURE;
    printStatus(currentStatus); // Pass enum to function
    return 0;    }
```



Coding Example_13: One of the short comings of Enumerated Types is that they don't print nicely.

To print the "String"(symbol) associated with the defined enumerated value, you must use the following cumbersome code.

```
#include<stdio.h>
enum example1{      abc=123, bef=345, cdt=555    };
void printing(enum example1 e1);
int main()
{
    enum example1 e1;
    // how to print abc, bef and cdt based on user choice?? printf("enter the number");
    scanf("%d",&e1); //enter any number
    printing(e1);     // user defined function to print the symbol name
    return 0;
}
void printing(enum example1 e1)
{
    switch(e1)
    {
        case abc: printf("abc");break;
        case bef: printf("bef");break;
        case cdt: printf("cdt");break;
        default: printf("no symbol in enum with this value"); break;
    }
}
```



```
C:\Users\Dell>a
enter the number6
no symbol in enum with this value
C:\Users\Dell>a
enter the number123
abc
C:\Users\Dell>a
enter the number456
no symbol in enum with this value
C:\Users\Dell>a
enter the number555
cdt
C:\Users\Dell>
```

Think about it!!

- Is pointer to enum same as pointer to integer?
- Can a pointer to integer hold the address of enum variable?
- Can a pointer to integer hold the address of enum constant?
- Can a pointer to enum hold the address of any integer?

Happy Mastering Enumerations in C!!



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE25CS151B**

***Lecture #19
Linked List Implementation in C***

**By,
Prof. Sindhu R Pai,
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: Text Processing and User-Defined Types****Topic: Linked List Implementation in C**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Introduction

A linked list is a collection of **nodes that are stored at different locations in memory but are logically connected using links** (pointers). Each node usually contains some set of data and a pointer that refers to the next node in the sequence. The node and link has a special meaning which we will be discussing in this chapter. Before diving deeper, let's first address a few points.

Point #1: Is a pointer allowed as a data member in a structure? Absolutely.

Consider the below structure. Let us see some of the versions of accessing the data members.

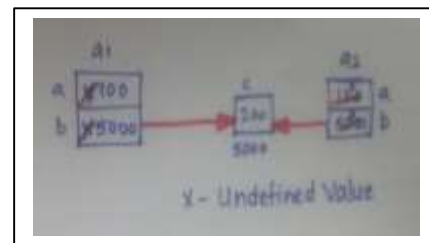
struct A

```
{    int a;    int *b;    };
```

Coding Example_1:

Version 1:

```
int main()
{
    struct A a1; struct A a2; a1.a = 100;
    int c = 200; a1.b = &c;
    printf("a1 values:%d and %d\n", a1.a, *(a1.b));
    a2 = a1;
    printf("a2 values:%d and %d\n", a2.a, *(a2.b));
    a1.a = 300;
    *(a1.b) = 400;
    printf("a2 values:%d and %d\n", a2.a, *(a2.b));
    return 0;
}
```



```
C:\Users\Dell>a
a1 values:100 and 200
a2 values:100 and 200
a2 values:100 and 400
```

Version 2:

```
#include<stdio.h>
int main()
{
    struct A s;    s.a = 100;    s.b = &(s.a);
    printf("%d %d",s.a,*(s.b));
    struct A s1;    s1.a = 100;    s1.b = &(s1.a);
    printf("%d %d\n",s1.a,*(s1.b));
    struct A s2 = s1;
    printf("%p %p\n",s1.b,s2.b);
    printf("%d %d\n",s2.a,*(s2.b));
}
```



```

s2.a = 200;    printf("%p %p\n",s1.b,s2.b);
printf("%d %d\n",s1.a,*(s1.b));
printf("%d %d\n",s2.a,*(s2.b)); // very imp
*(s2.b) = 300; printf("%p %p\n",s1.b,s2.b);
printf("%d %d\n",s1.a,*(s1.b));
printf("%d %d\n",s2.a,*(s2.b));
s2.b = &(s2.a);
*(s2.b) = 400;
printf("%p %p\n",s1.b,s2.b); printf("%d %d\n",s1.a,*(s1.b));
printf("%d %d\n",s2.a,*(s2.b));    return 0;
}

```

```

C:\Users\Dell>a
100 100100 100
0061FF10 0061FF10
100 100
0061FF10 0061FF10
100 100
                200 100
0061FF10 0061FF10
300 300
200 300
0061FF10 0061FF08
300 300
400 400

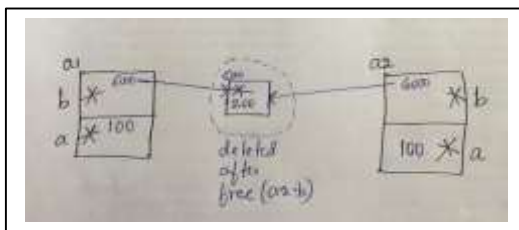
```

Version 3: Dynamic Memory Management and dereferencing the dangling pointer

```

#include<stdio.h>
#include<stdlib.h>
int main()
{
    struct A a1; struct A a2; a1.a = 100;
    a1.b = (int*) malloc(sizeof(int));
    *(a1.b) = 200;
    printf("a1 values:%d and %d\n", a1.a, *(a1.b)); // 100 200
    a2 = a1;
    printf("a2 values:%d and %d\n", a2.a, *(a2.b)); // 100 200
    free(a2.b); // a1.b too becomes dangling pointer
    printf("a1 values:%d and %d\n", a1.a, *(a1.b)); // 100 undefined behaviour
    return 0;
}

```



```

C:\Users\Dell>a
a1 values:100 and 200
a2 values:100 and 200
a1 values:100 and 14619624

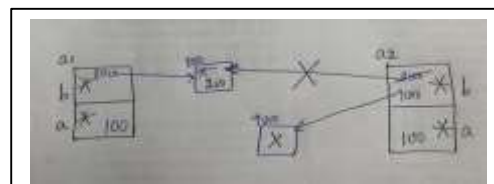
```

Version 4:

```

int main()
{
    struct A a1; struct A a2; a1.a = 100;
    a1.b = (int*) malloc(sizeof(int));
    *(a1.b) = 200;

```



```
printf("a1 values:%d and %d\n", a1.a, *(a1.b)); // 100 200
a2 = a1;
a2.b = (int*) malloc(sizeof(int)); // changing this to a1.b creates garbage and output differs
printf("a2 values:%d and %d\n", a2.a, *(a2.b));
printf("a1 values:%d and %d\n", a1.a, *(a1.b));
return 0;
}
```

```
C:\Users\Dell>a
a1 values:100 and 200
a2 values:100 and 12193256
a1 values:100 and 200
```

Point #2: Can a structure contain another structure as a member? Absolutely. This is one type of nesting the structure inside another – Nested structures

Coding Example_2:

```
struct A {    int a;    };
struct B{      int a;  struct A a1;
}; // structure variable a1 as a data member in B
int main()
{
    printf("sizeof A %d\n",sizeof(structA);
    printf("sizeof B %d\n",sizeof(struct B)); // more than A
    return 0;
}
```

```
C:\Users\Dell>a
sizeof A 4
sizeof B 8
```

Point #3: Can we have a structure variable inside the same structure? – No.

Coding Example_3: Results in Compiletime Error

```
struct A {    int a;  struct A a1;    };
```

```
C:\Users\Dell>gcc -c check.c -W
check.c:1:30: error: field 'a1' has incomplete type
struct A { int a; struct A a1;    };
                        ^
```

The solution to the earlier issue is to **include a pointer to the same structure type** within the structure itself. Since the **size of a pointer is fixed for any given system**, the **compiler can determine the total size of the structure at compile time**. This leads to the usage of **Self-Referential Structures**.

Self Referential Structures

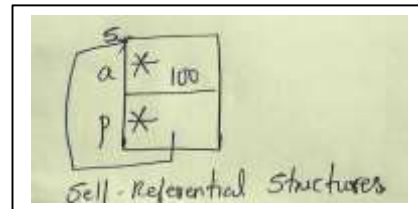
A structure which has a **pointer to itself as a data member** is called a self - referential structure. In this type of structure, the pointer inside the structure points to the same structure. Structure can have one or more pointers pointing to the same type of structure as their member. It is **widely used in building dynamic data models such as trees, linked lists, graphs** etc. where each element needs to connect to others of the same type. Usually, such a structure is called as a **Node**.

```
struct Node { int data;    struct Node *next;    };
```

This allows creating a chain of nodes dynamically linked together, forming the basis for structures like linked lists.

Coding Example_4: Recursive Pointer Access in Self-Referential Struct

```
struct Sample {      int a;    struct Sample *p;    };  
int main()  
{  
    printf("%d\n",sizeof(struct Sample) );  
    struct Sample s;  
    s.a = 100;  
    s.p = &s;  
    printf("%d %d %d\n", s.a, s.p->a, s.p->p->a);  
    return 0;  
}
```



```
C:\Users\Dell>a  
8  
100 100 100
```

Linked List in C

When self referential structures are linked together, they form a **Linked list** — a **dynamic and flexible way of storing and managing sequential data**. A linked list is a collection of **nodes that are stored at different locations in memory but are logically connected using links** (pointers). Each node usually contains some set of data and a pointer that refers to the next node in the sequence.

Characteristics of Linked List

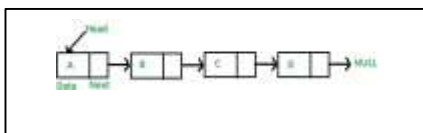
1. A data structure that consists of **zero or more nodes**. Every node is composed of two fields: a **data/component field** and a **pointer field**. The pointer field of every node point to the next node in the sequence.
2. We can access the nodes one after the other. There is **no way to access the node directly** as **random access is not possible in a linked list**. Lists have sequential access.
3. **Insertion and deletion in a list at a given position requires no shifting of elements.**

Operations on Linked List

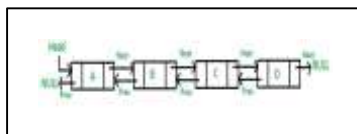
- **Insertion Operation on the list** includes Insertion at the beginning of the list, Insertion at the end of the list and Insertion at a Specific Node in the List
- **Deletion Operation on the list** includes Deletion at the beginning of the list, Deletion at the end of the list and Deletion of a Specific Node in the List.
- **Other Operations on the list** such as Traversing the list, Searching the list and Sorting the List, Merging two lists, Finding the Union, Set operations on nodes of the list, FindMin, FindMx, Find Repeated etc

Types of Linked List

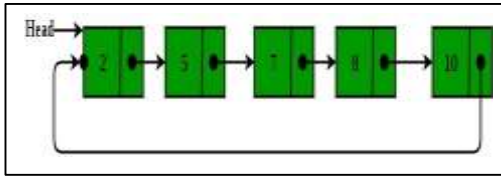
i. Singly Linked List



ii. Doubly Linked List



iii. Circular Linked List



Singly Linked List

A singly linked list consists of a **series of nodes, where each node holds data and a pointer to the next node** in the list. To build and work with a singly linked list efficiently, we'll focus on three fundamental operations: **insertion, traversal, and deletion**. These operations enable us **to add nodes, navigate through the list, and remove nodes**. As we implement these operations, it will become clear how **self-referential structures** enable flexible and dynamic handling of data in memory.

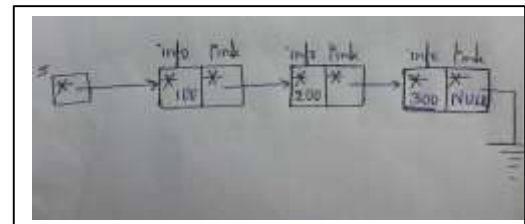
Implementation:

// Self referential structure template/entity

```
struct node
{
    int info; // component field
    struct node *link; // pointer field
};
typedef struct node NODE_T; // Creation of alias/type
```

// Client code

```
#include<stdio.h>
int main()
{
    NODE_T *s;
    s = (NODE_T*) malloc(sizeof(NODE_T));
    s->info = 100;
    s->link = (NODE_T*) malloc(sizeof(NODE_T));
    s->link->info = 200;
    s->link->link = (NODE_T*) malloc(sizeof(NODE_T));
    s->link->link->info = 300;
    s->link->link->link = NULL;
    display(s); // Function call to display all nodes
    freelist(s); // Function call to free all nodes
}
```

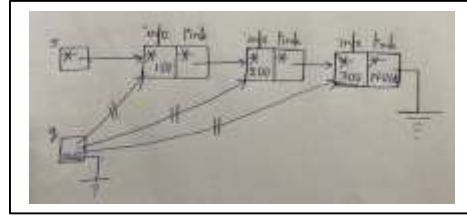


```
C:\Users\Dell>a
100      200      300
100 deleted
200 deleted
300 deleted
```

// Function Implementations

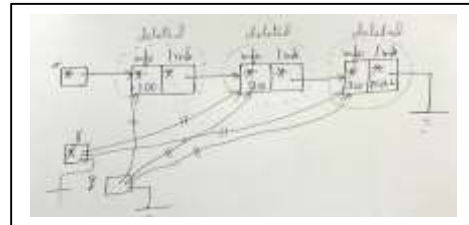
// To display all nodes in the list

```
void display(NODE_T* q)
{
    while(q != NULL)
    {
        printf("%d\t", q->info);
        q = q->link;
    }
}
```



// To delete all nodes in the list

```
void freelist(NODE_T* q)
{
    NODE_T* r;
    while(q != NULL)
    {
        printf("\n%d deleted", q->info);
        r = q->link;
        free(q);
        q = r;
    }
}
```



Points to Think!

- Why can't we say just free(s) rather than writing freelist function and calling it in the client?
- Is the client code creating any dangling pointer at the end of execution? If yes, what guideline to be followed to avoid this?
- Can we write a create_node() function which can take the data from the user and return the node?
- Can we include a definition of insert_to_list() function which can insert the node only if user wants to add a node to the list.
- To include these operations, menu driven application might help you. Try it yourself!!!

Enjoy Exploring Linked Lists!!

Unit #: 3

Unit Name: Text processing and user defined types

Topic: Implementation of Priority Queue

Course objectives:

The objective(s) of this course is to make students

COBj1: Acquire knowledge on how to solve relevant and logical problems using computing machine

COBj2: Map algorithmic solutions to relevant features of C programming language constructs

COBj3: Gain knowledge about C constructs and it's associated eco-system

COBj4: Appreciate and gain knowledge about the issues with C Standards and its respective behaviors

COBj5: Get insights about testing and debugging C Programs

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs

CO2: Understand, analyse and apply text processing and string manipulation methods using C Arrays, Pointers and functions

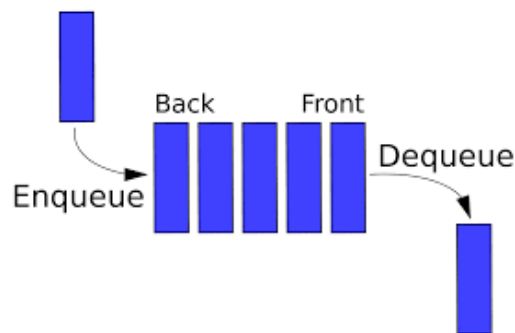
CO3: Understand prioritized scheduling and implement the same using C structures

CO4: Understand and apply sorting techniques using advanced C constructs

CO5: Understand and evaluate portable programming techniques using preprocessor directives and conditional compilation of C Programs

Introduction

Queue: A line or a sequence of people or vehicles waiting for their turn to be attended or to proceed. In computer Science, **a list of data items, commands, etc., stored so as to be retrievable in a definite order. A Non- Linear data structure which has 2 ends - Rear end and a Front end.** Data elements are inserted into the queue from the rear (back) end and deleted from the front end. Hence a queue is also known as **First In First Out (FIFO)** data structure.



Two major Operations on a queue:

- enqueue() – add (store) an item to the queue from rear end.
- dequeue() – remove (access) an item from the queue from front end.

Others operations may include,

- peek() – Gets the element at the front of the queue without removing it.
- isfull() – Checks if the queue is full.
- isempty() – Checks if the queue is empty.

Types of Queue:

- **Ordinary queue** - insertion takes place at rear and deletion takes place at front.
- **Priority queue** - special type of queue in which each element is associated with a priority and is served according to its priority. If elements with the same priority occur, they are served according to their order in the queue
- **Circular queue** - Last element points to first element of queue making circular link.
- **Double ended queue** - insertion and removal of elements can be performed from either from the front or rear.

Priority Queue

A data structure which is like a regular **queue** but where additionally each element has a "**priority**" associated with it. This priority decides about the deque operation. The enqueue operation stores the item and the priority information.

Types of Priority Queue:

- **Ascending priority queue:**
 - The smallest number, the highest priority.
- **Descending priority queue:**
 - The highest number, the highest priority.

Applications of Priority Queue:

1. Dijkstra's Algorithm
2. Prims Algorithm
3. Data Compression
4. Artificial Intelligence
5. Heap Sort

Priority Queue can be implemented using different ways.

- Using an Unordered Array
- Using an Ordered Array
- Using an Unordered Linked list
- Using an Ordered Linked list
- Using Heap



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE25CS151B**

***Lecture #1
File Handling in C***

**By,
Prof. Sindhu R Pai,
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Poojari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 4

Unit Name: File Handling and Portable Programming

Topic: File handling in C

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Dept. of CSE,

PES University

Introduction

Data created during program execution is stored temporarily in variables. However, once the program ends, this data is lost. To preserve data even after the program terminates, **files** are used. File handling allows a C program to **create, read, write, and modify files** stored on disk. This is crucial for tasks such as **saving user input, processing large datasets, logging system activities, or reading configuration files**.

File handling enables the below tasks:

- **Persistent data storage** beyond the life of a program.
- **Data sharing** between programs.
- **Handling large volumes of data** efficiently (which might not fit in memory)

File handling allows programs to **create, open, read, write, append, and close files** stored on the computer's storage device. File handling gives us the tools(functions) to manage the different types of files efficiently.

Real-Life Examples

1. Saving User Data in a Program
2. Creating Logs for a System - Record program events, errors, or status updates
3. Storing Game Scores - Save high scores or game progress that can be retrieved later.
4. Reading Configuration Settings - Load program preferences from a file during startup.
5. Writing Results from a Scientific Simulation
6. Handling Financial Transactions - Record billing or transaction data securely in a file.
7. Maintaining Audit Trails - Keep track of user actions and system access for security reviews.

File Classification

Files are referred to as data files in C. There are basically two distinct types of data files available in C.

- Text Files
- Binary Files

Text Files

The simplest type of file is a **text file**, typically created with a **.txt extension** using any basic text editor. Internally, **it stores data as ASCII characters**, making the content **easily readable by humans**. However, text files are not secure, as the information is stored in plain text, and they tend to occupy more storage space compared to binary files.

Binary Files

A binary file stores data using the binary number system (0s and 1s) and typically has a **.bin extension**. Since it saves information in the same format as it's stored in computer memory, it uses **less storage space**. Unlike text files, binary files are **not human-readable**, making them a safer option for **storing sensitive or structured data**.

Major Operations on Files

- Creation of a new file or Opening an existing file
- Reading data from a file
- Writing data in a file
- Closing a file
- Rewinding a File Pointer
- Moving the File Pointer
- Getting the Current File Position
- Checking for End-of-File
- Removing a File
- Renaming a File

To perform any operation on Files, the **physical filename**, the **logical filename** and the **mode** must be connected using `fopen()`

- **Physical Name:** A file is maintained by the OS. The OS decides the naming convention of a file.
- **Logical Name:** In a C Program, identifier is used to refer to a file. Also called as FileHandle.
- **Mode:** Can be read only, write only, append or a combination of these.

1. Creation of a new file or Opening an existing file

To create a new file, one has to use a **built-in FILE structure**. Create a pointer to FILE type. This must hold a logical reference to physical file on disk (hard disk) using the fopen() function.

```
FILE *fp;  
fp = fopen("filepath", "mode");
```

fopen(): Opens a file in the specified mode and **returns a FILE pointer — a reference to a structure that holds information about the file's attributes**. If the file is opened successfully, this pointer is returned; otherwise, NULL is returned. **File opening can fail for various reasons, such as the file not existing or lacking the necessary permissions for the requested mode**. The first argument, filePath, is a pointer to a string representing the name of the file to be opened. The second argument, mode, defines the type of access (read, write, append, etc.). It's important to note that initializing a FILE pointer does not load the entire file into memory — additional file handling functions must be used to access or modify the file's content.

A file in C can be opened using **different modes based on the desired operation**.

- **"r"** – **Opens an existing file for reading only**. The file must already be present; otherwise, the operation fails.
- **"w"** – **Opens a file for only writing**. If the file exists, its contents are erased. If it doesn't, a new empty file is created.
- **"a"** – **Opens a file to append data at the end**. If the file does not exist, it is created automatically.
- **"r+"** – **Opens an existing file for both reading and writing**. The file must exist; otherwise, the operation fails.
- **"w+"** – **Creates a new file for both reading and writing**. If the file exists, it is overwritten.
- **"a+"** – **Opens a file for both reading and appending**. Data is always added to the end of the file. If the file doesn't exist, it is created.

fclose(): When a file is opened, certain data structures are created in memory to manage the file operations. Once the file is no longer needed, these **resources should be released by closing the file using the fclose() function**. This action **closes the stream, flushes any remaining data in the buffers to the file, and frees associated memory**. If the file is closed **successfully, fclose() returns 0; otherwise, it returns EOF to indicate an error**.

Consider, **data.txt** contains the below data.
welcome to file
handling
in C

Coding Example_1: Demo of fopen and fclose.

```
#include<stdio.h>
int main()
{
    FILE* fp = fopen("data.txt","r"); //file must be existing to open in read mode. Else returns NULL
    if(fp== NULL)
        printf("file cannot be opened\n");
    else {
        //perform reading here in else using any functions .then close the file
        printf("File can be accessed for reading\n");
        fclose(fp);
    }
    return 0;
}
```



2. Read/Write data from/to a file

This can be categorized into the following types:

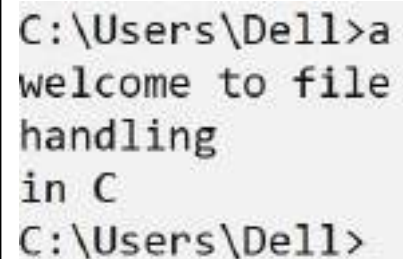
- **Character-based I/O** – Reading and writing one character at a time.
- **String-based I/O** – Handling entire lines or strings at once.
- **Formatted I/O** – Input and output using specific formats, similar to scanf() and printf().
- **Block I/O** – Reading from or writing to files in blocks of data, typically used for binary files.

Character based functions:

- **fputc(int ch, File_pointer):** Writes a character (an unsigned char) specified by the argument ch to the specified stream and advances the position indicator for the stream. If there are no errors, the same character that has been written is returned. If an error occurs, EOF is returned and the error indicator is set.
- **fgetc(file_pointer):** Reads a character from the file and returns the character read as an unsigned char cast to an int(ascii) and advances the file pointer to the next byte to be read or EOF on end of file **or error**.
- **getc(FILE *fp) and putc(char ch, FILE *fp):** The operation performed by getc() and putc() functions is the same as that performed by fgetc() and fputc() functions.

Coding Example_2: Demo of fgetc and fputc

```
#include<stdio.h>
int main()
{
    FILE* fp = fopen("data.txt","r");    // file must be existing
    if(fp== NULL)
        printf("file cannot be opened\n");
    else
    {
        char ch;
        while((ch=fgetc(fp)) != EOF)
        {
            //printf("character read is %c",ch);
            //printf("%c",ch);
            fputc(ch,stdout);// writing to standard output that is terminal
        }
        fclose(fp);
    }
    return 0;
}
```



```
C:\Users\Dell>a
welcome to file
handling
in C
C:\Users\Dell>
```

String based Functions:

- **fputs(const char *str, int n, FILE *fp):** Writes a string to the specified stream up to but not including the null character. Returns a non-negative value, or else on error it returns EOF.

- **fgets(char *str, FILE *fp):** Reads a line from the specified stream and stores it into the string pointed to by str. It stops when either (n-1) characters are read, the newline character is read, or the end-of-file is reached, whichever comes first. On success, the function returns the same str. On error or if EOF is encountered, it returns NULL.

Coding Example_3: Demo of fgets and fputs. out.txt created with the contents of data.txt

```
#include<stdio.h>
```

```
int main(){
```

```
    FILE *fp1 = fopen("data.txt","r");
```

```
    FILE *fp2 = fopen("out.txt","w");
```

```
    char ch[500];
```

```
    if(fp1==NULL || fp2 ==NULL)
```

```
        printf("issue in opening the file\n");
```

```
    else {
```

```
        while(fgets(ch,500,fp1) != NULL) // change 500 to 2 and check the output
```

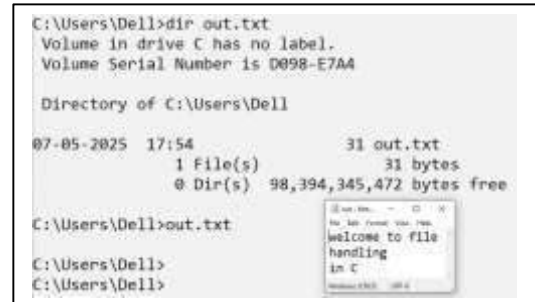
```
        {      fputs(ch, fp2);      }
```

```
        fclose(fp1); fclose(fp2);
```

```
    }
```

```
    return 0;
```

```
}
```



Formatted IO Functions:

When a data file is organized in a specific format—such as a line containing five integers separated by spaces—and the programmer knows this format in advance, it is more efficient to use formatted file I/O functions instead of reading character by character. These functions allow structured reading and writing based on expected data patterns. The `fscanf()` function works like `scanf()`, but it takes a file pointer as its first argument to read data directly from a file. `fprintf()` is similar to `printf()`.

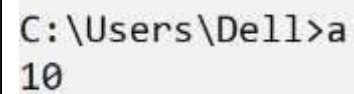
- **fprintf(file_pointer,"format_string", arg1,arg2,...):** Sends formatted output to a stream. If successful, the total number of characters written is returned otherwise, a negative number is returned .
- **fscanf(file_pointer,"format_string",&arg1,&arg2,...):** Reads formatted input from a stream and returns the number of input items successfully matched and assigned, which can be fewer than provided for, or even zero in the event of an early matching failure.

Assume, Formatted.txt file contains the below data.

```
10    20    30
40    10    20
```

Coding Example_4: Demo of fscanf and fprintf to read an integer from a file and write to the terminal

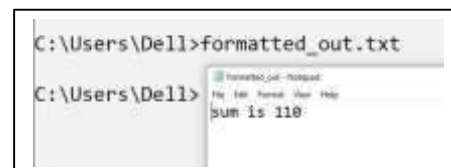
```
#include<stdio.h>
int main() {
    FILE *fp1 = fopen("Formatted_data.txt","r");
    if(fp1==NULL)
        printf("issue in opening the file\n");
    else {
        int a;          fscanf(fp1,"%d",&a);
        fprintf(stdout,"%d",a); fclose(fp1);
    }
    return 0;
}
```



Coding Example_5: Reading a set of integers from a file called numbers.txt and writing the sum of these integers to a new file.

Version 1: We know the number of integers to be read from the data file – n is hardcoded.

```
void read(int *a, int n, FILE *fp);
int find_sum(int *a, int n);
int main() {
    FILE *fp1 = fopen("formatted_data.txt","r");
    FILE *fp2 = fopen("formatted_out.txt","w");
    if(fp1==NULL || fp2 ==NULL)
        printf("issue in opening the file\n");
    else {
        int a[500];          int n = 5;
        read(a,n,fp1);
        int sum = find_sum(a,n);
        //fprintf(stdout,"sum is %d",sum);
        fprintf(fp2,"sum is %d",sum);
        fclose(fp1);    fclose(fp2);
    }
    return 0;
}
```



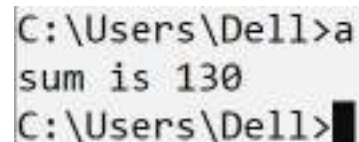
```
void read(int *a, int n, FILE *fp) {
    int i;
    for(i = 0; i < n; i++)
        fscanf(fp, "%d", a+i);
}
```

```
int find_sum(int *a, int n) {
    int i;
    int sum = 0;
    for(i = 0; i < n; i++)
        sum = sum + *(a+i);
    return sum;
}
```

Version 2: If we do not specify n in version 1, code fails. So use the return value of fscanf to find n

```
#include <stdio.h>
int read(int *a, FILE *fp);
int find_sum(int *a, int n);
int main() {
    FILE *fp1 = fopen("formatted_data.txt", "r");
    FILE *fp2 = fopen("formatted_out.txt", "w");
    if(fp1 == NULL || fp2 == NULL)
        printf("issue in opening the file\n");
    else {
        int a[500];
        int n = read(a, fp1);
        int sum = find_sum(a, n); // same as version1
        fprintf(stdout, "sum is %d", sum);
        fclose(fp1); fclose(fp2);
    }
    return 0;
}
```

```
int read(int *a, FILE *fp)
{
    int i = 0;
    while (fscanf(fp, "%d", a+i) != EOF)
        i++;
    return i;
}
```



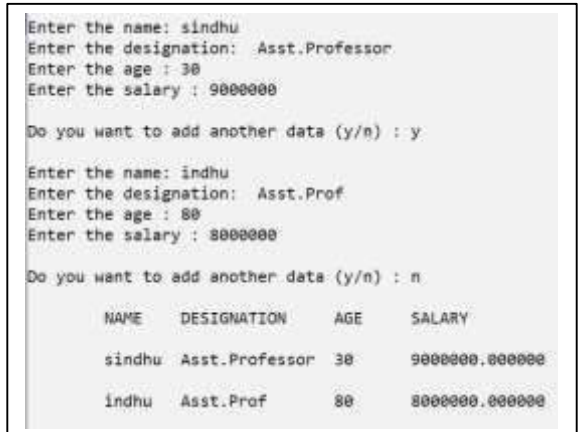
```
C:\Users\De11>a
sum is 130
C:\Users\De11>
```

Block read / write operation on File

- **fwrite(const void *ptr, size_t size, size_t n, FILE *fp):** This function writes the data specified by the **void pointer ptr** to the file. **On success, it returns the count of the number of items successfully written to the file. On error, it returns a number less than n.**
- **fread(void *ptr, size_t size, size_t n, FILE *fp):** This function is commonly used to read binary data. It accepts the same arguments as the fwrite() function does. The function reads n items from the file where each item occupies the number of bytes specified in the second argument. **The function on success reads n items from the file and returns n and on error or end of the file, it returns a number less than n.**

Coding Example_6: Demo of fwrite and fread functions. Employee details to be read from the user and writing a data file.

```
#include<stdio.h>
#include<stdlib.h>
struct employee
{
    char name[15];      char designation[15];      int age;      float salary;
}emp;
int main()
{
    FILE *fp;
    fp = fopen("employee_data.txt", "w"); // at the end of the program execution, open and
    see the contents of the data file. Not in readable format
    if(fp == NULL)
        printf("Error opening file\n");
    else
    {
        char ch;
        do
        {
            printf("\nEnter the name:\t");
            scanf("%s",emp.name);
            printf("Enter the designation:\t");
            scanf("%s",emp.designation);
            printf("Enter the age : ");
            scanf("%d",&emp.age);
```



```
Enter the name: sindhu
Enter the designation: Asst.Professor
Enter the age : 30
Enter the salary : 9000000

Do you want to add another data (y/n) : y

Enter the name: indhu
Enter the designation: Asst.Prof
Enter the age : 80
Enter the salary : 8000000

Do you want to add another data (y/n) : n
```

NAME	DESIGNATION	AGE	SALARY
sindhu	Asst.Professor	30	9000000.000000
indhu	Asst.Prof	80	8000000.000000

```
        printf("Enter the salary : ");
        scanf("%f",&emp.salary);
        fwrite(&emp,sizeof(emp),1,fp);
        printf("\nDo you want to add another data (y/n) : ");
        fflush(stdin);
        ch = getchar();
    } while(ch=='y' || ch=='Y');
    fclose(fp);
    fp = fopen("employee_data.txt","r");
    if(fp == NULL)
        printf("cannot open the file\n");
    else
    {
        printf("\n\tNAME\tDESIGNATION\tAGE\tSALARY\n");
        while(fread(&emp,sizeof(emp),1,fp)>0)
            printf("\n\t%s\t%s\t%d\t%f\n",emp.name,emp.designation
            ,emp.age,emp.salary);
        fclose(fp);
    }
}
return 0;
}
```

Random Access to a file

You can jump to a specific position in the file and read/write data — without reading everything sequentially. Files permit non-sequential or random access to its contents using C functions. To access a file randomly, you open the file, seek a particular location, and read from or write to that file. In order to access a particular file or record C supports below functions.

fseek(), ftell(), rewind()

- **int fseek(FILE *stream, long offset, int whence);**

Used to set the file position indicator for the stream to a new position. This function accepts three arguments. The first argument is the FILE stream pointer returned by the fopen() function. The second argument 'offset' tells the amount of bytes to seek. The third argument 'whence' tells from where the seek of 'offset' number of bytes is to be done. The available values for whence are as below. Returns zero if successful, else it returns a non-zero value.

	Value	whence
SEEK_SET	0	Beginning of file
SEEK_CUR	1	Current position
SEEK_END	2	End of file

- **ftell(FILE *stream)**

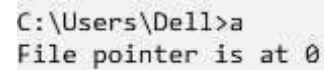
Returns the current file position of the given file_pointer. If an error occurs, -1 is returned.

- **rewind(FILE *stream)**

The function rewind sets the file_pointer position to the beginning of the file. Doesn't return any value.

Coding Example_7: Demo of ftell () . Pointer is at the beginning of the file just after opening the file in read mode

```
#include<stdio.h>
int main() {
    FILE *fp = fopen("data.txt","r"); // try w and a as well
    if(fp == NULL) printf("cannot open the file");
    else {
        printf("File pointer is at %d\n",ftell(fp));
        fclose(fp);
    }
    return 0;
}
```



Coding Example_8: Demo of fseek()

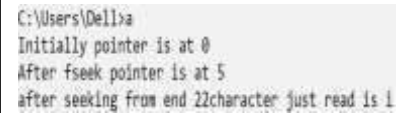
```
#include<stdio.h>
int main()
{
    FILE *fp = fopen("data.txt","r");
    if(fp == NULL)
        printf("cannot open the file");
    else
    {
        printf("Initially pointer is at %d\n",ftell(fp));
        fseek(fp,5,SEEK_SET); // start from the beginning
        printf("After fseek pointer is at %d\n",ftell(fp)); //5
        //fseek(fp,2,SEEK_SET);
        //printf("%d",ftell(fp)); // 2
    }
}
```



```

        //fseek(fp,2,SEEK_CUR); //start from the current position of the pointer
        //printf("%d",ftell(fp)); //7
        fseek(fp,-9,SEEK_END); //start from the end of the file
        printf("after seeking from end %d",ftell(fp));
        printf("character just read is ");
        putchar(fgetc(fp));
        fclose(fp);
    }
    return 0;
}

```



Coding Example_9: Copy the contents of one File to another

```

#include<stdio.h>
int main() {
    FILE *fp1 = fopen("data.txt","r");
    FILE *fp2 = fopen("data_new.txt","w");
    char ch;
    while((ch = fgetc(fp1)) != EOF)
        fputc(ch, fp2);    // copy character by character from 5th position
    fclose(fp1); fclose(fp2);
    return 0;
}

```



Coding Example_10: Usage of rewind()

```

#include<stdio.h>
int main() {
    FILE *fp = fopen("data.txt","r");
    if(fp == NULL)
        printf("cannot open the file");
    else{
        fseek(fp,-5,SEEK_END);    printf("%d\n",ftell(fp));
        fseek(fp,0,SEEK_SET)//rewind(fp); to move the pointer to the beginning
of the file.
        printf("%d\n",ftell(fp));    fclose(fp);
        }return 0;
}

```



Handle Files, Handle Data – Happy Coding!



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE25CS151B**

***Lecture #2
Error handling in File Handling***

**By,
Prof. Sindhu R Pai,
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 4

Unit Name: File Handling and Portable Programming

Topic: Error handling in File Handling

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Dept. of CSE,

PES University

Introduction

Errors are the problems or the faults that occur in the program, which makes the behavior of the program abnormal, and experienced programmers and developers can also make these faults. **Programming errors** are also known as the **bugs or faults**.

When working with files in C, it's crucial to ensure that operations such as **opening, reading, writing, or closing a file are executed safely**. However, various factors such as **missing files, access restrictions, or disk issues can cause these operations to fail**. If such failures are not handled properly, they may lead to incorrect program behavior, crash, or data loss. The most common errors that may occur are:

1. **Reading beyond the end-of- file:** EOF is a condition in a computer operating system where no more data can be read from a data source. The data source is usually a file or stream.
2. **Performing operations on the file that has not still been opened:** Before performing any operation on a file, you must first open it. An attempt to perform operations on a file that has not been opened yet will lead to error
3. **Writing to a file that is opened in the read mode:** A mode is used to specify whether you want to open a file to perform operations like read, write append etc.
4. **Opening a file with invalid filename:** If you attempt to upload a file that has an invalid file name or type you will receive error.
5. **Write to a write-protected file:** Write-protected file is the ability to prevent new information from being written or old information being changed. In other words, information can be read, but nothing can be added or modified.

Error handling allows the program to **detect if a file operation failed, respond gracefully (e.g., display an error message, attempt recovery, or exit safely)** and **maintain robustness and user-friendliness**. It helps during the processing of input-output statements by catching severe errors that might not otherwise be noticed. In C, standard library functions provide return values and flags that can be checked to identify and manage such issues effectively.

Types of Error Handling in C

C does not have built-in exception handling features. Instead, **it uses manual error handling techniques**. These can be broadly categorized into:

1. **Return value checking (Most Common in C):** Most functions return a value that indicates whether they succeeded or failed. These values must be explicitly checked to detect errors and handle them properly. This helps avoid unexpected behavior or program crashes by ensuring that operations only proceed when they are safe. This practice also simplifies the debugging process by making it easier to identify where and why something went wrong.

Case 1: `fopen()` returns a NULL pointer on failure.

```
FILE *fp = fopen("data.txt", "r");
if (fp == NULL) {
    printf("Error opening file");
    exit(1);
}
```

Case 2: `fscanf()` returns the number of items successfully read. Check if the expected number of items was read.

```
int num;
if (fscanf(fp, "%d", &num) != 1) {
    fprintf(stderr, "Error reading integer from file\n");
}
```

Case 3: closing File safely

```
if (fclose(fp) != 0) {
    perror("Error closing file");
}
```

PS: Think of return code checking like **checking a receipt after shopping** — it helps you know if everything went right or if there was a billing error.

2. **Global Variable errno:** C programming being a system programming language, it provides you access at lower level in the form of return values from the functions. Most of the C function calls return -1 or NULL in case of any error and set an error code **global variable errno**. When a function is called in C, a variable named as errno is automatically assigned a code (value) which can be used to identify the type of error that has been encountered. Its a global variable **indicating the error occurred during any function call** and defined in the header file **errno.h**. Different codes (values) for errno mean different types of errors.

errno value	Type of Error
0	No Error
1	Operation not permitted
2	No such file or directory
5	I/O error
7	Argument list too long
9	Bad file number
11	Try again
12	Out of memory
13	Permission denied

The errno value indicate the types of error encountered. If you want to show the **error description**, there are **two functions that can be used to display a text message that is associated with errno - perror() and strerror()**

3. **perror() function:** The function perror() **stands for print error**. In case of an error, the programmer can determine the type of error that has occurred using the perror() function. It displays **the string you pass to it, followed by a colon, a space, and then the textual representation of the current errno value**.

Syntax: void perror (const char *str) // (str: is a string containing a custom message to be printed before the error message itself.)

Coding Example_1: Demo of perror()

```
#include <stdio.h>
#include <errno.h>
#include <string.h>
int main () {
    FILE *fp = fopen ("data.txt", "r"); // File with this name exist
    //FILE *fp = fopen ("data1.txt", "r"); // File with this name doesn't exist
    printf("Value of errno: %d\n", errno);
    perror("Bad code");
    return 0;
}
```

```
C:\Users\Dell>a
Value of errno: 0
Bad code: No error
```

```
C:\Users\Dell>a
Value of errno: 2
Bad code: No such file or directory
```

4. **strerror() function:** The function used to get a **human-readable error message**. Returns a pointer to the textual representation of the current error code usually stored in the global variable `errno`. It is important to note that the **returned string is statically allocated** and should not be modified.

Syntax: `char *strerror(int errnum);`

Coding Example_2: Demo of strerror()

```
#include <stdio.h>
#include <string.h>
#include <errno.h>
int main() {
    FILE *fp = fopen("data1.txt", "r"); // Trying to open a file that may not exist
    if (fp == NULL) {
        printf("File open failed: %s\n", strerror(errno));
    }
    return 0;
}
```

```
C:\Users\Dell>a
File open failed: No such file or directory
```

Coding Example_3: Usage of perror() and strerror() while handling illegal operations on the file

```
#include <stdio.h>
#include <errno.h>
#include <string.h>
int main ()
{
    FILE *fp;
    fp = fopen ("data.txt", "r"); // File with this name exist
```

```
C:\Users\Dell>a
value of errno is 9 with info:Bad file descriptor
Bad code: Bad file descriptor
```

```
    fputc('A',fp); // writing to a file which is opened for read
    printf("value of errno is %d with info:%s\n",errno,strerror(errno));
    perror("Bad code");
    fclose(fp);
    return 0;
}
```

To ensure safe and error-free file operations in C, two standard library functions are provided to monitor the status of a file stream. These functions help detect specific conditions such as reaching the end of a file or encountering an error during input/output operations. The two commonly used status-checking functions are:

1. **fEOF()** – This function checks if the end of the file (EOF) has been reached. It is typically used in loops to prevent reading past the file's end.
2. **ferror()** – This function checks whether an error has occurred during a file operation, such as reading from or writing to the file. It helps identify issues like invalid file access or hardware failures.

fEOF():

In C, `getc()` returns EOF when end of file is reached. `getc()` also returns EOF when it fails. So, only comparing the value returned by `getc()` with EOF is not sufficient to check for actual end of file. To solve this problem, C provides `fEOF()` function used to check whether the end of a file has been reached during a file read operation. This **returns a non-zero value only if end of file has reached, otherwise it returns 0.**

Syntax: `int fEOF(FILE *stream);`

Coding Example_4: Usage of fEOF() in loops to detect when a read operation failed.

```
#include <stdio.h>
int main ()
{
    FILE *fp = fopen("data.txt", "r");
    char ch;
    if (fp != NULL) {
        while (!fEOF(fp)) {
            ch = fgetc(fp);
            if (!fEOF(fp)) // check again after reading
                putchar(ch);
        }
    }
}
```



Content of file has been read and displayed.


```
    }  
}  
fclose(fp);  
return 0;  
}
```

feof() does not predict or detect EOF in advance. Instead, it checks if the last file operation **attempted to read beyond the end of the file. Best Practice** is instead of checking feof() before reading, check the return value of the read function itself (like fgetc() or fscanf()), and use feof() only to confirm EOF after a failed read.

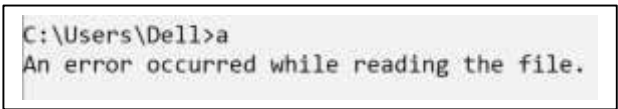
ferror():

This function checks for any error in the stream occurred during input or output operations. **It returns a value zero if no error has occurred and a non-zero value if there is an error.** The error indication will last until the file is closed unless it is cleared by the clearerr() function.

Syntax: int ferror (FILE *file_pointer);

Coding Example_5: Usage of ferror() to find the illegal operation on the file stream.

```
#include <stdio.h>  
int main() {  
    FILE *fp = fopen("data1.txt", "w"); // file is opened in write mode  
    char buffer[100];  
    if (fp != NULL) { // trying to read - illegal operation on the stream  
        if (fgetc(buffer, sizeof(buffer), fp) != NULL) {  
            printf("Read line: %s", buffer);  
        } else {  
            if (ferror(fp)) {  
                printf("An error occurred while reading the file.\n");  
            }  
        }  
    }  
    fclose(fp);  
}  
return 0;  
}
```



The `ferror()` is useful when a read or write operation fails. Instead of assuming the cause is end-of-file (EOF), this function helps determine whether the failure was due to an actual error such as:

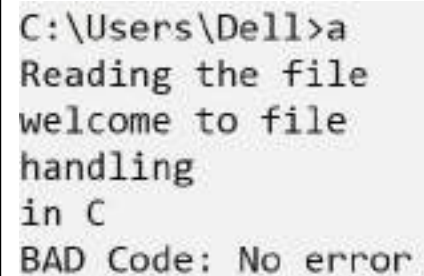
- Reading from a corrupted file
- Writing to a full disk
- Accessing a file in an invalid mode

Best practice is that the `ferror()` should only be used after performing input or output operations such as `fgets()`, `fscanf()`, `fread()`, or `fwrite()`. It checks whether an error occurred during those operations and returns a non-zero value if an error has indeed happened. **It does not initiate the check by itself—it must follow a failed I/O attempt.** Additionally, because `ferror()` only signals that an error occurred without giving details, it's **recommended to use it in combination with `perror()` or `strerror(errno)` to retrieve a human-readable error message.** This is particularly helpful for debugging or logging errors for user-friendly messages. To distinguish between an **end-of-file condition** and a **real error**, it's also good practice to use `feof()` alongside `ferror()`.

Coding Example_6: Displaying the contents of the file on the terminal.

Version 1: Reading after encountering EOF

```
#include<stdio.h>
int main()
{
    FILE *fp; char c;
    fp = fopen ( "data.txt", "r" ); // opening an existing file
    if ( fp != NULL )
    {
        printf( "Reading the file\n" );
        while ( !feof(fp) ) // returns nonzero if EOF encountered
        {
            c = fgetc(fp) ; // reading the file
            printf ("%c",c) ;
        }
        c = getc(fp); // doesn't show any error. so better to use feof function. See Version 2
        perror("\nBAD Code");
        fclose ( fp ) ; // Closing the file
    }
    return 0;
}
```



Version 2:

```
printf( "Reading the file\n" ) ;
while ( !feof(fp) ) // returns nonzero if EOF encountered
{
    c = fgetc(fp) ; // reading the file
    printf ("%c",c) ;
}
if(feof(fp))
{
    c = getc(fp);
    printf("\nEnd of file has reached. So illegal to read more contents\n");
}
fclose ( fp ) ; // Closing the file
```

**clearerr():**

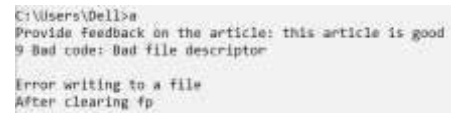
It is used to reset **the error and EOF indicators for a file stream**. After an error or end-of-file condition occurs on a stream, further operations might not proceed as expected until these indicators are cleared.

Coding Example_7: Writing to a file which is opened in read mode

```
#include <stdio.h>
#include <errno.h>
int main() {
    FILE* fp;
    char feedback[100];
    fp = fopen("data.txt", "r"); // opened in r mode. File must be existing
    printf("Provide feedback on the article: ");
    fgets(feedback, 100, stdin);
    for (int i = 0; i < feedback[i]; i++)
        fputc(feedback[i], fp); // writing to a file character by character
    printf("%d",errno);
    perror("Bad code"); //writing to a file is not allowed as the mode used is r while opening
    if (ferror(fp))
        printf("\nError writing to a file");
}
```

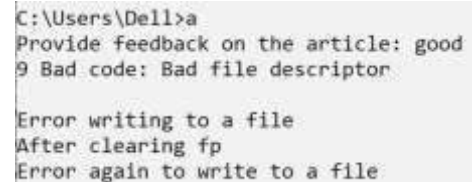
clearerr(fp); // error indication is stopped . next ferror() is returning 0

```
printf("\nAfter clearing fp\n");  
if (ferror(fp)) {  
    printf("Error again to write to a file\n");  
}  
return 0;  
}
```



```
C:\Users\Dell>a  
Provide feedback on the article: this article is good  
9 Bad code: Bad file descriptor  
  
Error writing to a file  
After clearing fp
```

Think about this - > If we comment clearerr(fp), what happens?



```
C:\Users\Dell>a  
Provide feedback on the article: good  
9 Bad code: Bad file descriptor  
  
Error writing to a file  
After clearing fp  
Error again to write to a file
```

The best practice is to use clearerr() **only after you have properly handled the error or end-of-file (EOF) condition**. This ensures that you're not ignoring unresolved issues that may affect the correctness of your program. This allows the file stream to be safely reused for further read or write operations without being blocked by previously set indicators.

Best practices to be followed during File handling

- Always Check File Pointers
- Validate Return Values of I/O Functions
- Use ferror() and feof() appropriately
- Clear Errors if needed
- Always Close Files
- Provide User-Friendly Error Messages
- Avoid Hard coding File Paths

Happy Error-Free File Handling implementations!!



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE25CS151B**

***Lecture #4
Searching on the file data using array of pointers***

**By,
Prof. Sindhu R Pai,
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit # 4

Unit Name: File Handling and Portable Programming

Topic: Searching on the file data using array of pointers

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Dept. of CSE,

PES University

Introduction

When working with files in C, efficient data handling is crucial, especially when the file contains multiple entries such as lines, records, or words. One powerful technique is using an **array of pointers** to store and access the data dynamically. This approach **allows flexible memory management, faster search operations, and easier data manipulation without altering the original file content**. By storing each line or record in a dynamically allocated memory block and keeping track of these blocks through an array of pointers, we can perform searches or any modifications in-memory, leading to optimized and structured file processing.

Let us consider a simple text file called numbers.txt which contains many integers in an ascending order, separated by any white space in between. The file is well structured and very well formatted. The requirement is to search for the given integer in the file and display appropriate message.

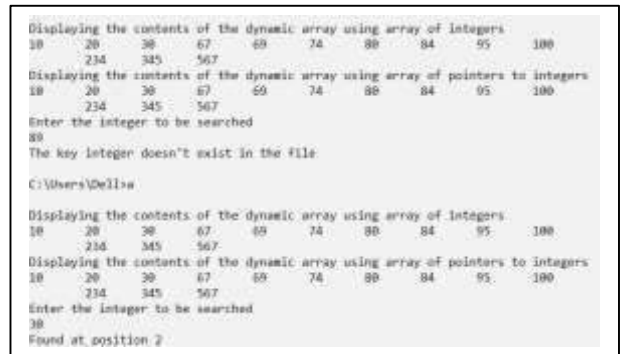
Coding Example_1: Recursive binary search on an array having the integers from the file.

```
#include <stdio.h>
#include <stdlib.h>
int Search(int **aop, int low, int high, int key);
int main(){
    FILE* fp;
    fp = fopen("numbers.txt", "r");
    if(fp == NULL)
        printf("Can't open the file\n");
    else {
        int a;          int n = 0;
        while(fscanf(fp, "%d", &a) != -1)
            n++;
        int *p = malloc(n*sizeof(int));
        int **aop = (int **)malloc(n * sizeof(int *));
        for(int i = 0; i < n; i++)
            aop[i] = &p[i];
        rewind(fp);
        int i = 0;
        while(fscanf(fp, "%d", &p[i]) != -1)
            i++;
        fclose(fp);
```

```

printf("\nDisplaying the contents of the dynamic array using array of integers\n");
for(int i = 0 ; i < n; i++)
    printf("%d\t", p[i]);
printf("\nDisplaying the contents of the dynamic array using array of pointers to
integers\n");
for(int i = 0 ; i < n; i++)
    printf("%d\t", *aop[i]);
printf("\nEnter the integer to be searched\n");
int key_int;
scanf("%d", &key_int);
int res = Search(aop,0, n, key_int);
if(res == -1)
    printf("The key integer doesn't exist in the file\n");
else
    printf("Found at position %d", res);
}
return 0;
}
int Search(int **aop, int low, int high, int key){
    if(low > high) // base condition
        return -1;
    else{
        int mid = (low+high)/2;
        if(*aop[mid] == key)
            return mid;
        else if(key < *aop[mid])
            return Search(aop,low,mid-1,key);
        else
            return Search(aop,mid+1,high,key);
    }
}

```



```

Displaying the contents of the dynamic array using array of integers
10      20      30      67      69      74      80      84      95      100
      234      345      567
Displaying the contents of the dynamic array using array of pointers to integers
10      20      30      67      69      74      80      84      95      100
      234      345      567
Enter the integer to be searched
89
The key integer doesn't exist in the file

C:\Users\De11>

Displaying the contents of the dynamic array using array of integers
10      20      30      67      69      74      80      84      95      100
      234      345      567
Displaying the contents of the dynamic array using array of pointers to integers
10      20      30      67      69      74      80      84      95      100
      234      345      567
Enter the integer to be searched
90
Found at position 2

```

Now let us consider the array of structures and array of pointers to array of structures. Given data file - > **student_rollno_name.csv**. If the roll_number of the student is given, display the details of that student.

Data file Description:

- Contains the information of students in two columns only.
- First row represents the column headers such as roll_number and name.
- Contains around 55 rows. Every data is stored with a comma in between.

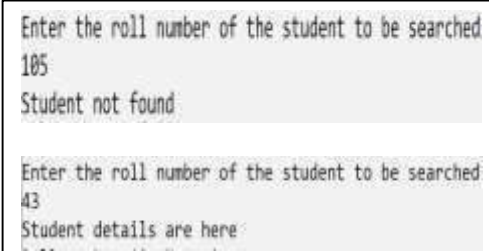
Coding Example_2: Searching the details of student based on roll_number

```
typedef struct student
{
    int roll;      char name[20];    }STUDENT;
void init_ptr(struct student s[], struct student *p[], int n);
void disp(struct student* p[], int n) ;
int Search(STUDENT **aop, int low, int high, int key);
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
int main(){
    FILE *fr=fopen("sorted_data.csv","r");
    if(fr == NULL) {
        printf("Cant access the file\n");exit(1);
    }
    char a[200];    fgets(a,200,fr);      char *item; int i=0;
    struct student s[100];                // Array of structures
    // Reading a line from the file and splitting it based on the delimiter comma(,), copying the details
    to array of student structure
    while(fgets(a,200,fr)) {
        item=strtok(a,",");      s[i].roll=atoi(item);
        item=strtok(NULL,",");   strcpy(s[i].name,item);
        i++;
    }
    int n = i;
    fclose(fr);
    struct student *p[100]; //Creation of array of pointers to point to array of structures
    init_ptr(s, p, n); // every element of array of pointer is made to point to every element of
    array of structure
    disp(p,n);      // to display the array of structures using array of pointers
    int key_rollno;
    printf("Enter the roll number of the student to be searched\n");
    scanf("%d",&key_rollno);
    int res = Search(p, 0, n-1, key_rollno);
    if(res == -1)
        printf("Student not found\n");
    else{
        printf("Student details are here\n");
        printf("Roll_number %d, Name is %s",p[res]->roll, p[res]->name);
    }
    return 0;
}
```

```
int Search(STUDENT **aop, int low, int high, int key){
    if(low > high) // base condition
        return -1;
    else {
        int mid = (low+high)/2;
        if(aop[mid]->roll == key)
            return mid;
        else if(key < aop[mid]->roll)
            return Search(aop,low,mid-1,key);
        else
            return Search(aop,mid+1,high,key);
    }
}

void init_ptr(struct student s[], struct student *p[], int n){
    for(int i = 0 ; i < n; i++)
        p[i] = &s[i];
}

void disp(struct student* p[], int n) {
    for(int i = 0 ; i < n; i++)
        printf("%d %s", p[i]->roll, p[i]->name);
}
```



```
Enter the roll number of the student to be searched
105
Student not found

Enter the roll number of the student to be searched
43
Student details are here
```

Think about this-> How to compare the strings if user wants to enter the student name and get other details of the student?

Happy Searching file contents using Array of Pointers!!



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE25CS151B**

***Lecture #5
Sorting file data using array of pointers***

**By,
Prof. Sindhu R Pai,
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 4

Unit Name: File Handling and Portable Programming

Topic: Sorting file data using array of pointers

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Dept. of CSE,

PES University

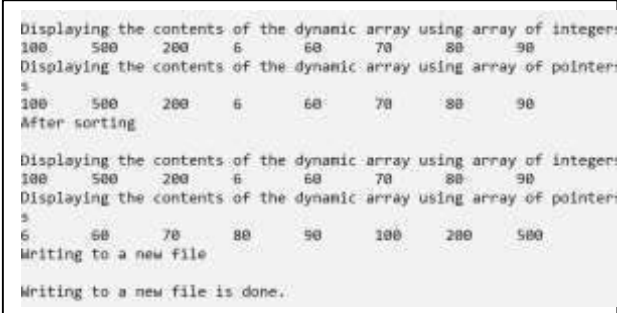
Introduction

When working with text files, sorting the content such as integers or words or line of strings can be efficiently managed using an **array of pointers**. Instead of moving entire content during sorting, we manipulate the pointers to those contents, which **reduces memory operations and improves performance**. This approach involves reading file contents into dynamically allocated memory blocks, storing addresses of these blocks in an array of pointers, and then applying a sorting algorithm (like selection sort) on the pointer array. This technique is especially useful for handling large text files with multiple lines or records, offering **both speed and flexibility in data processing**.

Let us consider a simple text file called numbers.txt which contains a lot of integers in a well structured manner. The requirement is to sort the integers in the file and write the sorted collection of integers to another file.

Coding Example_1:

```
#include <stdio.h>
#include <stdlib.h>
void swap( int** lhs, int** rhs );
void sort(int **aop, int n);
int main() {
    FILE* fp;
    fp = fopen("numbers.txt", "r");
    if(fp == NULL)
        printf("Can't open the file\n");
    else {
        int a;          int n = 0;
        while(fscanf(fp, "%d", &a) != -1)
            n++;
        int *p = malloc(n*sizeof(int));
        int **aop = (int **)malloc(n * sizeof(int *));
        for(int i = 0; i < n; i++)
            aop[i] = &p[i];
        rewind(fp);
        int i = 0;
        while(fscanf(fp, "%d", &p[i]) != -1)
            i++;
    }
```



```
Displaying the contents of the dynamic array using array of integers
100  500  200  6  60  70  80  90
Displaying the contents of the dynamic array using array of pointers
100  500  200  6  60  70  80  90
After sorting
Displaying the contents of the dynamic array using array of integers
100  500  200  6  60  70  80  90
Displaying the contents of the dynamic array using array of pointers
6  60  70  80  90  100  200  500
Writing to a new file
Writing to a new file is done.
```

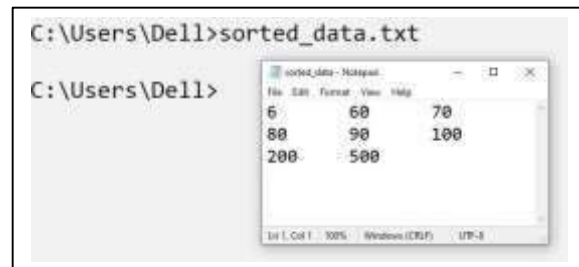
```

printf("\nDisplaying the contents of the dynamic array using array of integers\n");
for(int i = 0 ; i < n; i++)
    printf("%d\t", p[i]);
printf("\nDisplaying the contents of the dynamic array using array of pointers to
integers\n");
for(int i = 0 ; i < n; i++)
    printf("%d\t", *aop[i]);
sort(aop, n);
printf("\nAfter sorting\n");
printf("\nDisplaying the contents of the dynamic array using array of integers\n");
for(int i = 0 ; i < n; i++)
    printf("%d\t", p[i]);
printf("\nDisplaying the contents of the dynamic array using array of pointers to
integers\n");
for(int i = 0 ; i < n; i++)
    printf("%d\t", *aop[i]);
printf("\nWriting to a new file\n");
FILE *fpw = fopen("sorted_data.txt", "w");
for(int i = 0 ; i < n; i++)
    fprintf(fpw, "%d\t", *aop[i]);
printf("\nWriting to a new file is done.\n");
fclose(fp);
fclose(fpw);
}
return 0;
}

void sort(int **aop, int n){
    int i,pos,j;
    for(i = 0;i<n-1;i++)    {
        pos = i;
        for(j = i+1;j<n;j++)    {
            if(*aop[pos] > *aop[j])        pos = j;
        }
        if(pos != i)
            swap(&aop[pos],&aop[i]); // Swap the pointers not the integers in the array
    }
}

void swap( int** lhs, int** rhs){
    int* temp = *lhs;    *lhs = *rhs;    *rhs = temp;
}

```



Now let us consider the array of structures and array of pointers to array of structures. Given a data file **student_rollno_name.csv**, create a menu driven program to perform selection sort based on roll_number and name. Write the sorted record to a new file.

Data file Description:

- Contains the information of students in two columns only.
- First row represents the column headers such as roll_number and name.
- Contains around 55 rows. Every data is stored with a comma in between.

Coding Example_2:

```
struct student
{
    int roll;
    char name[20];
};

void init_ptr(struct student s[], struct student *p[], int n);
void swap( struct student** lhs, struct student** rhs);
void disp(struct student* p[], int n);
void selection_sort_roll_no(struct student* s[],int n);
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
int main() {
    FILE *fr=fopen("student_rollno_name.csv","r");
    char a[200]; fgets(a,200,fr);
    char *item;
    struct student s[100]; int i=0;
    // Reading a line from the file and splitting it based on the delimiter comma(,),
    copying the details to array of student structure
    while(fgets(a,200,fr)) {
        item=strtok(a,"");
        s[i].roll=atoi(item);
        item=strtok(NULL,"");
        strcpy(s[i].name,item);
        i++;
    }
    int n = i;
    fclose(fr);
    struct student *p[100]; //Creation of array of pointers to point to array of structures
    init_ptr(s, p, n); // every element of array of pointer is made to point to every element of
    array of structure
    disp(p,n); // to display the array of structures using array of pointers
```

// Menu driven code

```
int ch;
printf("enter the choice.\n 1.sort on roll\n");
printf("2. sort on name");
scanf("%d",&ch);
switch(ch) {
    case 1:selection_sort_roll_no(p,n); // array of pointer and n is the argument
            disp(p,n); break;
    case 2:selection_sort_names(p,n); disp(p,n); break; // same as above
    default: printf("exiting from the program"); exit(0);
}
```

// Writing the sorted record to a new file

```
FILE *fw=fopen("student_sorted.csv","w");
fprintf(fw,"Roll_number,Name\n");
i=0;
while(i<n){
    fprintf(fw,"%d,%s",p[i]->roll,p[i]->name); i++;
}
fclose(fw);
printf("sorted data is written to a file student_sorted.csv\n");
return 0;
}
```

// Initializing the pointer

```
void init_ptr(struct student s[], struct student *p[], int n){
    for(int i = 0; i < n; ++i)
        p[i] = &s[i];
}
```

// Swapping the pointers

```
void swap( struct student** lhs, struct student** rhs) {
    struct student* temp = *lhs; *lhs = *rhs; *rhs = temp;
}
```

//Displaying using array of pointers

```
void disp(struct student* p[], int n) {
    for(int i = 0; i < n; ++i)
        printf("%d %s\n", p[i]->roll, p[i]->name);
    //printf("%s %d\n", (s+i)->name, (s+i)->roll);
}
```


//Selection sort using roll_no on array of pointers to array of structures

```
void selection_sort_roll_no(struct student *s[],int n){
    int i,pos,j;
    for(i = 0;i<n-1;i++)    {
        printf("in for\n");
        pos = i;
        for(j = i+1;j<n;j++)    {
            if((s[pos]->roll) > (s[j]->roll)) // compare the roll numbers using pointers to
sort the record
                pos = j;
        }
        if(pos != i)
            swap(&s[pos],&s[i]);    // Swap the pointers not the structures
    }
}
```

//Selection sort using name on array of pointers to array of structures

```
void selection_sort_names(struct student* s[],int n){
    int i,pos,j;
    for(i = 0;i<n-1;i++)    {
        pos = i;
        for(j = i+1;j<n;j++)    {
            if(strcmp(s[pos]->name,s[j]->name)>0)
                pos = j; // compare the names using pointers to sort the record
        }
        if(pos != i)
            swap(&s[pos],&s[i]);
    }
}
```

Happy Sorting file contents using Array of Pointers!!



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE25CS151B**

***Lecture #7
Creation and Usage of Header files***

**By,
Prof. Sindhu R Pai,
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 4

Unit Name: File Handling and Portable Programming

Topic: Creation and Usage of Header files

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Dept. of CSE,

PES University

Introduction

In C programming, header files serve as a way to organize code for better **modularity, readability, and reusability**. A header file typically contains **function declarations, macro definitions, constants, and structure or type definitions that are used across multiple source files**. C provides several **predefined header files** like `<stdio.h>`, `<math.h>`, and also allows you to create **custom header files** to include user-defined functions and types. **Header files are included in the program using the `#include` directive**.

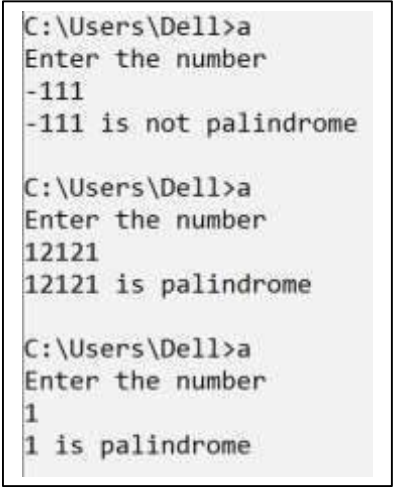
Advantages of using header files:

- Avoiding rewriting the code across different .c files.
- Separating declarations(interface) from definitions(implementation), keeping code clean and organized.
- Easier maintenance and collaboration in large projects.

Let us consider an example code related to **finding whether the number is a palindrome or not**.

Coding Example_1: All operations are in one single file.

```
#include<stdio.h>
int main()
{
    int n;
    printf("Enter the number\n");
    scanf("%d", &n);
    int rev = 0;    int number = n;
    while(n>0) // can be just while(n).
    {
        rev = rev * 10 + n % 10; n /= 10;
    }
    if(number == rev)
        printf("%d is palindrome\n",number);
    else
        printf("%d is not palindrome\n",number);
    return 0;
}
```



```
C:\Users\Dell>a
Enter the number
-111
-111 is not palindrome

C:\Users\Dell>a
Enter the number
12121
12121 is palindrome

C:\Users\Dell>a
Enter the number
1
1 is palindrome
```

Coding Example_2: Separating the interface and implementation - Function to check whether the given number is palindrome or not.

```
include<stdio.h>
int is_palin(int); // interface
int main(){
    int n;
    printf("Enter the number\n");
    scanf("%d", &n);
    if(is_palin(n))
        printf("%d is a palindrome\n", n);
    else
        printf("%d is not a palindrome\n", n);
}
int is_palin(int n) // implementation
{
    int rev = 0;
    int number = n;
    while(n>0){
        rev = rev * 10 + n % 10; n /= 10;
    }
    return number == rev;
}
```

```
C:\Users\Dell>a
Enter the number
123
123 is not a palindrome

C:\Users\Dell>a
Enter the number
-3443
-3443 is not a palindrome

C:\Users\Dell>a
Enter the number
15351
15351 is a palindrome
```

Now let us create separate files for interface and implementation to follow modular programming practices and enable code reuse and maintainability.

Coding Example_3: Creation of header file. Separating the client and server.

//palin.h

int is_palin(int); // interface

//client code – palin_client.c

#include<stdio.h>

```
int main(){
    int n;
    printf("Enter the number\n"); scanf("%d", &n);
    if(is_palin(n))
        printf("%d is a palindrome\n", n);
    else
        printf("%d is not a palindrome\n", n);
}
```

```
C:\Users\Dell>gcc -c palin_client.c
C:\Users\Dell>gcc -c palin.c
C:\Users\Dell>gcc palin_client.o palin.o
C:\Users\Dell>a.exe
Enter the number
121
121 is a palindrome

C:\Users\Dell>a.exe
Enter the number
-121
-121 is not a palindrome

C:\Users\Dell>a.exe
Enter the number
5
5 is a palindrome
```

//Server code Implementation – palin.c

```
int is_palin(int n) // implementation
{
    int rev = 0;
    int number = n;
    while(n>0){
        rev = rev * 10 + n % 10; n /= 10;
    }
    return number == rev;
}
```

Execution Steps when using Header and Implementation Files in C.

1. Compile Each .c File into an Object File (.o)
2. Link All .o Files to Create the Final Executable
3. Run the Executable

For the above codes,

```
gcc -c palin.c palin_client.c // results in 2 .o files
```

```
gcc palin.o palin_client.o // results in an executable file
```

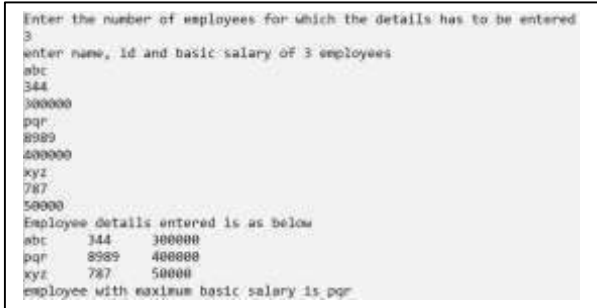
```
a.exe OR ./aout // Execute the executable
```

Let us consider an application in C to read and display the details of n Employees. Separate the interface and implementation.

Coding Example_4: Employee application segregating client and server and display the employee details whose salary is max.

//employee.h

```
typedef struct employee
{
    char name[20];
    int id;
    int basic_salary;
    int total_salary;
}employee_t;
void scan_details(employee_t*, int);
void display_details(const employee_t*, int);
int calculate_total_salary(employee_t*, int,int);
employee_t max(employee_t*, int);
```



```
Enter the number of employees for which the details has to be entered
3
Enter name, id and basic salary of 3 employees
abc
344
300000
pqr
8989
400000
xyz
787
50000
Employee details entered is as below
abc 344 300000
pqr 8989 400000
xyz 787 50000
Employee with maximum basic salary is pqr
```

```
// employee_server.c
```

```
#include "employee.h"
```

```
#include <stdio.h>
```

```
void scan_details(employee_t* e, int n){  
    for(int i = 0; i < n; i++)  
        scanf("%s %d %d", (e+i)->name, &(e+i)->id, &(e+i)->basic_salary);  
}
```

```
void display_details(const employee_t* e, int n){  
    for(int i = 0; i < n; i++)  
        printf("%s\t%d\t%d\n", (e+i)->name, (e+i)->id, (e+i)->basic_salary);  
}
```

```
employee_t max(employee_t* e, int n){  
    int i;    int max = 0;    employee_t e_max;  
    for(i = 0; i < n; i++){  
        if((e+i)->basic_salary > max) {  
            max = (e+i)->basic_salary;    e_max = *(e+i);  
        }  
    }  
    return e_max;  
}
```

```
//employee_client.c
```

```
#include <stdio.h>
```

```
#include "employee.h"
```

```
int main()
```

```
{  
    employee_t e[1000]; int n, id;  
    printf("Enter the number of employees for which the details has to be entered\n");  
    scanf("%d", &n);  
    printf("enter name, id and basic salary of %d employees\n", n);  
    scan_details(e, n);  
    printf("Employee details entered is as below\n");  
    display_details(e, n);  
    employee_t e_max = max(e, n);  
    printf("employee with maximum basic salary is %s", e_max.name);  
    return 0;  
}
```

Keep your code neat — headers to the rescue!!

Introduction

Environment variables are a set of dynamic named values that can affect the way running processes will behave on a computer. They are part of the environment in which a program executes and programs executed from the shell inherit all of the environment variables from the shell. The information about the context of the program under execution can be known using two ways:

- 1) Command line arguments
- 2) Environment variables

```
int main(int argc, char*argv[], char*envp[])
```

Standard environment variables

These are used for information about the user's home directory, terminal type, current locale, and so on. You can define additional variables for other purposes. The set of all environment variables that have values is collectively known as the **environment**. **Names of environment variables are case-sensitive and must not contain the character '='.** The values of environment variables can be anything that can be represented as a string. A value must not contain an embedded null character, since this is assumed to terminate the string.

Environment Access

Let us discuss few functions used to perform read and write Environment Variables. All these functions are available in **stdlib.h**

- **int setenv (const char *name, const char *value, int replace)**
 - The setenv function can be used to add a new definition to the environment. The entry with the name name is replaced by the value '**name=value**'
 - A null pointer for the value parameter is illegal
 - The value of third argument decides whether the name to be overwritten with the given value or not. If the third argument is non-zero, overwrite the name with the value. If it is 0, do not over write.
 - If the function is successful it returns 0. Otherwise the environment is unchanged and the return value is -1 and errno is set

- **char *getenv(const char *name)**
 - The getenv() function searches the environment list to find the environment variable name, and returns a pointer to the corresponding value string
 - The getenv() function returns a pointer to the value in the environment, or NULL if there is no match.
- **int putenv(char *string)**
 - The putenv() function adds or changes the value of environment variables. The argument string is of the form **name=value**. If name does not already exist in the environment, then string is added to the environment. If name does exist, then the value of name in the environment is changed to value. The string pointed to by string becomes part of the environment, so altering the string changes the environment
 - The putenv() function returns zero on success, or nonzero if an error occurs

Program2 : Demo of getenv and setenv functions

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
int main()
{
    char mypath[10000];
    //printf("%s\n", getenv("PATH"));
    char *path = getenv("PATH");
    /*
    printf("Before setting path is %s\n",path);           // the entire PATH
    setenv("PATH", ".", 1);
    printf("Before setting path is %s\n",path);           // new path, that is .
    */
    /*
    // But we should never change the environment variables this way. We need to concatenate
    the current directory path with existing PATH.
    path = getenv("PATH");
    printf("Before setting path is %s\n",path);
```

```
strcpy(mypath, path);
strcat(mypath, ".");    // concatenating current directory to mypath
                        // all paths are separated with : in ubuntu
                        // all paths are separated with ; in windows
setenv("PATH", mypath, 1); // setting mypath to new mypath
                        // last argument can be any non-zero value to overwrite

path = getenv("PATH");
printf("After setting path is   s\n",path); // the entire pvalue of PATH with ;. At the end
*/
/*
// Can we create new environment variable?
setenv("HOME","Sindhu",1); // HOME=Sindhu
path = getenv("HOME");
printf("after setting %s",path);
*/
printf("execution is over");
return 0;
}
```

Program2 : Demo of getenv and putenv functions

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
int main()
{
    char *p = getenv("PATH");
    if(p==NULL)
        printf("NOT AVAILABLE\n");
    else
    {
        /*
        // updating the existing PATH
        printf("before setting %s\n",p);
        strcat(p, "."); // : used as separator in ubuntu
```

```
//putenv(PATH = p);// error //putenv takes the string in the form of name=value
char path_v[100000];
strcpy(path_v,"PATH=");
strcat(path_v,p);
putenv(path_v); // correct way
p = getenv("PATH");
printf("after setting %s\n",p);
*/

//adding new env variable
/*
putenv("HOME=sindhu");
p = getenv("HOME");
printf("home is %s",p);
*/

}
return 0;
}
```

We can display all the environment variables using an array of character pointers specified in main()

```
int main(int argc,char* argv[],char* envp[])
```

Program 3: Using envp

```
#include <stdio.h>

int main(int argc, char *argv[], char * envp[])
{
    int i;
    for (i= 0; envp[i] != NULL; i++)
        printf("\n%d %s", i,envp[i]);
}
```

```
    return 0;  
}
```

Introduction

The execution of a 'C' program takes place as a multi stage process. The stages are: **Pre-processing, Compilation, Loading, Linking and Execution**. The first stage is Pre - processing, which is performed by the 'C' pre-processor. It is not a part of the compiler, but is a separate step in the compilation process. In simple terms, a C Preprocessor is just a **text substitution tool** and it instructs the compiler to do **required pre-processing before the actual compilation**. We refer to the C Preprocessor as CPP.

All preprocessor commands begin with a hash symbol (#). It must be the first nonblank character, and for readability, a preprocessor directive should begin in the first column. Possible to see the **effect of the pre-processor on source files** directly by using the **-E option of gcc**

Sr.No.	Directive & Description
1	#define -> Substitutes a preprocessor macro.
2	#include -> Inserts a particular header from another file.
3	#undef -> Undefines a preprocessor macro.
4	#ifdef -> Returns true if this macro is defined.
5	#ifndef -> Returns true if this macro is not defined.
6	#if -> Tests if a compile time condition is true.
7	#else -> The alternative for #if.
8	#elif -> #else and #if in one statement.
9	#endif -> Ends preprocessor conditional.
10	#error-> Prints error message on stderr.
11	#pragma ->Issues special commands to the compiler, using a standardized method.



Macro in C

#define: Used to define a Macro. Macro is a piece of code in a program which has been given a name. During preprocessing, when the 'C' preprocessor encounters the name, it substitutes the name with the piece of code.

Program 1: Macros doesn't judge anything

```
#include<stdio.h>

#define PI 3.14          // symbolic constant, macro, plain text substitution

// It doesn't judge anything
// No memory is allocated for this

int main()
{
    printf("%f",PI);
    return 0;
}
```

Program 2: No Memory allocation for Macro

// name given to 10 is MAX. No memory is allocated for MAX. So &MAX is an error.

```
#define MAX 10

int main()
{
    int a[MAX];

    scanf("%d", &MAX);          // Error
    return 0;
}
```



Unit 4: Preprocessor Directives

Program 3: Defining a string using #define

```
#include<stdio.h>

#define STR "Hello All"           //define can be used for strings as well

int main()

{

    printf("The string is %s\n",STR);    // The string is Hello All
    return 0;

}
```

Program 4: Macro with expression

```
#include<stdio.h>

#define STR 2+5*1                //macro with expression

int main()

{

    printf("Value is %d\n",STR); //STR replaced with 2+5*1 in preprocessing stage. During
    execution, expression is evaluated
    return 0;

}
```

Program 5: Macro with parameter

```
#include<stdio.h>

#define SUM(a,b) a+b // SUM is not a function. It is a macro.

int main()

{

    int a,b;

    printf("Enter two numbers:\n");
    scanf("%d%d",&a,&b);

}
```

```
printf("The sum of two numbers is %d\n",SUM(a,b));  
return 0;  
}
```

Program 6: Demo of plain text substitution

```
#include<stdio.h>  
  
#define sqr(x) (x*x) //change this to (x)*(x).  
  
int main()  
{  
  
    int y=8;  
  
    printf("%d",sqr(2+3));      // 11(2+3*2+3)      // Not (2+3)*(2+3)  
  
    return 0;  
}
```

What changes in Program 6 to perform $(2+3)*(2+3)$?

#define sqr(x) (x)*(x) – Think about this.

Program 7: Macro in another macro

```
#include<stdio.h>  
  
#define sqr(x) x*x  
#define cube(x) sqr(x)*x          // using sqr in cube  
  
int main()  
{  
  
    printf("The cube of 9 is d\n",cube(9));  
    return 0;  
}
```


Program 8: #define is used to define constants. These constants cannot be changed using assignment operator(=). Code results in error

```
#include<stdio.h>
#define MAX 200
int main()
{
    if(MAX==20)
        printf("hello");
    else
    {
        printf("U here??");
        MAX=20;// Error because no memory allocated for MAX
        // To avoid this, use #define and check
    }
    printf("MAX is %d",MAX);
    return 0;
}
```

Program 9: Redefining the macro with #define is allowed. But not advisable

//Results in warning

```
#include<stdio.h>
#define MAX 200
int main()
{
    if(MAX==20)
        printf("hello");
    else
    {
        printf("U here??");
        #define MAX 20
    }
    printf("MAX is %d",MAX);//20
    return 0;
}
```

Pre-defined Macros in C

Below program demonstrates few pre-defined constants in C. Comments in the program helps you to understand.

```
#include<stdio.h>

int main()
{
    printf("file name is %s",__FILE__);          // current file name
    printf("\nDate is %s",__DATE__);             // current date
    printf("\nTime is %s",__TIME__);             // time during preprocessing
    printf("\nC version is %d",__STDC_VERSION__); //This macro
    expands to the C Standard version number, a long integer constant of
    the form yyyyymmL where yyyy and mm are the year and month of the
    Standard version
    //Example: 199901L signifies the 1999 revision of the C standard

    printf("\nC version is %d",__STDC__); //In normal operation, this macro expands to the
    constant 1, to signify that this compiler conforms to ISO Standard C

    printf("\nLine number is %d",__LINE__);
    if( __LINE__ == 12)
        printf("hello");
    else
        printf("%d",__LINE__);
    return 0;
}
```

Enum v/s Macros

- ✓ Macro doesn't have a type and enum constants have a type int.
- ✓ Macro is substituted at pre-processing stage and enum constants are not.
- ✓ Macro can be redefined using #define but enum constants cannot be redefined.
However assignment operator on a macro results in error.

File Inclusion Directives

This type of preprocessor directive tells the compiler to include a file in the source code program. There are two types of files which can be included by the user in the program

- **Header File or Standard files:** These files contain definition of pre-defined functions like `printf()`, `scanf()` etc. To work with these functions, header files must be included. Different functions are declared in different header files. These files can be added using **`#include <file_name>`** where `file_name` is the name of the header file to be included. The angular brackets '`<`' and '`>`' tells the compiler to look for the file in standard directory(PATH).

Example: Standard I/O functions are in '`stdio.h`' file whereas functions which perform string operations are in '`string.h`'

- **User- defined files:** When a program becomes very large, it is a good practice to divide it into smaller files and include whenever needed. These types of files are user defined files. These files can be included as **`#include "filename"`**

Example: `#include "sort.h"`

Note: It is not compulsory to have standard file names using angular brackets and user defined files using double quotes. If we change the environment variable `PATH` to have current directory also, then we can use angular brackets for user defined files too. Demo is available in the live video.

Conditional Compilation Directives

Implies that in a particular program, all blocks of code will not be compiled. Rather, a few blocks of code will be compiled based on the result of some condition. Conditional Compilation in 'C' is performed with the help of the following Preprocessor Directives -> **`#ifdef`**, **`#ifndef`**, **`#if`**, **`#else`**, **`#elif`**, **`#else`**, **`#endif`**

Program 1: This program demonstrates *#ifdef*

If the identifier checked by *#ifdef* is defined, only then, statements followed by *#ifdef* will be included in the file to be compiled.

```
#define MAX 20
#include<stdio.h>

int main()
{
    #ifdef MAX    //No parentheses for #ifdef
        printf("MAX defined\n");    // If #define above is removed, this line is not
    included for compilation.
    #endif        // compulsory for #ifdef
    printf("Outside the scope of #ifdef\n");
}
```

Output: MAX defined

Outside the scope of *#ifdef*

Program 2: This program demonstrates *#ifndef*

If the identifier checked by *#ifndef* is defined, only then, statements followed by *#ifndef* will be included for compilation.

```
#include<stdio.h>

int main()
{
    #ifndef MAX    //No parentheses for #ifndef
        printf("MAX defined\n"); // not included for compilation
    #endif        // compulsory for #ifndef
    printf("Outside the scope of #ifndef\n");
    return 0;
}
```

Output: Outside the scope of *#ifndef*

Program 3: This program demonstrates #ifndef

If the identifier checked by #ifndef is not defined, only then, statements followed by #ifndef will be included for compilation.

```
#include<stdio.h>

int main()
{
    #ifndef MAX    //No parentheses for #ifndef
        printf("MAX not defined\n");    // If #define above is added, this line is not
included for compilation.
    #endif        // compulsory for #ifndef
    printf("Outside the scope of #ifndef\n");
    return 0;
}
```

Output: MAX not defined

Outside the scope of #ifndef

Program 4: This program demonstrates #if

```
#include<stdio.h>

int main()
{
    #if (2>1)    //Syntax:  #if Constant_Expression
        // #if 2<1. True will not be printed
        printf("True\n");
    #endif
    printf("Outside if\n");
    return 0;
}
```

Output: True

Outside if

Program 5: This program demonstrates #if and #else

```
#include<stdio.h>

int main()
{
    #if ((2+3) < 2)
        printf("Hello\n");
    #else
        printf("Hi\n");

    #endif

    return 0;
}
```

Program 6: This program demonstrates #elif

If the result of #if or #ifdef or #ifndef is not true, then the control moves to #elif Preprocessor Directive where it has another constant expression to test. If the constant expression is a non - zero value, then the statements within the scope of #elif will be included for compilation.

```
#include<stdio.h>
```

```
int main()
```

```
{      #if (12%5 == 0)

        printf("Multiple of 5\n");

    #elif (12 % 3 == 0)

        printf("Multiple of 3\n");

    #endif

}
```

Output: Multiple of 3

Other directives

#undef: Used to undefine an existing macro. This directive works as: #undef LIMIT. Using this statement will undefine the existing macro LIMIT. After this statement every “#ifdef LIMIT” statement will evaluate to false.

Pragma directives :

This directive is a special purpose directive and is used to turn on or off some features. This type of directives are compiler-specific i.e., they vary from compiler to compiler. Some of the #pragma directives are discussed below.

1. **#pragma startup and #pragma exit:** These directives helps us to specify the functions that are needed to run before program startup(before the control passes to main()) and just before program exit (just before the control returns from main()).

Pragma 1.c

```
void func1();
void func2();
#pragma startup func1
#pragma exit func2
void func1()
{
    printf("Inside func1()\n");
}
void func2()
{
    printf("Inside func2()\n");
}
int main()
{
    printf("Inside main()\n");
    return 0;
}
```

Output:

```
Inside func1()
Inside main()
Inside func2()
```

2. **#pragma warn Directive:** This directive is used to hide the warning message which are displayed during compilation. We can hide the warnings as shown below:
- **#pragma warn -rvl:** This directive hides those warning which are raised when a function which is supposed to return a value does not returns a value.
 - **#pragma warn -par:** This directive hides those warning which are raised when a function does not uses the parameters passed to it.

- `#pragma warn -rch`: This directive hides those warning which are raised when a code is unreachable. For example: any code written after the *return* statement in a function is unreachable.

Pragma 2.c: If your version of gcc compiler supports `pragma warn`, then comment the first pragma and second pragma separately and observe the output of compilation

```
#include<stdio.h>

#pragma warn -rvl /* return value */
#pragma warn -par /* parameter never used */
#pragma warn -rch /*unreachable code */

int show(int x)
{
    // parameter x is never used in the function
    printf("PESUNIVERSITY");
    // function does not have a return statement
}

int main ()
{
    show (10);
    return 0;
}
```

Output: PESUNIVERSITY

3. #pragma pack(n) where *n* is the alignment in bytes, valid alignment values being 1, 2, 4 and 8. This pragma aligns members of a structure to the minimum of *n* and their natural alignment. Packed objects are read and written using unaligned accesses.

Pragma 3.c

```
#include<stdio.h>
//#pragma pack(n) //n=1 2 or 4
#pragma pack(1)
struct sample
{
    int a; //4 bytes
    char b; //1 byte + 3 padding
    int x; //4 byte
};
int main()
{
    printf("%lu\n",sizeof(struct sample));
    return 0;
}
```

Output: 12

Note: If the compiler doesn't support pragma, then it ignores that statement. No error

Portable program development

A portable application is a software product designed to be easily moved from one computing environment to another. To make the part of the code to be compiled and executed, we check whether the below macros are set or not.

If **mingw in windows system**, the value of `__MINGW32` will be 1

If **unix/Linux system** is used, the value of `__unix` is 1

If **mac system** is used, the value of `APPLE` is 1

The following program demonstrates the **real use of conditional compilation and pre-defined macros that makes the code portable across different platforms.**

Program:

// Use gcc -E option with this code initially to check which part of the code is provided for compilation

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    #ifdef __MINGW32__ // No ( and ) // if mingw in windows system is used, the value of this macro is 1
```

```
        printf("windows system");
```

```
        char c,d;
```

```
        printf("enter the character");
```

```
        scanf("%c",&c);
```

```
        fflush(stdin);
```

```
        printf("enter one more character");
```

```
        scanf("%c",&d);
```

```
        printf("entered characters are %c %c",c,d);
```

```
    #elif __unix__ // if unix/Linux system is used, the value of this macro is 1
```

```
        char c;
```

```
        printf("unix system");
```

```
#include<stdio_ext.h>
printf("enter the character");
scanf("%c",&c);
__fpurge(stdin);
printf("enter one more character");
scanf("%c",&d);
printf("entered characters are %c %c",c,d);
#ifdef __APPLE // if MAC system is used, the value of this macro is 1
printf("mac system");
#else
printf("other system");
#endif
return 0;
}
```